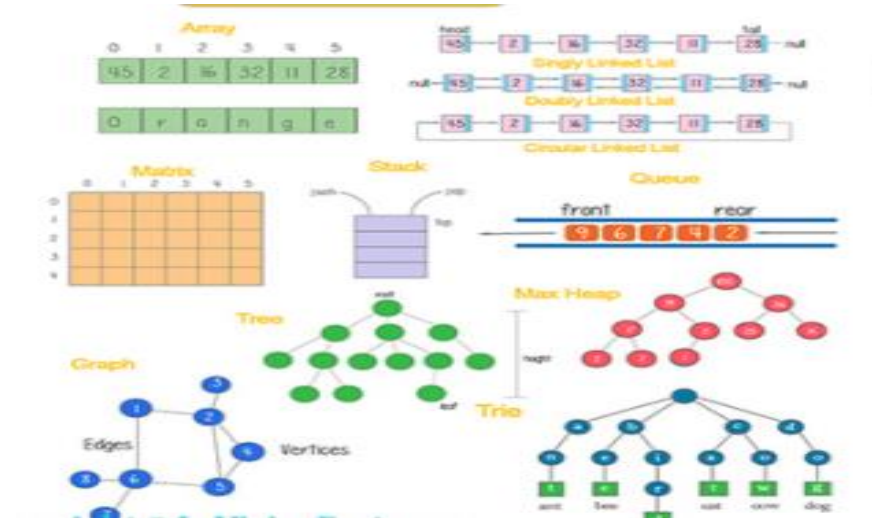


Data Structures

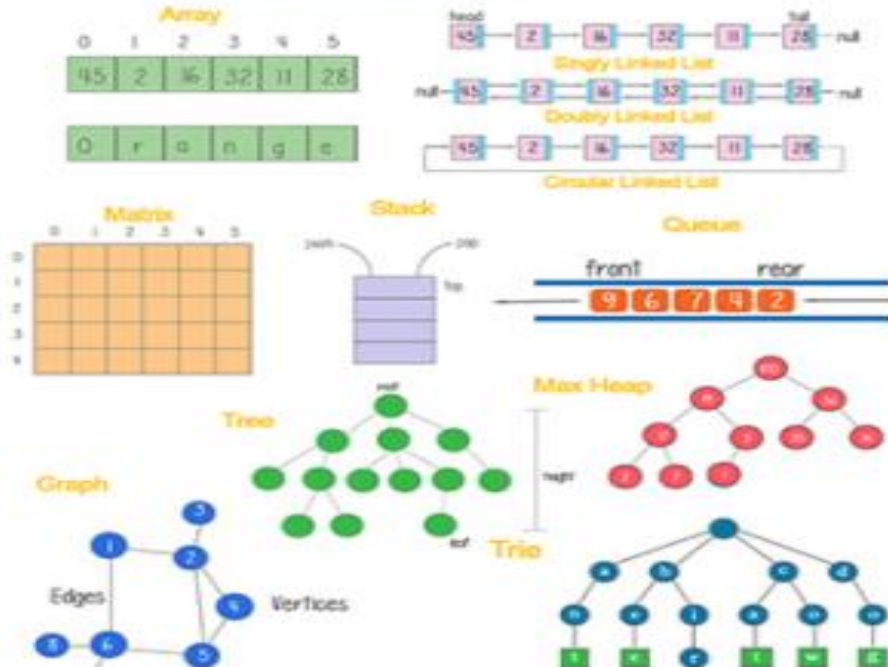


Dr. Pandu Sowkuntla
Asst. Professor,
Department of CSE



SRM
UNIVERSITY AP
— Andhra Pradesh

Data structures



UNIT-I



INTRODUCTION TO DATA STRUCTURES

Outline of Unit-I

Data Structures

Unit-I

Unit-II

Unit-III

Unit-IV

Unit-V

1. Introduction to Data Structures

- ▶ Why Data Structures?
- ▶ What is Data Structure?
- ▶ Types of Data Structures

2. Abstract Data Types (ADT)

- ▶ Why ADT?
- ▶ What is ADT?
- ▶ ADT implementation

3. Algorithm Efficiency

- ▶ What is an Algorithm?
- ▶ Pseudocode?
- ▶ Complexity Analysis

- Big-O notation
- Time Complexity
- Space Complexity

4. Applications of Array ADT

- ▶ Sparse matrix representation & implementation
- ▶ **Polynomial Expressions representation & implementation**

5. Stack ADT

- ▶ Stack representation
- ▶ Array implementation of Stacks
- ▶ Applications of Stacks

- Infix, prefix, postfix Conversions
- Evaluation of Postfix and Prefix expressions

6. Queue ADT

- ▶ Queue representation
- ▶ Array implementation of Queues
- ▶ Circular Queues
- ▶ Applications of Queues

What is a variable?

`int a;`

a



2 Bytes

`float b;`

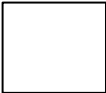
b



4 Bytes

`char c;`

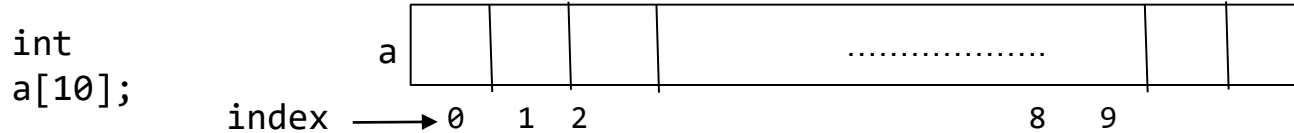
c



1 Byte

A variable is a name of the **memory location**.

○ What is an Array?



Array is a data structure used to store homogeneous elements at **consecutive memory locations**.

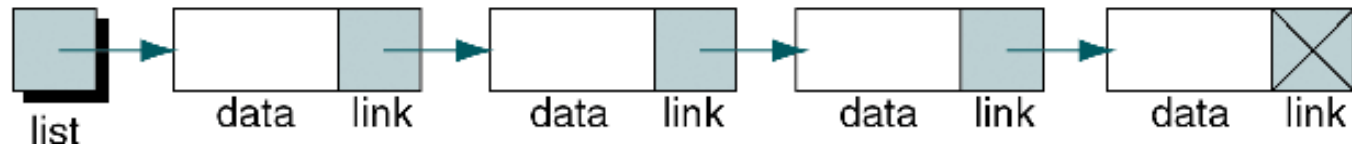
1. Is it possible to store element at the location of `a[9]` (at the end)?

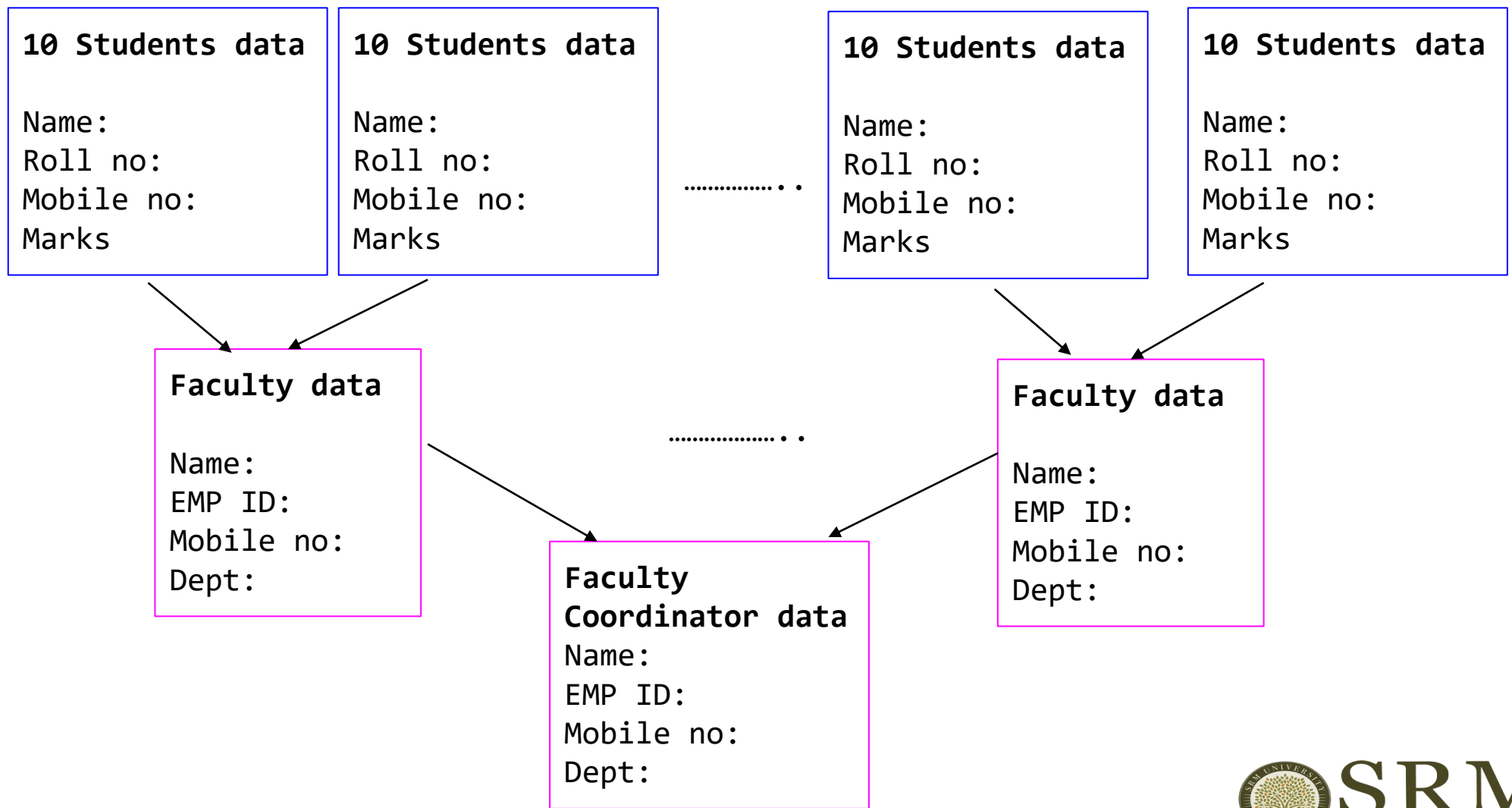
YES

2. Is it possible to insert the elements at the middle or at the beginning locations of the array? **NO**

3. Is it possible to store the elements into the array beyond the size of the array? **NO**

If array is stored as **Linked List**





Why Data Structures?

- ▶ To **store**, **process** and **retrieve** the large amounts of complex data efficiently.
- ▶ To design **efficient algorithms** (programs).
- ▶ Programmers can save **time and memory space** in storing, accessing and processing large amounts of data.

✓ **Efficiency**

✓ **Reusability**

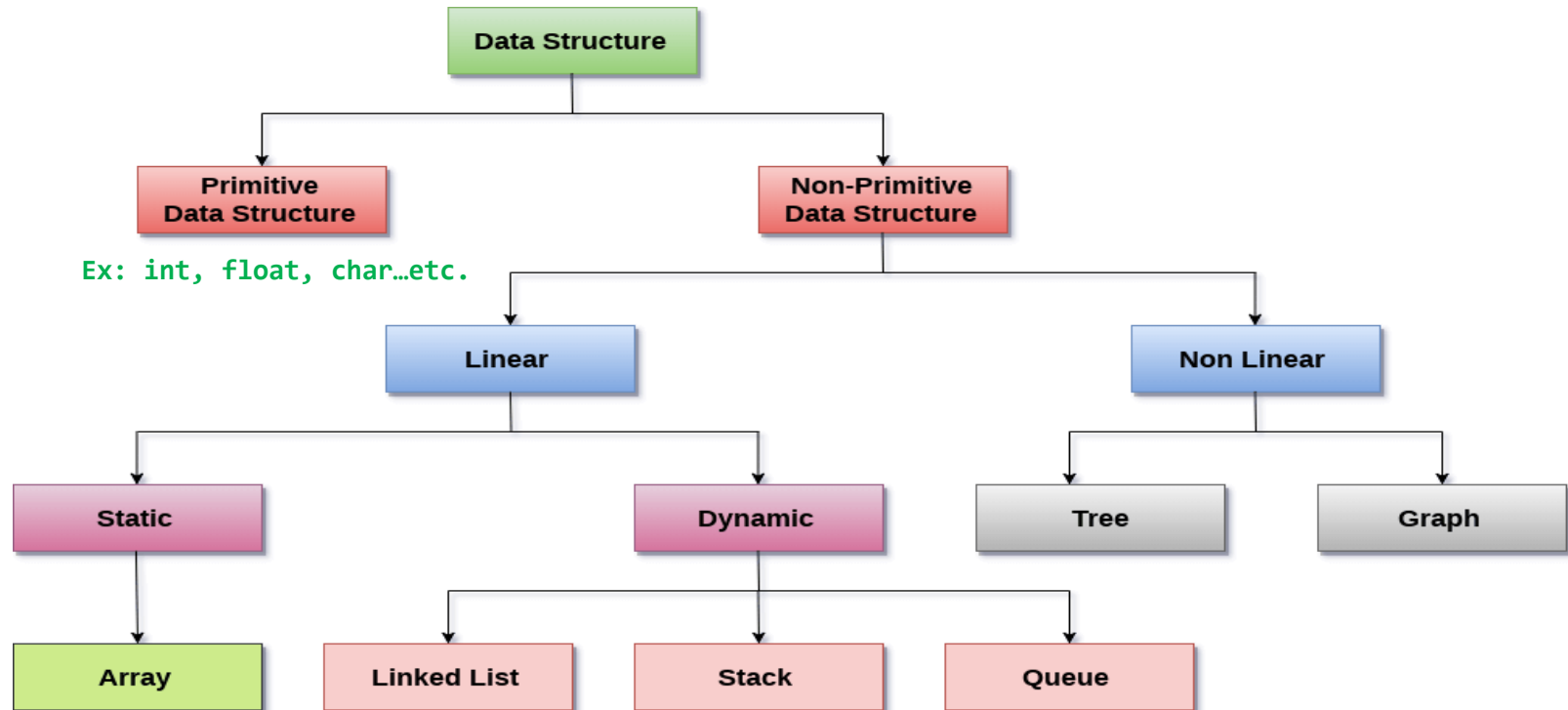
✓ **Abstraction**

What is Data Structure?

- ▶ A data structure is an aggregation of atomic and composite data into a set with the defined relationships.

OR

- ▶ A data structure is a particular way of organizing data in a computer so that it can be used effectively.



○ What is Data?

Collections of facts from which conclusions **may be** drawn is known as data.

○ What is Information?

Data from which **decisions** can be made is known as information.

► Atomic Data

The data that consists of single piece of information.

```
int a=2314;
```

(Data can not be divided further)

► Composite Data

The data that can be divided (not an atomic data)

+91 9231093219

Country code

Mobile no

Date: **dd/mm/yyyy**

What is a Data Type?

Type	Values	Operations
integer	$-\infty, \dots, -2, -1, 0, 1, 2, \dots, \infty$	$*, +, -, \%, /, ++, --, \dots$
floating point	$-\infty, \dots, 0.0, \dots, \infty$	$*, +, -, /, \dots$
character	$\backslash 0, \dots, 'A', 'B', \dots, 'a', 'b', \dots, \sim$	$<, >, \dots$

A Data Type consists of two parts: **a set of data**, and the **operations** that can be performed on the data.

We got answers for following questions

What is Data?

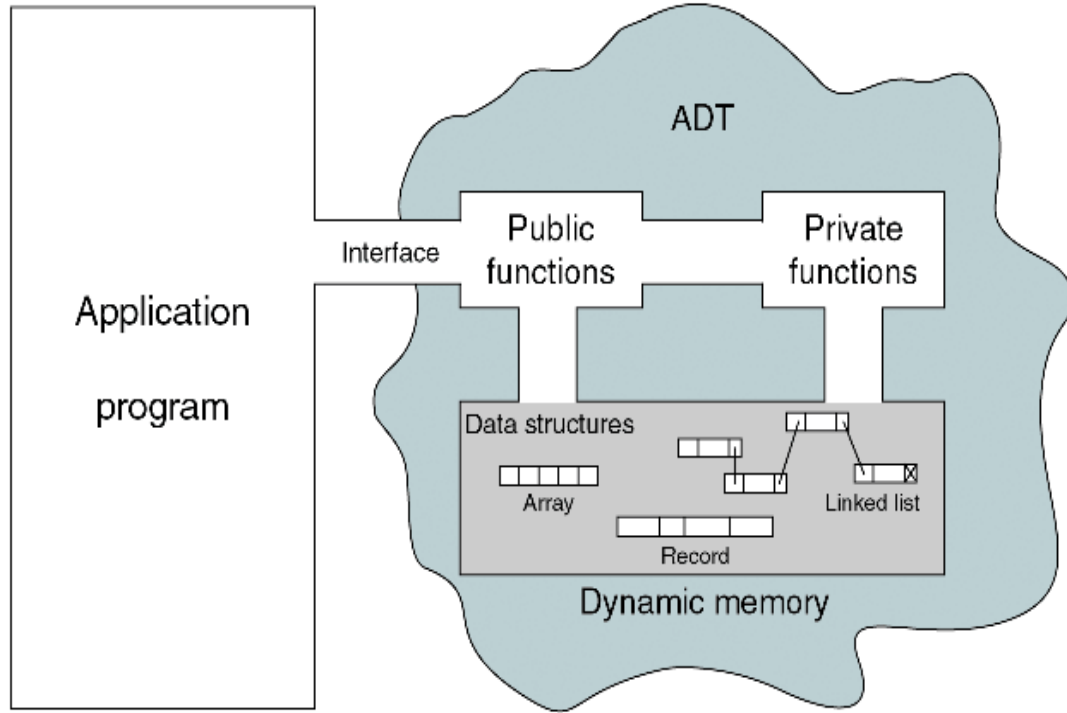
What is Data Type?

What is Data Structure?

Why Abstract Data Types?

1. Can I retrieve the elements from the array in last in first out (LIFO) or first in last out manner?
 2. Can I retrieve the first stored element first (first in first out (FIFO)) manner?
 3. Can I store the data of DATE in the format dd/mm/yyyy?
 - Can I do addition, subtraction operations on date data
 - Can I add some days?
 - Can I find the day for the specified date?
- Can a programmer create his own data types (user defined data types) for solving above type of problems?

An abstract data type is a data declaration packaged together with the operations that are meaningful for the data type.



Abstract Data Type Model

➤ The concept **Abstraction** means:

- We know what a data type can do
- How it is done is hidden (i.e., implementation details are hidden)

➤ Programmer **encapsulates** the data and operations on the data and hides the implementation details.

There are two basic structures we can use to implement an ADT:

1. **Array implementations**
2. **Linked list implementations**

Algorithm Efficiency

► Why Algorithm?

- Provides a design to solve a problem.
- Improves the efficiency in problem solving.
- To utilize the resources efficiently in problem solving.

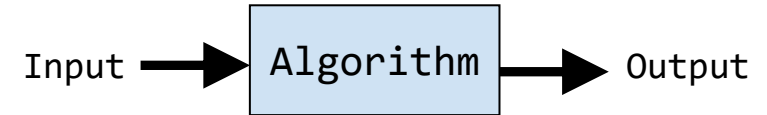


al-Khwarizmi

9th-century's Arab scholar

► What is an Algorithm?

An algorithm is finite set of well-defined instructions, if followed, accomplishes a particular task.



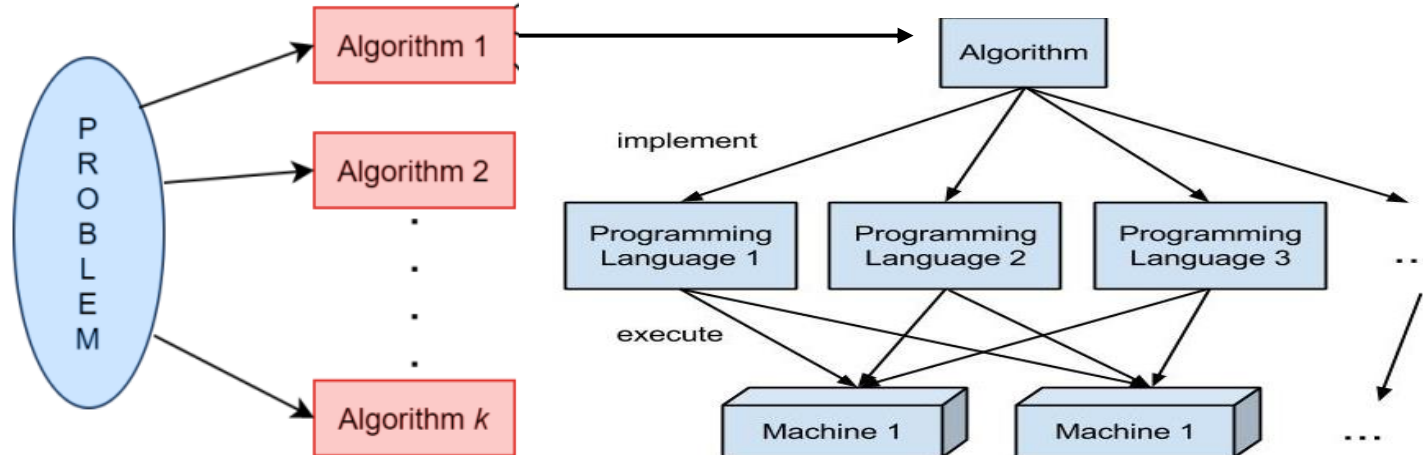
► An Algorithm must satisfy the following criteria.

- **Input:** Zero or more quantities which are externally supplied.
- **Output:** At least one quantity is generated.
- **Finiteness:** Execution terminates after a finite number of steps for all possible inputs.
- **Definiteness:** Each step must be clear and unambiguous.
- **Effectiveness:** Every step must be given enough information to produce results.

Algorithm v/s Program

- ▶ An algorithm is a well-defined, systematic logical approach that comes with a step-by-step procedure for computers to solve any given problem.
- ▶ We can express algorithms using flowcharts, natural language, and many more.
- ▶ Algorithm should be finite.

- ▶ It is a programming code written using a programming language to implement the algorithm of a given problem.
- ▶ Program can be written in a programming language and then use a compiler or interpreter (for compiling and interpreting) so that it becomes understandable for any computer system.
- ▶ Program need not be finite.



- ▶ We can define or describe an algorithm in two ways:

1. Pseudocode
2. Flowchart

- ▶ Pseudocode is an English like representation of the algorithm logic. It is part English, and part structured code (syntax).

Conventions used in writing Pseudocode

- `for`, `if`, `else`, `return` and `while` (similar to C, Java, Python, etc.)
- `//` is used for commenting.
- `and`, `or` & `not` are boolean operators similar to `&&`, `||` & `!`
- Indentation can be used to represent the body block.
- `==`, `!=`, `<`, `>`, `<=` and `>=` are used to compare values.
- `A[l,r]` means a subarray of A, indexed from 'l' to 'r'.

- ▶ **Note:** In this Data Structures course, we use the C programming language syntax in writing pseudocodes.

Example

Algorithm name

Algorithm Max(*a*, *n*)

Algorithm header

//Algorithm used to find the maximum element in the given array *a*
of size *n*

Comments

1. **begin** // Beginning of the algorithm

2. *max* = *a*[0];

3. **for** *i* = 2 to *n* do

4. **if** *a*[*i*] > *max* then

5. *max* = *a*[*i*];

6. **end if**

7. **end for**

8. **return** *max*

9. **end** // End of the algorithm

'{' can be used to begin

Loop
statement

Conditional
statement

'}' can be used to end

Statement number

a, *i*, and *max* are variables

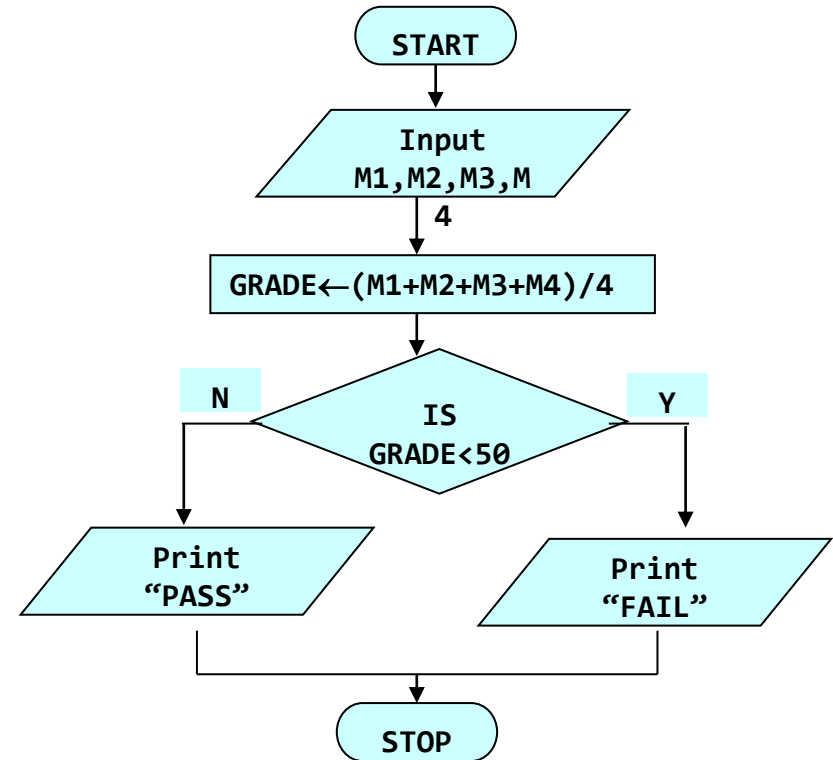
Flowchart

- Flow chart is the graphical representation of an algorithm
- Different symbols exist to represent different operations of the algorithm.

Pseudocode

```
Step 1:  Input M1,M2,M3,M4
Step 2:  GRADE = (M1+M2+M3+M4)/4
Step 3:  if (GRADE < 50) then
                                Print "FAIL"
                                else
                                Print "PASS"
                                end if
```

Flowchart



Need of analyzing efficiency of the Algorithm

```
for i=2 to n-1 do
    if i divides n
        n is not prime
```

```
for i=2 to  $\sqrt{n}$  do
    if i divides n
        n is not prime
```

If **division** operation takes **1 msec** time

Loop executes **n-2** times

If n=12, loop executes 10 times--→10 msec

If n=102, loop executes 100 times----→ 100 msec

If n=1002, loop executes 1000 times---→1000 msec=1 sec

n=1000002, loop executes 1000000 times--→1000*1000
=1000 sec
=1000/60=**16.6 min**

Loop executes $\sqrt{n}$ -1 times

If n=12, loop executes 3 times--→2 msec

If n=102, loop executes 10 times----→ 10 msec

If n=1002, loop executes 31 times---→31 msec

n=1000002, loop executes 1000 times--→1000 msec
=**1 sec**

(Values are approximate results)

Main criteria for judging the merits of algorithms:

- ▶ **Correctness:** Does algorithm give solution to the problem in a finite no. of steps?
- ▶ **Efficiency:** How much resources (memory, time, ..etc.) does it take to execute.
- ▶ **Execution time:** It is specific to a particular computer (Not a suitable measure).
- ▶ **Number of statements in algorithm:** Not a good measure, since the no. of statements varies with the programming language and the style of the individual programmer.

Ideal measure:

If the running time (or memory space) of a given algorithm is a function of the input size n (i.e., $f(n)$) and compared to different functions corresponding to running times (or memory spaces).

This kind of comparison is independent of machine time, space and programming style, ...,etc.

The RAM model of computation:

The RAM (Random Access Machine) model is used for analyzing algorithms without running them on a physical machine.

The RAM model has the following properties:

- A simple operation (+, \, *, -, =, if) takes one time step.
- Loops and subroutines are comprised of simple operations.
- Memory is unlimited and access takes one time step (the RAM model does not consider whether data is on cache or disk).

Using the RAM model, we measure the running time of an algorithm by counting the number of steps an algorithm takes on a given input.

Asymptotic Analysis and notations

To analyze an algorithm, we need **some kind of syntax**, and that forms the base for **asymptotic analysis/notation**.

Worst case (O)

Defines the input for which the algorithm takes a long time (slowest time to complete). Input is the one for which the algorithm runs the slowest.

Best case (Ω)

Defines the input for which the algorithm takes least time (fastest time to complete). Input is the one for which the algorithm runs the fastest.

Average case (Θ)

Run the algorithm many times, using many different inputs that come from some distribution that generates these inputs, compute the total running time (by adding the individual times) and divide by the number of trials.
Assumes that the input is random.

$$\text{Lower Bound} \leq \text{Average Time} \leq \text{Upper Bound}$$

- ▶ **Time complexity** of an algorithm is the amount of computer time it needs to run to completion.
- ▶ **Space complexity** of an algorithm is the amount of computer memory it needs to run to completion.

Rate of Growth

The rate at which the running time (or memory space) increases as a function of input is called rate of growth or order of growth.

Example:
$$f(n) = n^5 + \frac{n^2}{2} + \frac{n}{2} + 1000 = n^5$$

Big-O Notation

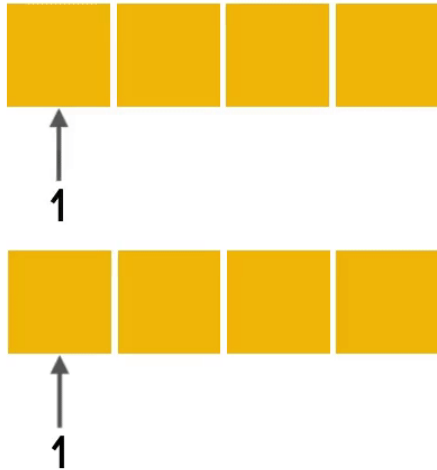
We do not need to determine the complete measure of the efficiency, only the factor that determines the magnitude is considered.

The factor is denoted with **big-O**, and read it as “in the order of.”

Algorithm (Efficiency) Analysis ↔ Big-O Analysis

EXAMPLE1 (A, n)

for i in 1 to A.length	c_1	$n + 1$
print i	c_2	n
for i in 1 to A.length	c_3	$n + 1$
print i	c_4	n



The growth (cost) function:

$$C = c_1 * (n + 1) + c_2 * n + c_3 * (n + 1) + c_4 * n$$

$$C = (c_1 + c_3 + c_2 + c_4)n + c_1 + c_3$$

Time complexity is $c = O(n)$

Time Complexity

1. Linear Complexity

```
for(i=1; i<=1000; i++)  
{  
    ----- Code  
}
```

Loop executes 1000 times

```
for(i=1; i<=n; i++)  
{  
    ----- Code  
}
```

Loop executes n times

Complexity is $O(n)$ (in the order of n)

```
for(i=1; i<=1000; i+=2)  
{  
    ----- Code  
}
```

Loop executes 500 times

```
for(i=1; i<=n; i+=2)  
{  
    ----- Code  
}
```

Loop executes $n/2$ times

Complexity is $O(n/2)=O(n)$

Time Complexity

2. Logarithmic Complexity

```
for(i=1; i<=1000; i*=2)
{
    ----- Code
}
```

i=1, iteration=1
i=2, iteration=2
i=4, iteration=3
i=8, iteration=4
i=16, iteration=5
:
:
i=512, iteration=10
i=1024, Exit

$$2^{10} \leq 1000$$

$$\log 1000 = \log 2^{10} = 10$$

```
for(i=1; i<=n; i*=2)
{
    ----- Code
}
```

```
for(i=1000; i>=1; i/=2)
{
    ----- Code
}
```

i=1000, iteration=1
i=500, iteration=2
i=250, iteration=3
i=125, iteration=4
i=62, iteration=5
:
:
i=1, iteration=10
i=0, Exit

$$1000 / 2^{10} >= 1$$

$$\log 1000 = \log 2^{10} = 10$$

```
for(i=n; i>=1; i/=2)
{
    ----- Code
}
```

Complexity= **$O(\log n)$**

3. Quadratic Complexity

EXAMPLE2 (A, n)

for i in 1 to A.length

for j in 1 to A.length

print j

→ c_1

→ c_2

→ c_3

$$\sum_{i=1}^n \sum_{j=1}^n (n+1)$$

Time complexity is $c = O(n^2)$

Time Complexity

3. Quadratic Complexity

```
for(i=1; i<=n; i++)  
{  
    for(j=1; j<=n; j++)  
    {  
        ----- Code  
    }  
}
```

Code executes $n*n$ times

Time Complexity = $O(n^2)$

```
for (i = 1; i <=n; i++)  
{  
    for (j = 1; j <=i; j++)  
    {  
        ----- Code  
    }  
}
```

$i=1, j=1$, code executes=1 time

$i=2, j=1,2$ ----→2 times

$i=3, j=1,2, 3$ ----→2 times

:

:

$i=n, j=1,2, 3...n$ ----→n times

Total no. of times = $1 + 2 + 3 + \dots + n$
 $= \frac{n(n+1)}{2}$
 $= \frac{(n^2+n)}{2}$
 $= \frac{n^2}{2} + \frac{n}{2}$

Time Complexity = $O(n^2)$

Time Complexity

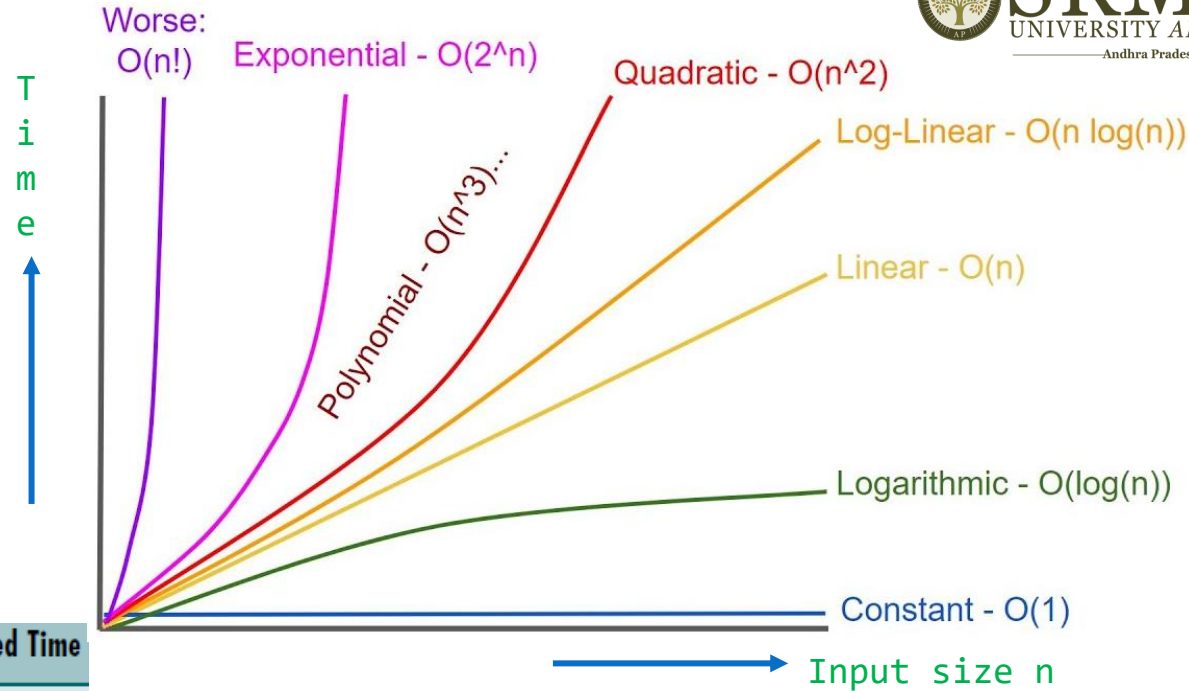
4. Linear logarithmic Complexity

```
for(i=1; i<=n; i++)  
{  
    for(j=1; j<=n; j++)  
    {  
        -----  
        -----  
    }  
}
```

Time Complexity = $O(n \log n)$

Algorithm Efficiency

Practical Time Complexities



Efficiency	Big-O	Iterations	Estimated Time
Logarithmic	$O(\log n)$	14	microseconds
Linear	$O(n)$	10,000	seconds
Linear logarithmic	$O(n(\log n))$	140,000	seconds
Quadratic	$O(n^2)$	$10,000^2$	minutes
Polynomial	$O(n^k)$	$10,000^k$	hours
Exponential	$O(c^n)$	$2^{10,000}$	intractable
Factorial	$O(n!)$	$10,000!$	intractable

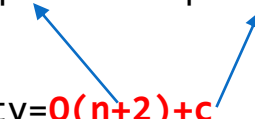
► Space Complexity

```
Algorithm Sum(a, n)
// To find the sum of array elements
1. begin
2.     sum=0.0;
3.     for i=1 to n do
4.         sum=sum+a[i];
5.     end for
6.     return sum;
7. end
```

Variable `i` takes 1 space
Variable `sum` takes 1 space
Variable `a` takes `n` spaces

Total space= `n+2` spaces + Space for the code

Space complexity= $O(n+2)+c$
 $=O(n)$



Space complexity=Fixed part+ Variable part



Space for the code Depends on variation in problem

*Important Note

Types of complexities:

1. Worst case (Big O)
2. Best case
3. Average

Give worst case and best-case time complexities for the following code

```
int isPrime(int n)
{
    int i, n;
    for (i=2; i<= $\sqrt{n}$  ; i++)
    {
        if( n % i ==0)
        {
            printf("Not Prime")
            return 0;
        }
    }
}
```

Worst case:

Best case:

Array ADT

- A matrix is a two-dimensional data structure consisting of m rows and n columns, with a total of $m \times n$ values.
- 2D-Arrays are used to represent matrices.
- A sparse matrix is one in which the majority of the elements have 0 (zero) value.

► Why would you want to use a new representation?

- To save the storage space.
- To reduce the computing time.

► Sparse Matrix can be represented in two ways:

- Array representation.
- Linked list representation.

`a[0][2]=9,`
`a[1][1]=1,`
`a[2][0]=5,`
`a[2][2]=9,`
`a[3][0]=8,`
`a[3][3]=6`

Example

	0	1	2	3
0	0	0	9	0
1	0	1	0	0
2	5	0	9	0
3	8	0	0	6

Sparse Matrix

A matrix contains zero values more than half of its size.
(if (no.of zeros > Size(Array)/2))

Sparse Matrix using Array ADT

	0	1	2	3
0	0	0	9	0
1	0	1	0	0
2	5	0	9	0
3	8	0	0	6

Sparse Matrix:a

First row contains
no. of rows and cols
in the matrix

We store the non-zero elements
with
triple: <row, column, value>.

Sparse Matrix representation:b

	0	1	2
row	col	value	
0	4	4	6
1	0	2	9
2	1	1	1
3	2	0	5
4	2	2	9
5	3	0	8
6	3	3	6

Total no. of
non-Zero values

Row indices

Col indices

non-Zero values

Operations performed on Sparse matrices

1. Creation
2. Transpose
3. Addition
4. Multiplication

Algorithm for representing Sparse Matrix

Sparse matrix: a

	0	1	2	3
0	0	0	9	0
1	0	1	0	0
2	5	0	9	0
3	8	0	0	6

Sparse matrix representation: b
row size varies but col size fixed(=3)

	0	1	2
	row	col	value
0	4	4	6
1	0	2	9
2	1	1	1
3	2	0	5
4	2	2	9
5	3	0	8
6	3	3	6

Algorithm SparseM(a, row, col, b)

// Algorithm to represent Sparse matrix

begin

b[0][0] = row;

b[0][1] = col;

k = 1;

for i = 0 to row do

for j = 0 to col do

if (a[i][j] != 0) then

b[k][0] = i;

b[k][1] = j;

b[k][2] = a[i][j];

k++;

end if

end for

end for

b[0][2]=k-1;

return b;

end

Algorithm to Transpose a Sparse Matrix

Original matrix

row	col	val
4	4	5
1	2	10
1	4	12
3	3	5
4	1	15
4	2	12

transpose

Transposed matrix

row	col	val
4	4	5
2	1	10
4	1	12
3	3	5
1	4	15
2	4	12

```
for each row i
    take element <i, j, value> and
    store it as element <j, i, value> in the
    transpose
```

Required transposed matrix

transpose

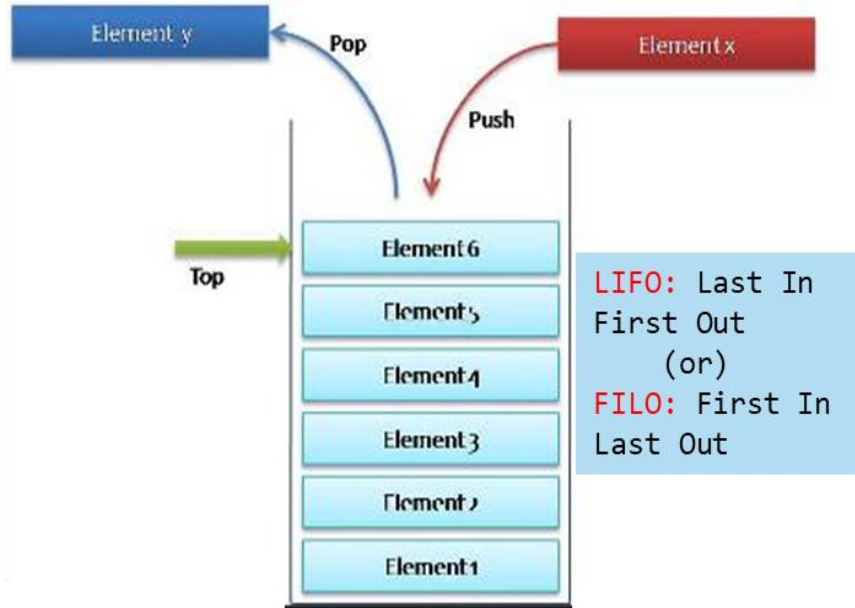
row	col	val
4	4	5
1	4	15
2	1	10
2	4	12
3	3	5
4	1	12

```
for all elements in j
    place element <i, j, value> as
    element <j, i, value> in the transpose
```

Stack Data Structure

Stacks

- Stack is a linear data structure in which items can be **inserted and removed only at one end, called the top.**
- The last item inserted into stack, is the first item to be taken out from the stack.



➤ Operations on Stack

- **push(data):** adds an element to the top of the stack
- **pop():** removes an element from the top of the stack
- **peek() or top():** returns a copy of the element on the top of the stack without removing it.
- **is_empty():** checks whether a stack is empty.
- **is_full():** checks whether a stack is at maximum capacity **when stored in a static (fixed-size) structure.**

Stack implementation

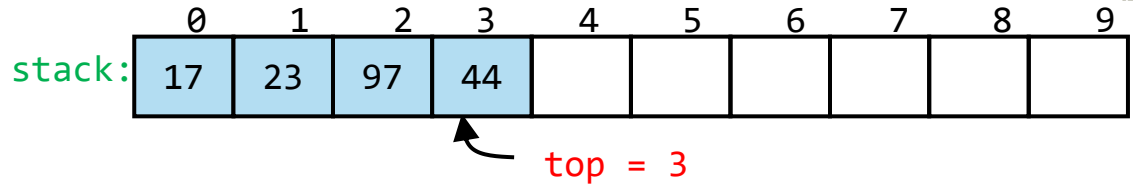
Stacks can be implemented in two ways:

1. Arrays.

- ▶ Easy to implement, but the size of the array is fixed (static)
- ▶ Extra memory is not needed for pointer variables.
- ▶ It is not dynamic. It doesn't grow and shrink depending on needs at runtime.
- ▶ Performing operation **is_full()** is required.

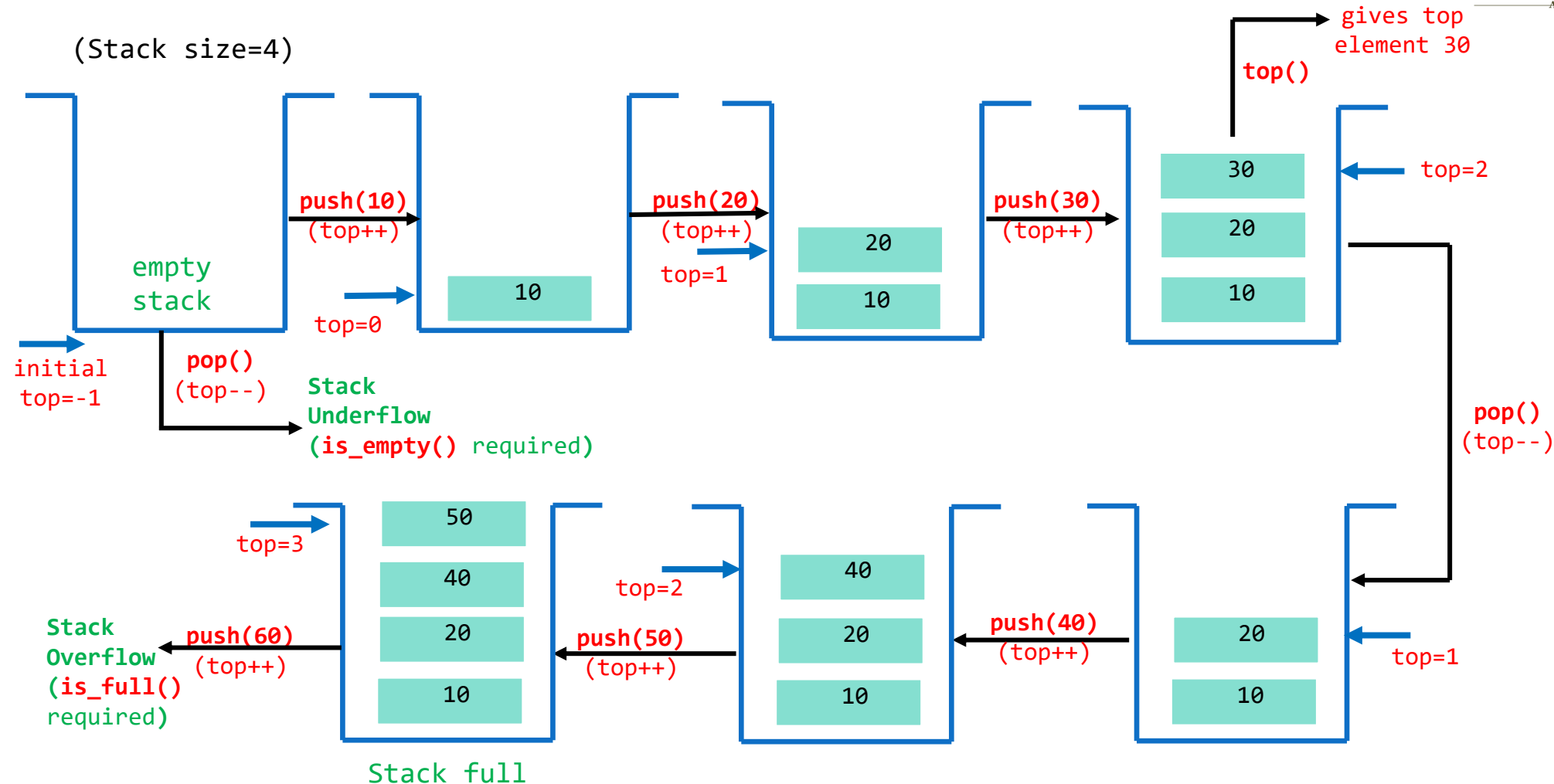
2. Linked lists.

- ▶ This implementation of a stack can grow and shrink according to the needs at runtime.
- ▶ Extra memory is needed for pointer variables.



Operations on Stack (Array implementation)

(Stack size=4)



```
MAX_SIZE = 100  
ARRAY stack[MAX_SIZE]  
top = -1;
```

Pseudocodes for Stack Operations (Array implementation)

```
Algorithm: is_full(top)  
begin  
  
    if (top == MAX_SIZE - 1) then  
        return True;  
    else  
        return False;  
    end if  
  
end
```

```
Algorithm: push(stack, top, data)  
begin  
  
    if (is_full(top) == True) then  
        print("Stack is full");  
    else  
        top = top + 1;  
        stack[top] = data;  
    end if  
    return top;  
  
end
```

```
Algorithm: is_empty(top)  
begin  
  
    if (top == -1) then  
        return True;  
    else  
        return False;  
    end if  
  
end
```

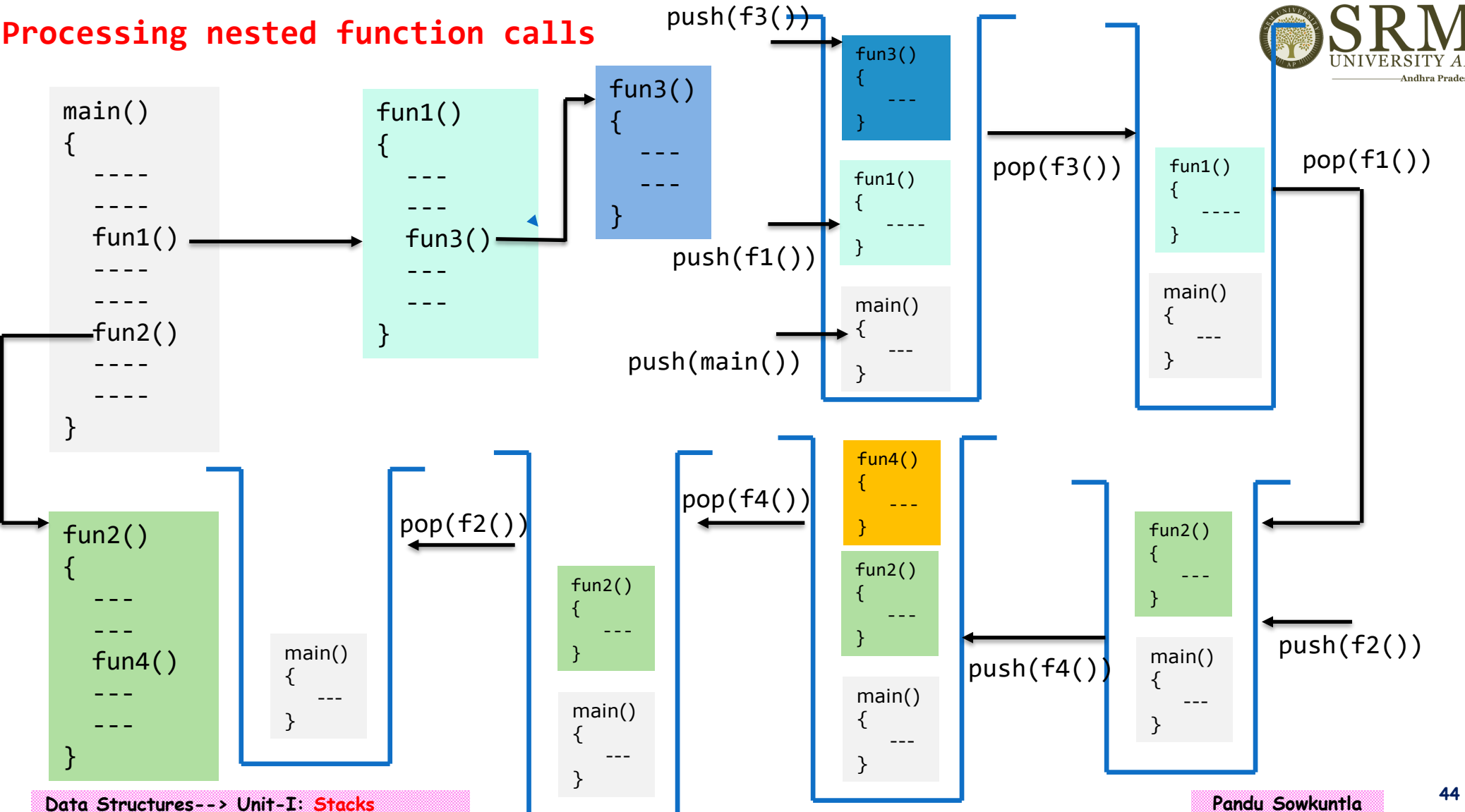
```
Algorithm: pop(stack, top)  
begin  
  
    if (is_empty(top) == True) then  
        print("Stack is empty");  
    else  
        popped_item = stack[top];  
        top = top - 1;  
    end if  
    return (popped_item, top);  
  
end
```

Linked list implementation of Stack

Applications of Stacks

- ▶ Arithmetic Expression Evaluation.
 - ▶ Expression Conversion.
 - ▶ Processing function calls.
 - ▶ Processing recursive calls (Recursion).
 - ▶ Parenthesis Checking.
 - ▶ Backtracking: To track the recursive calls in building optimal solutions.
 - ▶ Syntax Parsing (in Compilers).
 - ▶ Memory Management(in Operating Systems).
 - ▶ String Reversal.
 - ▶ Text Editors: Undo() or Redo () mechanism (Excel, Notepad or WordPad etc.)
 - ▶ Backward() and forward() buttons in browser.
- Infix expression to Prefix expression.
 - Infix expression to Postfix expression.
 - Evaluating postfix expression.
 - Evaluating prefix expression.

Processing nested function calls



Processing recursive calls (Recursion)

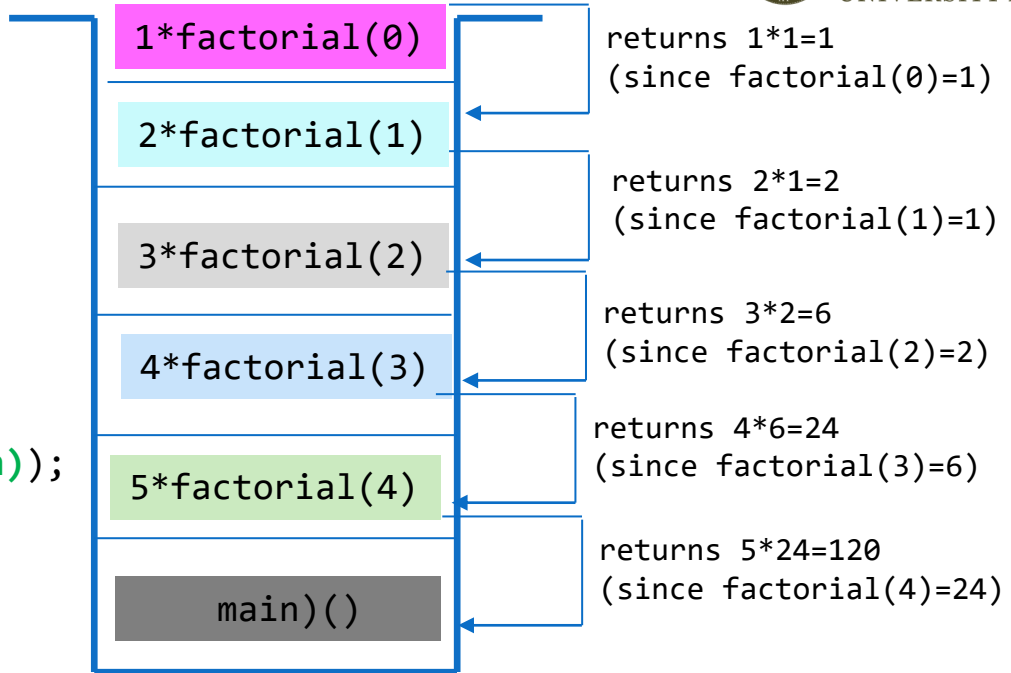
```
#include<stdio.h>

int factorial(int n);

int main()
{
    int n;
    printf("Enter a positive integer: ");
    scanf("%d",&n);
    printf("Factorial of %d = %d", n, factorial(n));
}

int factorial(int n)
{
    if (n>=1)
        return n*factorial(n-1);
    else
        return 1;
}
```

Example: $n=5$



$\text{main}()$ function prints **Factorial of 5=120**

Space complexity of recursive algorithm is proportional to maximum depth of recursion tree generated. If each function call takes $O(m)$ space and if the maximum depth of recursion tree is ' n ' then space complexity is $O(nm)$. Here depth indicates the number of calls.

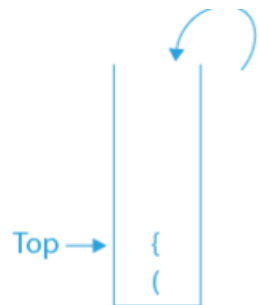
For the factorial algorithm = $O(n)$, because here m is $O(1)$

Balanced Parenthesis check using Stack

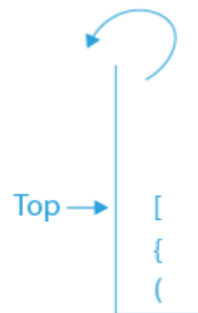
Step-1



Step-2



Step-3



Step-4



Step-5



Expression Representation

Popular methods used for representation of an expression →

Infix	$x + y$	Operator between operands.
Prefix	$+ x y$	Operator before operands.
Postfix	$x y +$	Operator after operands.

Precedence/Associativity table

Operators	Associativity	Precedence
$^$ exponentiation	Right to left	Highest followed by *Multiplication and /division
* Multiplication, / division	Left to right	Highest followed by + addition and - subtraction
+ addition, - subtraction	Left to right	lowest

Converting Infix to Postfix

Algorithm:

Example: $3*3/(4-1)+6*2$

Step 1: Consider the next element in the input.

Step 2: If it is operand, display it.

Step 3: If it is opening parenthesis, insert it on stack.

Step 4: If it is an operator, then

- If stack is empty, insert operator on stack.
- If the top of stack is opening parenthesis, insert the operator on stack
- If it has higher priority than the top of stack, insert the operator on stack.
- Else, delete the operator from the stack and display it, repeat Step 4.

Step 5: If it is a closing parenthesis, delete the operator from stack and display them until an opening parenthesis is encountered. Delete and discard the opening parenthesis.

Step 6: If there is more input, go to Step 1.

Step 7: If there is no more input, delete the remaining operators to output.

Converting Infix to Postfix

Example-1: $3*3/(4-1)+6*2$

Expression	Stack	Output
3	Empty	3
*	*	3
3	*	33
/	/	33*
(	/(	33*
4	/(	33*4
-	/(-	33*4
1	/(-	33*41
)	/	33*41-
+	+	33*41- /
6	+	33*41- /6
*	+	33*41- /6
2	+	33*41- /62
	Empty	33*41- /62*+

Example-2: $3 * 3 / (4 - 1) + 6 * 2$

Example-3: $(a*b) + (d-c)$

$ab^* + dc-$

ab^*dc-+

Example-4: $a + b / c + d * (e-f) ^ g$

$a + b / c + d * ef- ^g$

$a + b / c + d * ef-g^$

$a + bc/ + def-g^*$

$abc/+ + def-g^*$

$abc/+def-g^*+$

Example-5: $K + L - M*N + (O^P) * W/U/V * T + Q$

Converting Infix to Prefix

Algorithm for Infix to Prefix Conversion:

Step 1: Insert “)” onto stack, and add “(” to end of the expression .

Step 2: Scan expression from right to left and repeat Step 3 to 6 for each element of A until the stack is empty .

Step 3: If an operand is encountered, add it to output .

Step 4: If a right parenthesis is encountered, insert it onto stack .

Step 5: If an operator is encountered then,
 a. Delete from stack and add to output (each operator on the top of stack) which has same or higher precedence than the operator.
 b. Add operator to stack.

Step 6: If left parenthesis is encountered then ,
 a. Delete from the stack and add to B (each operator on top of stack until a left parenthesis is encountered).
 b. Remove the left parenthesis.

Step 7: Exit

Converting Infix to Prefix

Example-1: $(a * b) + (d - c)$

Example-2: $a + b / c + d * (e - f) ^ g$

Example-3: $K + L - M * N + (O ^ P) * W / U / V * T + Q$

Postfix expression evaluation

Algorithm for evaluating postfix expression

Step-1: Read all the symbols one by one from left to right in the given Postfix Expression

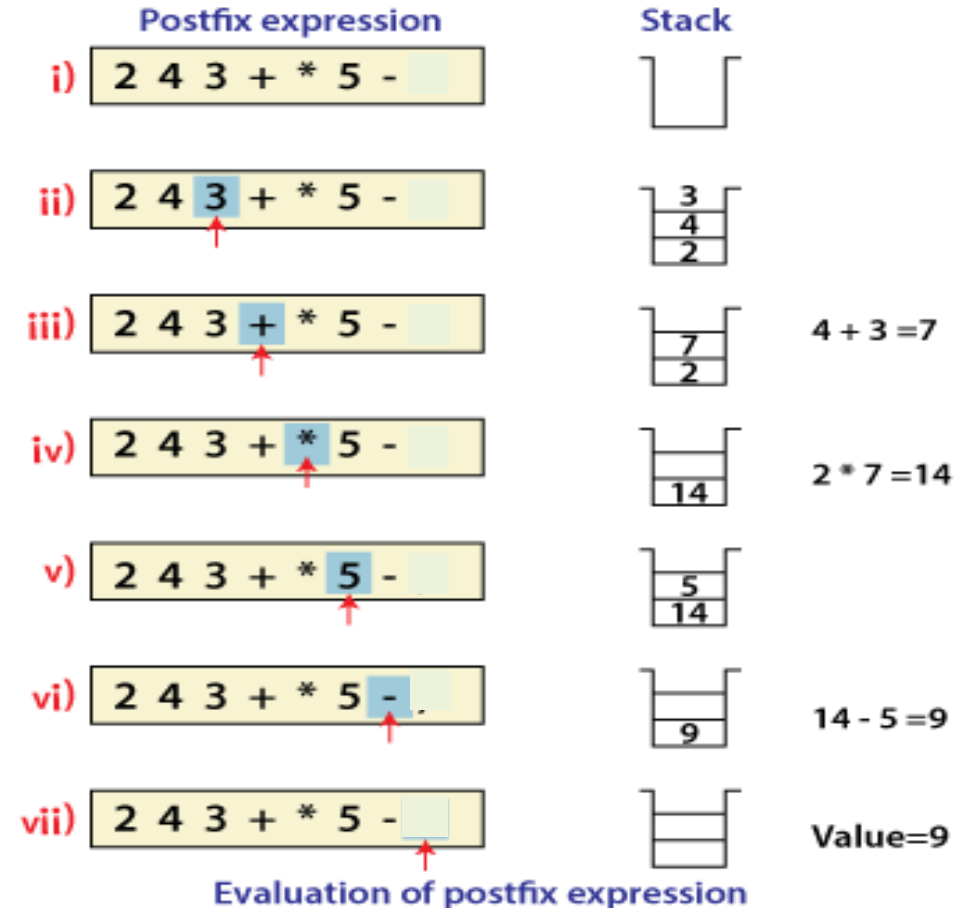
Step-2: If the reading symbol is **operand**, then push it on to the Stack.

Step-3: If the reading symbol is **operator** (+ , - , * , / etc.), then perform **TWO** pop operations and store the two popped operands in two different variables (operand1 and operand2). Then perform reading symbol operation using operand1 and operand2 and push result back on to the Stack.

Step-4: Finally! perform a pop operation and display the popped value as final result.

Example infix expression: $2 * (4 + 3) - 5$

Equivalent postfix: $2\ 4\ 3\ +\ *\ 5\ -$



Postfix expression evaluation

Example infix expression: $2 * (4 + 3) - 5$

Equivalent postfix: $2\ 4\ 3\ +\ *\ 5\ -$

Input	Stack	Operation
2	2	
4	2 4	
3	2 4 3	
+	2 2 7	$4+3=7$
*	empty 14	$2*7=14$
5	14 5	
-	empty 9	$14-5=9$
No i/p	empty	9

Prefix expression evaluation

Example infix expression: $2 * (4+3) - 5$

Equivalent prefix: $- * 2 + 4 3 5$

Reverse of prefix: $5 3 4 + 2 * -$

Reverse of prefix: $5 3 4 + 2 * -$

Above expression is evaluated according the algorithm of evaluating postfix expressions.

Note: Prefix notation also known as Polish Notation or Warsaw notation, and Postfix notation is also known as Reverse Polish notation

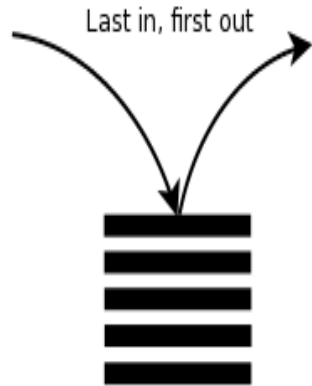
Queue Data Structure

Queues

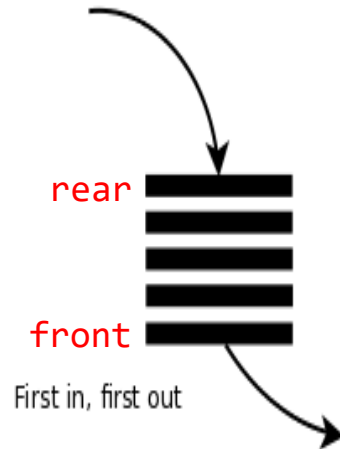
- **Queue** is a linear data structure, where a data item insertion is done at one end (called **rear**) and deletion is performed at the other end (called **front**).
- Accessing the elements of queue follows a **First In, First Out (FIFO)** order.



Stack:



Queue:



➤ Operations on Queue

- **enqueue(data)**: This is used to add elements into the queue at the rear (back) end.
- **dequeue()**: removes an element from the queue at the front end of the queue.
- **is_empty()**: checks whether a queue is empty.
- **is_full()**: checks whether a queue is at maximum capacity when stored in a static (fixed-size) structure.

Queue implementation

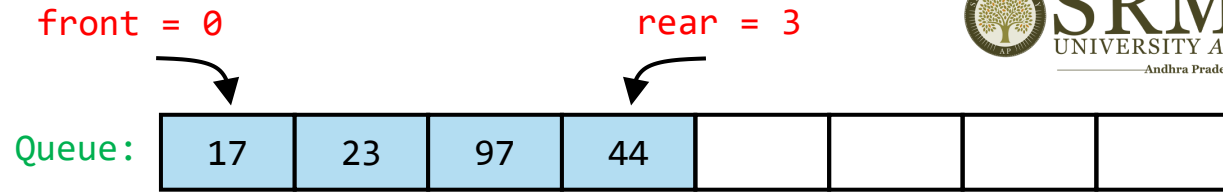
Queues can be implemented in two ways:

1. Arrays.

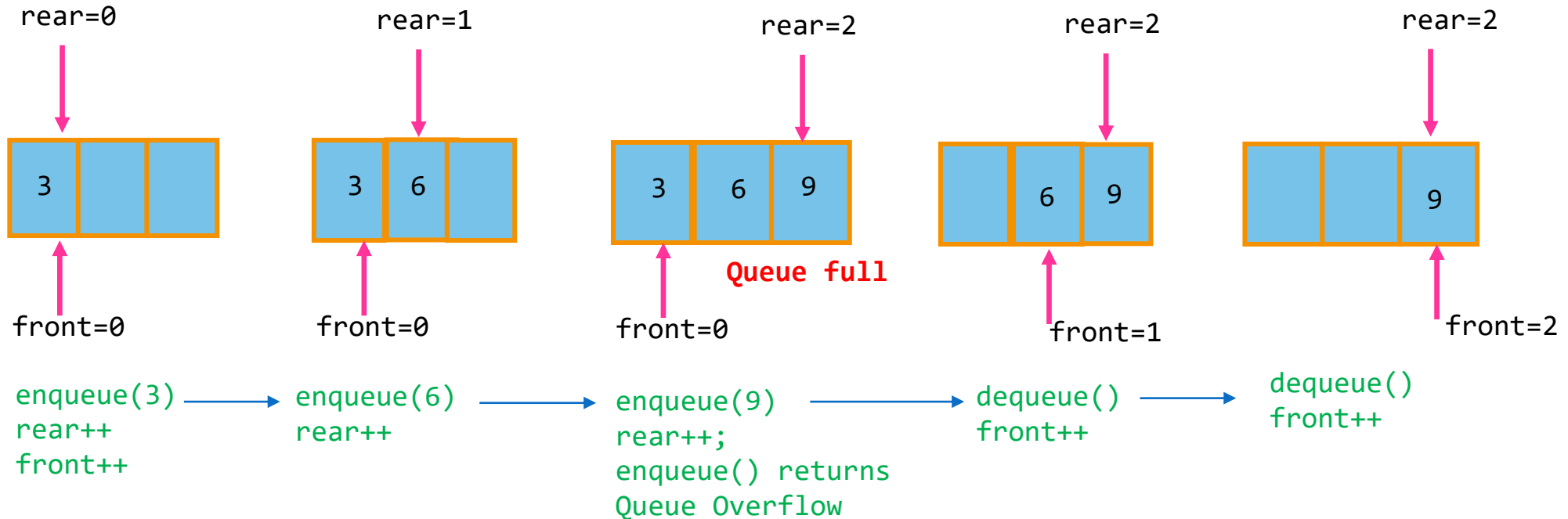
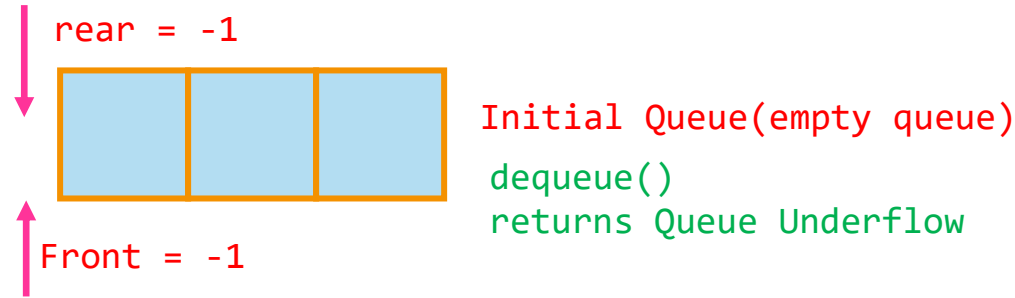
- ▶ Easy to implement, but the size of the array is fixed (static)
- ▶ Extra memory is not needed for pointer variables.
- ▶ It is not dynamic. It doesn't grow and shrink depending on needs at runtime.
- ▶ Performing operation **is_full()** is required.

2. Linked lists.

- ▶ This implementation of a stack can grow and shrink according to the needs at runtime.
- ▶ Extra memory is needed for pointer variables.



Operations on Queue (Array implementation)



Array implementation of Queue

```
#define SIZE 20
void enqueue(int);
void dequeue();
void display();
int queue[SIZE], front = -1, rear = -1;
```

```
void enqueue(int value)
{
    if(rear == SIZE-1)
        printf("\n Queue is full");
    else
    {
        if(front == -1)
            front = 0;
        rear++;
        queue[rear] = value;
        printf("\n Successfully inserted!!!");
    }
}
```

```
void dequeue()
{
    if(front == -1)
        printf("\nQueue is empty");
    else
    {
        printf("Successfully deleted\n");
        printf("Deleted ele : %d", queue[front]);
        front++;
        if(front == SIZE)
            front = rear = -1;
    }
}
```

```
void display()
{
    if(rear == -1)
        printf("\nQueue is Empty!!!");
    else
    {
        int i;
        printf("\nQueue elements are:\n");
        for(i=front; i<=rear; i++)
            printf("%d\t", queue[i]);
    }
}
```

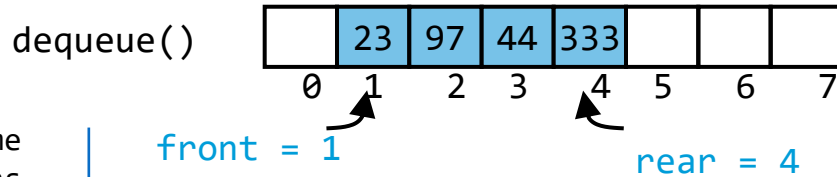
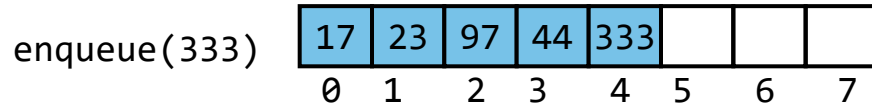
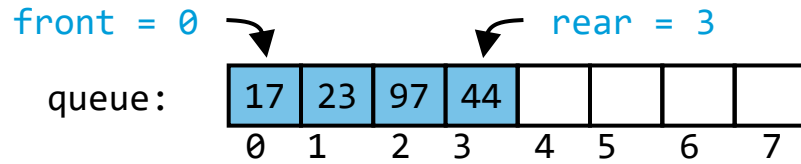
Linked list implementation of Queue

```
#include < stdio.h >
#include < stdlib.h >
struct node { int data;
struct node * next;
};
struct node * front = NULL;
struct node * rear = NULL;
```

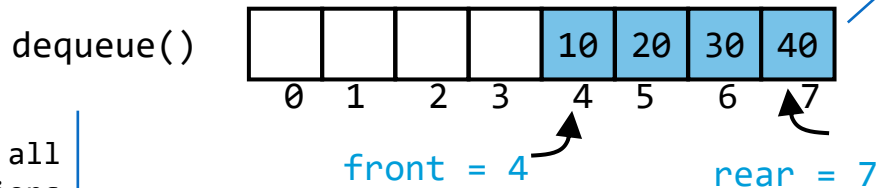
```
int dequeue() {
if (front == NULL)
{
printf("\nUnderflow\n");
return -1; }
else{
struct node * temp = front;
int temp_data = front -> data;
front = front -> next;
free(temp);
return temp_data; } }
```

```
void enqueue(int value)
{
struct node * ptr;
newnode=(struct node*)malloc(sizeof(struct node));
newnode -> data = value;
newnode -> next = NULL;
if ((front == NULL) && (rear == NULL)){
front = rear = newnode;
}
else {
rear -> next = newnode;
rear = newnode;
}
printf("Node is Inserted\n\n"); }
```

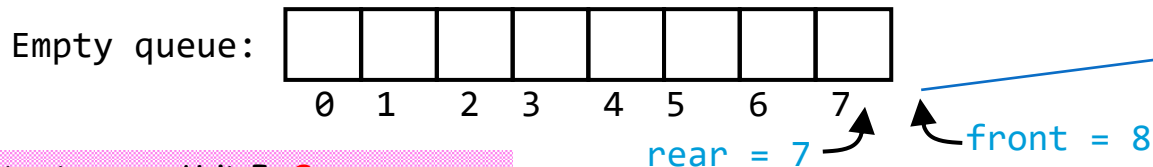
Problems with Linear queues



After some
insertions
and deletions



After all
deletions

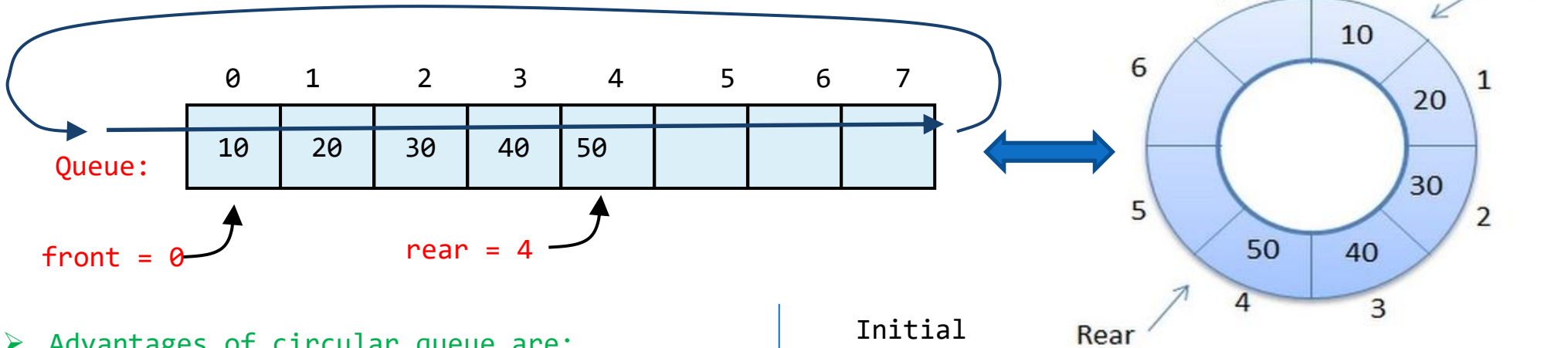


- If the **rear** of the queue is at the end of the array (the **highest index**).
- Even if there are empty cells at the beginning of the array, **rear** can't go any further.
- Because all the elements are removed, the **rear** points to end of queue (the **highest index**).
- Hence a new element can not be inserted.

**if front=SIZE of array then
rear=front=-1;**

Circular queues

An array holding the queue elements as circular (joined at the ends)



➤ Advantages of circular queue are:

- **Easier for insertion-deletion:** Elements can be inserted easily if there are vacant locations.
- **Efficient utilization of memory:** No wastage of memory as it uses the unoccupied space.

Initial

$\text{rear} = -1, \text{front} = -1$

(when Queue is empty)

- Elements were added to this queue in the order 10, 20, 30, 40, 50, and will be removed in the same order
 - Index of $\text{front} = (\text{front} + 1) \% \text{Queue.length};$
 - Index of $\text{rear} = (\text{rear} + 1) \% \text{Queue.length};$

Array implementation of Circular queue

```
#define SIZE 20
void enqueue(int);
void dequeue();
void display();
int cqueue[SIZE], front = -1, rear = -1;
```

```
void enqueue(int value)
{
    if(front == (rear+1)%SIZE)
        printf("\n Circular queue is full");
    else
    {
        if(front == -1)
            front = 0;

        rear=(rear+1)%SIZE;
        cqueue[rear] = value;
        printf("\n Successfully inserted!!!");
    }
}
```

```
void dequeue()
{
    if(front == -1)
        printf("\nQueue is empty");
    else
    {
        printf("Successfully deleted\n");
        printf("Deleted ele : %d", cqueue[front]);
        if(front==rear)
            front=rear=-1;
        else
            front = (front+1)%SIZE;
    }
}
```

Linked list implementation of Circular queue

- ▶ When a resource is shared among multiple consumers.

Examples: CPU scheduling, Disk Scheduling.

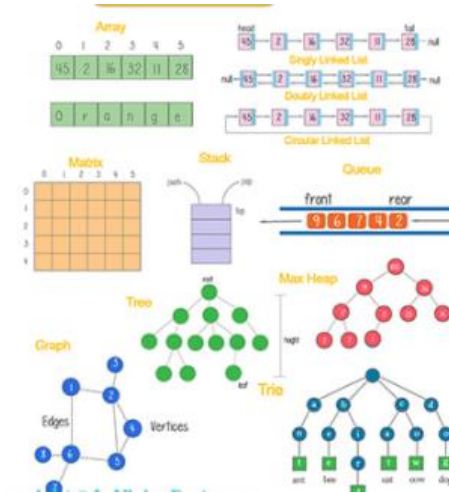
- ▶ When data is transferred asynchronously (data not necessarily received at same rate as sent) between two processes.

Examples: IO Buffers, pipes, file IO, printer queue etc.

- ▶ Data packets waiting to be transmitted over the Internet.
- ▶ Queue used to model real-world situations.

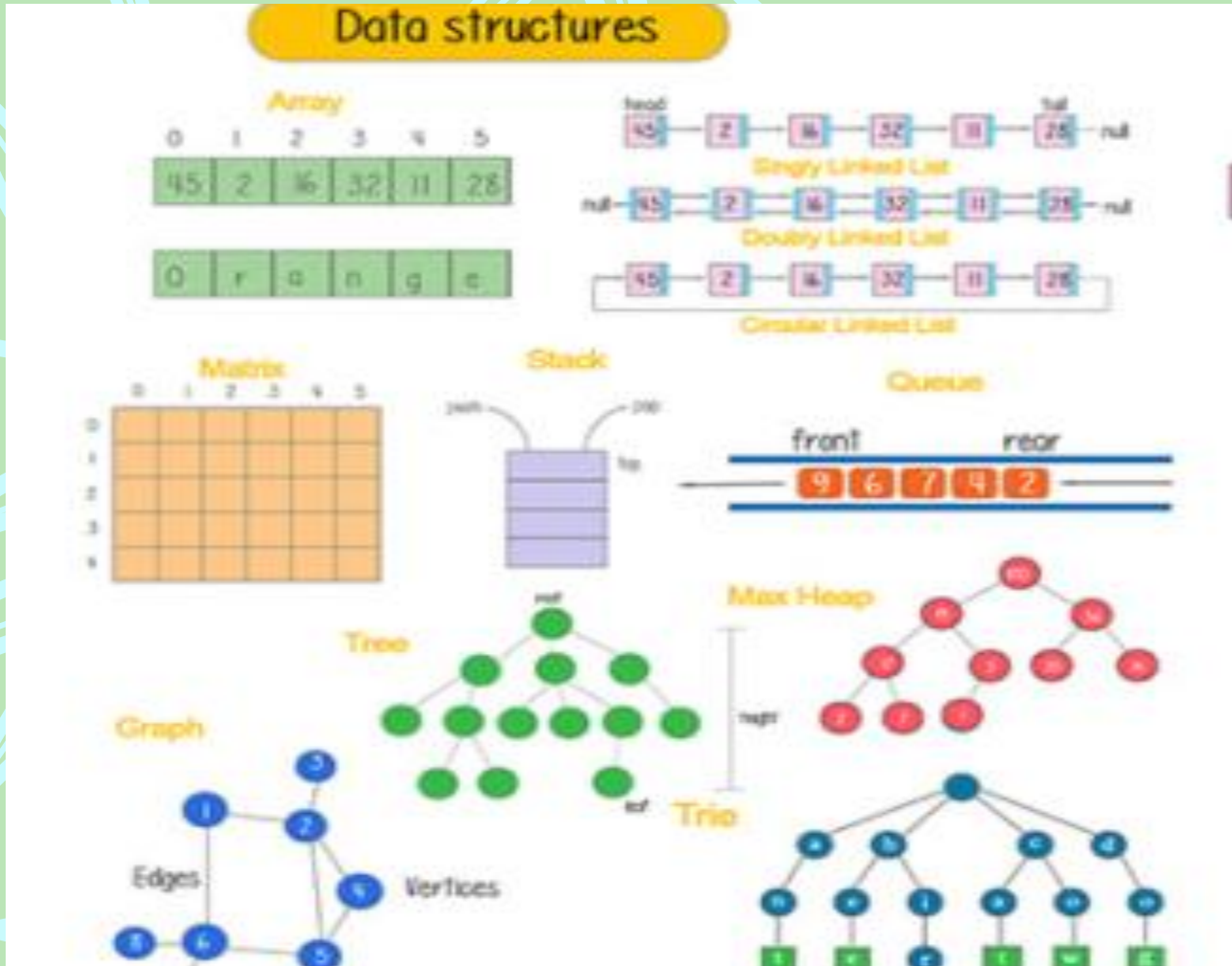
Examples: People waiting in line at a bank, airplanes waiting to take off, stores and reservation centres,. Etc.

Data Structures



Dr. Pandu Sowkuntla
Asst. Professor,
Dept. of CSE, SRM University AP





LINKED LISTS

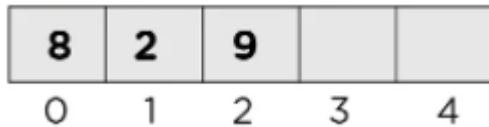
UNIT-II

➤ Operations performed on Arrays

1. Create
2. Insert
3. Delete
4. Traverse
5. Search
6. Display

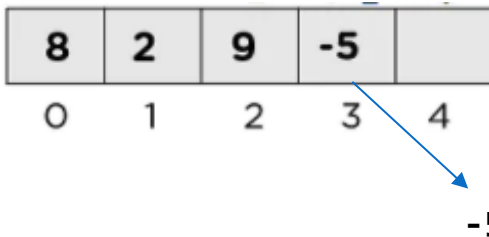
➤ Problems with Arrays

1. Memory requirements



If I store only 3 elements,
2 memory locations will be wasted

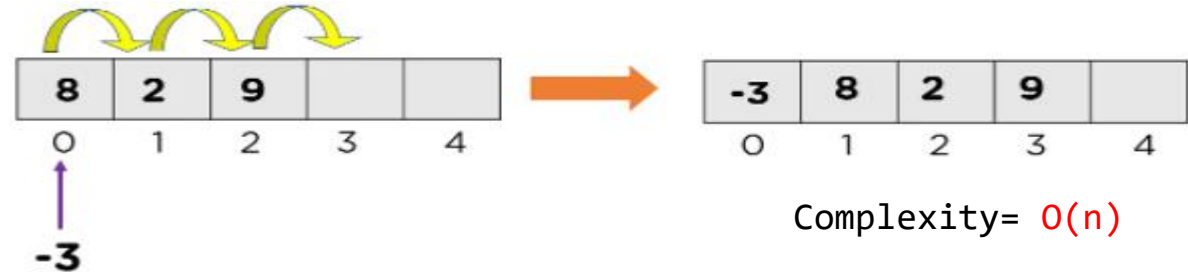
2. Cost of accessing an element



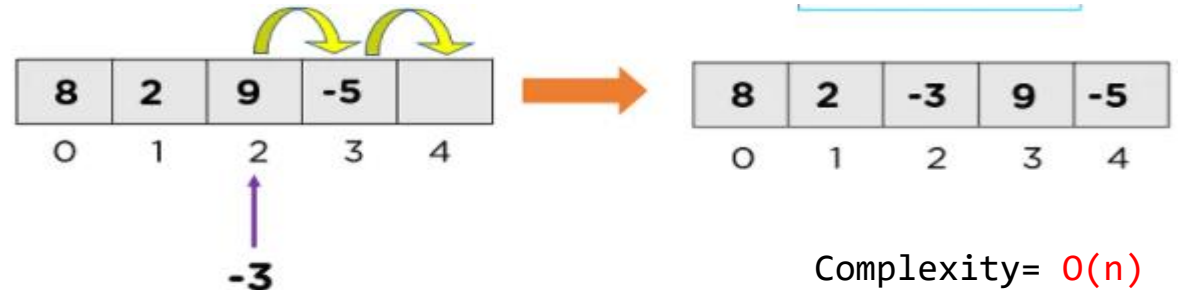
Time complexity= $O(1)$

3. Cost of Inserting an Element

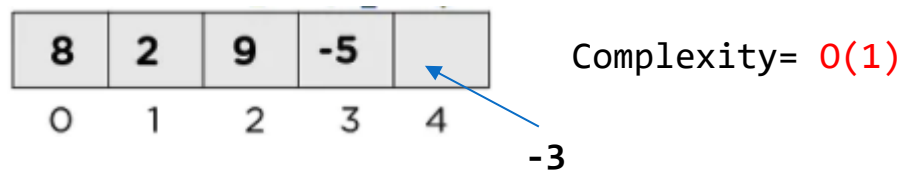
Inserting at the beginning



Inserting at the middle



Inserting at the end



Why Linked Lists?

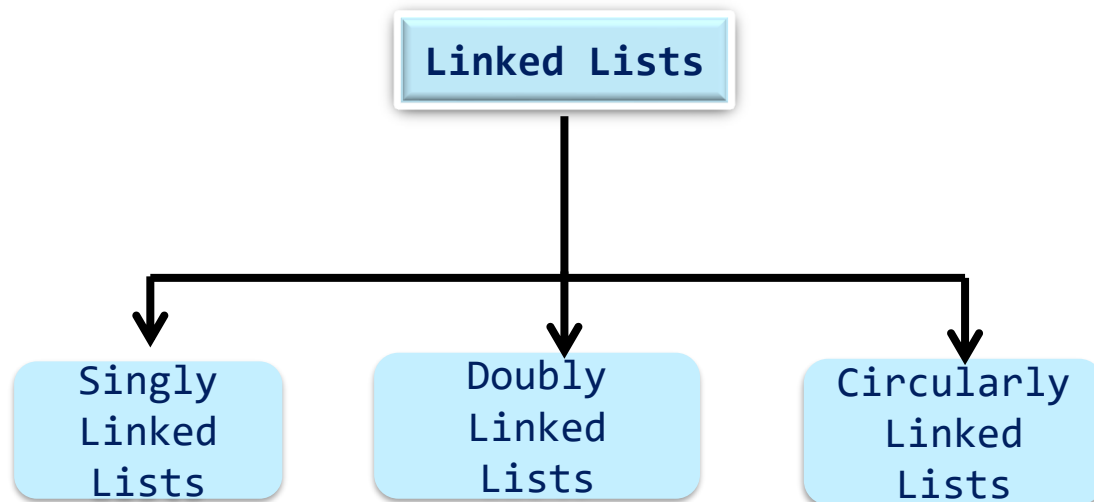
1. An array is a collection of elements of a similar data type.
2. Array elements store in a contiguous memory location.
3. Array elements are independent of each other.
4. Array works with a static memory.
5. Memory is allocated at compile-time.
6. Accessing any element in an array is faster ($O(1)$)
7. Cost is more for performing insertion, deletion, operations ($O(n)$).
8. Memory utilization is inefficient. (For example, if the size of the array is 6, and array consists of 3 elements only then the rest of the space will be unused).

1. A linked list is a collection nodes where a node consists of two parts, i.e., data and address.
2. Linked list elements can be stored anywhere in the memory or randomly stored.
3. Linked list elements are dependent on each other.
4. The Linked list works with dynamic memory.
5. Memory is allocated at run time.
6. Accessing an element in a linked list is slower.
7. Cost is less for performing insertion, deletion operations ()
8. Memory utilization is efficient (memory can be allocated or deallocated at the run time according to our requirement).

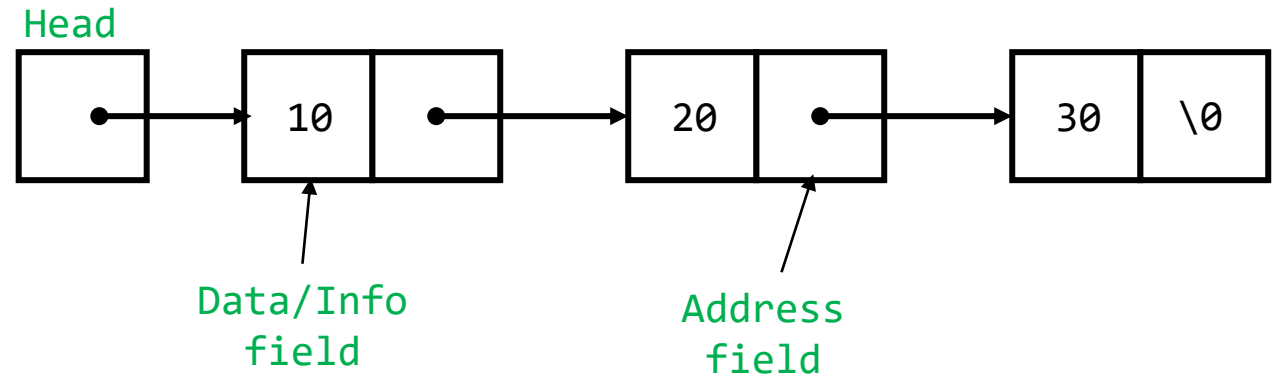
What is Linked List?

- A **Linked List** is a series of connected **nodes**
- Each node contains at least
 - A piece of **data** (any type)
 - **Pointer** to the next node in the list
- **Head**: pointer to the first node
- **NULL**: The last node points to NULL

In linked list, each node consists of its own data and the address of the next node and forms a chain.



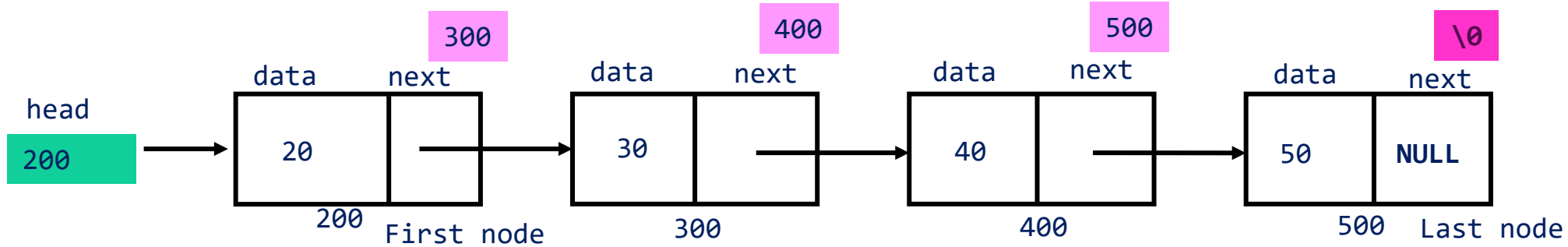
Singly Linked Lists



Operations performed on Singly Linked Lists

- Create
- Traversal and Display
- Search
- Insertion
 - At the begin
 - At the end
 - At the middle
- Deletion
 - At the begin
 - At the end
 - At the middle

Operations on Singly Linked List



1. Creation of Linked List

- Empty Linked list is a single pointer with the value of **NULL**.

```
head = NULL;
```

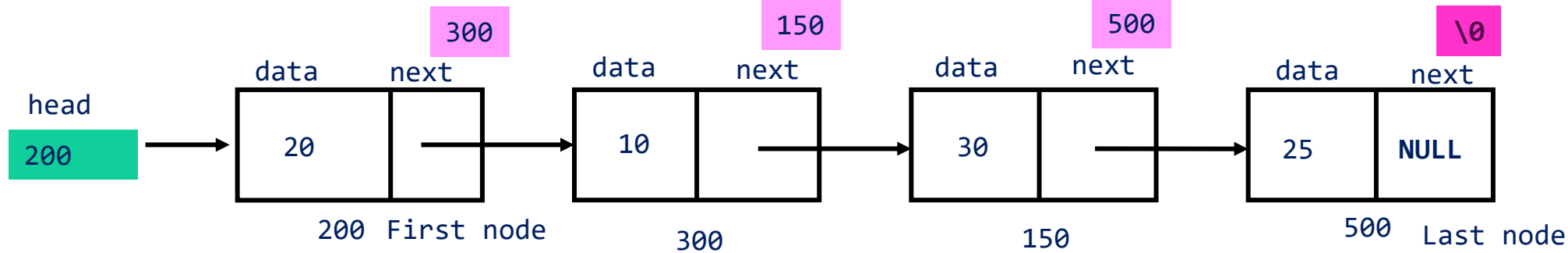
- A node can be declared as::

```
struct Node
{
    int data;
    struct Node *next;
};
struct Node *head;
```

- Create the first node and make **head** point to it.

```
head = (struct Node*)malloc(sizeof(struct Node));
```

Operations on Singly Linked List



1. Creation of Linked List

- Empty Linked list is a single pointer with the value of **NULL**.

```
head = NULL;
```

- A node can be declared as::

```
struct Node
{
    int data;
    struct Node *next;
};
```

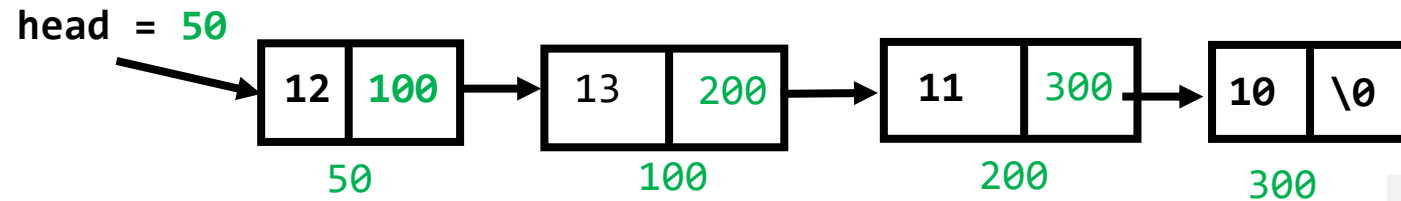
- Create the first node and make **head** point to it.

```
head = (struct Node*)malloc(sizeof(struct Node));
```

```
struct Node *create_SLL(int n)
{
    for (i=1; i<=n; i++)
    {
        if (i == 1)
        {
            //Allocate the memory space at runtime for first node
            head = (struct Node*)malloc(sizeof(struct Node));
            ptr = head;
        }
        else
        {
            //Allocate the memory space at runtime for remaining nodes
            ptr->next = (struct Node*)malloc(sizeof(struct Node));
            ptr = ptr->next;
        }
        //Read the data into each node of SLL
        printf("\n Enter an %dth element",i);
        scanf("%d",&ptr->data);
    }

    //Assign null pointer to last node of SLL
    ptr->next = NULL;
    return(head);
}
```

Operations on Singly Linked List



3. Searching an element in linked list

```
void main()
{
    struct Node *head;
    head=create_SLL();
    display(head);
    search(head);
}
```

```
void search_SLL(struct Node *head)
{
    struct Node *ptr;
    int key, flag=0;

    printf("Enter an element to search:");
    scanf("%d",&key);

    for(ptr = head; ptr!= NULL; ptr = ptr->next)
    {
        if ((ptr->data)==key)
        {
            flag=1;
            break;
        }
    }
    if(flag==1)
        printf("found");
    else
        printf("not found");
}
```

2. Traversing a linked list

```
void display_SLL(struct Node *head)
{
    struct Node *ptr;
    int count=0;
    for(ptr=head; ptr!= NULL; ptr = ptr->next)
    {
        printf ("\n Node data= %d", ptr->data);
        count++;
    }
    printf ("\n");
}
```

Inserting node at the beginning of the List

```
struct Node *insertBeg_SLL(struct Node *head)
{
    //Step-1. create a new node

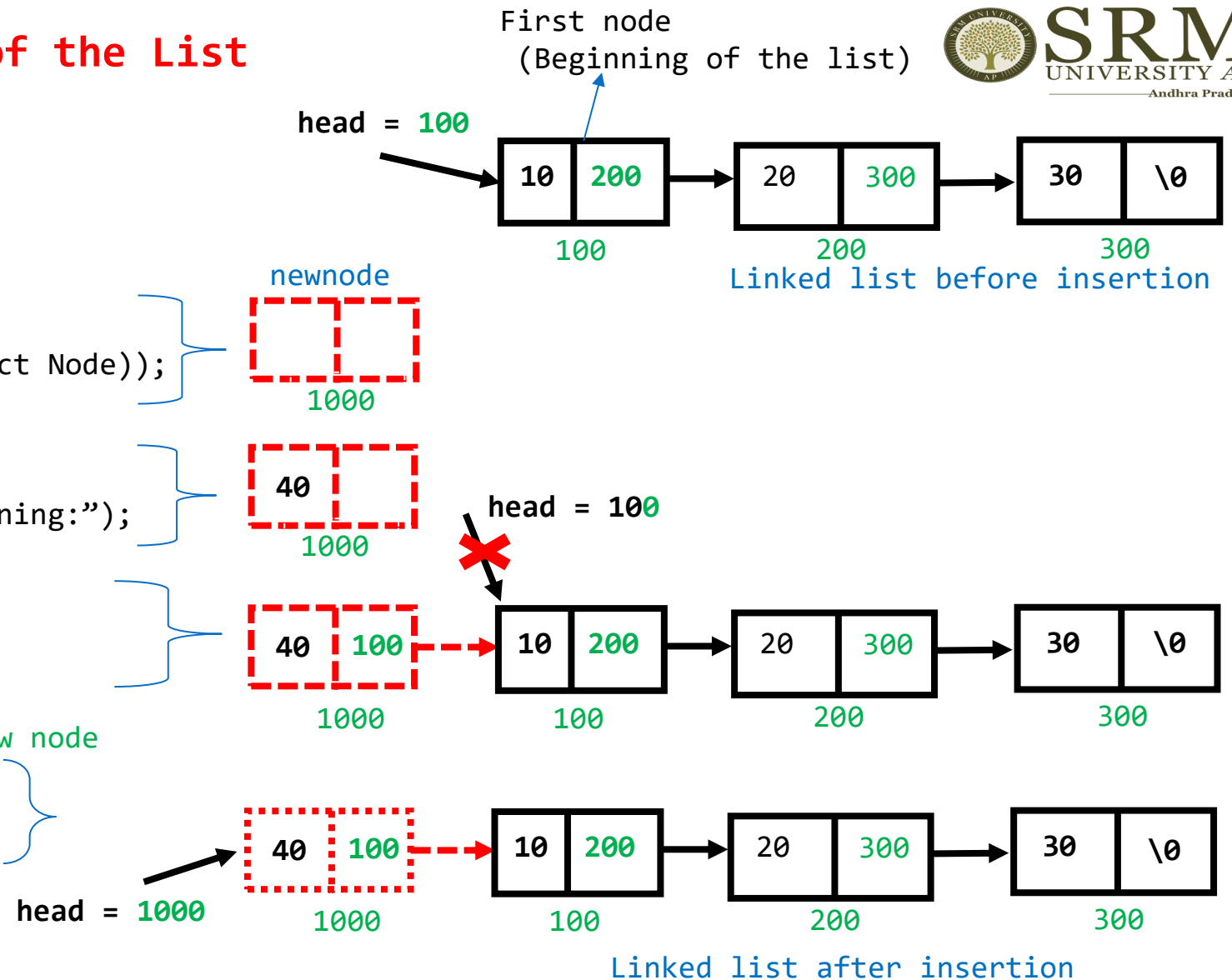
    struct Node* newnode;
    newnode = (struct Node*) malloc(sizeof(struct Node));

    //Step-2. Store the data

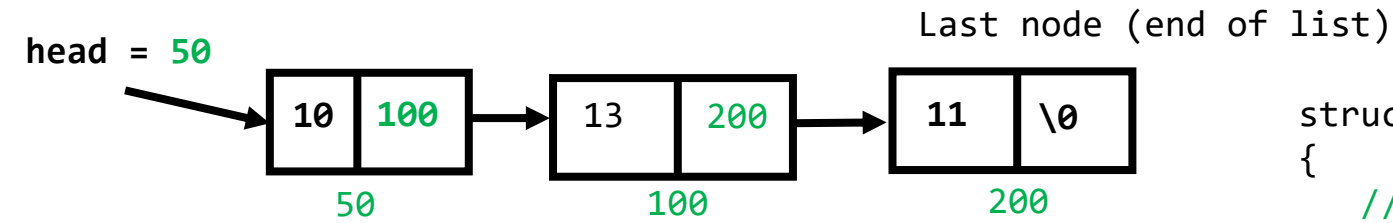
    printf("Enter an element to insert at beginning:");
    scanf("%d",&newnode->data);

    //Step-3. Make next of new node as head
    newnode->next = head;

    //Step-4. Move the head to point to the new node
    head=newnode;
    return(head);
}
```



Inserting node at the end of the List



```
struct Node *insertEnd_SLL(struct Node *head)
{
```

//Step-1. Create new node

```
struct Node* newNode;
newNode = (struct Node*) malloc(sizeof(struct Node));
```

//Step-2. Store the data

```
printf("Enter an element to insert:");
scanf("%d",&newNode->data);
```

//Step-3. New node Should be inserted last
// so make next of it as NULL

```
newNode->next = NULL;
```

//Step-4. If list is empty, then make newNode as head

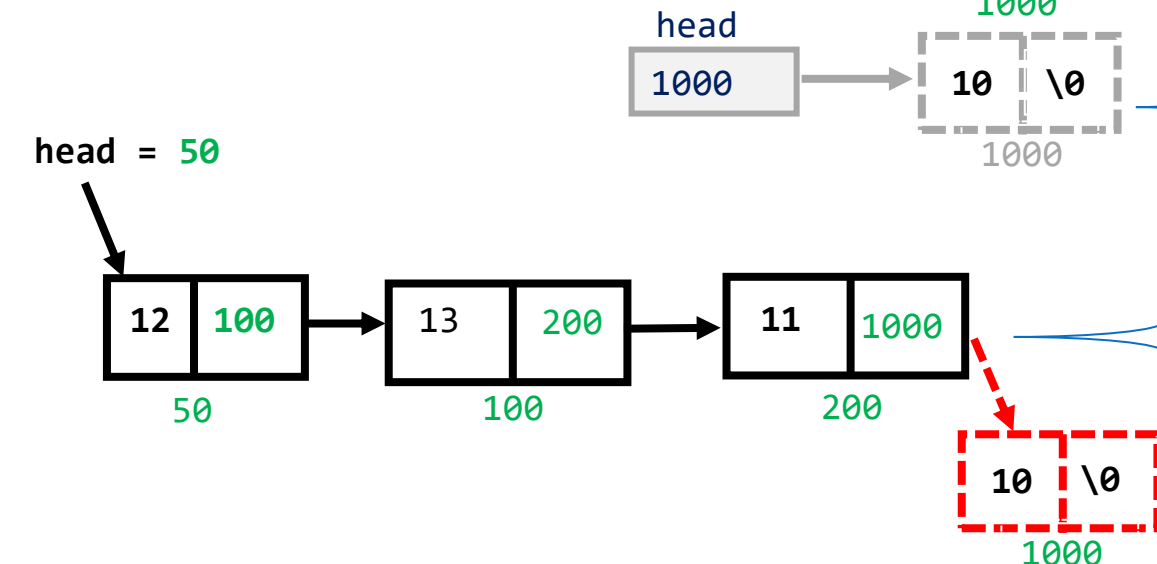
```
if (head == NULL){
    head = newNode;
    return(head);
}
```

//Step-5. Else traverse till the last node

```
struct Node *ptr=head;
while (ptr->next != NULL){
    ptr = ptr->next;
}
```

//Step-6. Change the next of last node

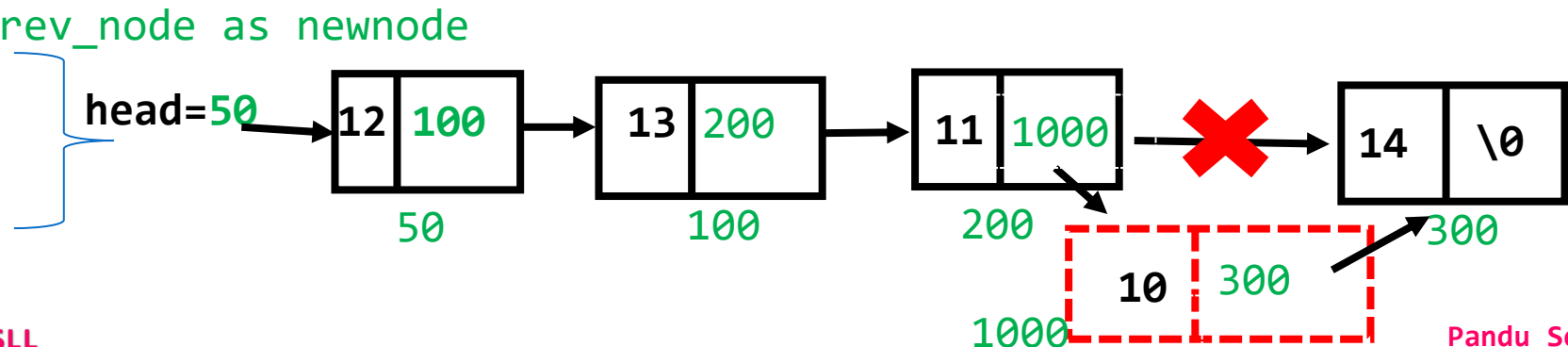
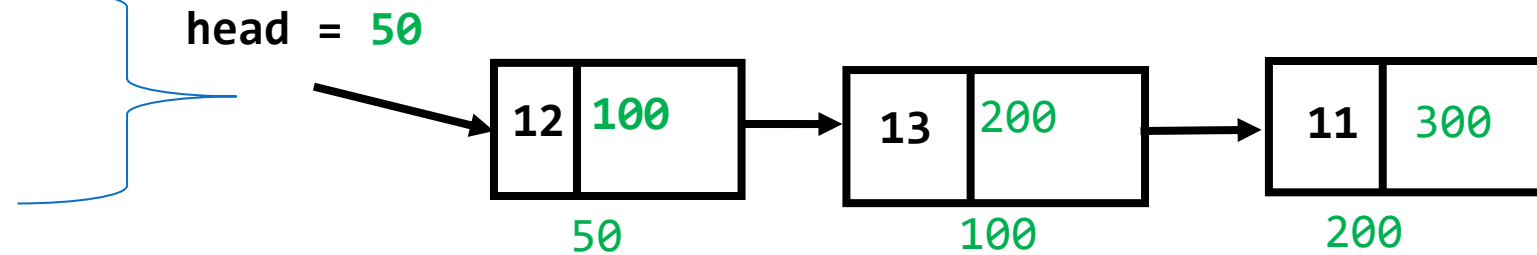
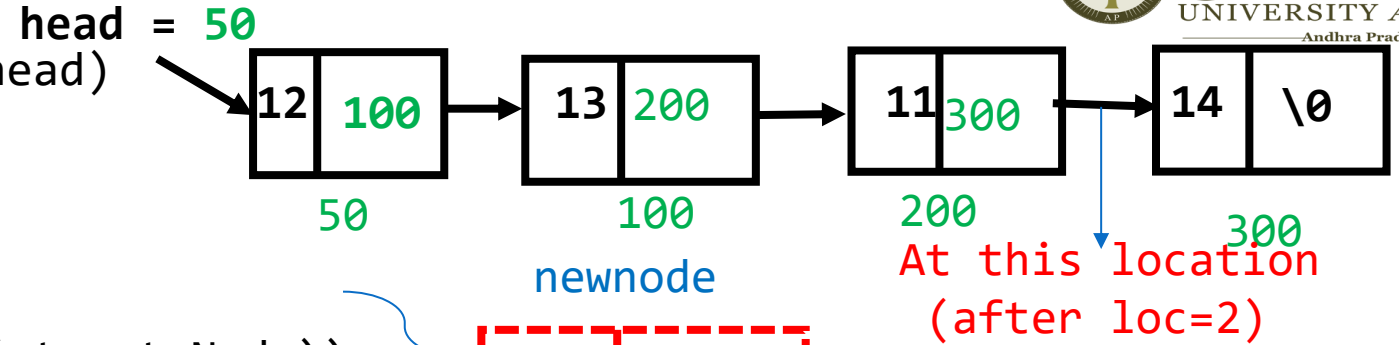
```
ptr->next = newNode;
return(head);
}
```



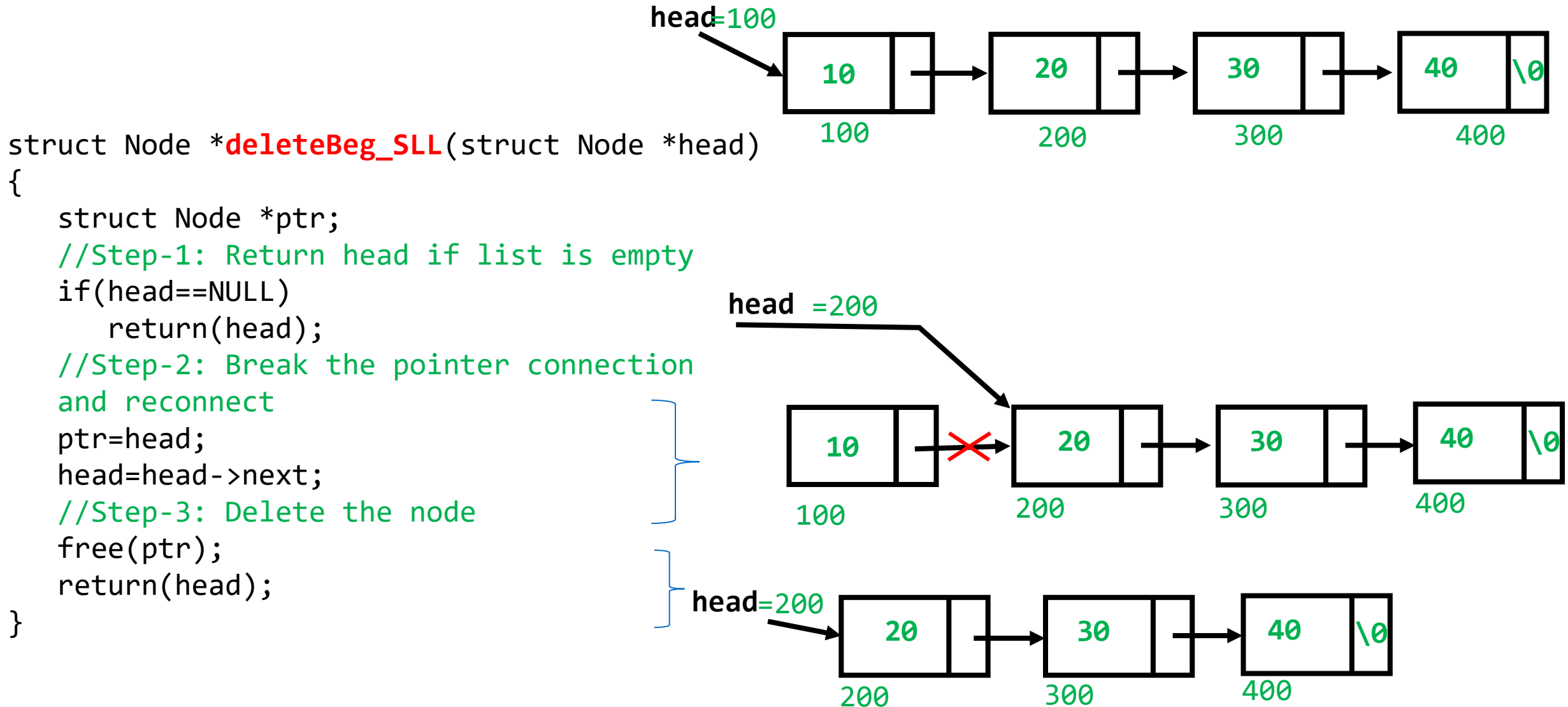
Inserting node at the middle of the List

```

struct Node *insertMid_SLL(struct Node *head)
{
    int newdata, loc=3, count=1;
    //Step-1. Create a newnode
    struct Node *newnode, *ptr;
    newnode = (struct Node*)malloc(sizeof(struct Node));
    //Step-2. Store the data
    printf("Enter an element to insert:");
    scanf("%d",&newnode->data);
    ptr=head;
    //Step-3. Traverse list, until required location
    while(ptr!= NULL && count<loc)
    {
        ptr = ptr->next;
        count=count+1;
    }
    //Step-4. Move the next of prev_node as newnode
    newnode->next=ptr->next;
    ptr->next=newnode;
    return(head);
}
  
```



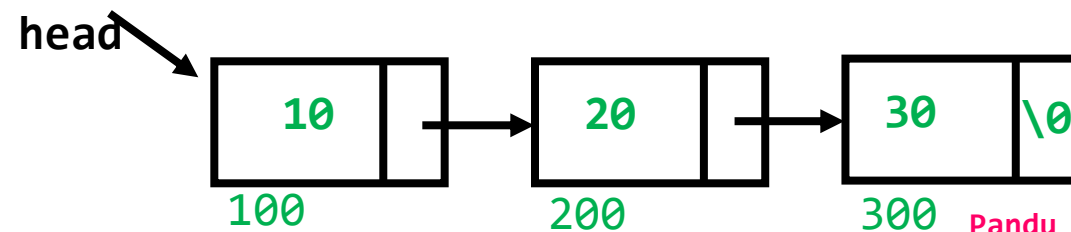
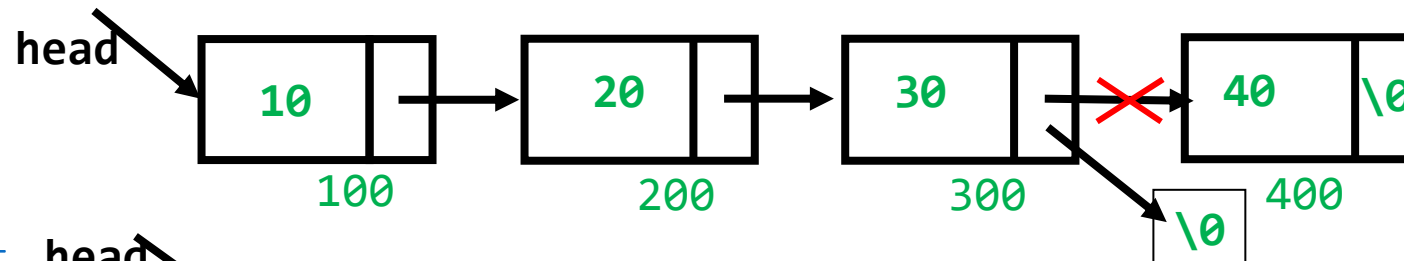
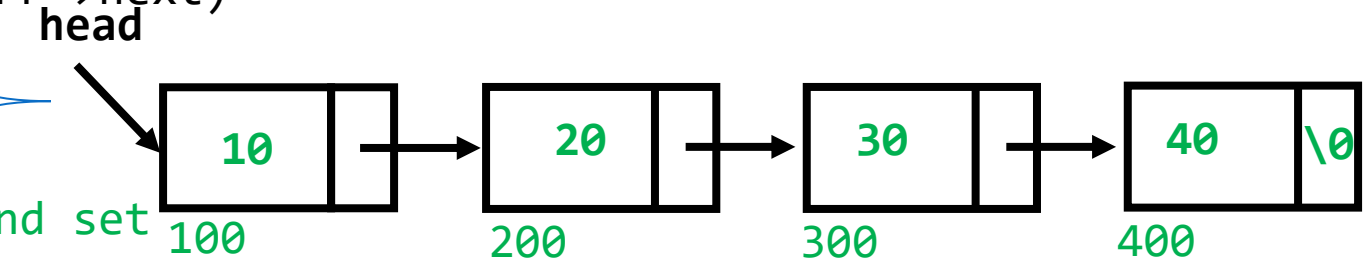
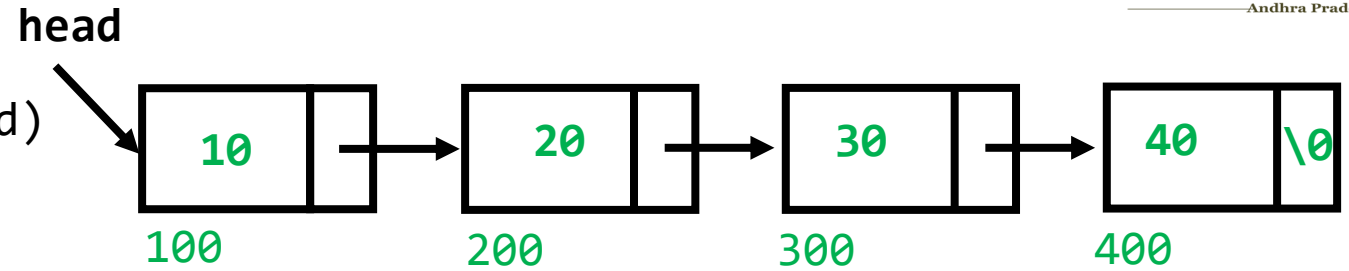
Delete node at beginning of Singly Linked List



Delete node at the end of Singly Linked List

```
struct Node *deleteEnd_SLL(struct Node *head)
{
    //Step-1: Return head if list is empty
    if(head==NULL)
        return(head);
    //Step-2: Traverse the list until end
    struct Node *curr, *prev;
    for(curr=head; curr->next!=NULL; curr=curr->next)
    {
        prev=curr;
    }
    //Step-3: Break the pointer connection and set
    //previous node pointer to null
    prev->next=NULL;

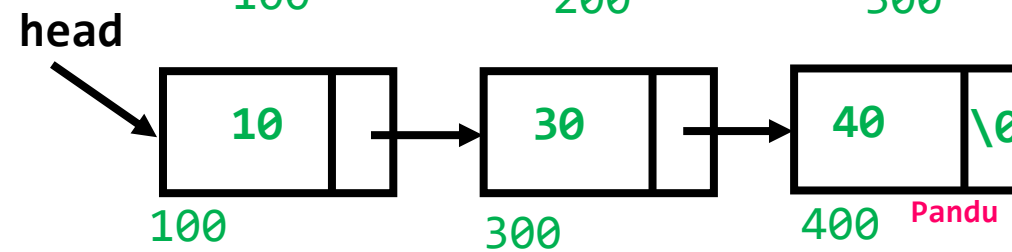
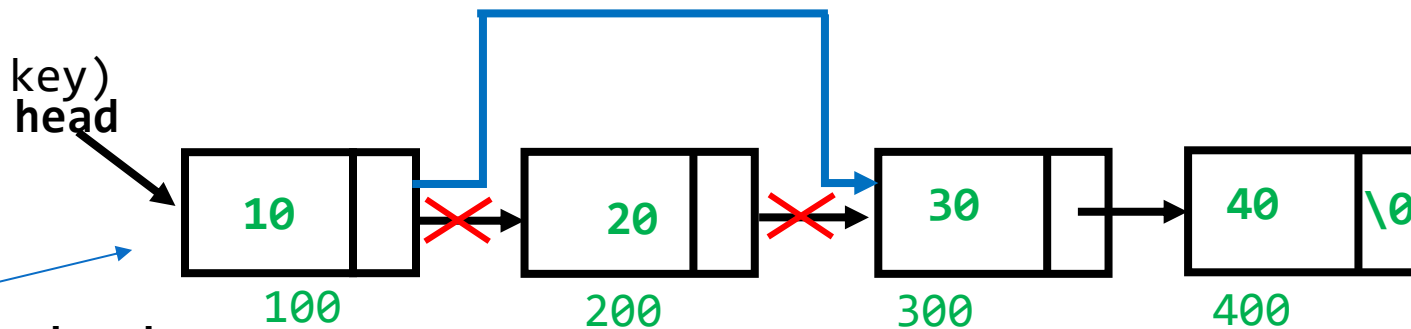
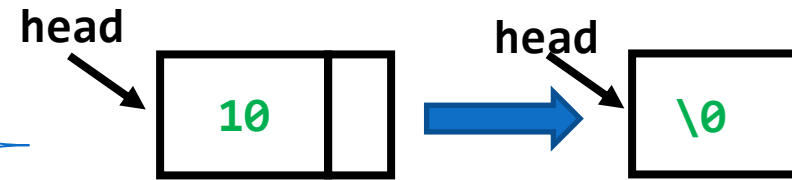
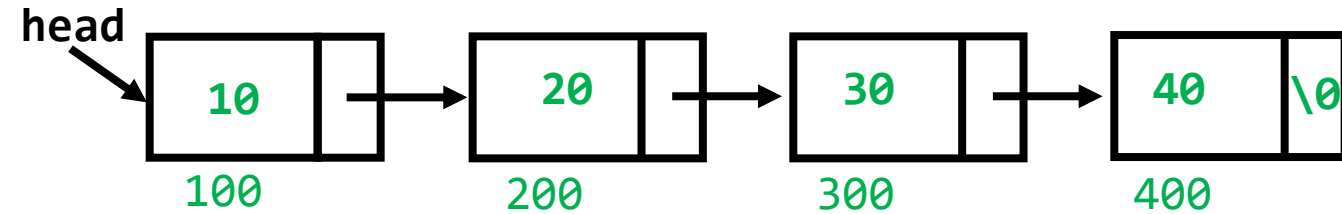
    //Step-2: Delete the node
    free(curr);
    return(head);
}
```



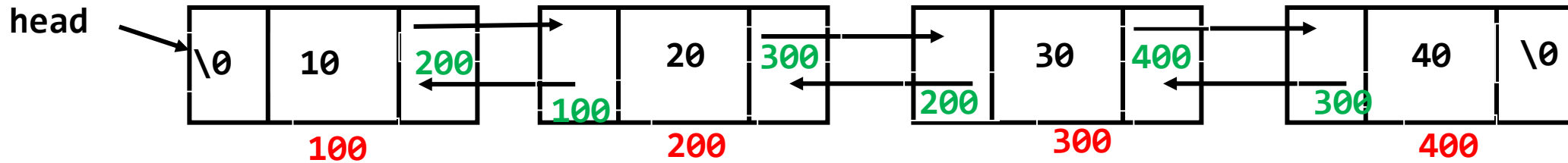
Delete node at the middle of Singly Linked List

```
struct Node *deleteMid_SLL(struct Node *head)
```

```
{  
    int key;  
    printf("Enter an element to delete:");  
    scanf("%d",&key);  
    struct Node *curr = head, *prev;  
    if(curr == NULL)  
        return(head);  
    if(curr!= NULL && curr->data == key)  
    {  
        head= curr->next;  
        free(curr);  
        return(head);  
    }  
    while (curr != NULL && curr->data != key)  
    {  
        prev = curr;  
        curr = curr->next;  
    }  
    prev->next = curr->next;  
    free(curr);  
}
```

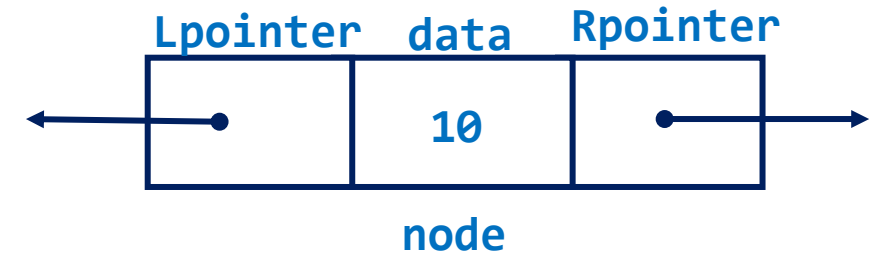


Doubly Linked List (DLL)



Advantages:

- Doubly linked list can be traversed forward and backward.
- Simplifies insertion and deletion because you no longer have to refer to the previous node.
- A node in a singly linked list cannot be removed unless we have the pointer to its predecessor.
- In a doubly linked list, we can delete a node even if we don't have the previous node's address (since each node has a left pointer pointing to the previous node and can move backward).



Disadvantage:

- Increase in space requirements.

Operations on Doubly Linked List (DLL)

- Empty Linked list is a single pointer having the value of NULL.

```
head = NULL;
```

- A node in DLL can be declared as:

```
struct Node
```

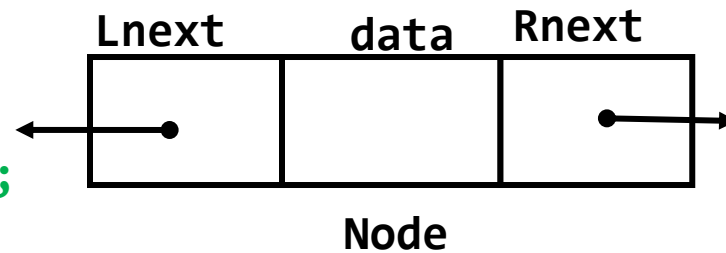
```
{
```

```
    struct Node *Lnext;
```

```
    int data;
```

```
    struct Node *Rnext;
```

```
};
```



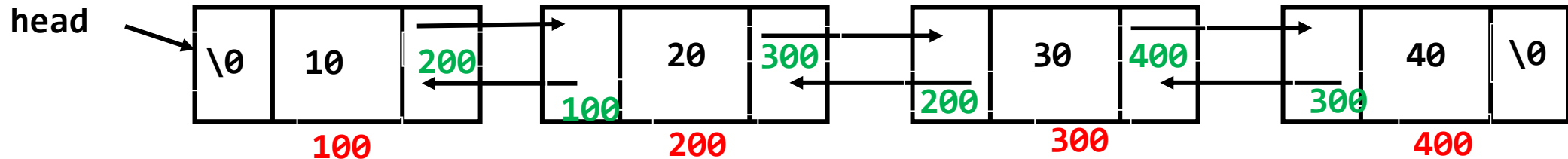
- A node in DLL can be created as:

```
head = (struct Node *)malloc(sizeof(struct Node));
```

Operations performed on DLL

- Create
- Traversing
- Search
- Insertion
 - At the begin
 - At the end
 - At the middle
- Deletion
 - At the begin
 - At the end
 - At the middle
- Display

Creation of the Doubly Linked List

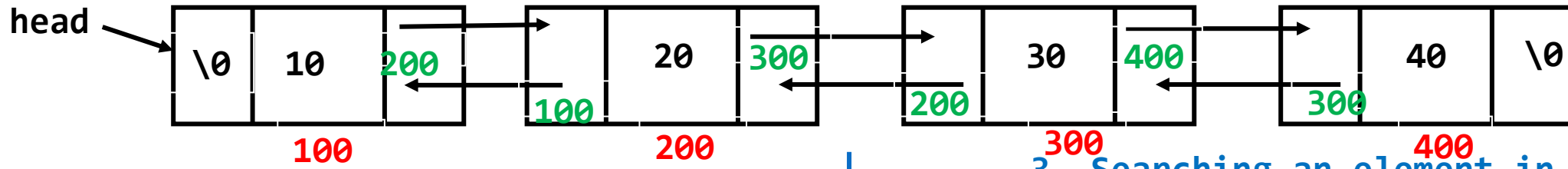


```
struct Node
{
    struct Node *Lnext;
    int data;
    struct Node *Rnext;
};

void main()
{
    struct Node *head;
    head=create_DLL();
    display(head);
}
```

```
struct Node *create_DLL()
{
    int i, n;
    struct Node *ptr, *newnode, *head;
    printf("\n How many elements you want to enter?");
    scanf("%d", &n);
    for (i=1; i<=n; i++)
    {
        if (i == 1){
            head = (struct Node*)malloc(sizeof(struct Node));
            head->Lnext=NULL;
            ptr = head;
        }
        else{
            newnode = (struct Node*)malloc(sizeof(struct Node));
            ptr->Rnext=newnode;
            newnode->Lnext=ptr;
            ptr=newnode;
        }
        printf("Enter the %d th element: ", i);
        scanf("%d",&ptr->data);
    }
    ptr->Rnext = NULL;
    return(head);
}
```

Search and Traverse operations on DLL



```
void main()
{
    struct Node *head;
    head=create_DLL();
    display_DLL(head);
    search_DLL(head);
}
```

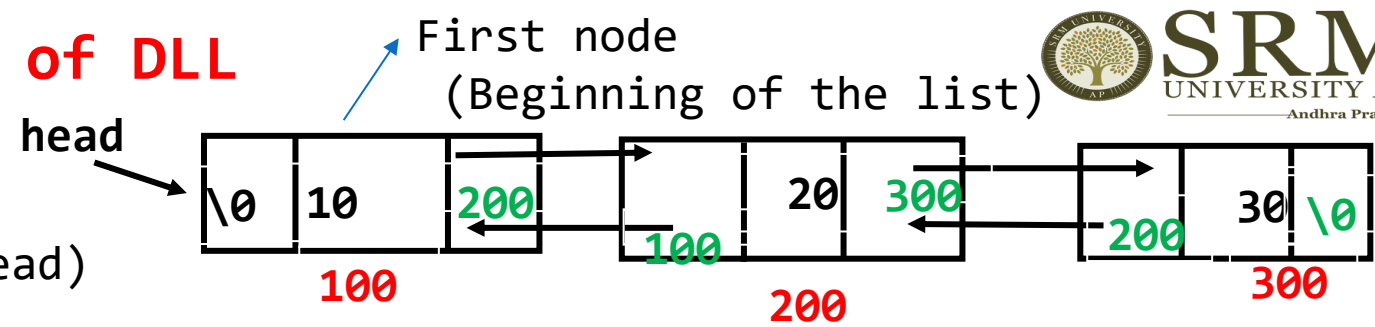
2. Traversing a DLL

```
void display_DLL(struct Node *head)
{
    struct Node *ptr;
    int count=0;
    for(ptr=head; ptr!= NULL; ptr = ptr->Rnext)
    {
        printf ("\n Node data= %d", ptr->data);
    }
    printf ("\n");
}
```

3. Searching an element in DLL

```
void search_DLL(struct Node *head)
{
    struct Node *ptr;
    int key, count=0;
    printf("Enter an element to search:");
    scanf("%d",&key);
    for(ptr = head; ptr!= NULL; ptr = ptr->Rnext)
    {
        if ((ptr->data)==key)
        {
            count=1;
            break;
        }
    }
    if(count==1)
        printf("found");
    else
        printf("not found");
}
```

Inserting node at the beginning of DLL



```

struct Node *insertBeg_DLL(struct Node *head)
{
    //Step-1. create a new node
    struct Node* newnode;
    newnode = (struct Node*) malloc(sizeof(struct Node));

    //Step-2. Store the data

    printf("Enter an element to insert at beginning:");
    scanf("%d",&newnode->data);

    //Step-3. Make next of new node as head
    newnode->Rnext = head;
    newnode->Lnext = NULL;
    head->Lnext = newnode;

    //Step-4. Move the head to point to the new node
    head=newnode;
    return(head);
}
  
```

Inserting node at the end of DLL

```
struct Node *insertEnd_DLL(struct Node *head)
{
```

//Step-1. Create new node

```
struct Node *newnode, *ptr;
```

```
newnode = (struct Node*) malloc(sizeof(struct Node));
```

//Step-2. Store the data

```
printf("Enter an element to insert:");
```

```
scanf("%d",&newnode->data);
```

//Step-3. New node Should be inserted last

// so make next of it as NULL

```
newnode->Rnext = NULL;
```

//Step-4. If list is empty,

//then make newnode as head

```
if (head == NULL){
```

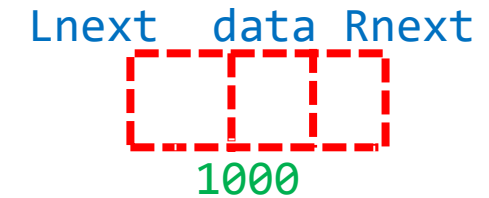
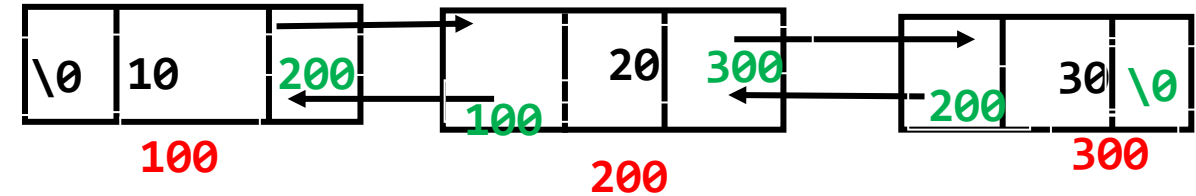
```
    newnode->Lnext = NULL;
```

```
    head = newnode;
```

```
    return(head);
```

```
}
```

head



//Step-5. Else traverse till the last node

```
ptr=head;
```

```
while (ptr->Rnext != NULL){
```

```
    ptr = ptr->Rnext;
```

```
}
```

//Step-6. Change the next of last node

```
ptr->Rnext = newnode;
```

```
newnode->Lnext = ptr;
```

```
return(head);
```

```
}
```

Inserting node at the middle of DLL

```
struct Node *insertMid_DLL(struct Node *head)
```

```
{
    int newdata, loc=2, count=1;
    struct Node *newnode, *ptr;
    newnode = (struct Node*)malloc(sizeof(struct Node));
```

```
    printf("Enter an element to insert:");
    scanf("%d",&newnode->data);
```

```
    //Step-3. If List is empty, then make newnode as head
```

```
    //head=newnode;
```

```
    //newnode->next=NULL
```

```
    ptr=head;
```

```
    while(ptr!= NULL && count!=loc)
```

```
    {
```

```
        ptr = ptr->Rnext;
```

```
        count=count+1;
```

```
    }
```

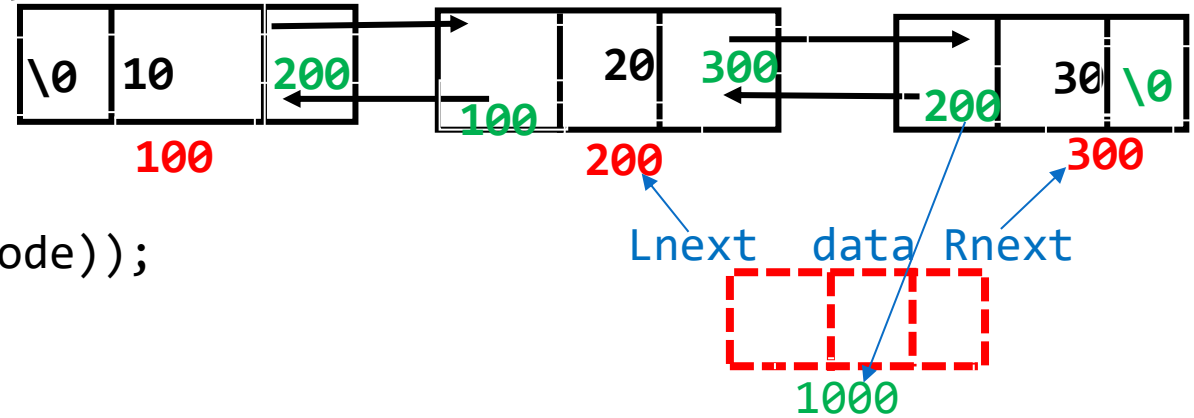
```
    newnode->Rnext=ptr->Rnext
```

```
    newnode->Lnext=ptr;
```

```
    ptr->Rnext=newnode;
```

```
    return(head);
```

head



Delete node at beginning of DLL

```
struct Node *deleteBeg_DLL(struct Node *head)
{
```

```
    struct Node *ptr;
```

```
    //Step-1: Return head if list is empty
```

```
    if(head==NULL)
```

```
        return(head);
```

```
    //Step-2: Break the pointer connection and
    //reconnect
```

```
    ptr=head;
```

```
    head=head->Rnext;
```

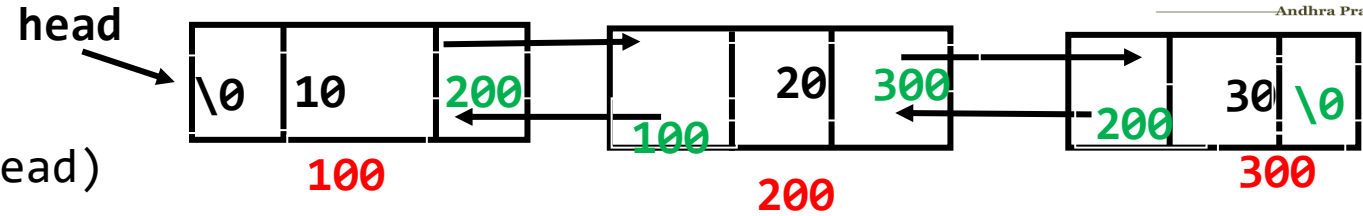
```
    head->Lnext=NULL;
```

```
    //Step-3: Delete the node
```

```
    free(ptr);
```

```
    return(head);
```

```
}
```

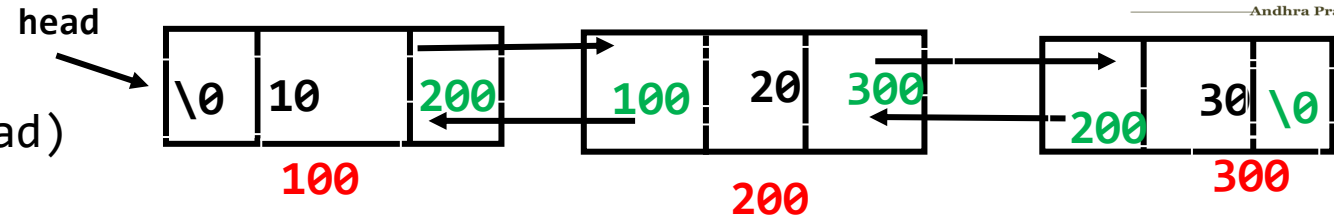


Delete node at the end of DLL

```

struct Node *deleteEnd_DLL(struct Node *head)
{
    //Step-1: Return head if list is empty
    if(head==NULL)
        return(head);
    //Step-2: Traverse the list until end
    struct Node *curr, *prev;
    for(curr=head; curr->Rnext!=NULL; curr=curr->Rnext)
    {
        prev=curr;
    }
    //Step-3: Break the pointer connection and set
    //previous node pointer to null
    prev->Rnext=NULL;

    //Step-2: Delete the node
    free(curr);
    return(head);
}
  
```



Delete node at the middle of DLL

```
struct Node *deleteMid_DLL(struct Node *head)
```

```
{
```

```
    printf("Enter an element to delete:");
```

```
    scanf("%d",&key);
```

```
    if(curr == NULL)
```

```
        return(head);
```

```
    curr=head;
```

```
    if(curr!= NULL && curr->data == key){
```

```
        head= curr->Rnext;
```

```
        free(curr);
```

```
        return(head);
```

```
    }
```

```
    while (curr != NULL && curr->data != key){
```

```
        prev = curr;
```

```
        curr = curr->Rnext;
```

```
    }
```

```
    prev->Rnext = curr->Rnext;
```

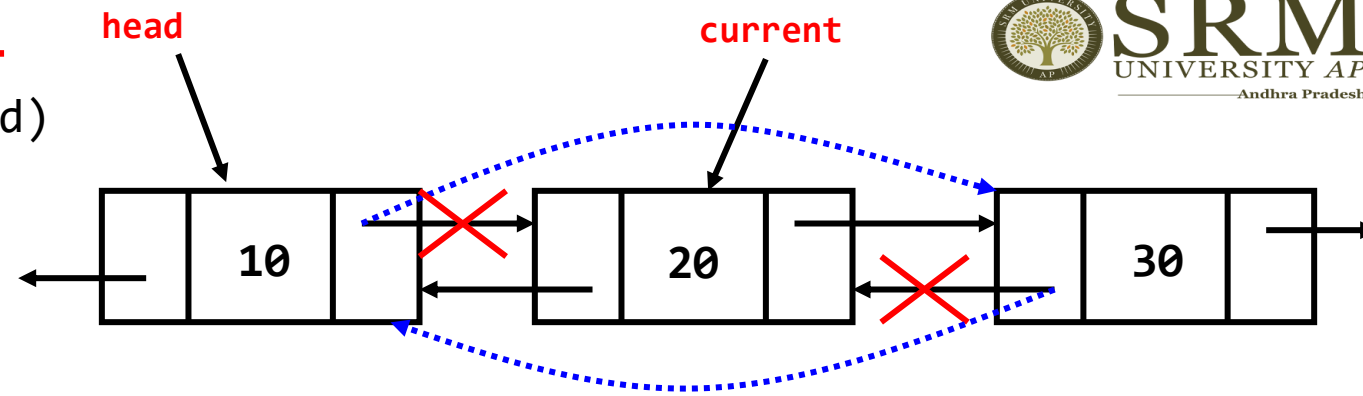
```
    temp=curr;
```

```
    curr=curr->Rnext;
```

```
    curr->Lnext=Prev
```

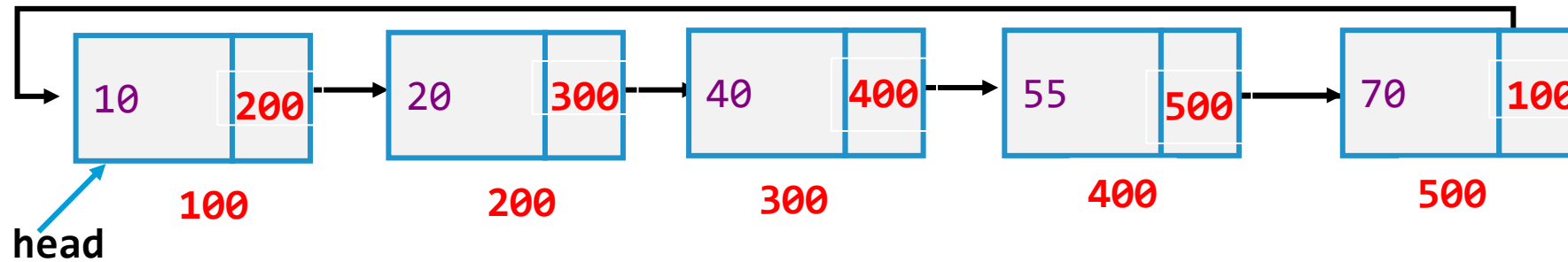
```
    free(temp);
```

```
}
```



Circularly Linked Lists (CLL)

- It supports traversing from the end of the list to the beginning by making the last node point back to the head of the list.



```
struct Node
{
    int data;
    struct Node *next;
};
```

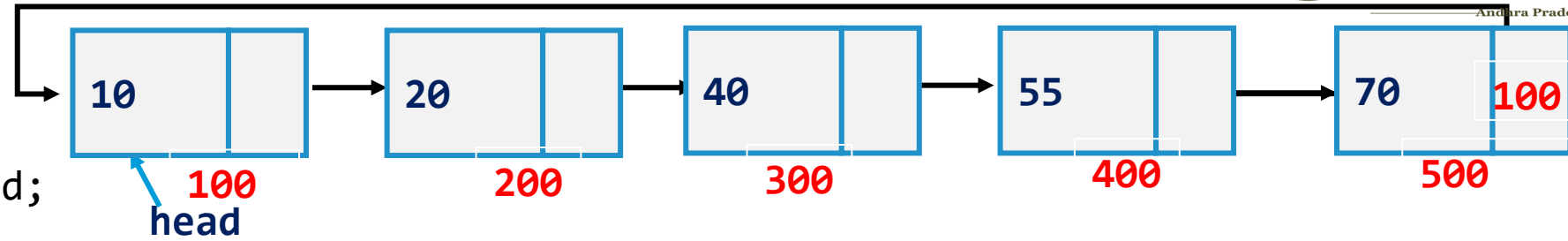
Operations on Circularly Linked Lists (CLL)

```
struct Node *create_CLL()
{
    int i, n;
    struct Node *ptr, *head;

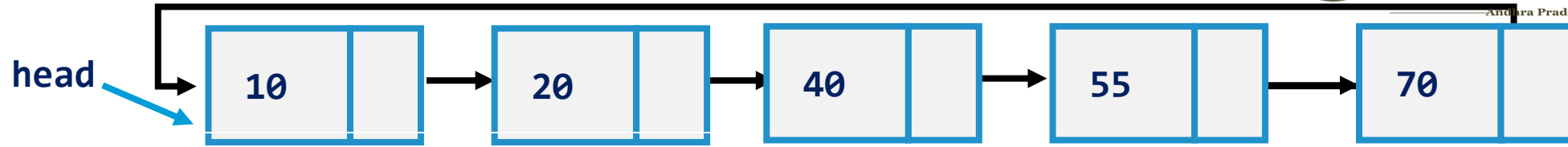
    printf("\n How many elements to enter?");
    scanf("%d", &n);

    for (i=1; i<=n; i++)
    {
        if (i == 0)
        {
            head = (struct Node*)malloc(sizeof(struct Node));
            ptr = head;
        }
        else
        {
            ptr->next = (struct Node*)malloc(sizeof(struct Node));
            ptr = ptr->next;
        }
        printf("Enter the value of %d node: ", i);
        scanf("%d",&ptr->data);
    }

    ptr->next = head;
    return(head);
}
```



Operations on Circular Linked Lists (CLL)



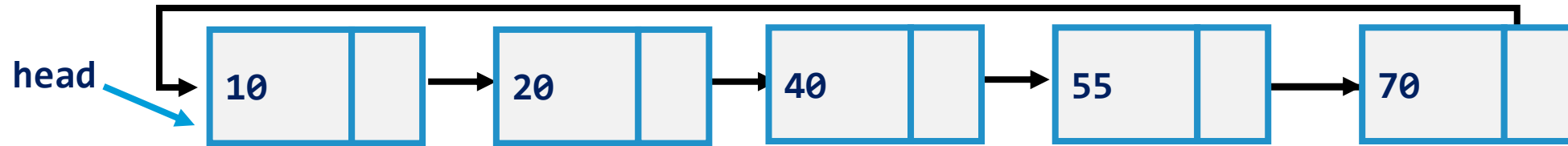
```
void search_CLL(struct Node *head)
{
```

```
    struct Node *ptr;
    int key, count=0;
    printf("Enter an element to search:");
    scanf("%d",&key);
    if(head->data==key){
        printf("Found"); return; }
    for(ptr = head->next; ptr!= head; ptr = ptr->next)
    {
        if ((ptr->data)==key){
            count=1;
            break;
        }
    }
    if(count==1)
        printf("found");
    else
        printf("not found");
}
```

Data Structures--> Unit-II:

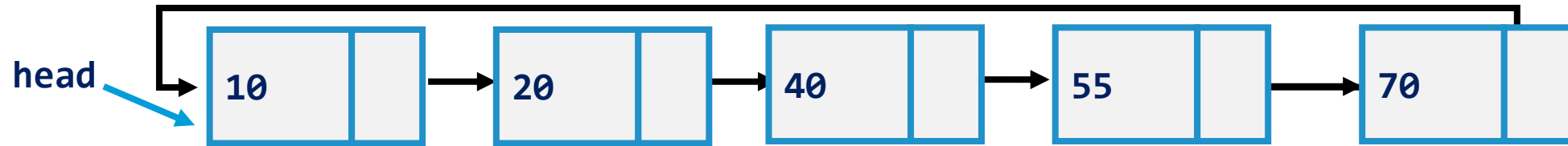
```
void display_CLL(struct Node *head)
{
    struct Node *ptr;
    printf("\nNode data %d", head->data);
    for(ptr=head->next; ptr!= head; ptr = ptr->next)
    {
        printf("\nNode data=%d", ptr->data);
    }
    printf ("\n");
}
```

Display the content of the CLL



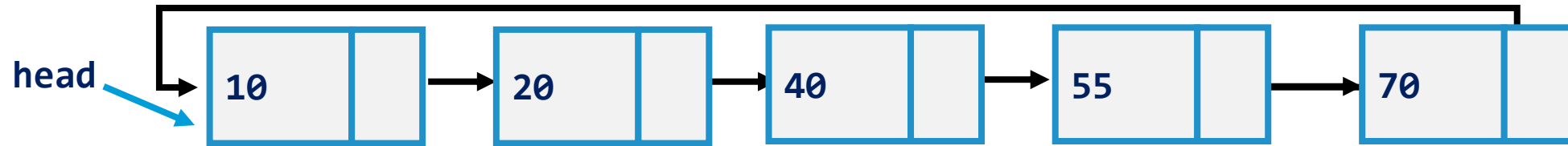
```
void displayList(struct Node* head)
{
    if (head == NULL) {
        printf("List is empty.\n");
        return;
    }
    struct Node* temp = head;
    do
    {
        printf("%d -> ", temp->data);
        temp = temp->next;
    }while (temp != head);
    printf("(Back to head: %d)\n", head->data);
}
```

Inserting node at beginning of CLL



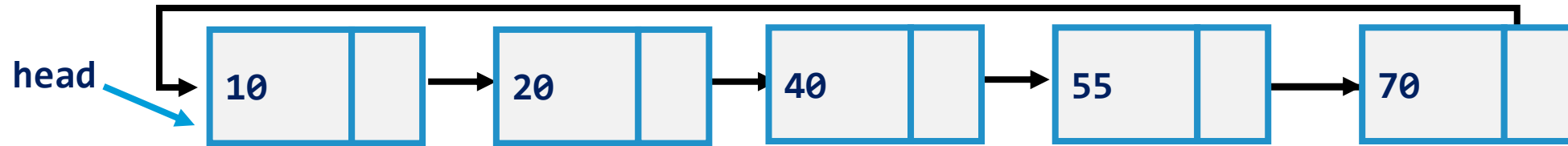
```
newnode->next = head;
ptr=head;
while(ptr->next !=head)
{
    ptr=ptr->next;
}
ptr->next=newnode
head=newnode;
return(head);
```

Inserting node at Middle of CLL



```
ptr=head;
while(ptr!= head && count!=loc)
{
    ptr = ptr->next;
    count=count+1;
}
newnode->next=ptr->next;
ptr->next=newnode;
return(head);
```

Delete node at beginning of CLL



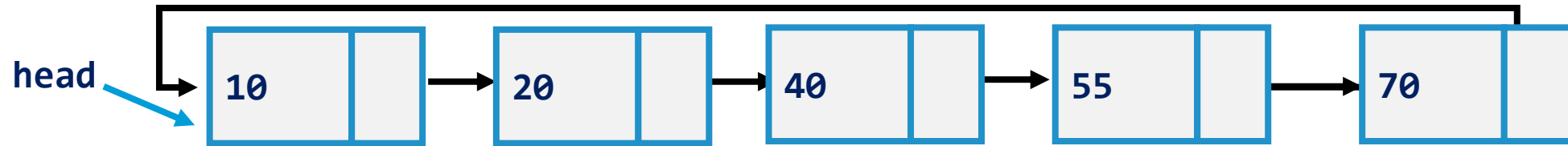
```
void deleteAtBeg(struct Node* head) {
    if (head == NULL) {
        printf("List is empty.\n");
        return;
    }
    struct Node* temp = head;
    struct Node* last = head;

    // If only one node exists
    if (temp->next == head) {
        free(temp);
        head = NULL;
        return;
    }
```

```
    // Find the last node
    while (last->next != head) {
        last = last->next;
    }

    head = temp->next;
    last->next = head;
    free(temp);
}
```

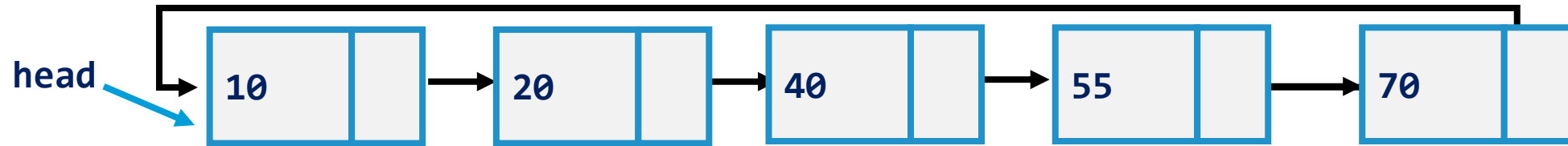
Delete node at the end of CLL



```
void deleteAtEnd(struct Node* head) {  
    if (head == NULL) {  
        printf("List is empty.\n");  
        return;  
    }  
    struct Node* temp = head;  
    struct Node* prev = NULL;  
  
    // If only one node exists  
    if (temp->next == head) {  
        free(temp);  
        head = NULL;  
        return;  
    }  
}
```

```
// Traverse to the last node  
while (temp->next != head) {  
    prev = temp;  
    temp = temp->next;  
}  
  
prev->next = head;  
free(temp);  
}
```

Delete node at the middle of CLL



```
void deleteAtMid(struct Node* head, int pos) {
    if (head == NULL) {
        printf("List is empty.\n");
        return;
    }

    struct Node* temp = head;
    struct Node* prev = NULL;

    // If deleting the first node
    if (pos == 1) {
        deleteAtBeg(head);
        return;
    }
```

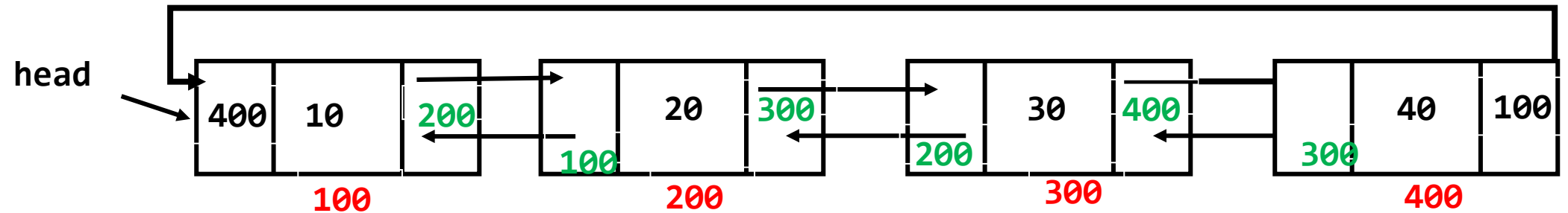
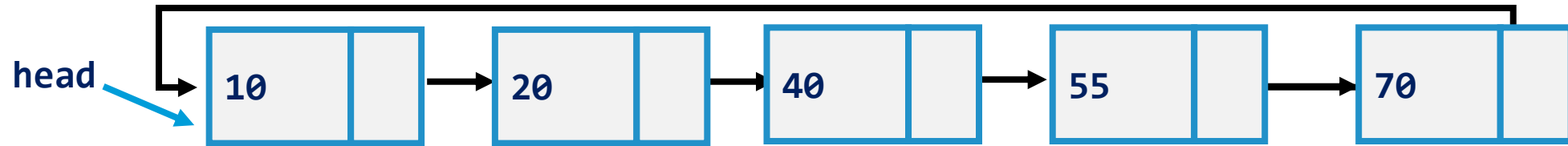
```
    int count = 1;
    while (count < pos && temp->next != head) {
        prev = temp;
        temp = temp->next;
        count++;
    }

    if (count < pos) {
        printf("Invalid position.\n");
        return;
    }

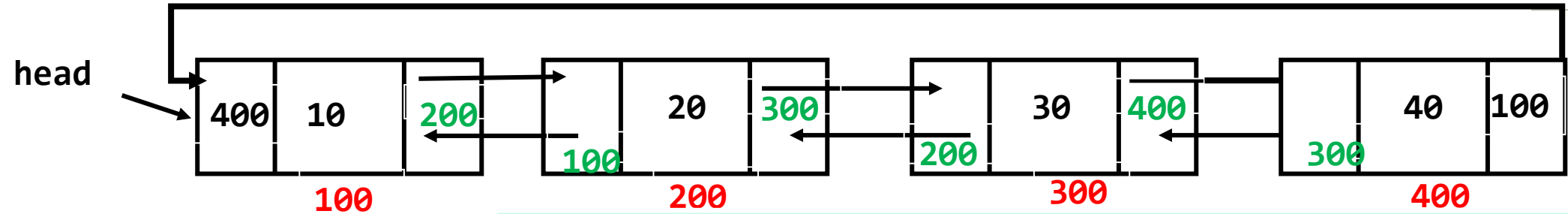
    prev->next = temp->next;
    free(temp);
```

```
}
```

Circular Doubly Linked Lists (CDLL)

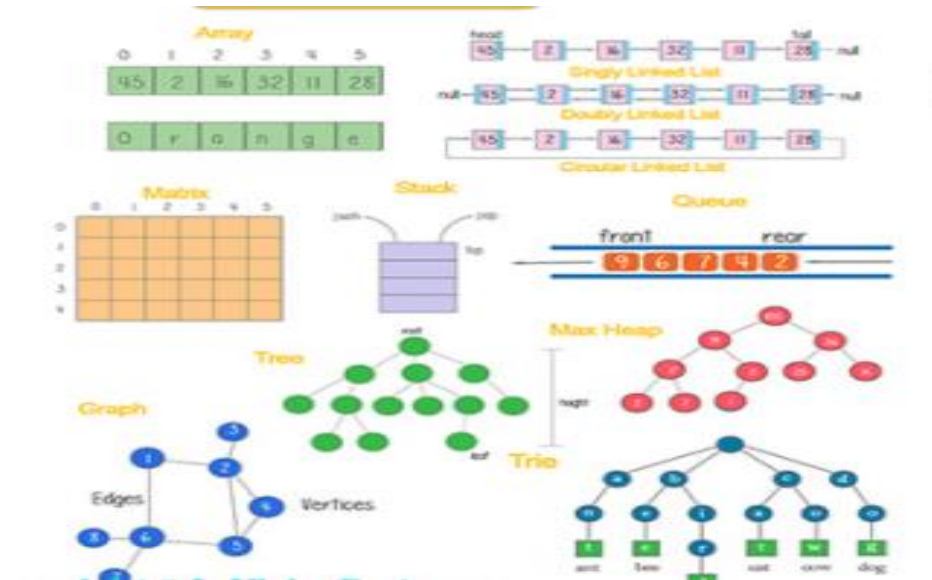


Circular Doubly Linked Lists (CDLL)



```
struct Node *create_CDLL()
{
    int i, n;
    struct Node *curr, *temp, *head;
    printf("\n How many elements you want to enter?");
    scanf("%d", &n);
    for (i=1; i<=n; i++)
    {
        if (i == 1){
            head = (struct Node*)malloc(sizeof(struct Node));
            head->Lnext=NULL;
            curr = head;
        }
        else{
            temp = (struct Node*)malloc(sizeof(struct Node));
            curr->Rnext=temp;
            temp->Lnext=curr;
            curr=temp;
        }
        printf("Enter the %d th element: ", i);
        scanf("%d",&curr->data);
    }
    curr->Rnext = head;
    head->Lnext = curr;
    return(head); }
```

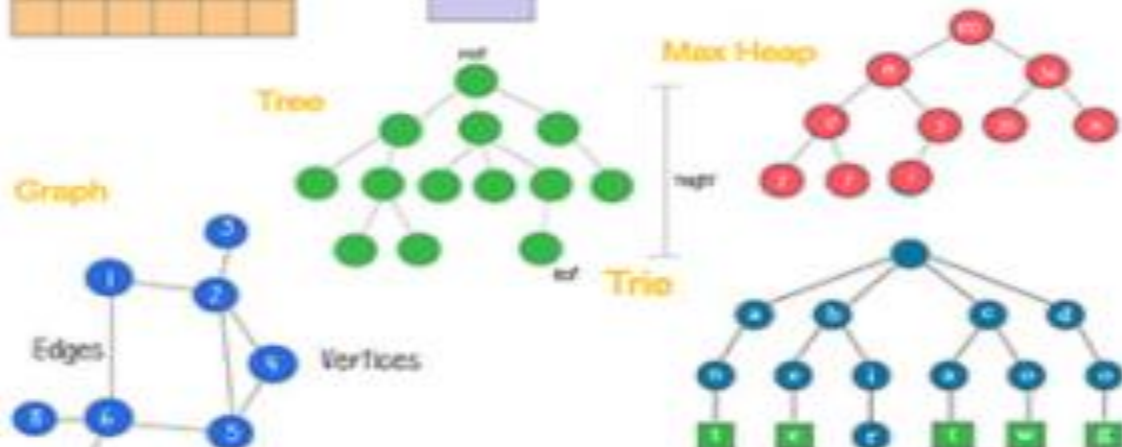
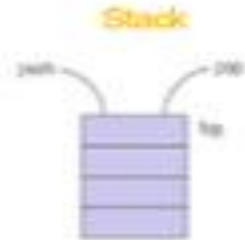
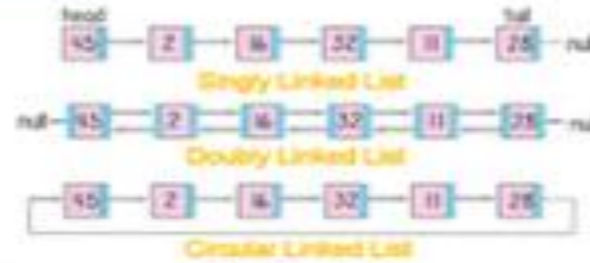

Data Structures



Mr. Pandu Sowkuntla
Asst. Professor,
Dept. of CSE, SRM University AP



Data structures



UNIT-III

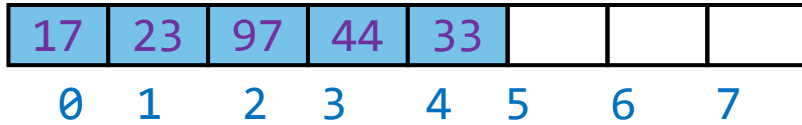


TREES

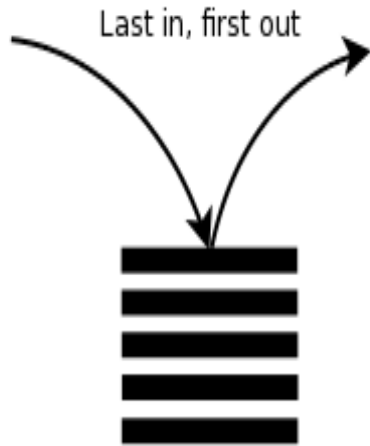
Introduction to Trees

Why Tree data structure?

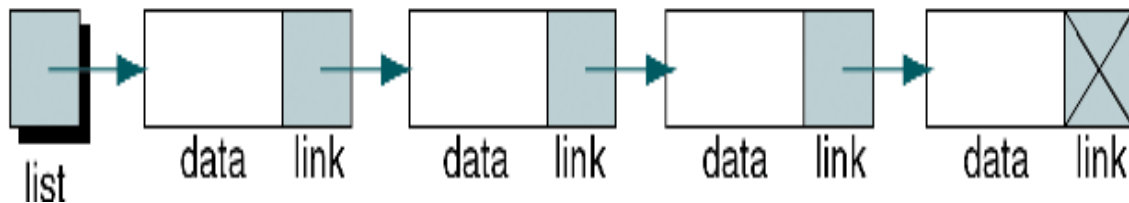
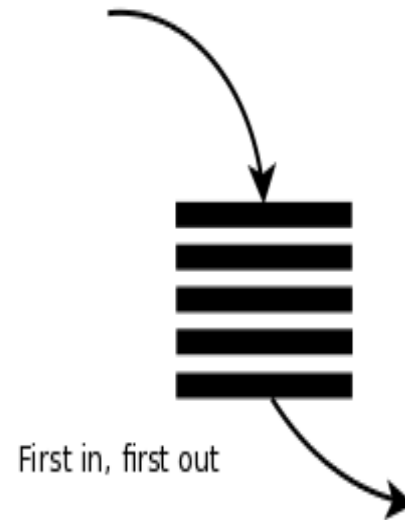
Array



Stack:



Queue:



▶ Linear data structure:

- Stores the data one after the other (for an element we can find before element and after element).

▶ Choosing an appropriate data structure:

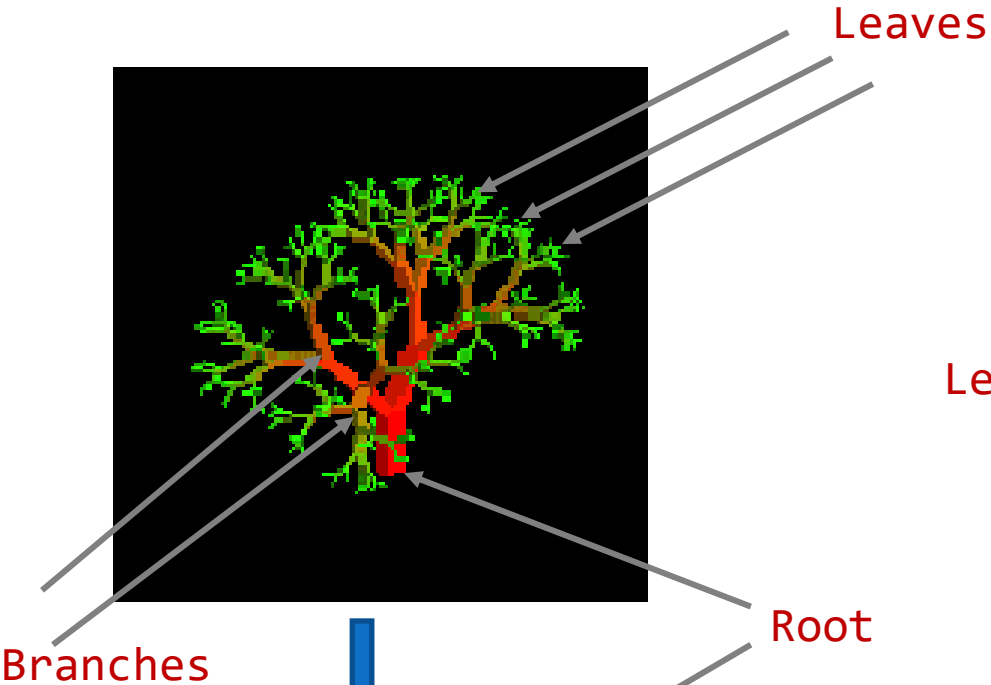
- Best suited for problem scenario
- Cost of common operations
- Space utilization

▶ Tree data structure used to:

- Store the data hierarchical (Ex: File System in an Operating System)
- Organize the data for quick insertion, deletion and search (Ex: Binary Search Algorithm)
- Solve complex problems (Ex: Network routing Algorithms)

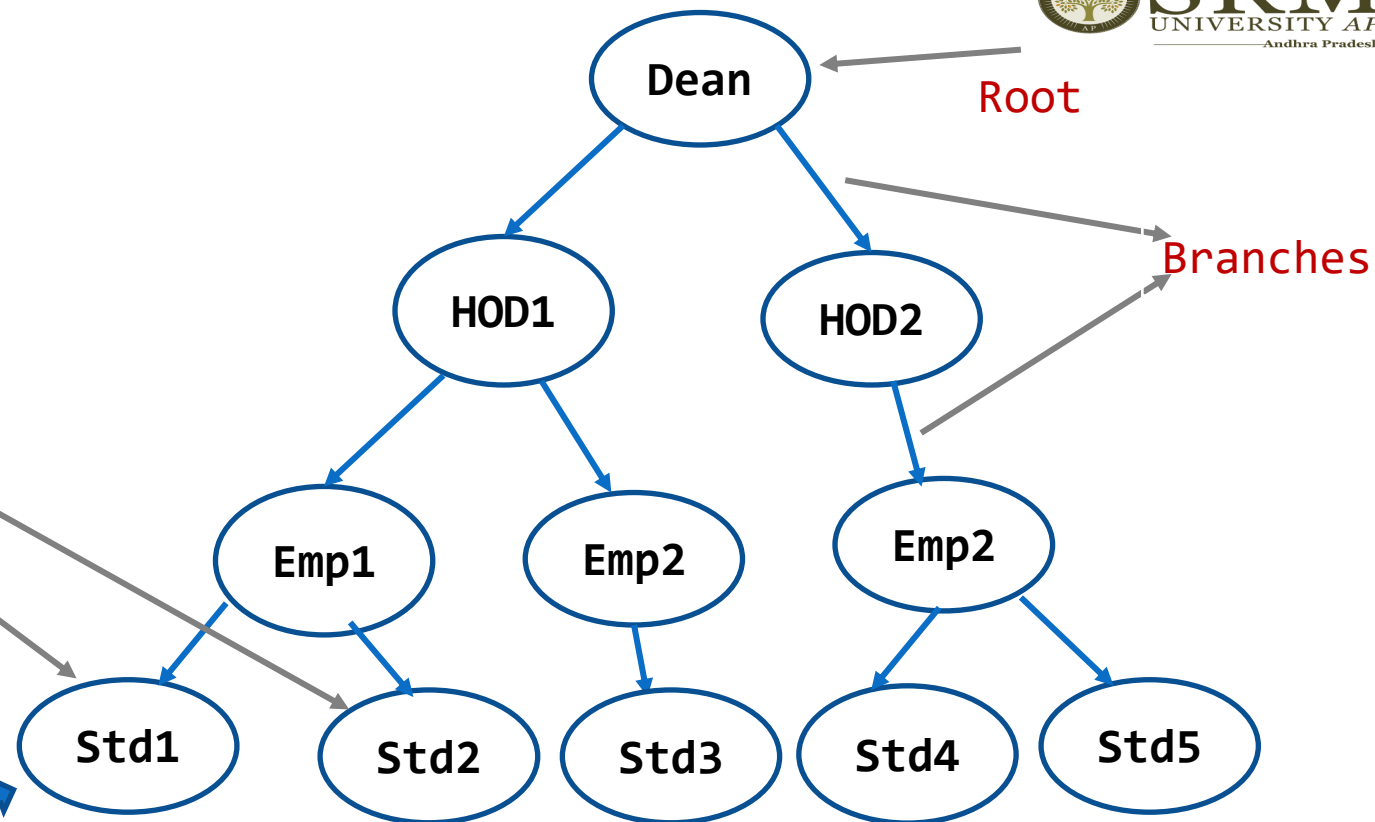
Introduction to Trees

What is Tree data structure ?



Leaves

Root



Logical representation of Tree data structure

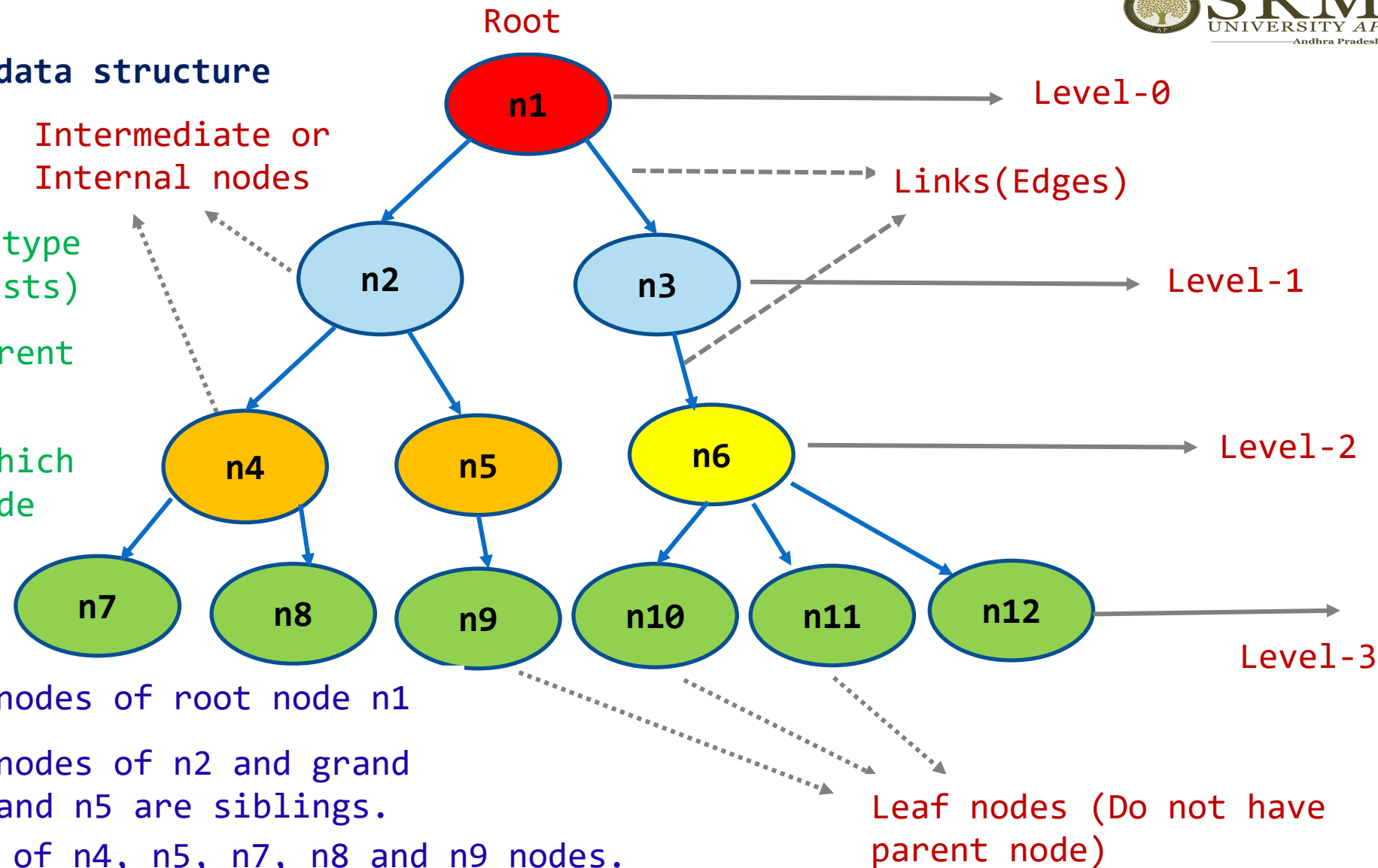
Tree data structure is an efficient way of storing and organizing the data that is naturally hierarchical.

OR

Tree data structure is a non-linear (hierarchical) data structure that consists of nodes with parent-child relationship.

Terminology used in Tree data structure

- ▶ A node can contain any type of data (Like Linked lists)
- ▶ The root (n1) is the parent of n2 and n3 nodes
- ▶ Root is the only node which does not have parent node

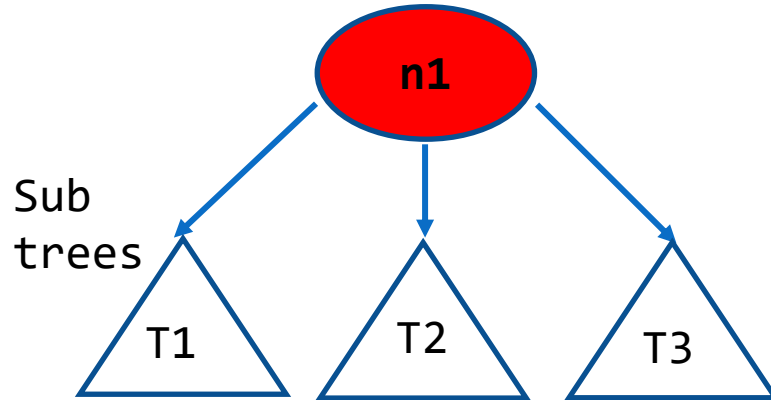


- ▶ n2 and n3 are children nodes of root node n1
- ▶ n4 and n5 are children nodes of n2 and grand children of n1. And n4 and n5 are siblings.
- ▶ n1 and n2 are ancestors of n4, n5, n7, n8 and n9 nodes.
- ▶ n4 and n5 are descendants of n2 node.
- ▶ Nodes not having same parent with same grand parent are called cousins (Ex: n8 and n9)

Introduction to Trees

Properties of Tree data structure

1. Tree is a recursive data structure



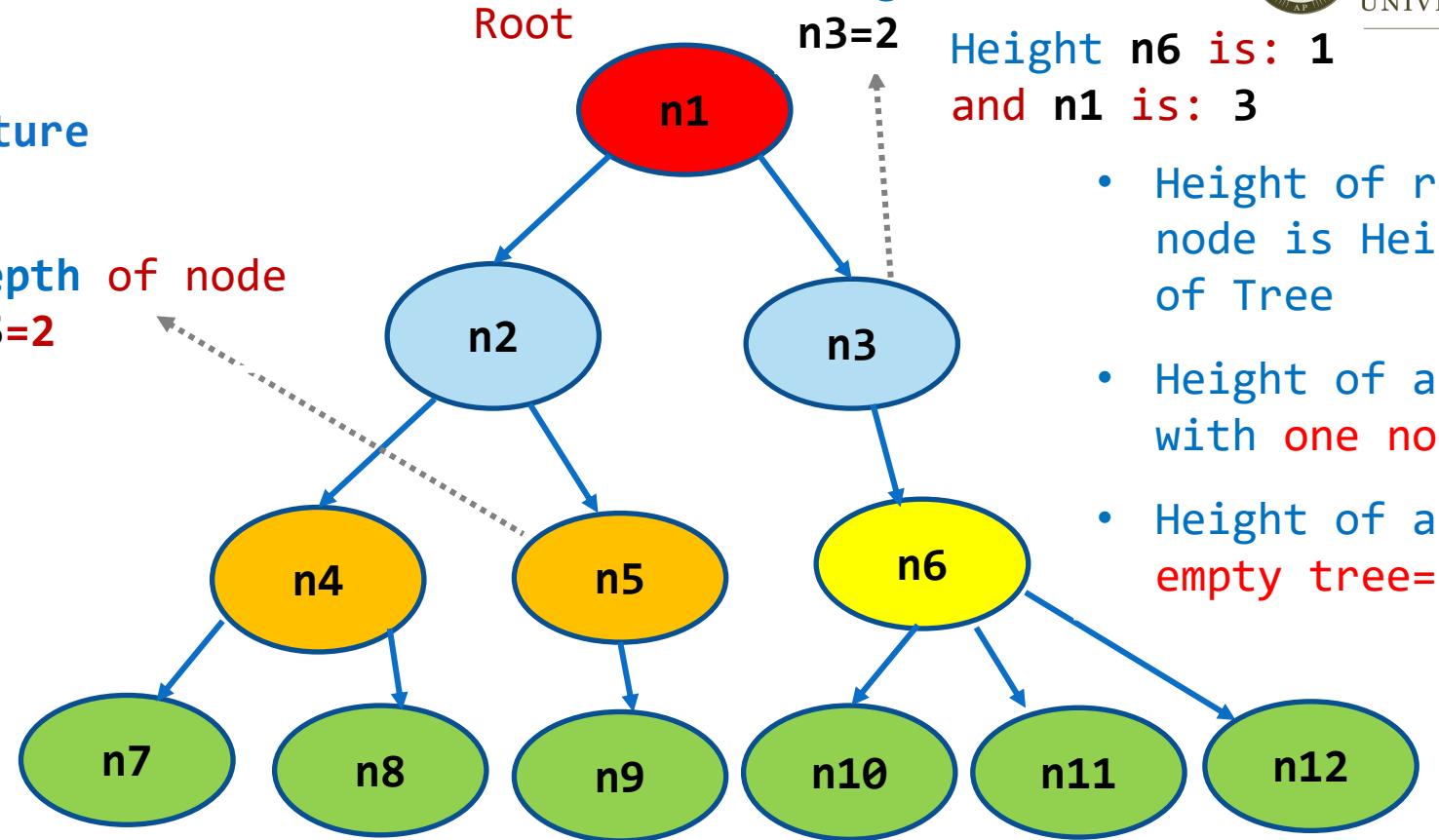
2. **Degree** of a node is the no. of children that a node has.

3. If Tree contains n nodes then it contains $n-1$ edges
(A tree does not have cycles).

4. **Depth** of a node X is length (number of edges) of the path from the **root node** to X

5. **Height** of a node X is number of edges in longest path from the node X to a **leaf node**

Depth of node
 $n5=2$

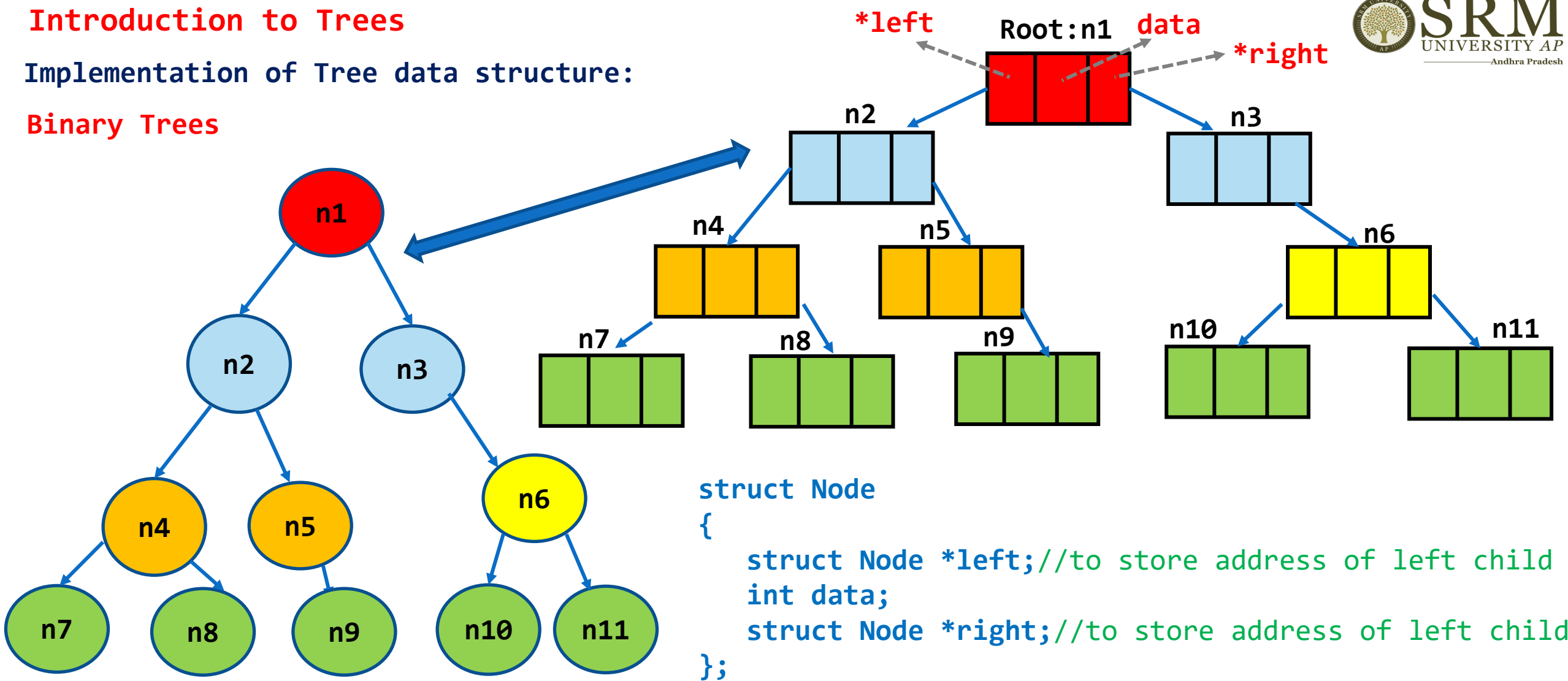


- Height of root node is Height of Tree
- Height of a tree with one node = 0
- Height of an empty tree = -1

Introduction to Trees

Implementation of Tree data structure:

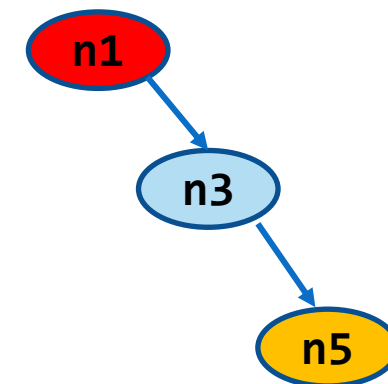
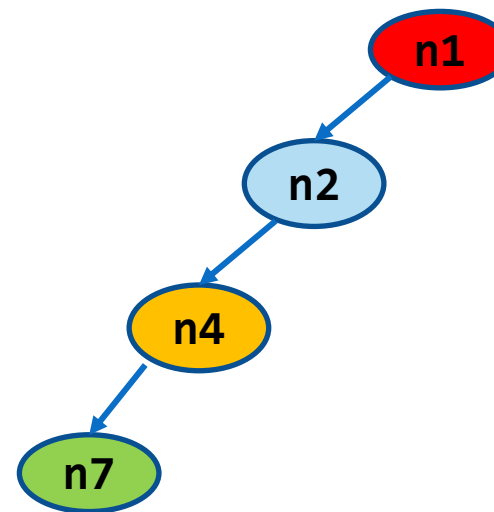
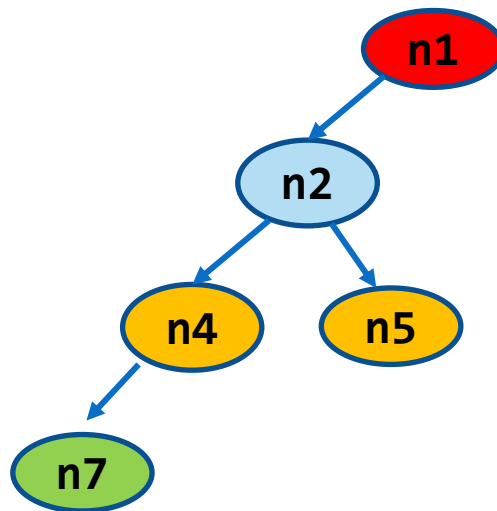
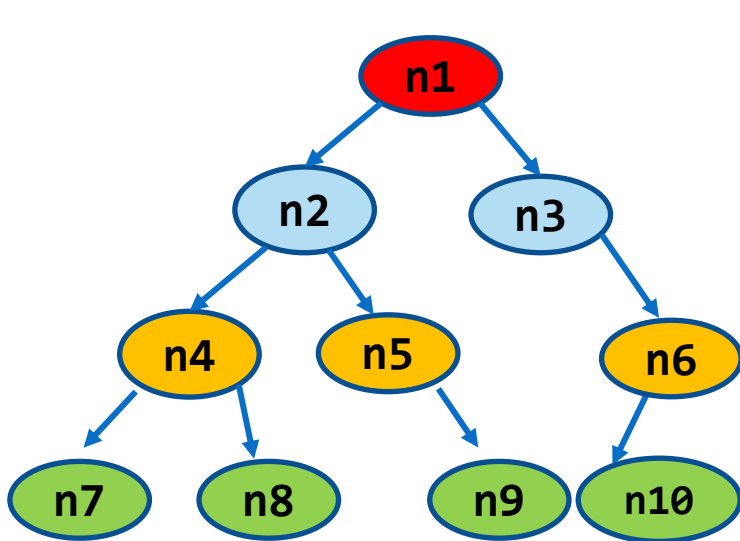
Binary Trees



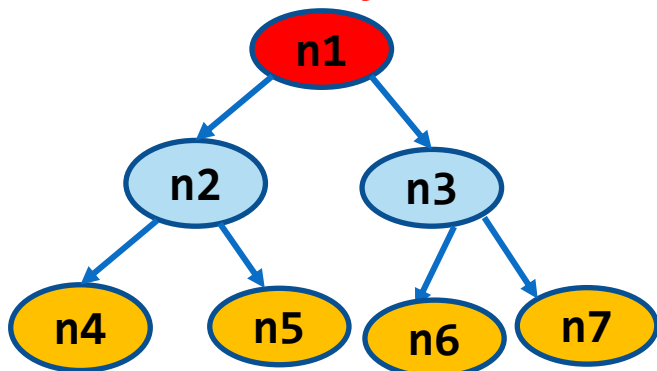
Note: Nodes are created dynamically

- A tree in which each node has at most two children is called as **Binary Tree**.
(No. of Children ≤ 2)

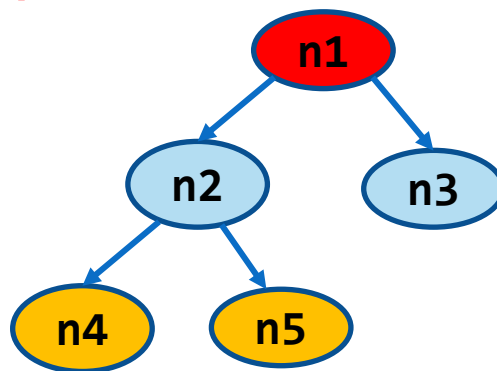
Binary Trees



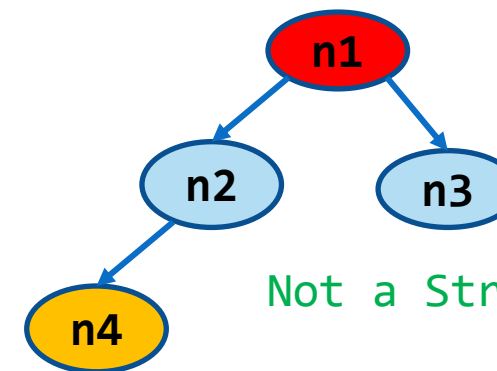
Strict Binary Tree or Proper or Full Binary Tree: In Strict Binary Tree, each node can have either zero or 2 nodes



Strict Binary Tree



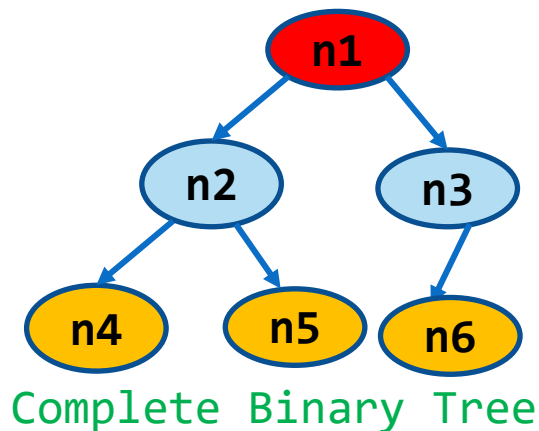
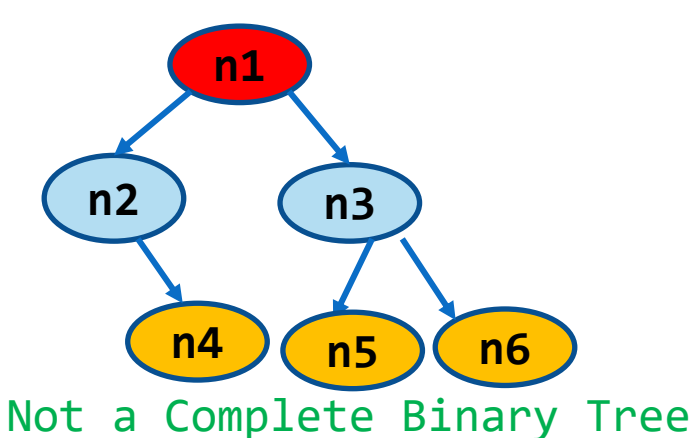
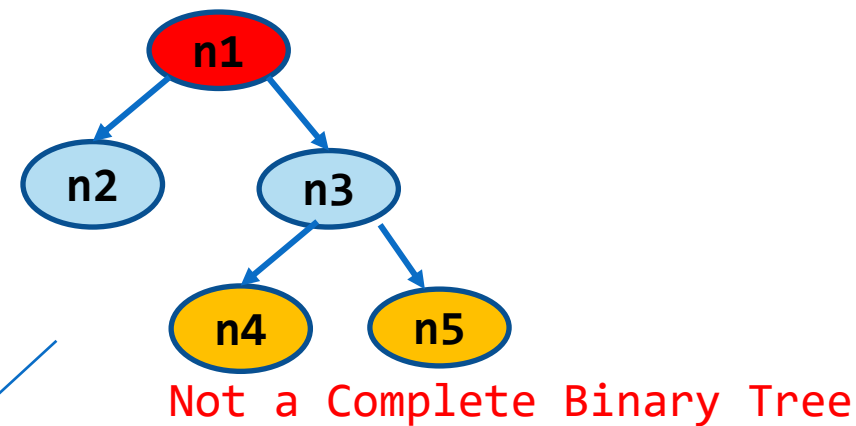
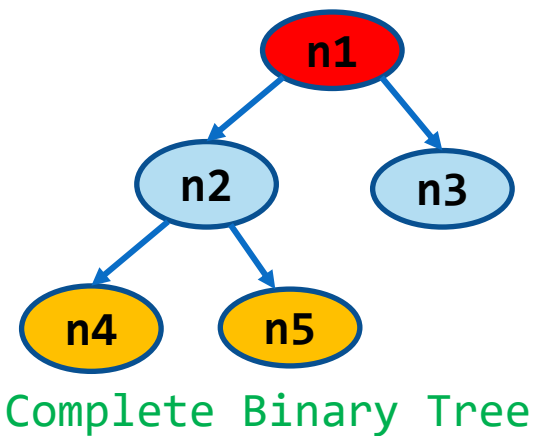
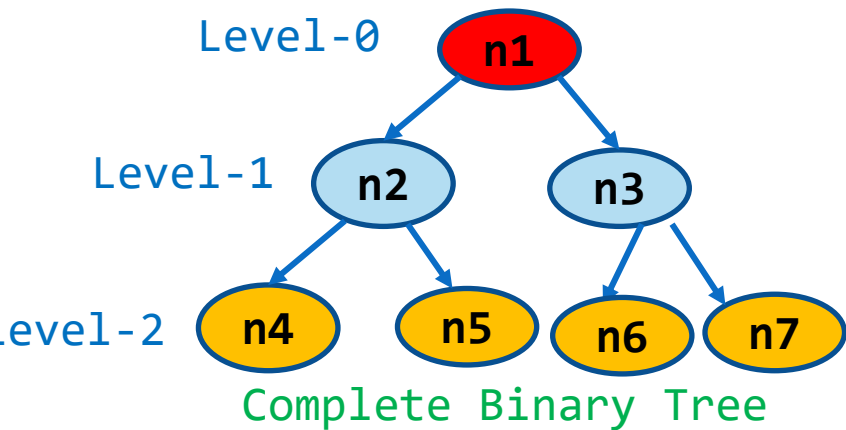
Strict Binary Tree



Not a Strict Binary Tree

Complete Binary Trees

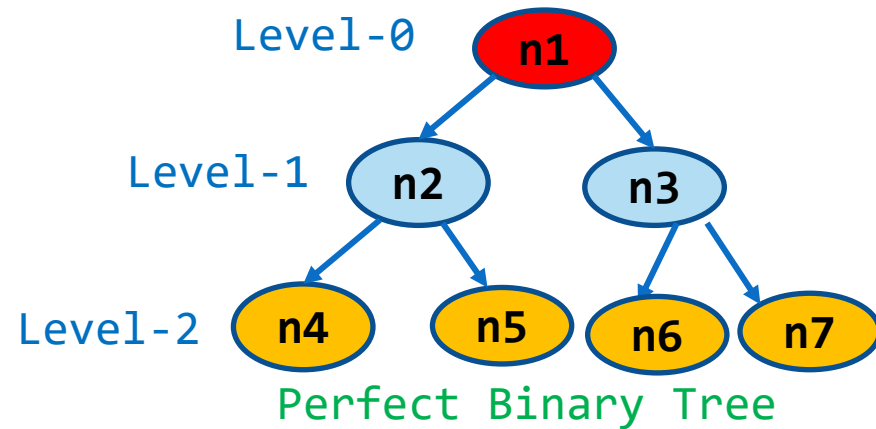
In Complete Binary Tree, all the levels except possibly last are filled and all nodes are as left as possible.



Maximum number of nodes in complete Binary tree at level-i = 2^i

Perfect Binary Trees

In Perfect Binary Tree, all the levels are filled.



With “n” nodes in Perfect binary tree, what is the height (h) of the tree?

Maximum no. of nodes in a binary tree with the height “h”,

$$\begin{aligned}
 &= 2^0 + 2^1 + \dots + 2^h \\
 &= 1 + 2^{h+1} - 2 \\
 &= 2^{h+1} - 1
 \end{aligned}$$

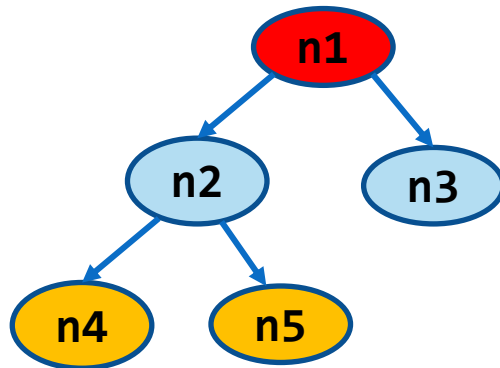
$$n = 2^{h+1} - 1$$

$$n+1 = 2^{h+1}$$

$$\log(n+1) = \log(2^{h+1})$$

$$\log(n+1) = (h+1)$$

$$h = \log(n+1) - 1$$



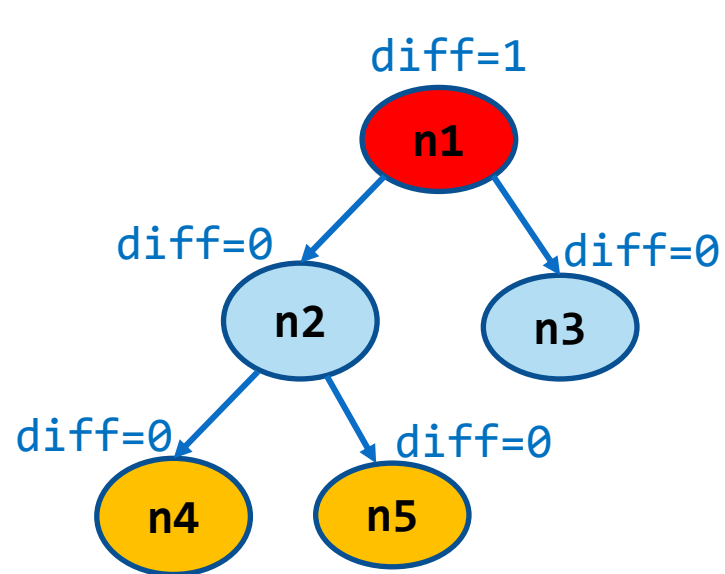
Complete Binary Tree
but not Perfect

Balanced Binary Trees or Height balanced binary trees

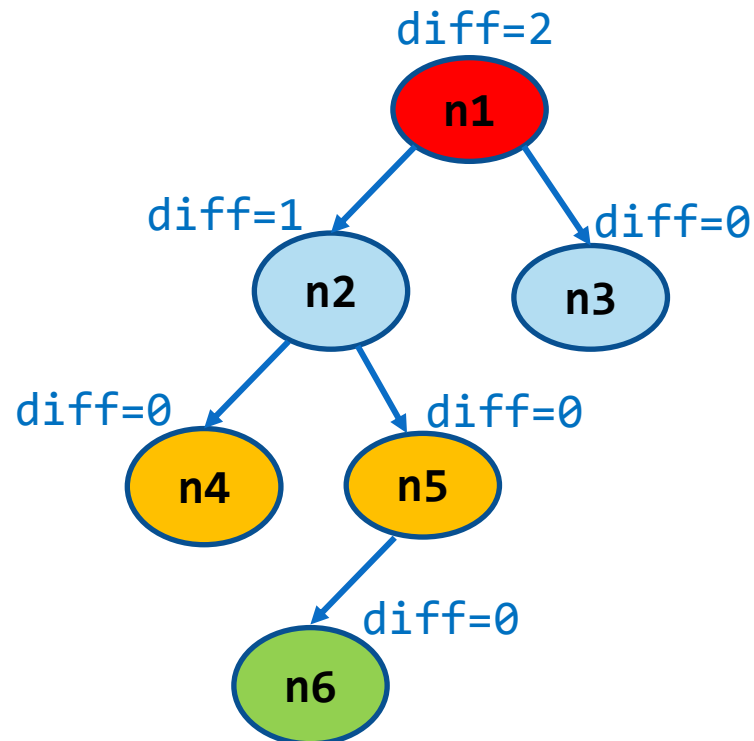
A Binary tree said to be if:

1. The height difference between the left and the right subtree for any node is not more than K (usually **K=1**).
2. The left subtree is balanced.
3. The right subtree is balanced.

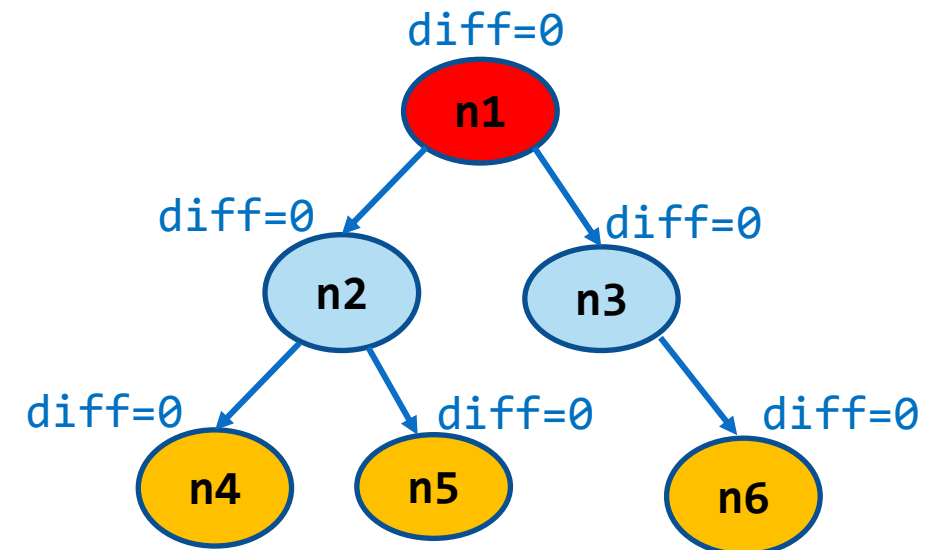
$$\text{Height diff} = | \text{Height of left child} - \text{Height of right child} |$$



Balanced binary tree



Not a Balanced binary tree

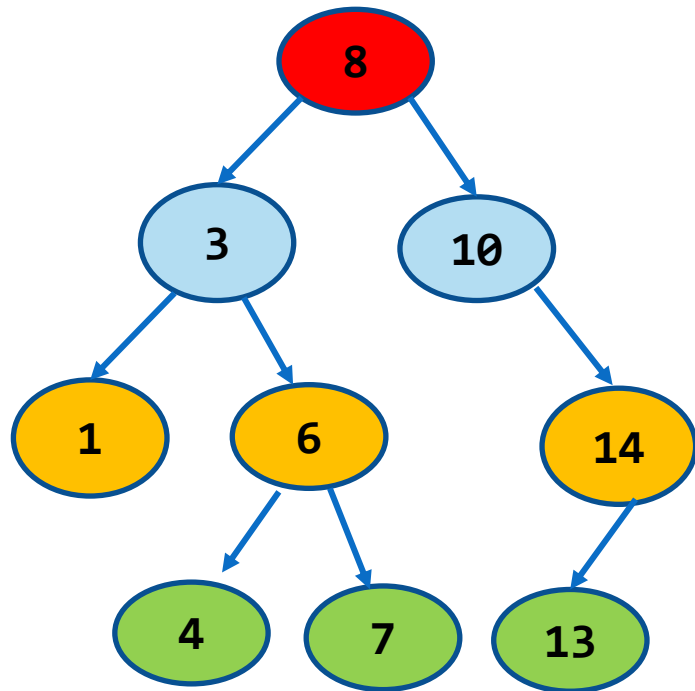


Balanced binary tree

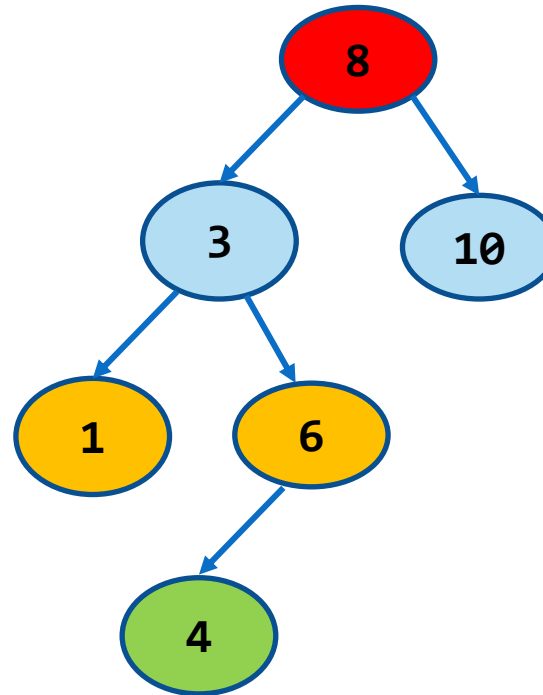
Binary Search Trees (BST)

BST is a binary tree where **each node n** satisfy the following:

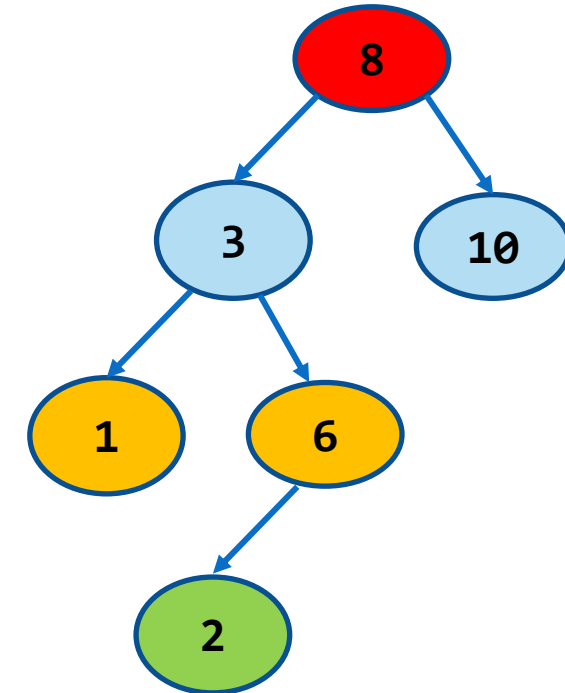
- Every node in the **left subtree of n** contains a value which is **smaller** than the value in **n**.
- Every node in the **right subtree of n** contains a value which is **larger** than the value in **n**.



Binary Search Tree



Binary Search Tree



Not a Binary Search Tree

Binary Search Trees (BST)

- Empty BST is a single pointer with the value of NULL.

```
root = NULL;
```

- A node in BST can be declared as:

```
struct Node
```

```
{
```

```
    struct Node *leftChild; //to store address of left child
```

```
    int data;
```

```
    struct Node *rightChild; //to store address of left child
```

```
};
```

- Create a node of BST as:

```
struct Node *newNode(int item)
```

```
{
```

```
    struct node *newnode = (struct Node*) malloc(sizeof(struct Node));
```

```
    newnode->data = item;
```

```
    newnode->leftChild = NULL;
```

```
    newnode->rightChild = NULL;
```

```
    return newnode;
```

```
}
```

```
int main()
```

```
{
```

```
    struct node *root;
```

```
    root=NULL;
```

```
    insert(root,45);
```

```
    insert(root,15);
```

```
    insert(root,79);
```

```
    insert(root,90);
```

```
    insert(root,10);
```

```
    insert(root,55);
```

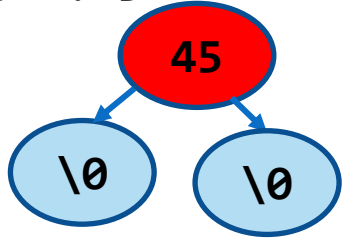
```
    insert(root,12);
```

```
    insert(root,50);
```

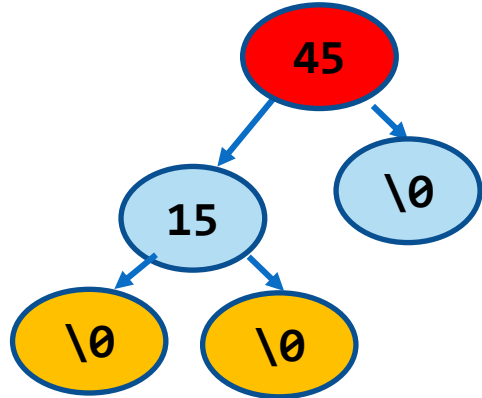
```
}
```

Elements: 45, 15, 79, 90, 10, 55, 12, 50

Step-4: 90,
90>45,
90>79



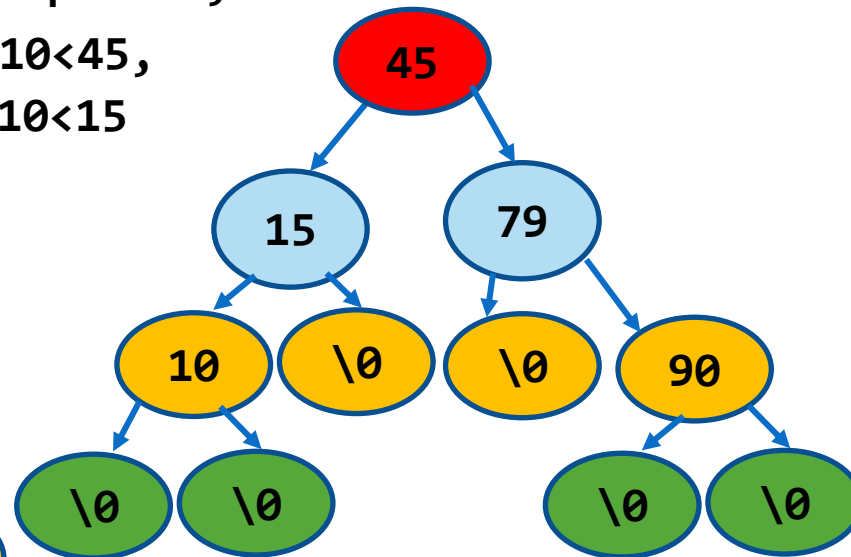
**Step-5:10,
10<45,
10<15**



$10 < 45$,
 $10 < 15$

```

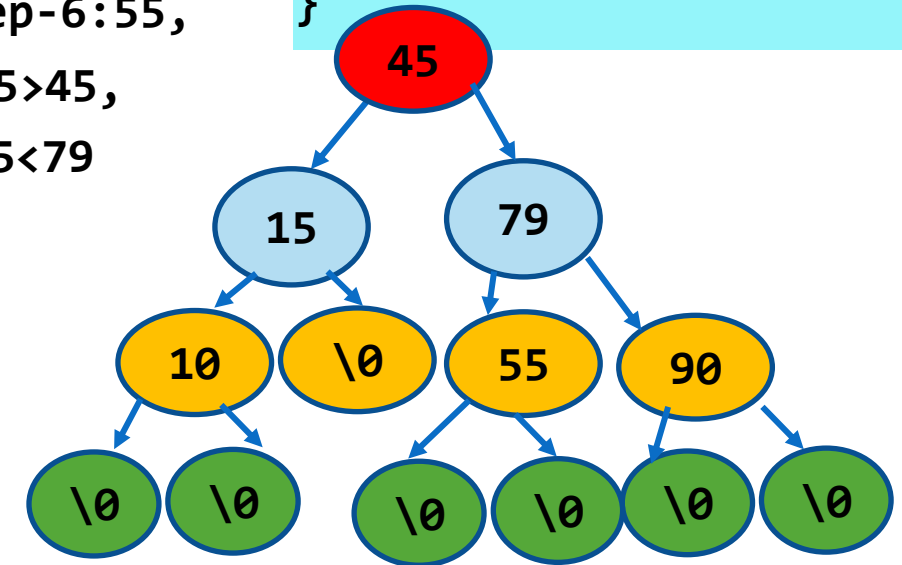
graph TD
    45((45)) --> 15((15))
    45 --> 79((79))
    15 --> 10((10))
    15 --> n15_0((\0))
    10 --> n10_0((\0))
    10 --> n10_1((\0))
    79 --> n79_0((\0))
    79 --> 90((90))
    90 --> n90_0((\0))
    90 --> n90_1((\0))
    style 45 fill:#ff0000,stroke:#0000ff,stroke-width:2px
    style 15 fill:#add8e6,stroke:#0000ff,stroke-width:2px
    style 79 fill:#add8e6,stroke:#0000ff,stroke-width:2px
    style 10 fill:#ffff00,stroke:#0000ff,stroke-width:2px
    style n15_0 fill:#ffff00,stroke:#0000ff,stroke-width:2px
    style n79_0 fill:#ffff00,stroke:#0000ff,stroke-width:2px
    style 90 fill:#ffff00,stroke:#0000ff,stroke-width:2px
    style n10_0 fill:#008000,stroke:#0000ff,stroke-width:2px
    style n10_1 fill:#008000,stroke:#0000ff,stroke-width:2px
    style n90_0 fill:#008000,stroke:#0000ff,stroke-width:2px
    style n90_1 fill:#008000,stroke:#0000ff,stroke-width:2px
  
```



p-6:55,
 5>45,
 5<79

```

graph TD
    45((45)) --> 15((15))
    45 --> 79((79))
    15 --> 10((10))
    15 --> N1((\0))
    10 --> N2((\0))
    10 --> N3((\0))
    79 --> 55((55))
    79 --> 90((90))
    55 --> N4((\0))
    55 --> N5((\0))
    90 --> N6((\0))
    90 --> N7((\0))
  
```

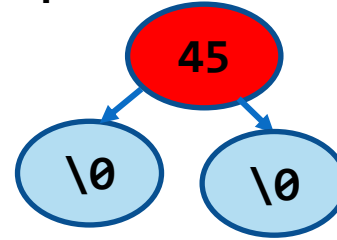


Binary Search Trees (BST)

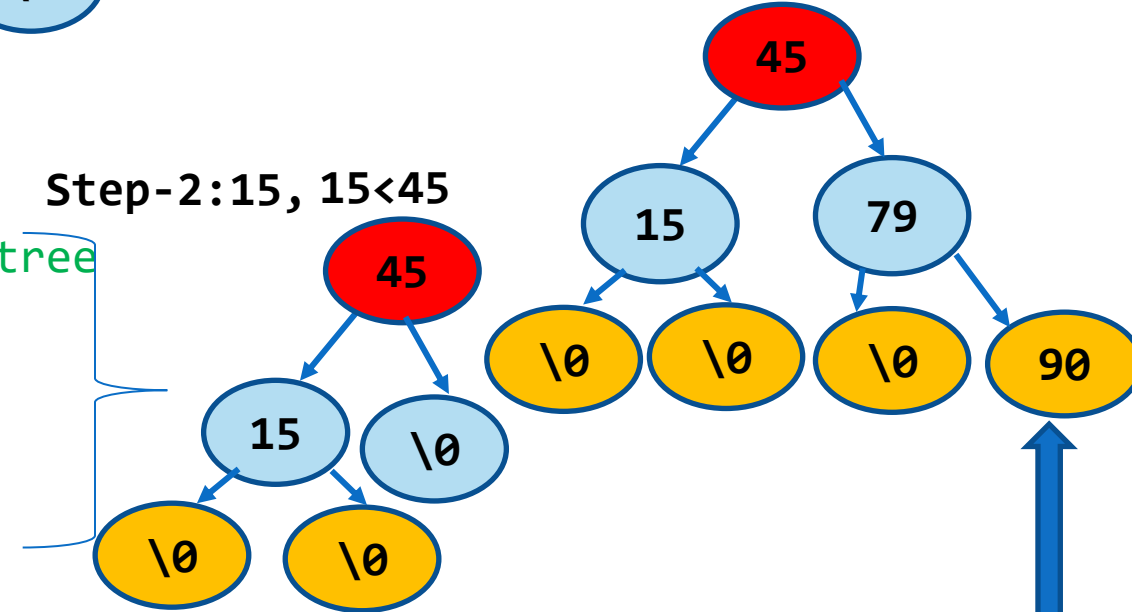
Elements: 45, 15, 79, 90, 10, 55, 12, 50

```
void insert_BST(struct Node *root,int item){  
    if(root == NULL){ //if tree is empty  
        root = newNode(item);  
    }  
    else{  
        current = root;  
        parent = NULL;  
        while(1){  
            parent = current;  
            if(item < parent->data){//go to left of the tree  
                current = current->leftChild;  
                if(current == NULL){ //insert to the left  
                    parent->leftChild = newNode(item);  
                    return;  
                }  
            }  
            else{ //go to right of the tree  
                current = current->rightChild;//insert to the right  
                if(current == NULL){  
                    parent->rightChild = newNode(item);  
                    return;  
                }  
            }  
        }  
    }  
}
```

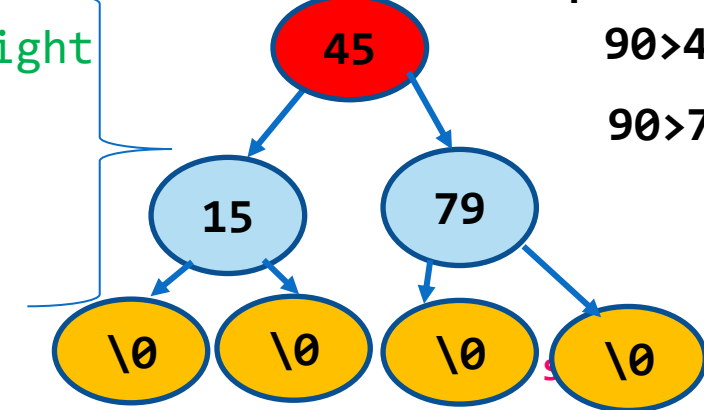
Step-1:45 root



Step-2:15, 15<45



Step-3:79, 79>45

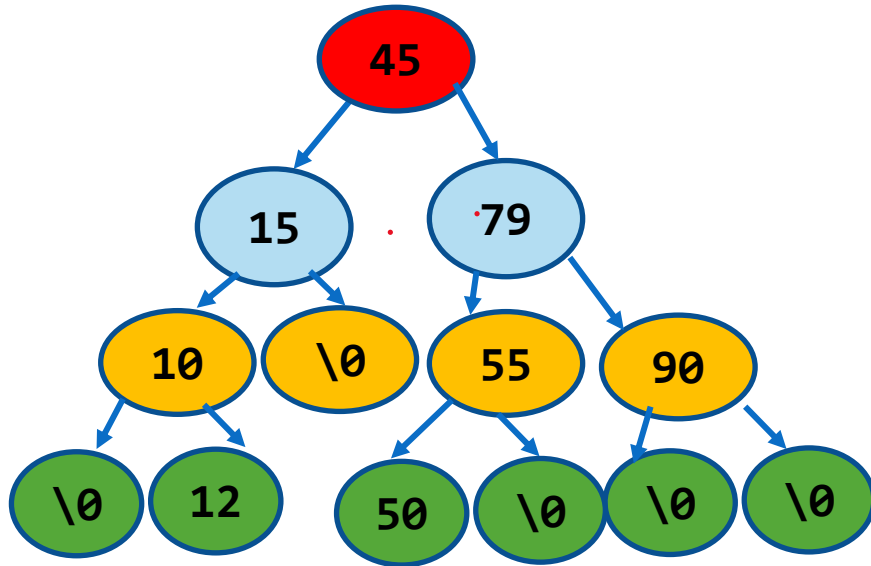


Step-4:90,

90>45

90>79

Binary Search Trees (BST)



```
void displayBST(struct Node *root)
{
    printf("BST elements are:")
    if(root!=NULL)
    {
        displayBST(root->leftChild);
        printf("%d\n", root->data);
        displayBST(root->rightChild);
    }
}
```

```
struct node *root;
int main()
{
    displayBST(root);
    if(searchBST(root, item)==TRUE)
        printf("Found");
    else
        printf("Not Found");
}
```

```
bool searchBST(struct Node *root, int item)
{
    if(root==NULL)
        return FALSE;
    else if(root->data==item)
        return TRUE;
    else if(item<root->data)
        searchBST(root->leftChild, item);
    else
        searchBST(root->rightChild, item);
}
```

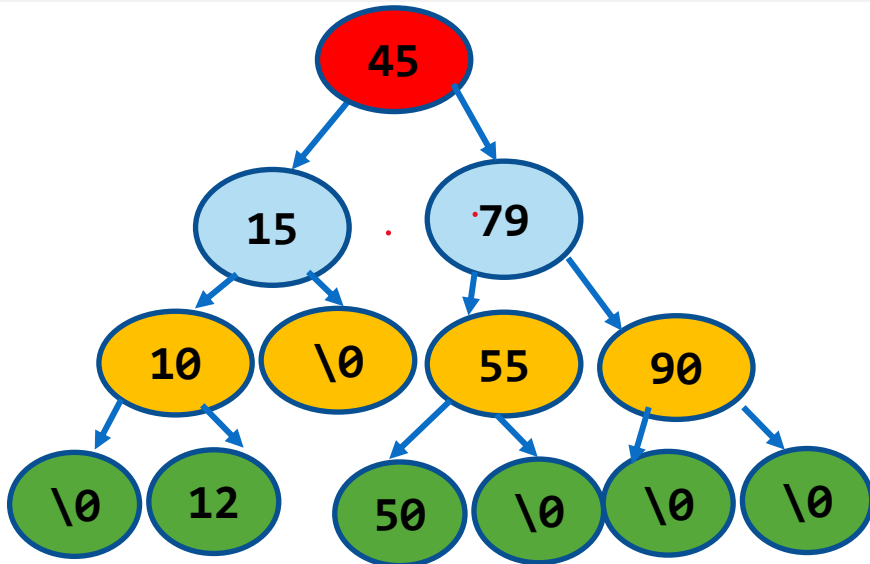
Binary Search Trees (BST)

```
void minBST(struct Node *root)
{
    if(root==NULL)
        printf("Empty Tree");
        return -1;
    else if(root->left==NULL)
        return root->data;
    else
        minBST(root->left);
}
```

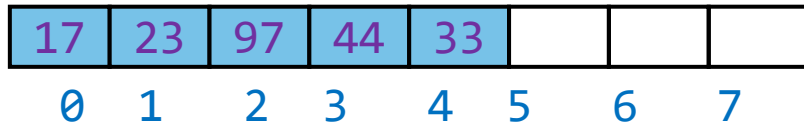
```
struct node *root;
int main()
{
    min=minBST(root);
    printf("Min val=%d", min);
    ht=maxBST(root);
    printf("Height=%d", ht);
}
```

```
void maxBST(struct Node *root)
{
}

}
```

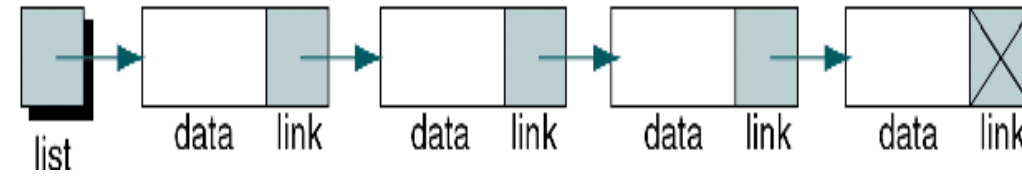


Binary Tree Traversal



Traversal in an array:
from $a[0]$ to $a[n-1]$

Linear Traversal



Traversal in a Linked list: from **head** to **NULL**

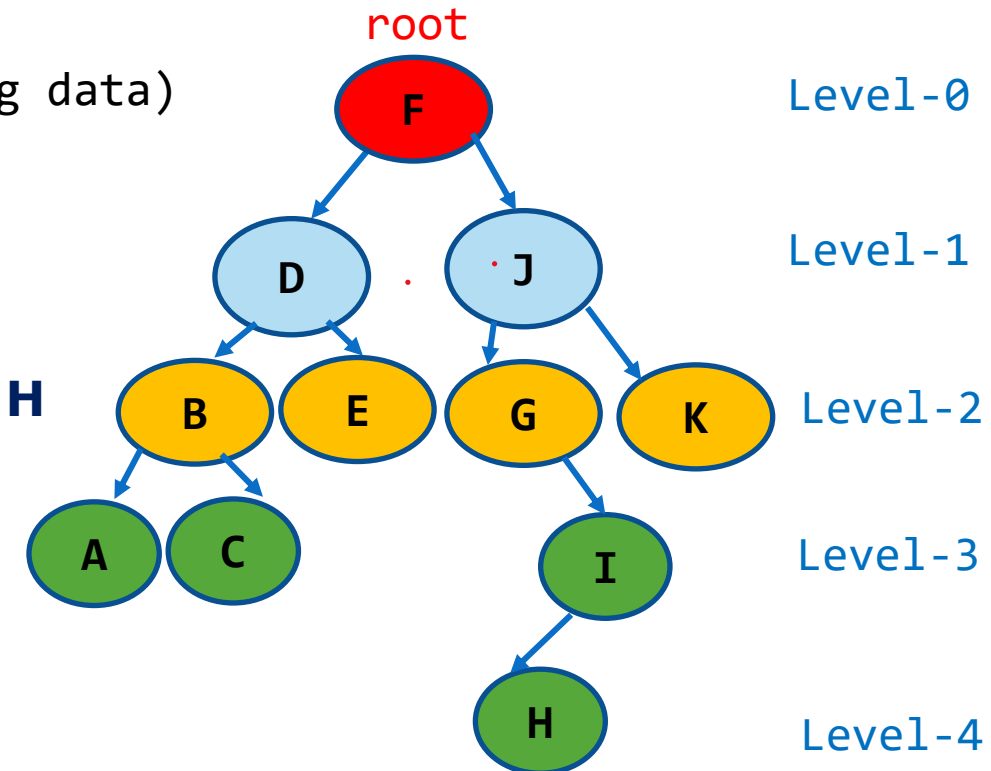
Tree traversal: Process of visiting (reading or processing data) each node in the tree exactly once in some order.

Tree traversal

→ **Breadth First Traversal**
(Level-order traversal): **F, D, J, B, E, G, K, A, C, I, H**

→ **Depth First Traversal**

1. Preorder
<root, left, right>: **F, D, B, A, C, E, J, G, I, H, K**
2. Inorder
<left, root, right>: **A, B, C, D, E, F, G, H, I, J, K**
3. Postorder
<left, right, root>: **A, C, B, E, D, H, I, G, K, J, F**



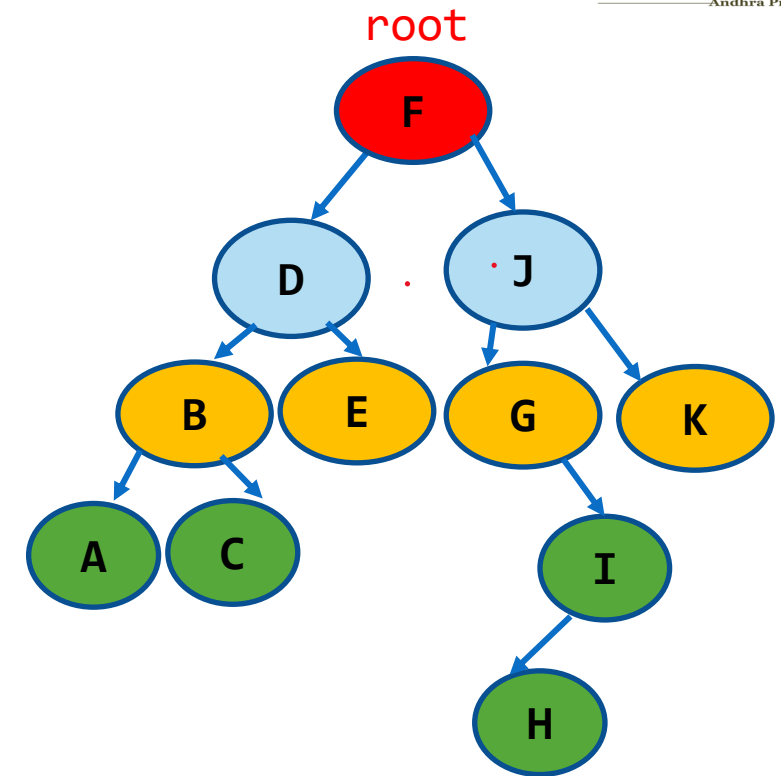
root-→ Print the value,
left-→ Traverse left sub tree,
right-→ Traverse right subtree

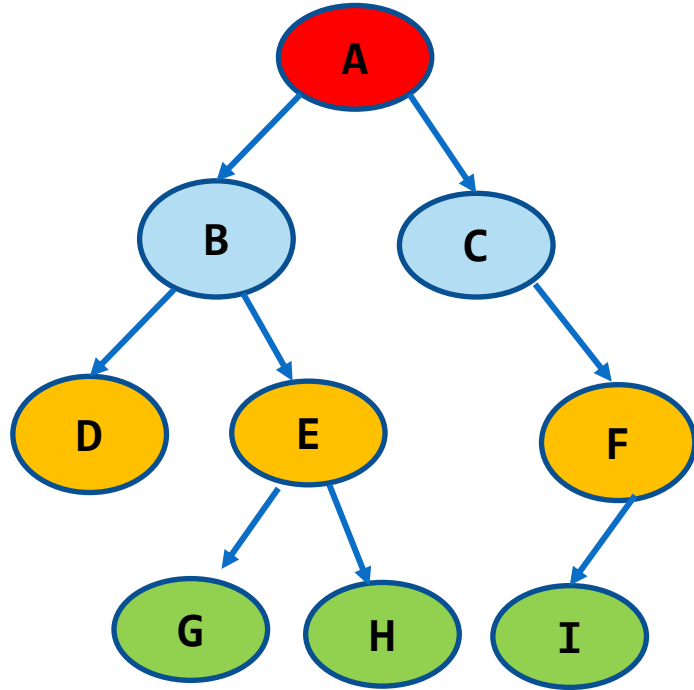
Binary Tree Traversal

```
void preorder(struct Node *root)
{
    if(root == NULL)
        return;
    printf("%d ",root->data); //visit the node
    preorder(root->left); //traverse the left subtree
    preorder(root->right); //traverse the right subtree
}

void inorder(struct Node *root)
{
    if(root == NULL)
        return;
    inorder(root->left); //traverse the left subtree
    printf("%d ",root->data); //visit the root
    inorder(root->right); //traverse the right subtree
}

void postorder(struct Node *root)
{
    if(root == NULL) return;
    postorder(root->left); //traverse the left subtree
    postorder(root->right); //traverse the right subtree
    printf("%d ",root->data); //visit the root
}
```





Preorder:

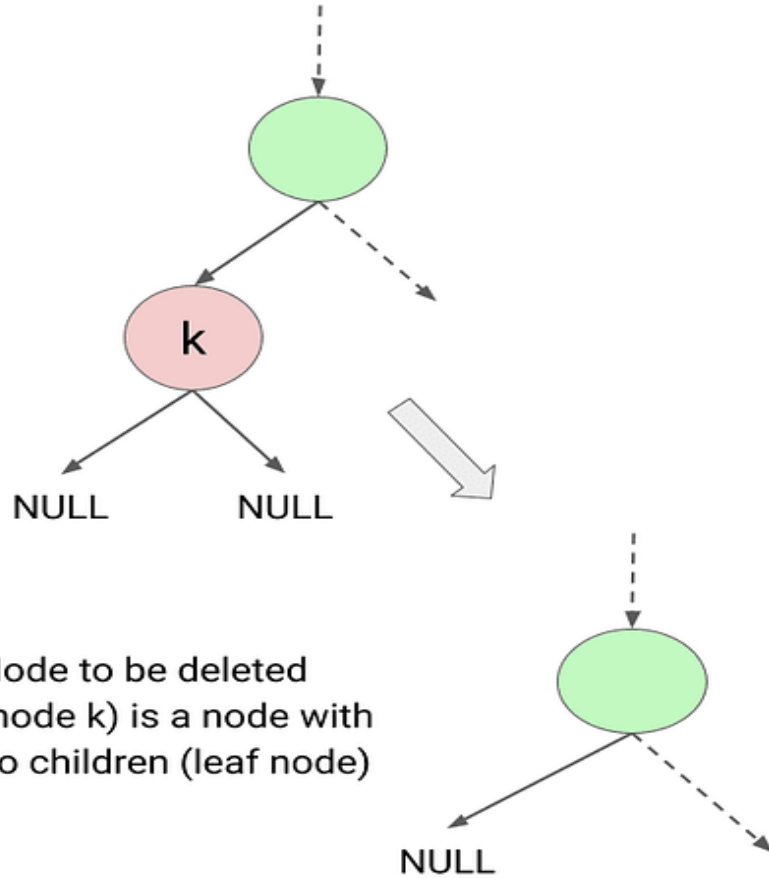
Inorder:

Postorder:

Deletion in Binary Search Tree

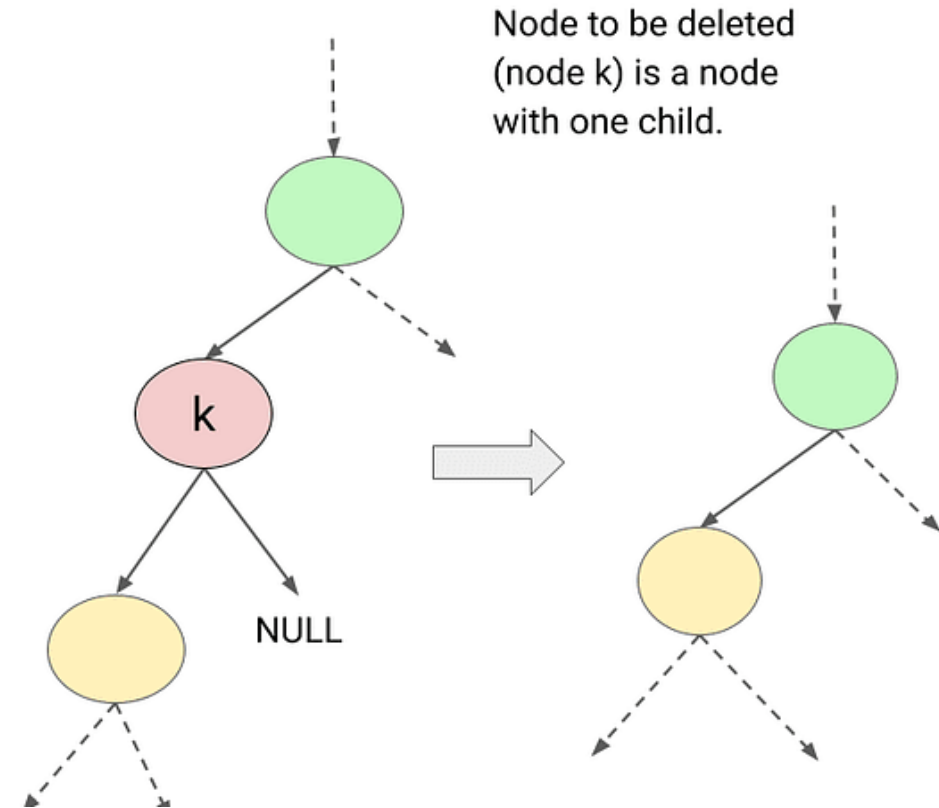
Three situations arise in deleting a node from binary search tree.

Case-I: The node to be deleted is a leaf node



Step: Replace the leaf node with the NULL and simple free the allocated space.

Case-II: The node to be deleted has only one child.



Step-1: Replace the node with its child and delete the child node (which now contains the value which is to be deleted).

Step-2: Simply replace it with the NULL and free the allocated space.

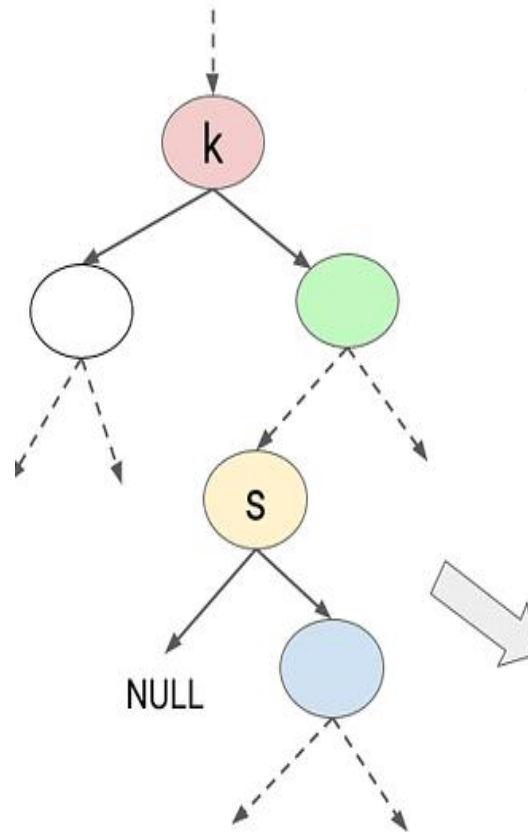
Deletion in Binary Search Tree

Case-III: The node to be deleted has two children.

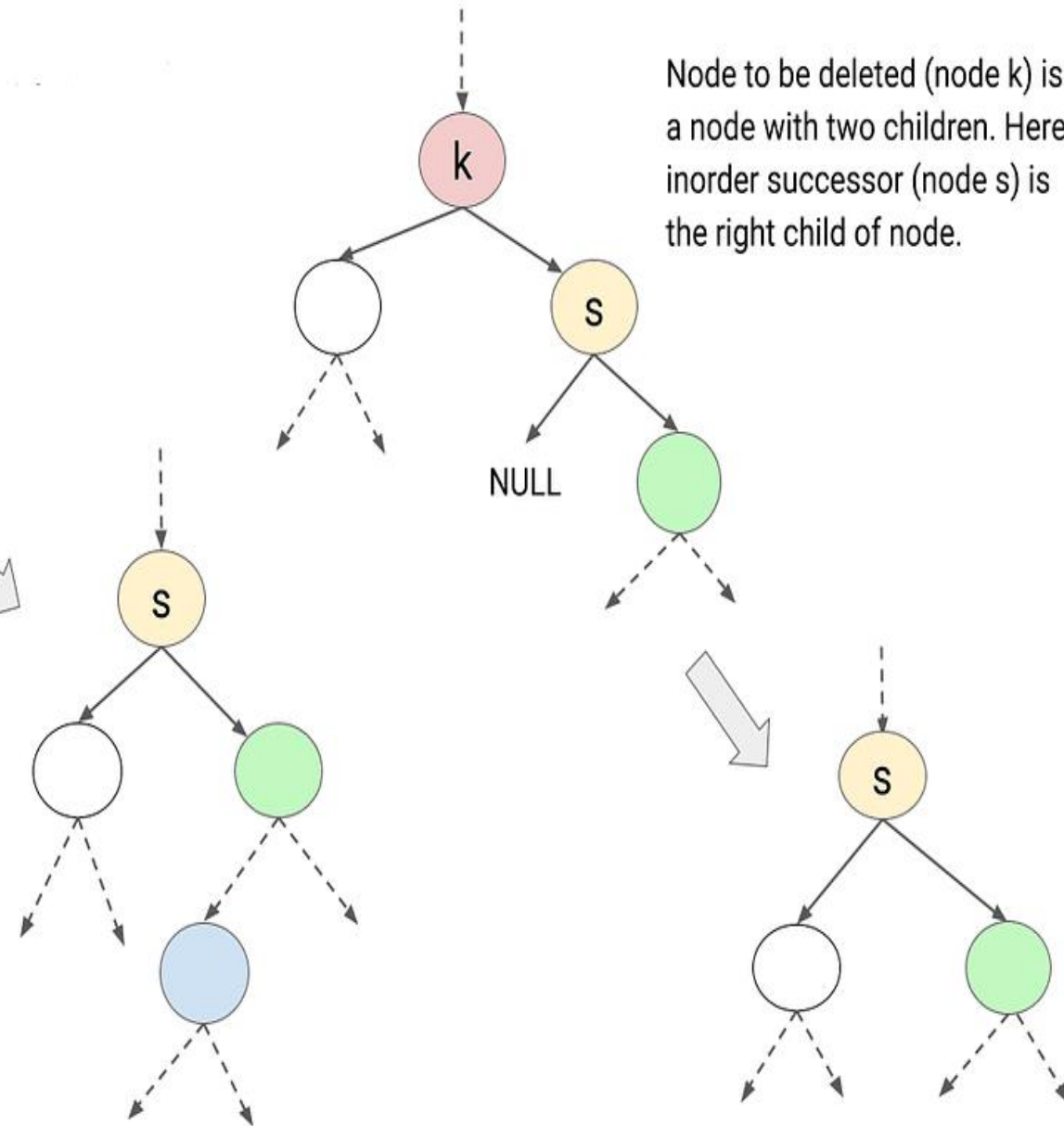
Step-1: Identify the in-order successor of the given node (i.e., a node just greater than the given node)

Step-2: Replace the given node with the in-order successor node.

Step-3: Delete in-order successor node



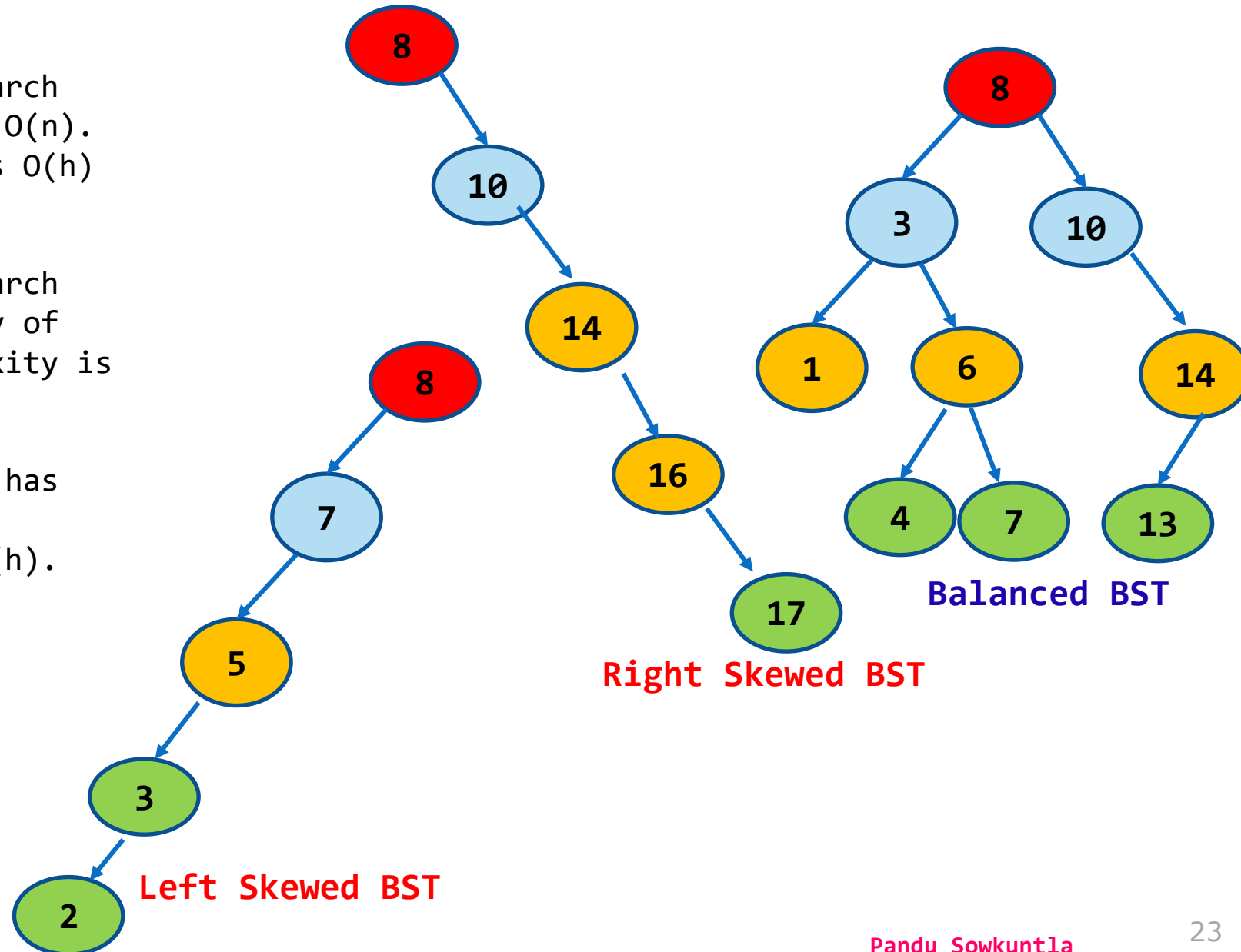
Node to be deleted (node k) is a node with two children. Here inorder successor (node s) is not the right child of node.



Node to be deleted (node k) is a node with two children. Here inorder successor (node s) is the right child of node.

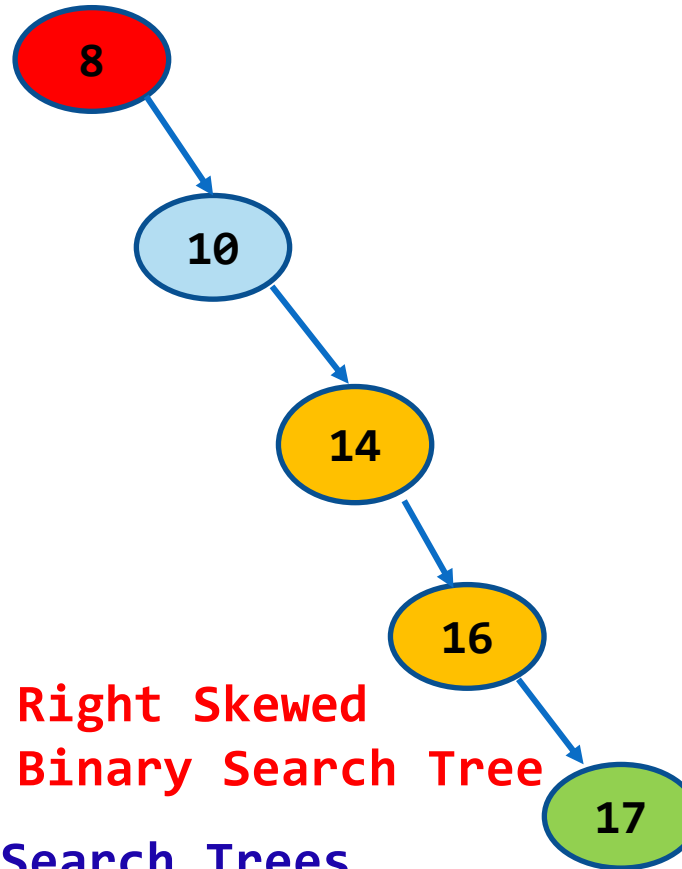
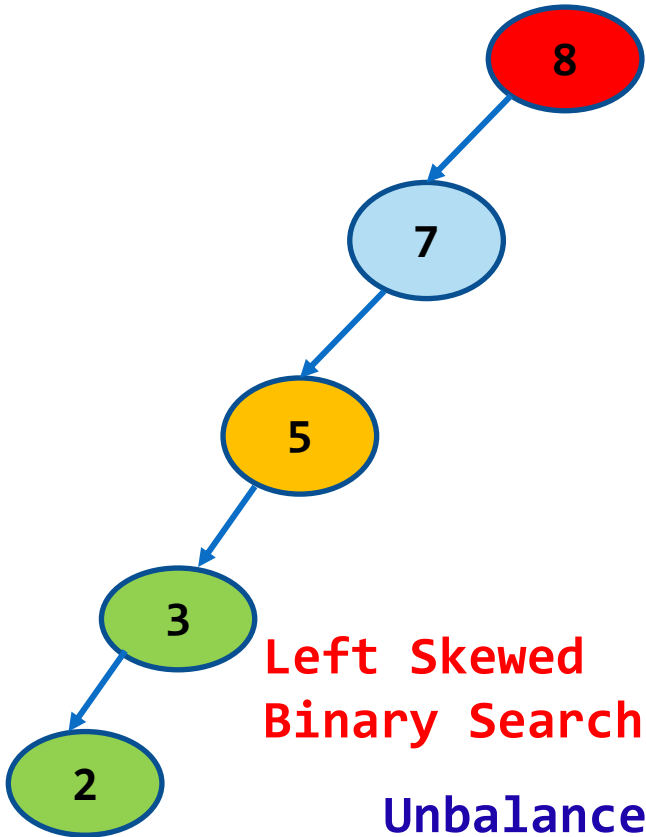
Complexity Analysis:

- ▶ **Searching:** Searching in binary search tree has worst case complexity of $O(n)$. In general, the time complexity is $O(h)$ where h is the height of BST.
- ▶ **Insertion:** Insertion in binary search tree has the worst-case complexity of $O(n)$. In general, the time complexity is $O(h)$.
- ▶ **Deletion:** Deletion in binary tree has worst case complexity of $O(n)$. In general, the time complexity is $O(h)$.

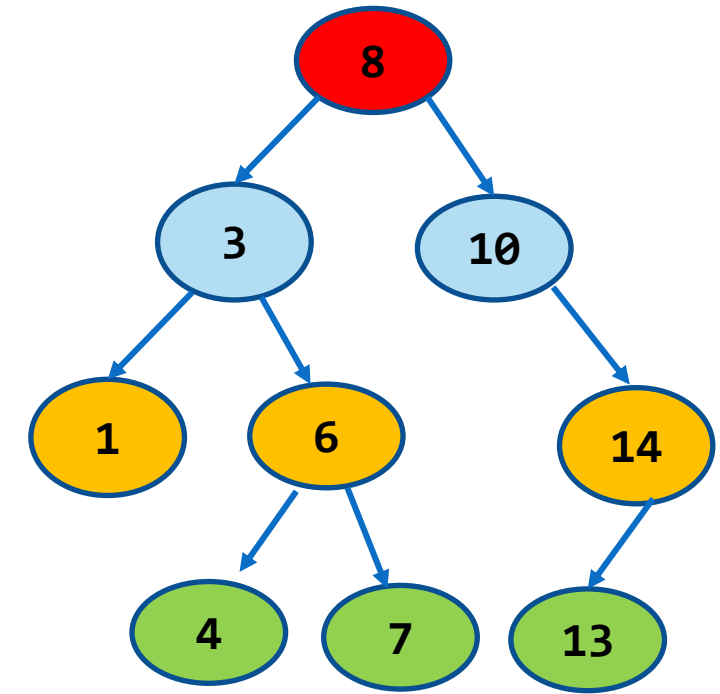


AVL Trees

Why AVL Trees ?



Unbalanced Binary Search Trees



Balanced Binary Search Tree

Insertion, Deletion or Search operations
take $O(n)$ time in unbalanced BSTs.

Cost can be reduced to $O(\log n)$ with
balanced BSTs.

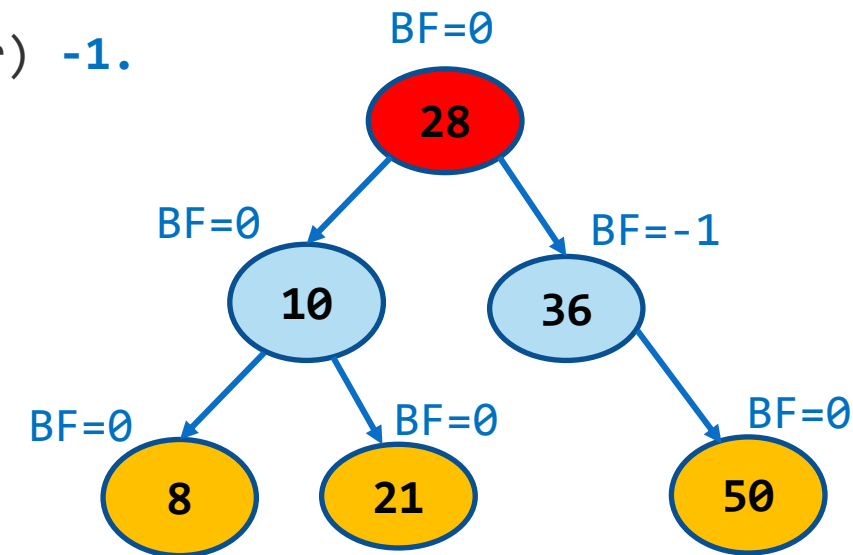
What is AVL Tree (or Height balanced binary search tree) ?

AVL tree is a balanced BST in which the height of left and right subtrees differ by **no more than one**.

- ▶ AVL tree is a BST that is either empty or consists of two AVL subtrees, Left subtree T_L and right subtree T_R whose heights differ by ≤ 1 .
- ▶ The height difference is called **Balance Factor**.
- ▶ Balance factor for any node in AVL tree must be **+1, 0, (or) -1**.

$$H_L - H_R \leq 1$$

AVL tree ensures that Insertion, Deletion and Search operations take **$O(\log n)$** .



AVL Tree

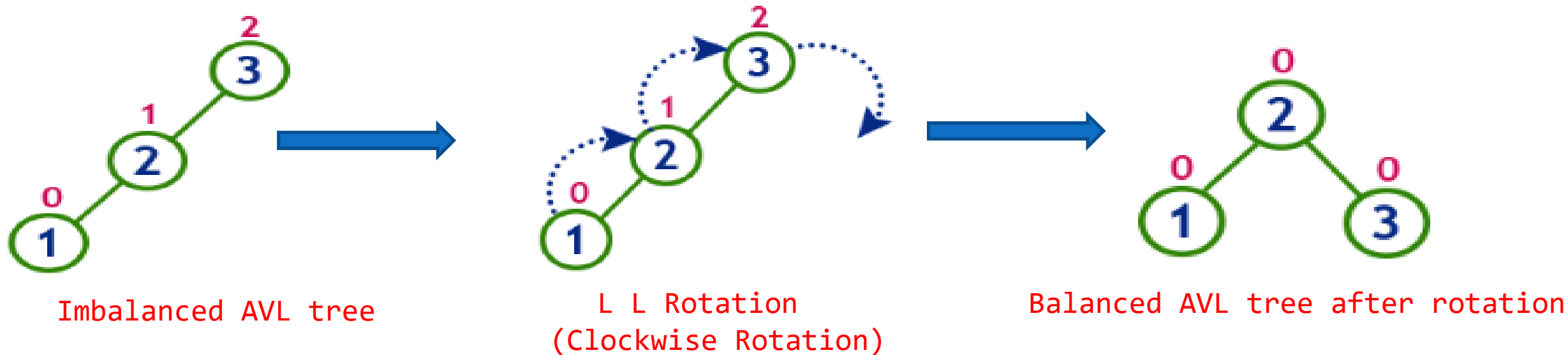
- ▶ In AVL tree, after performing operations like insertion or deletion, the **balance factor** of every node in the tree should be maintained.
- ▶ If tree becomes imbalanced due to any operation, the **rotation** operations are performed to make it balanced.

- | | |
|---|-------------------|
| 1) L L rotation : Inserted node is in the left subtree of left subtree of A | } Single rotation |
| 2) R R rotation : Inserted node is in the right subtree of right subtree of A | |
| 3) L R rotation : Inserted node is in the right subtree of left subtree of A | } Double rotation |
| 4) R L rotation : Inserted node is in the left subtree of right subtree of A | |

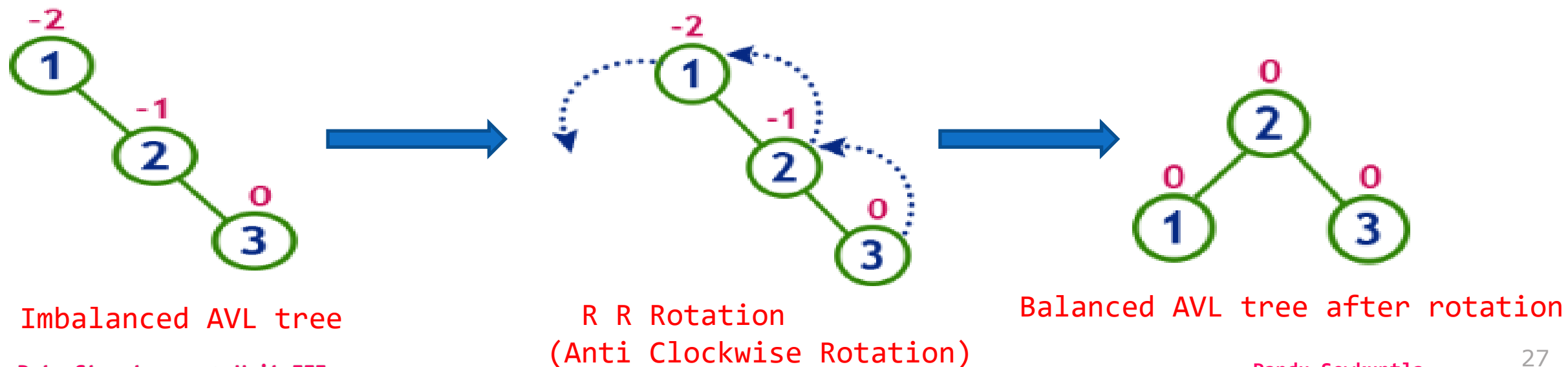
Here node **A** is the **critical node (closest ancestor)** whose balance Factor is other than **-1, 0, 1**

AVL Tree Rotations

1. L L Rotation (Single Left Rotation)

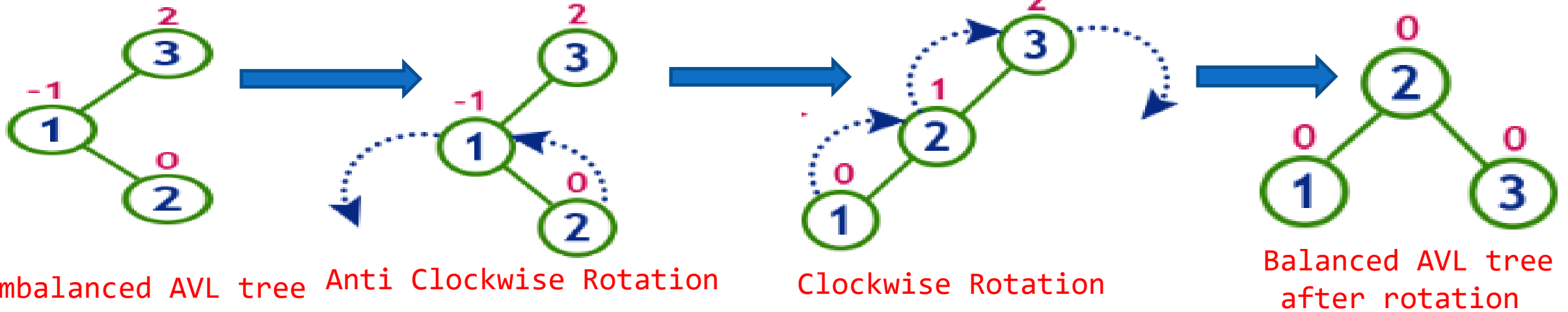


2. R R Rotation (Single Right Rotation)

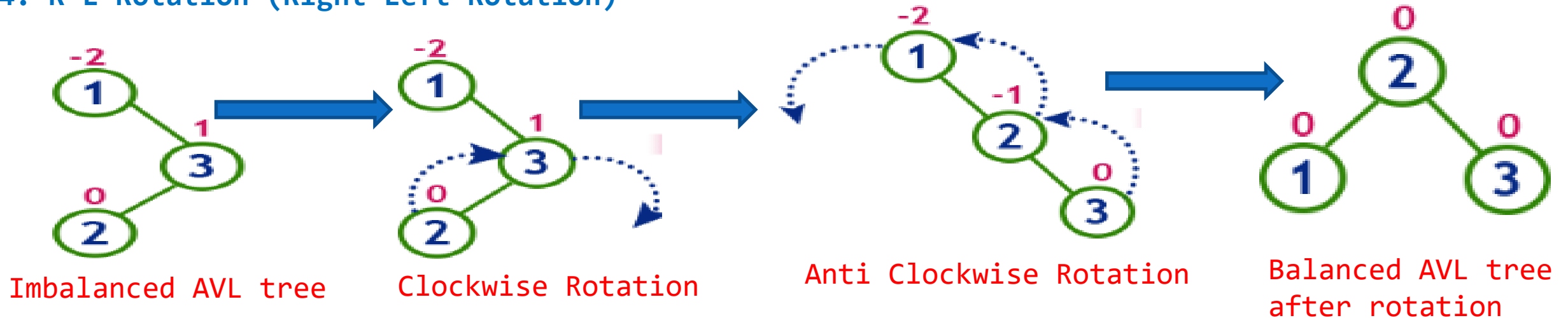


AVL Tree Rotations

3. L R Rotation (Left Right Rotation)

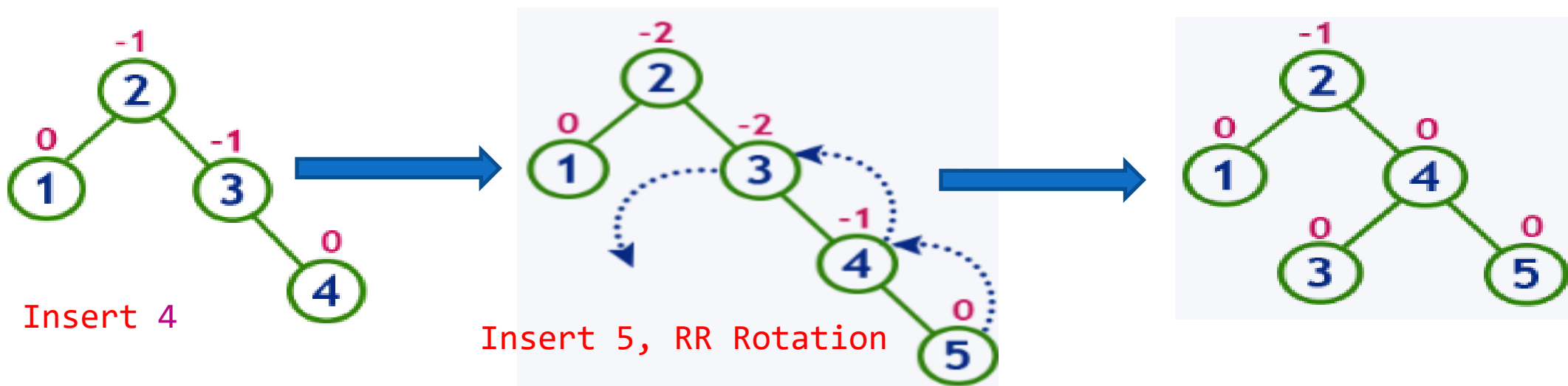
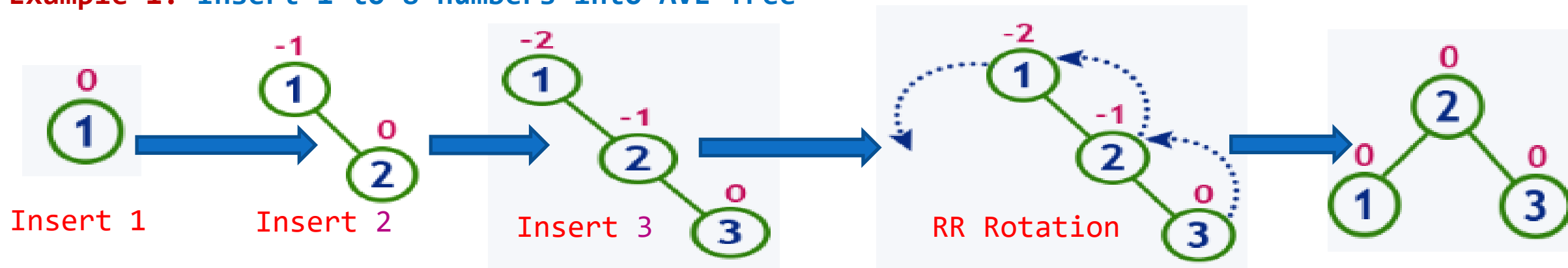


4. R L Rotation (Right Left Rotation)

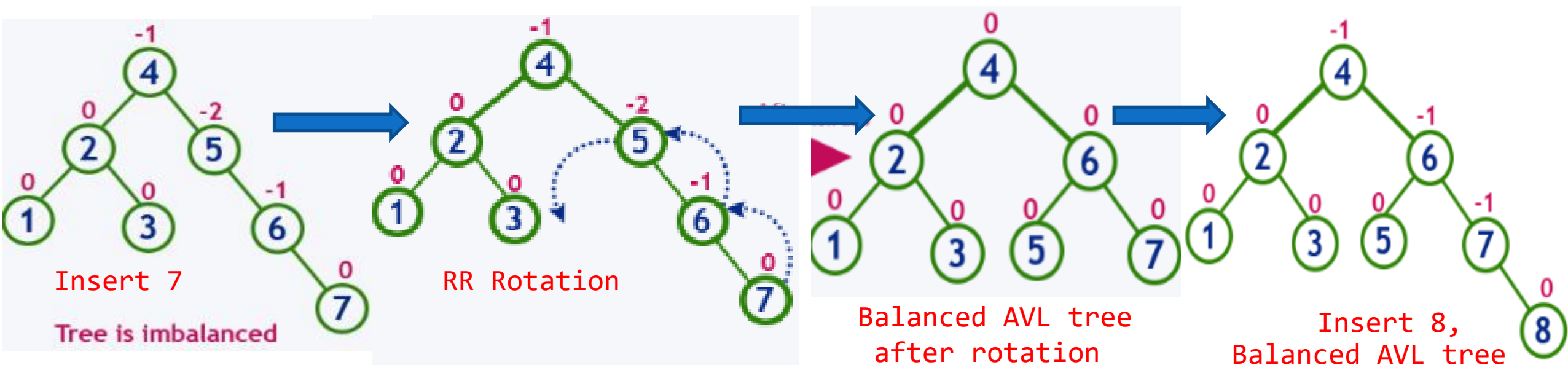
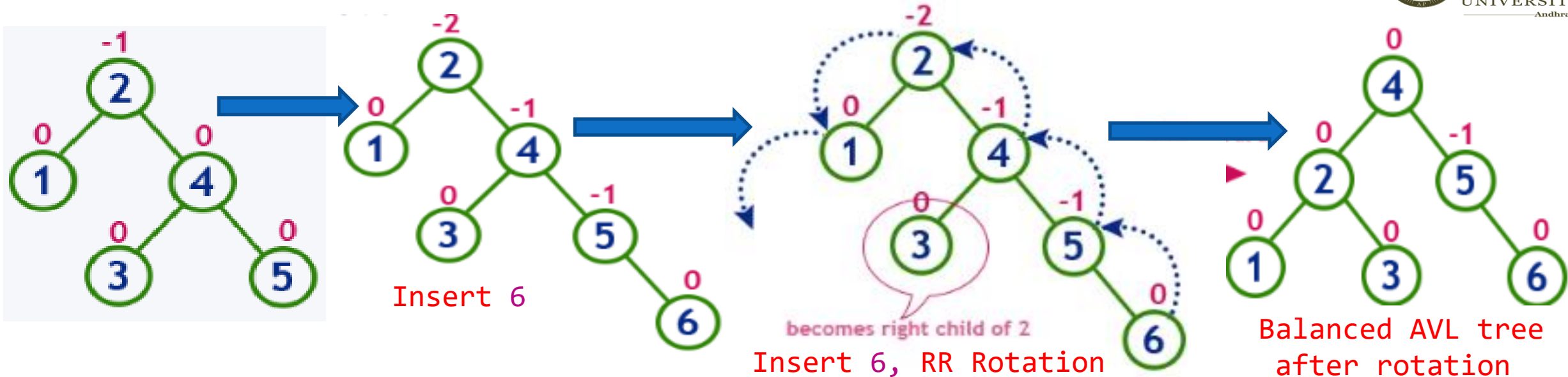


Insertion on AVL Tree

Example-1: Insert 1 to 8 numbers into AVL Tree



Insertion on AVL Tree



Example-2: Insert the following elements into AVL Tree: 50, 20, 60, 10, 8, 15, 32, 46, 11, 48.

Deletion on AVL Tree

Steps for deleting a node in an AVL tree.

Step-1: Search node in the tree by the BST logic. If the element is not present, then return.

- **Case-I:** The node to be deleted is a leaf node
 - Replace the leaf node with the NULL and simple free the allocated space.
- **Case-II:** The node to be deleted has only one child.
 - Replace the node with its child and delete the child node (which now contains the value which is to be deleted).
- **Case-III:** The node to be deleted has two children.
 - Replace the given node with the in-order successor node.

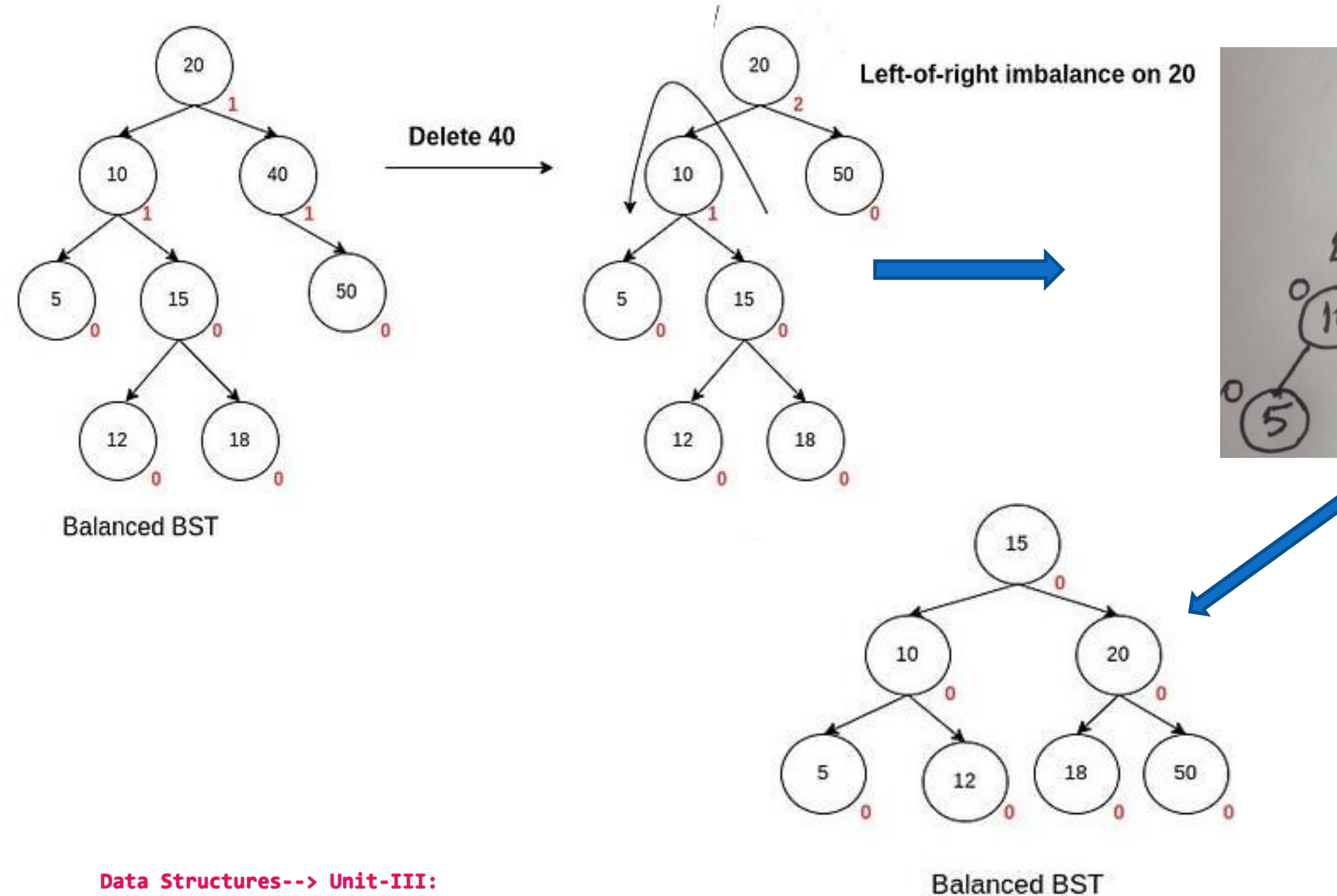
Step-2: Delete the node.

Step-3: While retracing, calculate and check the balance factor of each ancestor node.

Step-4: Remove the imbalance using rotations

Step-5: Repeat the step 3 till the root of the tree

Deletion on AVL Tree



Complexity Analysis:

	AVL Tree		Binary Search Tree	
	Average	Worst	Average	Worst
Insertion	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(n)$
Deletion	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(n)$
Searching	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(n)$
Traversal	$O(n)$	$O(n)$	$O(n)$	$O(n)$

Expression Trees

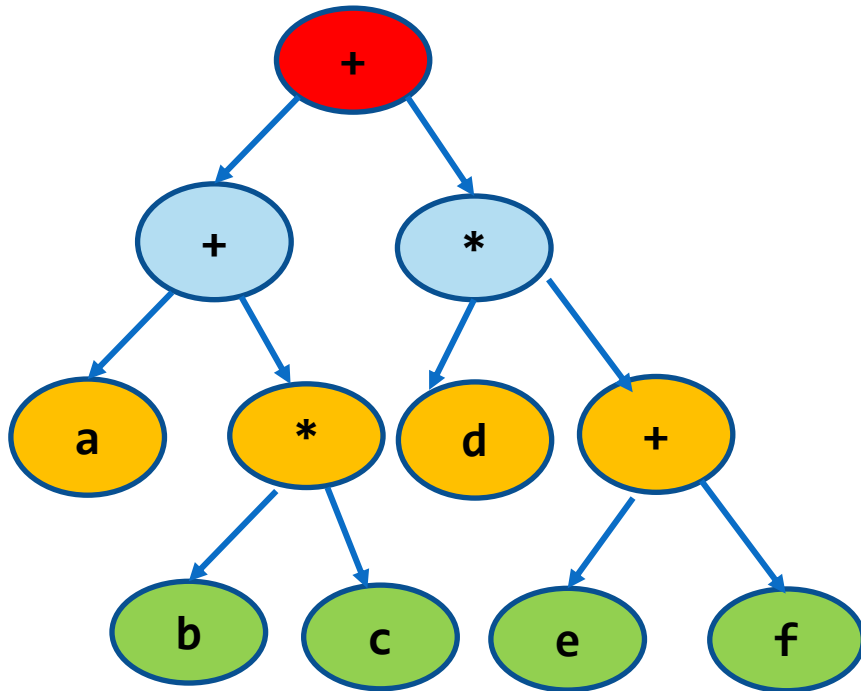
It is a binary tree that represents an expression with the following properties:

- Each leaf is an operand. Ex: **a, b, c, 5, 10**
- The root and internal nodes are operators. Ex: **+, -, *, /, ^**
- Subtrees are subexpressions with the root being an operator.

Example expression:

a + (b * c) + d * (e + f)

Equivalent Expression Tree:



An **infix expression** can be produced by performing **in-order traversal** on expression tree

(a+(b*c))+(d*(e + f))

A **prefix expression** can be produced by performing **pre-order traversal** on expression tree

+ + a * b c * d + e f

A **postfix expression** can be produced by performing **post-order traversal** on expression tree

a b c * + d e f + * +

Constructing Expression tree from postfix expression

Step-1: Read one symbol at a time from the postfix expression.

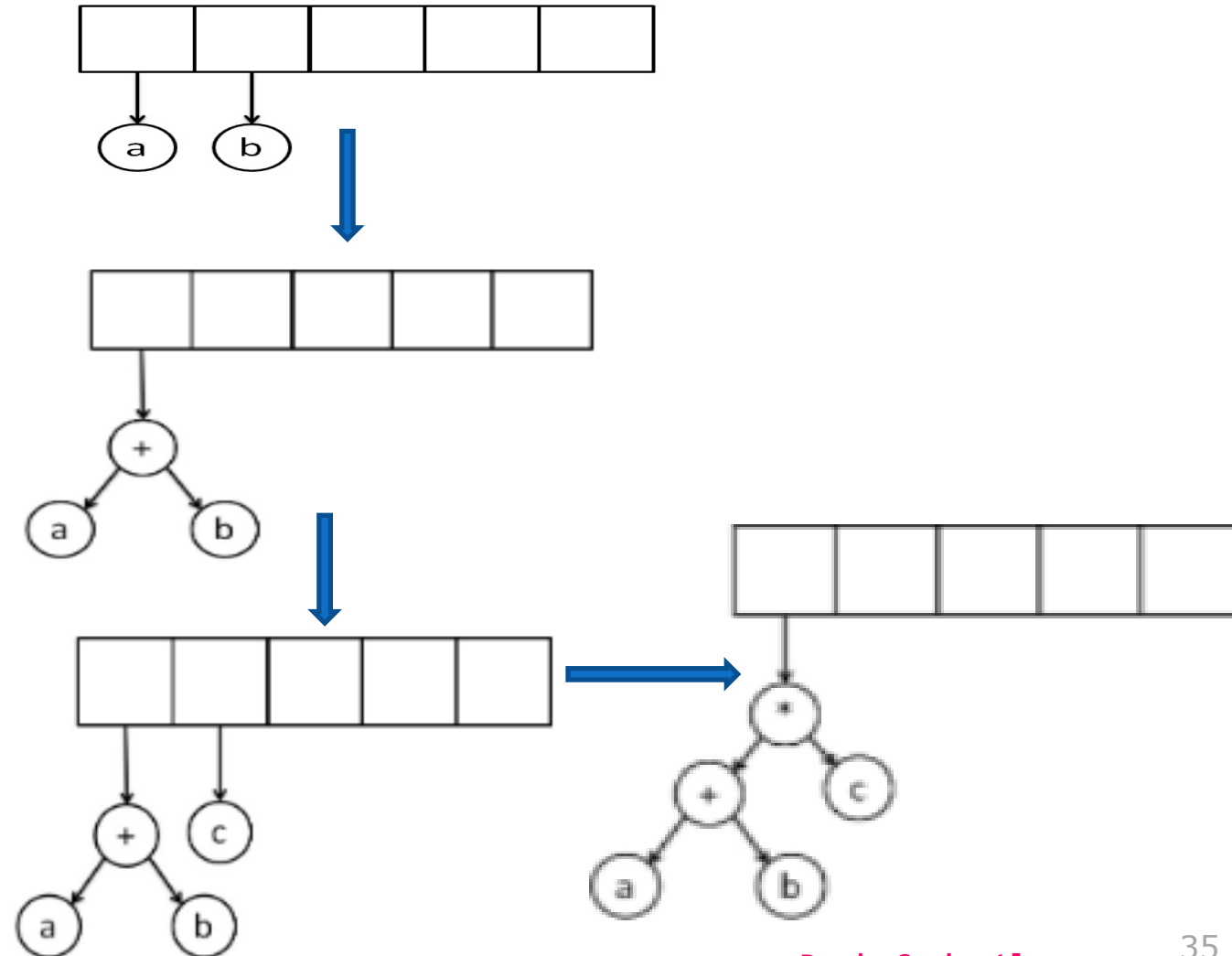
Step-2: Check if the symbol is an operand or operator.

Step-3: If the symbol is an operand, create a node tree and push it onto a stack

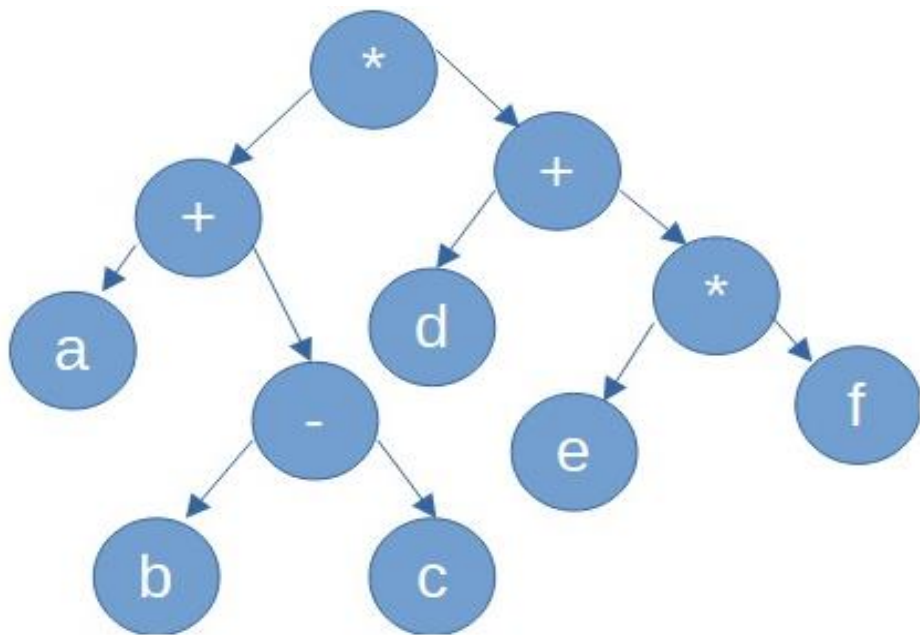
Step-4: If the symbol is an operator, pop two pointers from the stack namely T_1 & T_2 and form a new tree with root as the operator, T_1 & T_2 as a left and right child

Step-5: A pointer to this new tree is pushed onto the stack

Example: $a \ b \ + \ c \ *$



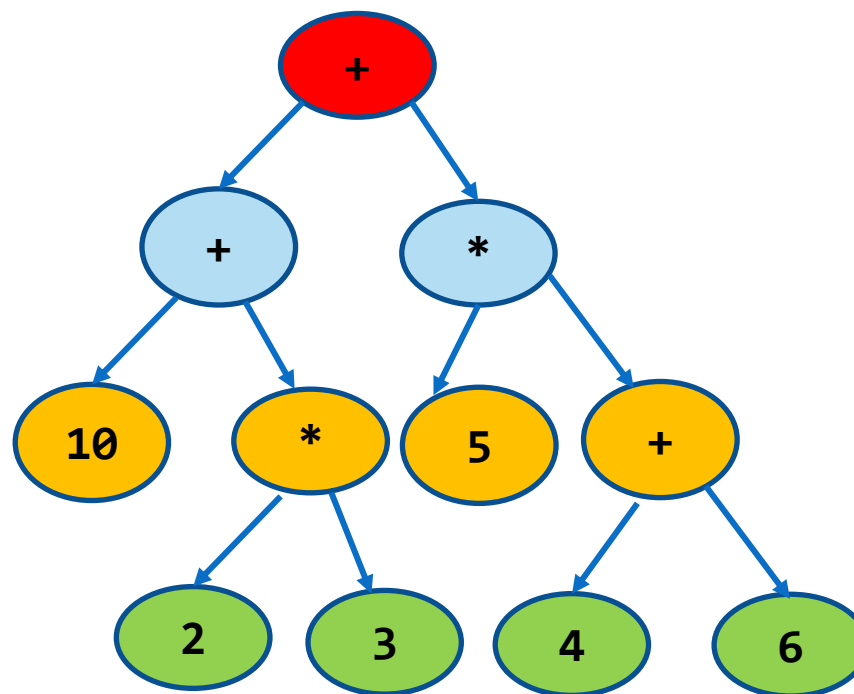
Expression Trees



Infix expression:

Prefix expression:

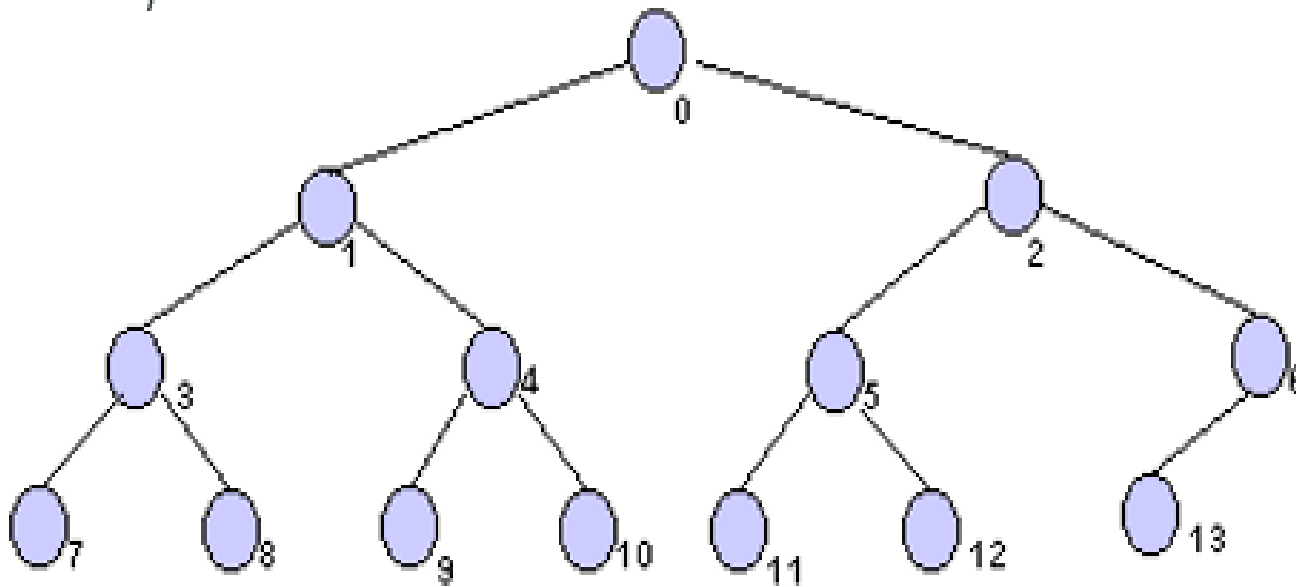
Postfix expression:



Postfix expression evaluation:

Array Representation of Complete Binary Tree

Tree Structure: **Array** indices carry tree structure



Parent of $a[k] = a[(k-1)/2]$

Left child of $a[k] = a[2*k+1]$

Right child of $a[k] = a[2*k+2]$

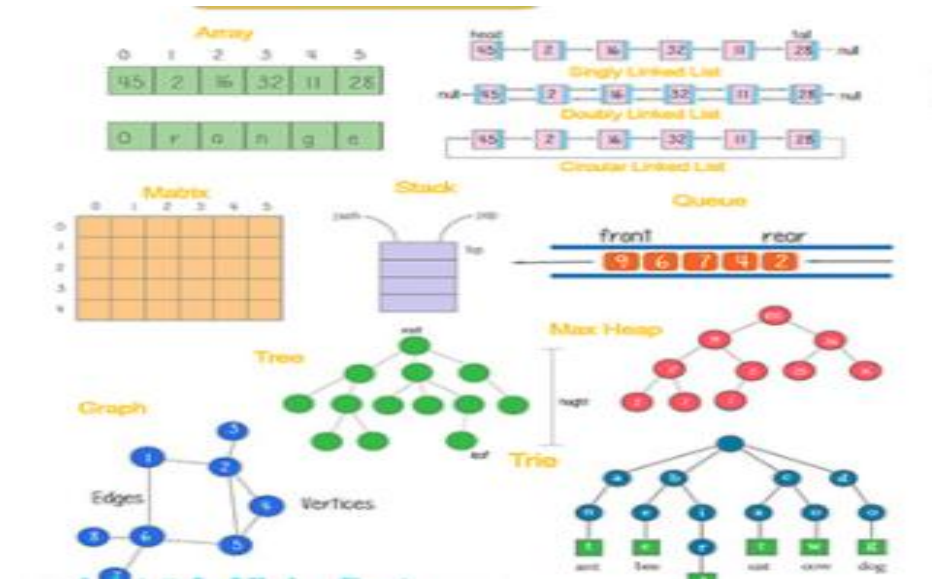
Array **index order** = level order (beginning with 0)

If parent node is at index k in the array then

left child of that node is at index $(2*k + 1)$

right child is at index $(2*k+ 2)$ in the array.

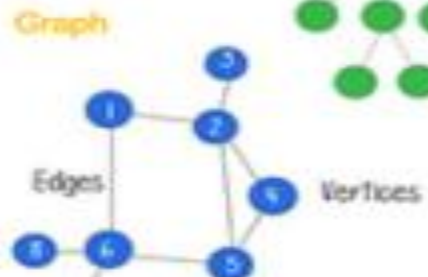
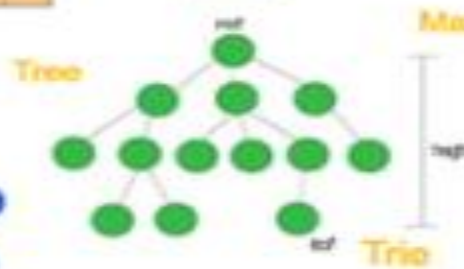
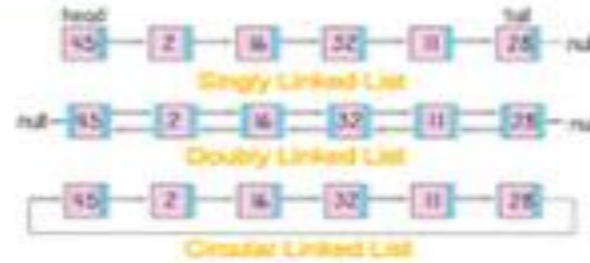
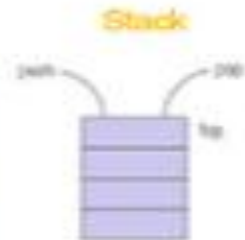
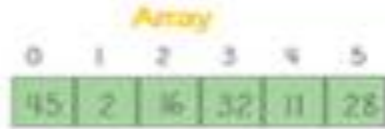
Data Structures



Dr. Pandu Sowkuntla
Asst. Professor,
Dept. of CSE, SRM University AP



Data structures



UNIT-IV



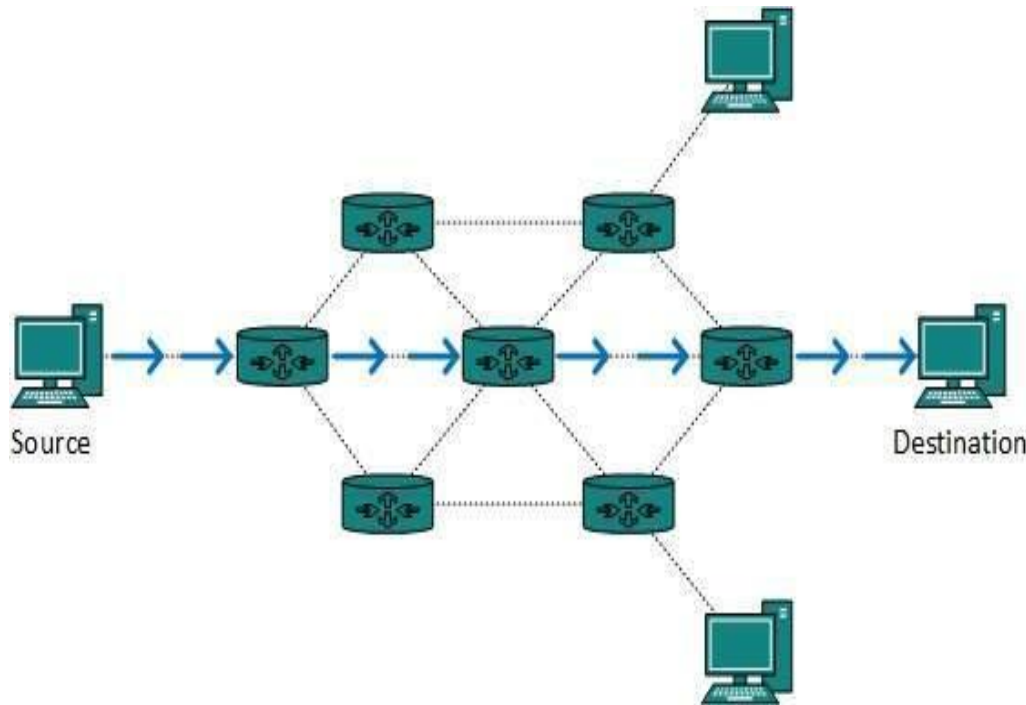
GRAPHS

Why Graphs ?

Graphs are widely used to model any situation where entities or things are related to each other in pairs or in network.

■ Communication networks :

- Routers in wired and wireless networks.



■ Social networks:

- Graphs are also used in social network like LinkedIn, Facebook, twitter etc.



■ Electrical circuits:

- The networks may include paths in a telephone network or circuit network.

■ Task schedules:

- Resource Allocation Graph where each process and resources are considered to be vertices in operating system

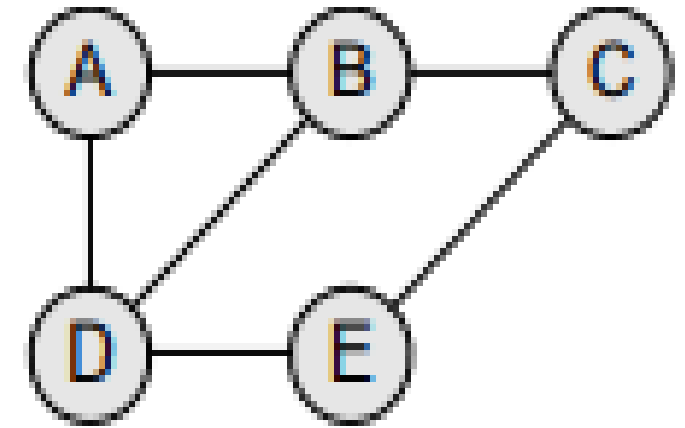
What is Graph ?

A **graph G** is defined as an ordered set (V, E) , here $V(G)$ represents the set of vertices and $E(G)$ represents the edges that connect these vertices.

$V(G) = \{A, B, C, D, E\}$

$E(G) = \{(A, B), (B, C), (A, D), (B, D), (D, E), (C, E)\}.$

Note: There are **five vertices** or **nodes** and **six edges** in the graph.



Undirected graph

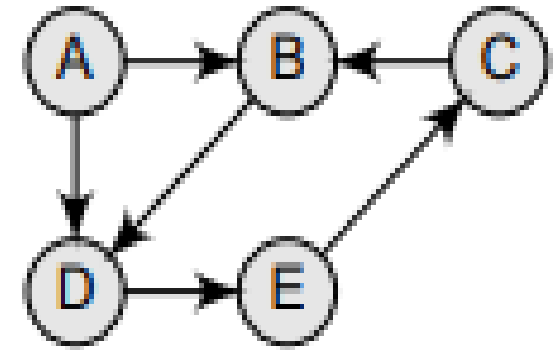
Graph terminology

Undirected graph

- ▶ Edges do not have any direction associated with them.
- ▶ If an edge is drawn between nodes A and B, then the nodes can be traversed from A to B as well as from B to A.

Directed graph (Digraph)

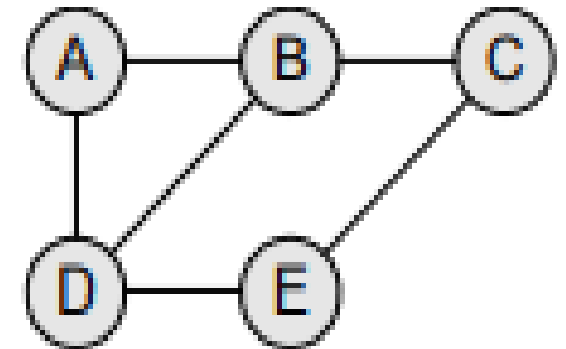
- ▶ Edges form an ordered pair.
- ▶ If there is an edge from A to B, then there is a path from A to B but not from B to A.
- ▶ The edge (A, B) is said to initiate from node A (also known as **initial node**) and terminate at node B (**terminal node**).



Directed graph

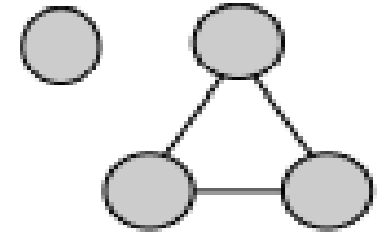
Adjacent nodes or neighbors

For every edge, $e = (u, v)$ that connects nodes u and v , the nodes u and v are said to be the adjacent nodes or neighbors.



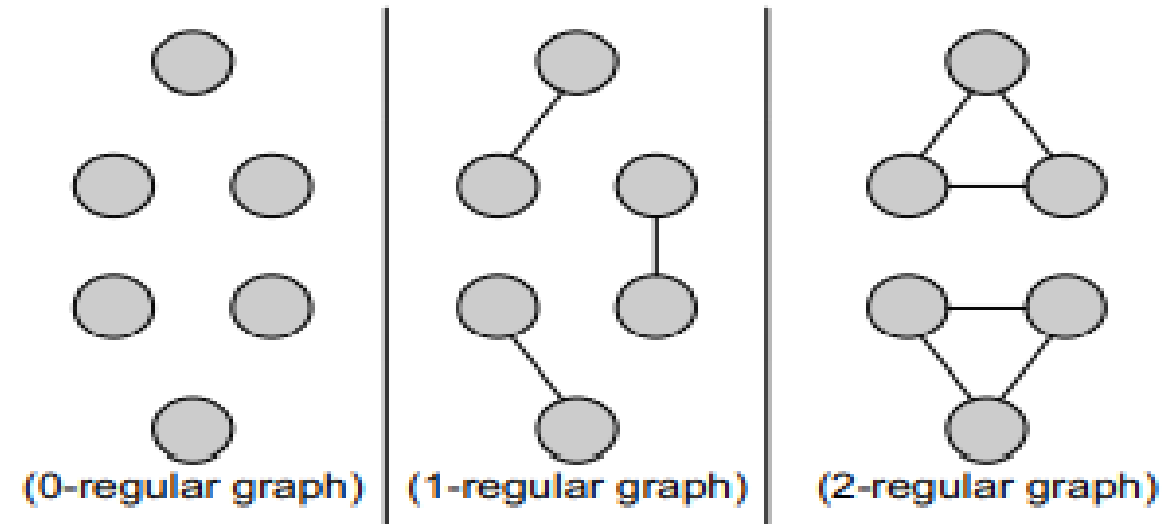
Degree of a node

- ▶ Degree of a node u , $\deg(u)$, is the total number of edges containing the node u .
- ▶ If $\deg(u) = 0$, it means that u does not belong to any edge and is known as an **isolated node**.



Regular graph

- ▶ It is a graph where **each vertex has the same number of neighbors**.
- ▶ Every node has the same degree.
- ▶ A regular graph with vertices of degree k is called a k -regular graph or a regular graph of degree k .



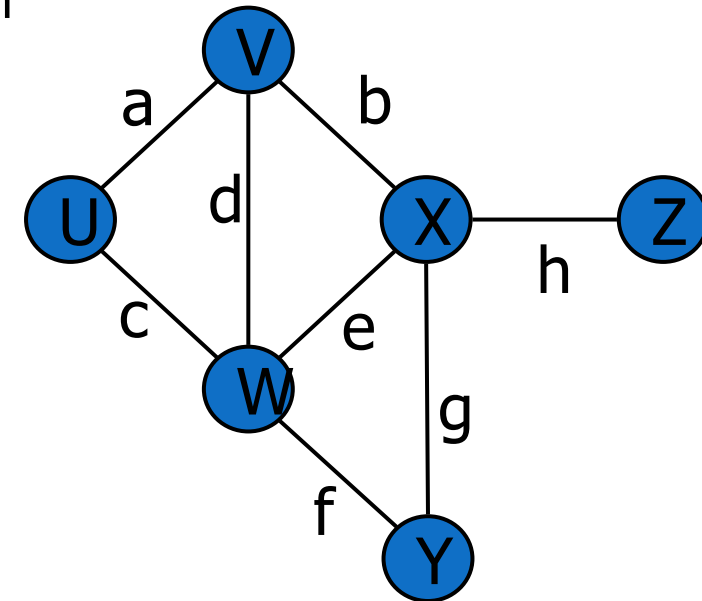
Path

A path P written as $P = (v_0, v_1, v_2, \dots, v_n)$, of length n from a node u to v is defined as a sequence of $(n+1)$ nodes.

Here, $u = v_0$, $v = v_n$

v_{i-1} is adjacent to v_i for $i = 1, 2, 3, \dots, n$.

Ex: $P_1 = (U, W, X, Y, W, V)$ is a path



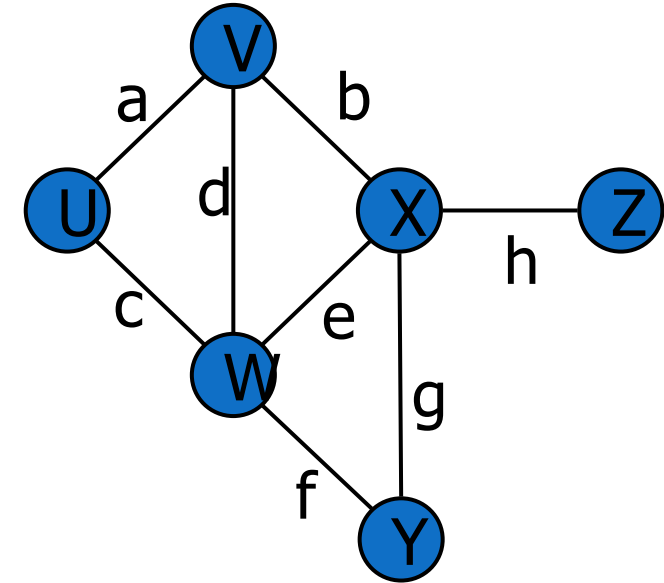
Closed path

A path P is known as a closed path if the edge has the same end-points (if $v_0 = v_n$.)

Ex: $P_1 = (U, W, X, Y, W, V, U)$ is a closed path

Simple path

- ▶ A path P is known as a simple path if all the nodes in the path are distinct with an exception that v_θ may be equal to v_n .
- ▶ Ex: $P_1=(V, X, Z)$ is a simple path, $P_2=(U, W, X, Y, W, V)$ is not a simple path
- ▶ If $v_\theta = v_n$, then the path is called a closed simple path,
Ex: $P_3=(W, X, Y, W)$

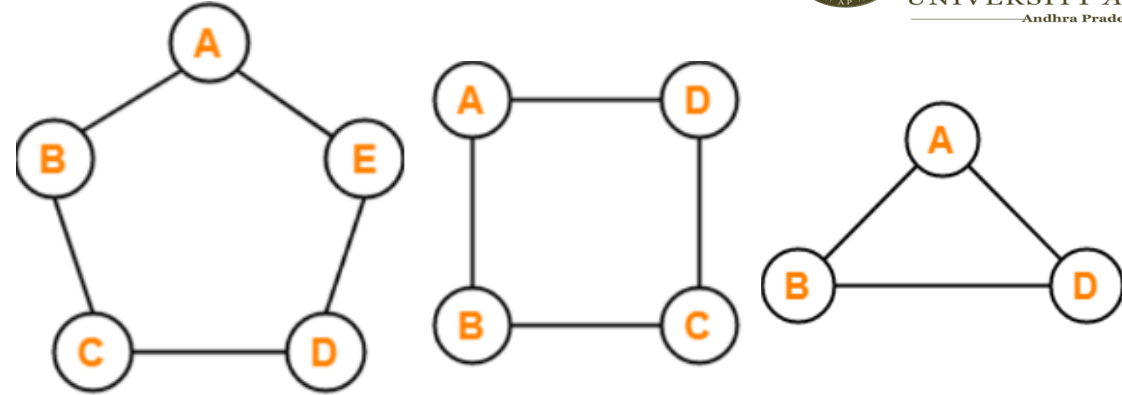


Cycle

- ▶ A path in which the first and the last vertices are same.
- ▶ A **simple cycle** has no repeated edges or vertices (except the first and last vertices).

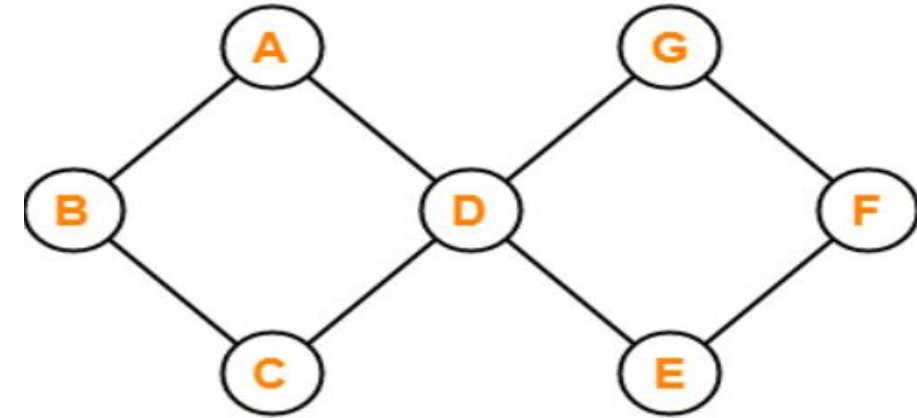
Cycle Graph

- ▶ If a graph with many vertices greater than three and edges form a cycle, then the graph is called a **cycle graph**.



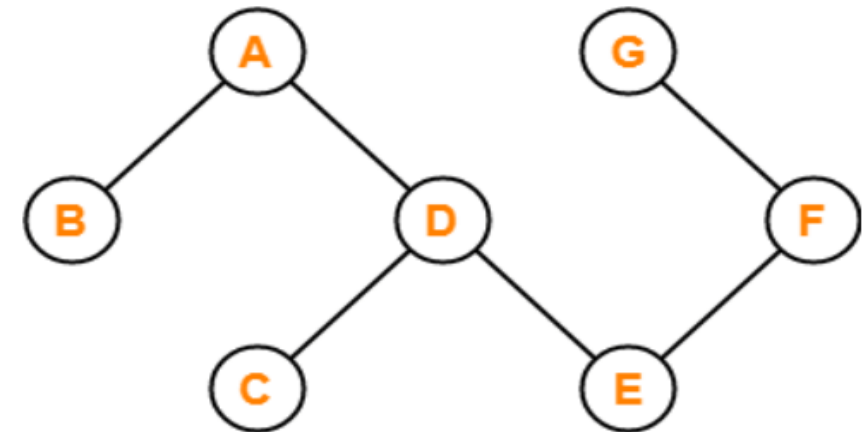
Cyclic Graph

- ▶ For a graph to be called a cyclic graph, it should consist of at least one cycle.
- ▶ If a graph has a minimum of one cycle present, it is called a cyclic graph.



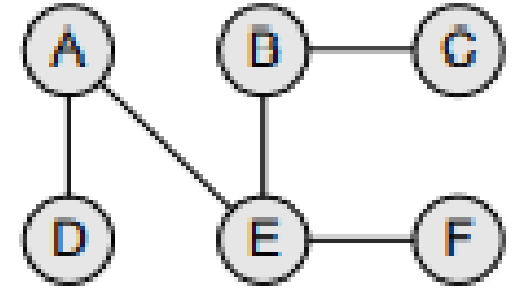
Acyclic Graph

- ▶ A graph is called an acyclic graph if zero cycles are present, and an acyclic graph is the complete opposite of a cyclic graph.



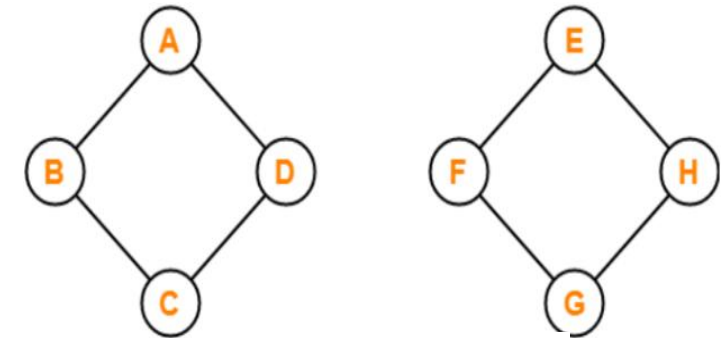
► Connected graph:

- For any two vertices (u, v) in V there is a path from u to v
- There are no isolated nodes in a connected graph.
- A connected graph that **does not have any cycle** is called a **tree**.



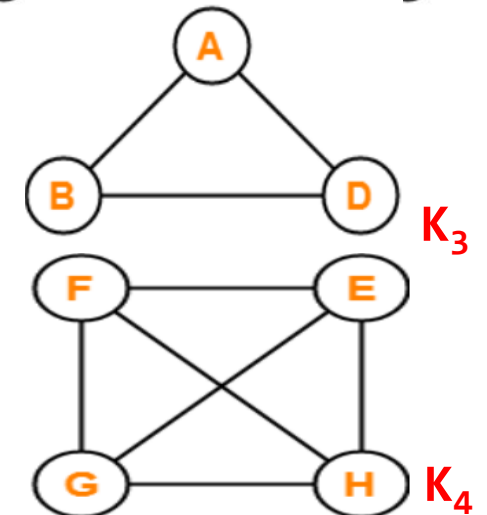
► Disconnected Graph

- A graph is said to be a disconnected graph where there does not exist any path between at least one pair of vertices.



► Complete or Fully connected or Strongly connected graph (K_n):

- G is said to be complete if all its nodes are fully connected.
- Fully connected: Each vertex directly connected every other vertex in the graph.
- A complete graph has $n(n-1)/2$ edges, where n is the number of nodes in G .



Graph terminology

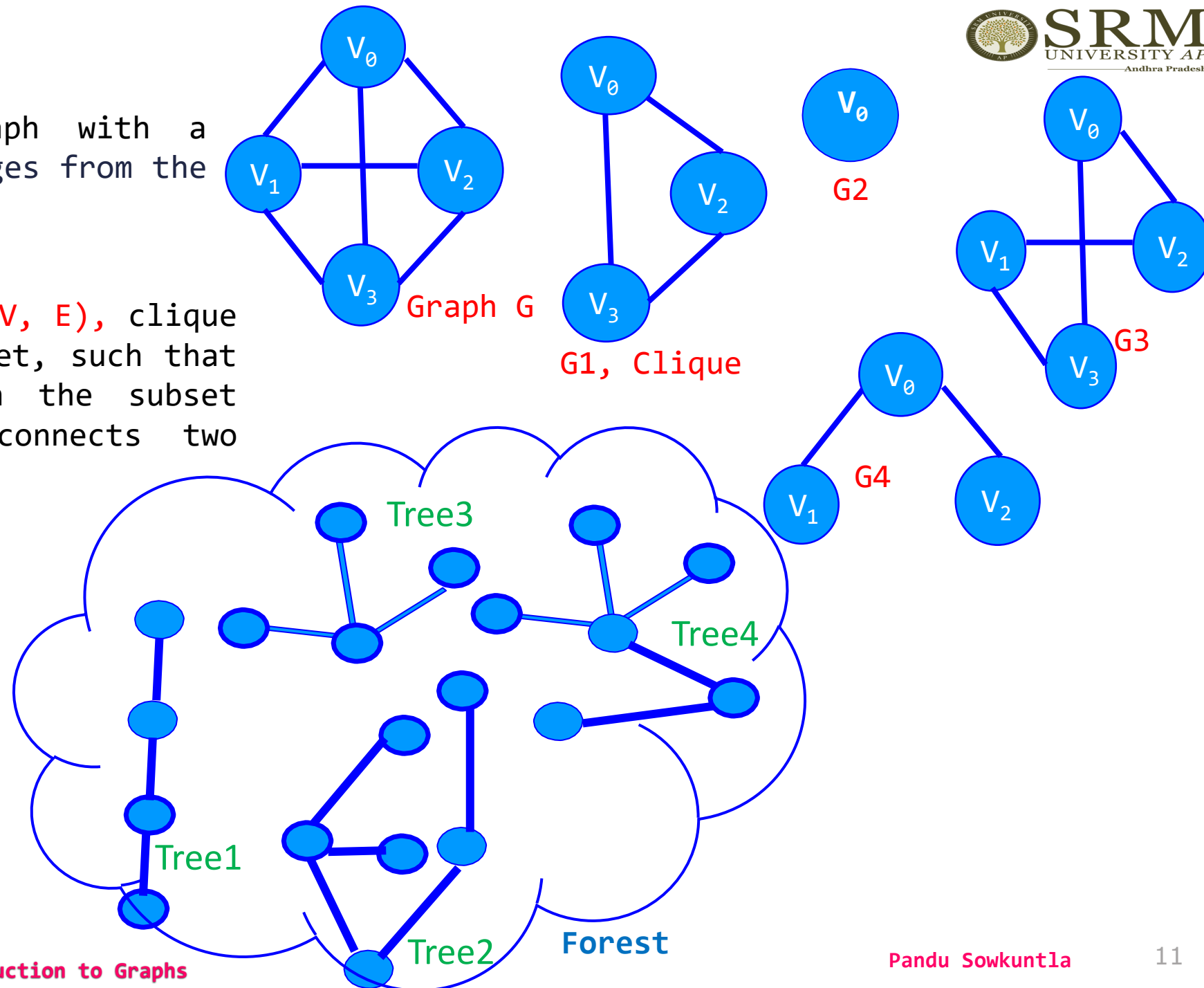
- ▶ **Subgraph:** Forming a graph with a subset of vertices and edges from the main graph.

- ▶ **Clique:**

In an undirected graph $G = (V, E)$, clique is a subset of the vertex set, such that for every two vertices in the subset there is an edge that connects two vertices.

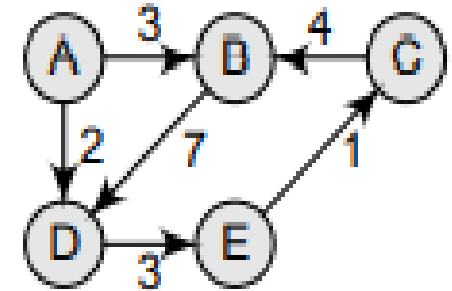
- ▶ **Trees and Forest**

- **Tree** - connected graph without cycles
- **Forest** - collection of trees.



► Labelled graph or weighted graph:

- A graph is said to be labelled if every edge in the graph is assigned some data.
- The edges of the graph are assigned some **weight or length**.



► Multiple edges or Parallel edges:

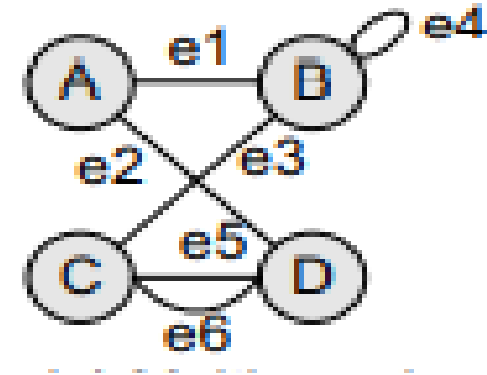
- Distinct edges which connect the same end-points are called multiple edges.
- $e = (u, v)$ and $e' = (u, v)$ are known as multiple edges of G .

► Loop:

- An edge that has identical end-points is called a loop ($e = (u, u)$).

► Multi-graph:

- A graph with multiple edges and/or loops is called a multi-graph.



► Size of a graph:

- The size of a graph is the total number of edges in it.

Terminology of a Directed Graph

► Out-degree of a node:

- The out-degree of a node u , written as $\text{outdeg}(u)$, is the number of edges that originate at u .

► In-degree of a node:

- The in-degree of a node u , written as $\text{indeg}(u)$, is the number of edges that terminate at u .

► Degree of a node:

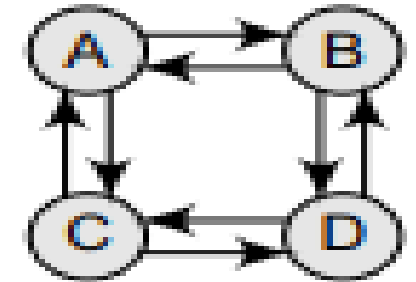
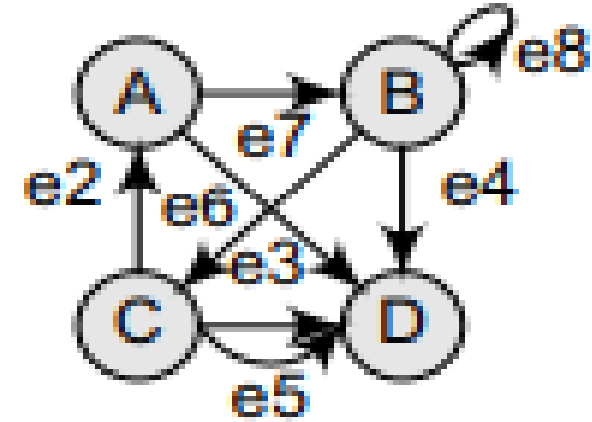
- The degree of a node, written as $\text{deg}(u)$, is equal to the sum of in-degree and out-degree of that node.
- $\text{deg}(u) = \text{indeg}(u) + \text{outdeg}(u)$.

► Null Graph

- The Null Graph is also known as the order zero graph. The term "*null graph*" refers to a graph with an empty edge set.

► Trivial Graph

- A graph is called a trivial graph if it has only one vertex present in it.

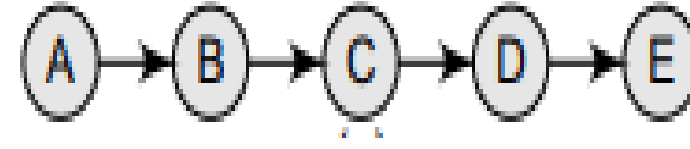


Null Graph



Trivial Graph

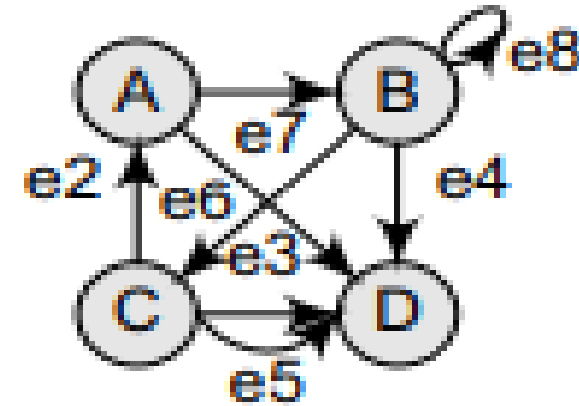
Terminology of a Directed Graph



- ▶ **Pendant vertex** (also known as leaf vertex): A vertex with degree one.
- ▶ **Cut vertex**: A vertex which when deleted would disconnect the remaining graph.

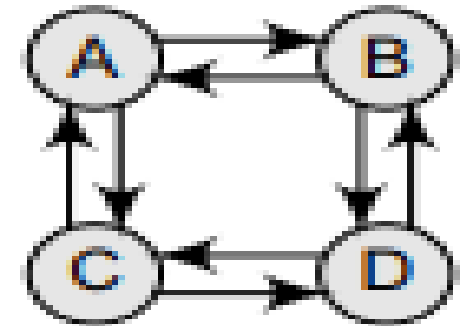
▶ **Reachability:**

- A node **v** is said to be reachable from node **u**, if and only if there exists a (directed) path from node **u** to node **v**.



▶ **Strongly connected directed graph:**

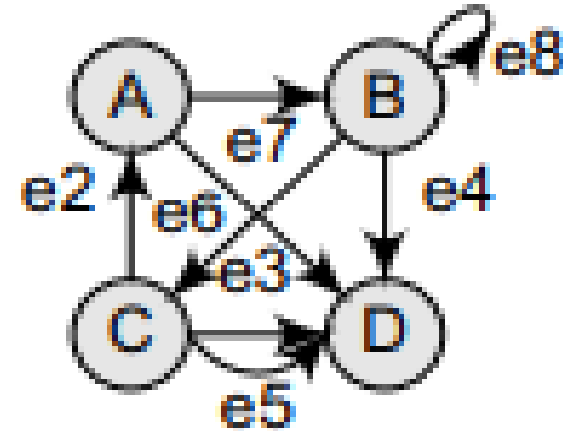
- A digraph is said to be strongly connected if and only if there exists a path between every pair of nodes in G.
- If there is a path from node **u** to **v**, then there must be a path from node **v** to **u**.



Terminology of a Directed Graph

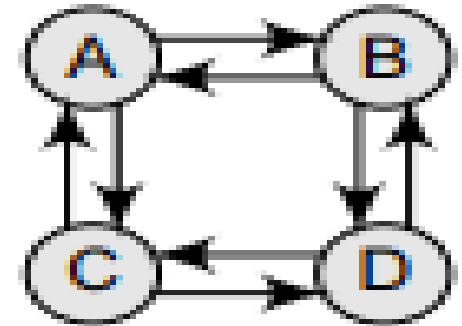
► Unilaterally connected graph:

- A digraph is said to be unilaterally connected if there exists a path between any pair of nodes u, v in G such that there is a path from u to v or a path from v to u , but not both.



► Simple directed graph:

- A directed graph G is said to be a simple directed graph if and only if it has no parallel edges.
- A simple directed graph may contain cycles with an exception that it cannot have more than one loop at a given node.



Representation of Graphs

- ▶ Two main ways of storing graphs in the computer's memory.
 - Sequential representation by using an adjacency matrix.
 - Linked representation by using an adjacency list that stores the neighbors of a node using a linked list.
- ▶ Adjacency Matrix Representation
 - An adjacency matrix is used to represent which nodes are adjacent to one another.
 - Two nodes are said to be adjacent if there is an edge connecting them.
 - In a directed graph G , if node v is adjacent to node u , then there is an edge from u to v .
 - For any graph G having n nodes, the adjacency matrix will have the dimension of $n \times n$.

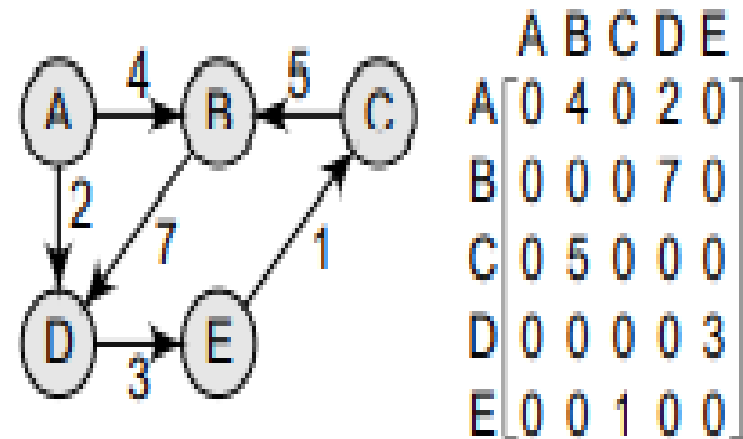
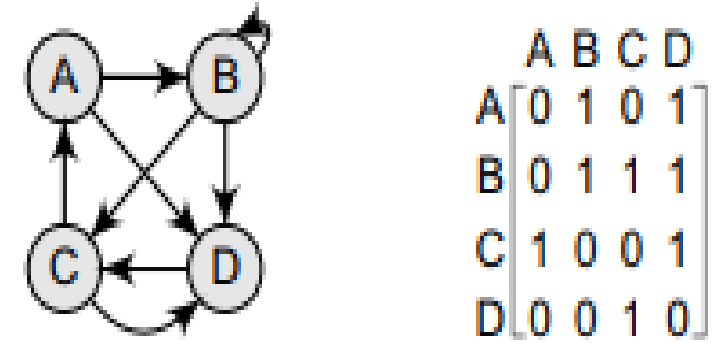
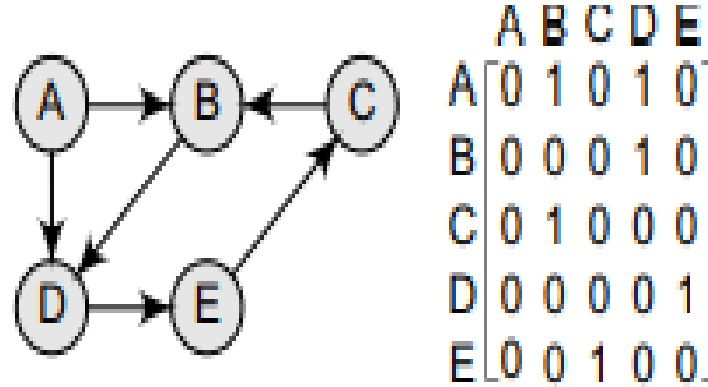
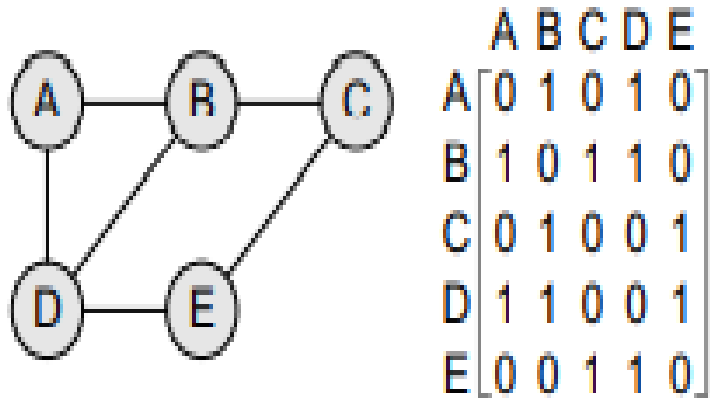
An entry a_{ij} in the adjacency matrix

1, if V_i is adjacent to V_j

0, otherwise

Note: Since the matrix contains only 0s and 1s, it is called a bit matrix or a boolean matrix.

Adjacency Matrix Representation



- ▶ For a **simple graph** (that has no loops), the adjacency matrix has **0s on the diagonal**.
- ▶ The adjacency matrix of an **undirected graph** is **symmetric**.
- ▶ The **memory use** of an adjacency matrix is **$O(n^2)$** , where n is the number of nodes in the graph.
- ▶ **Number of 1s** (or non-zero entries) in an adjacency matrix is equal to the **number of edges** in the directed graph.
- ▶ The adjacency matrix for a **weighted graph** contains the **weights** of the edges connecting the nodes.

```
addEdge(int arr[][V], int i, int j) {
    arr[i][j] = 1;
    arr[j][i] = 1; //for undirected graph
}

printAdjMatrix(int arr[][V]) {
    int i, j;
    for (i = 0; i < V; i++) {
        printf("Neighbours of %d: ", i);
        for (j = 0; j < V; j++) {
            printf("%d ", arr[i][j]);
        }
        printf("\n");
    }
}
```

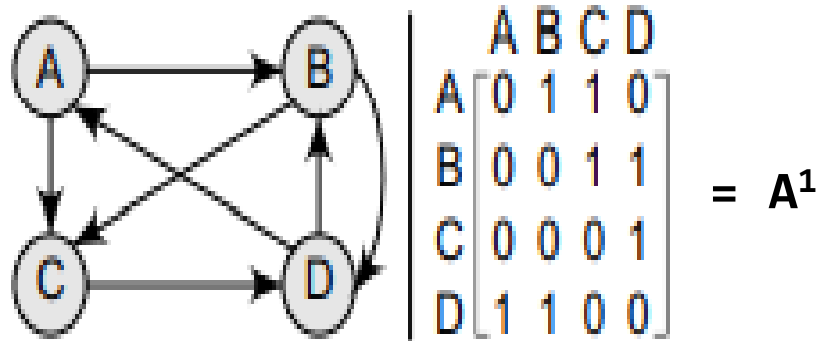
```
int main() {
    int adjMatrix[V][V];

    initialize(adjMatrix);
    addEdge(adjMatrix, 0, 1);
    addEdge(adjMatrix, 0, 2);
    addEdge(adjMatrix, 1, 2);
    addEdge(adjMatrix, 2, 0);
    addEdge(adjMatrix, 2, 3);

    printAdjMatrix(adjMatrix);

    return 0;
}
```

Path Matrix



$$A^2 = \begin{bmatrix} 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 \\ 1 & 1 & 0 & 0 \end{bmatrix} \times \begin{bmatrix} 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 \\ 1 & 1 & 0 & 0 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 1 & 2 \\ 1 & 1 & 0 & 1 \\ 1 & 1 & 0 & 0 \\ 1 & 1 & 2 & 1 \end{bmatrix}$$

$$A^3 = \begin{bmatrix} 0 & 0 & 1 & 2 \\ 1 & 1 & 0 & 1 \\ 1 & 1 & 0 & 0 \\ 1 & 1 & 2 & 1 \end{bmatrix} \times \begin{bmatrix} 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 \\ 1 & 1 & 0 & 0 \end{bmatrix} = \begin{bmatrix} 2 & 2 & 0 & 1 \\ 1 & 2 & 2 & 1 \\ 0 & 1 & 2 & 1 \\ 1 & 2 & 2 & 3 \end{bmatrix}$$

- From A^1 , an entry 1 in the i th row and j th column means that there exists a path of length 1 from v_i to v_j .

For A^2 we perform $(a_{ij})^2 = \sum a_{ik} * a_{kj}$
 Any entry $a_{ij} = 1$ if $a_{ik} = a_{kj} = 1$.

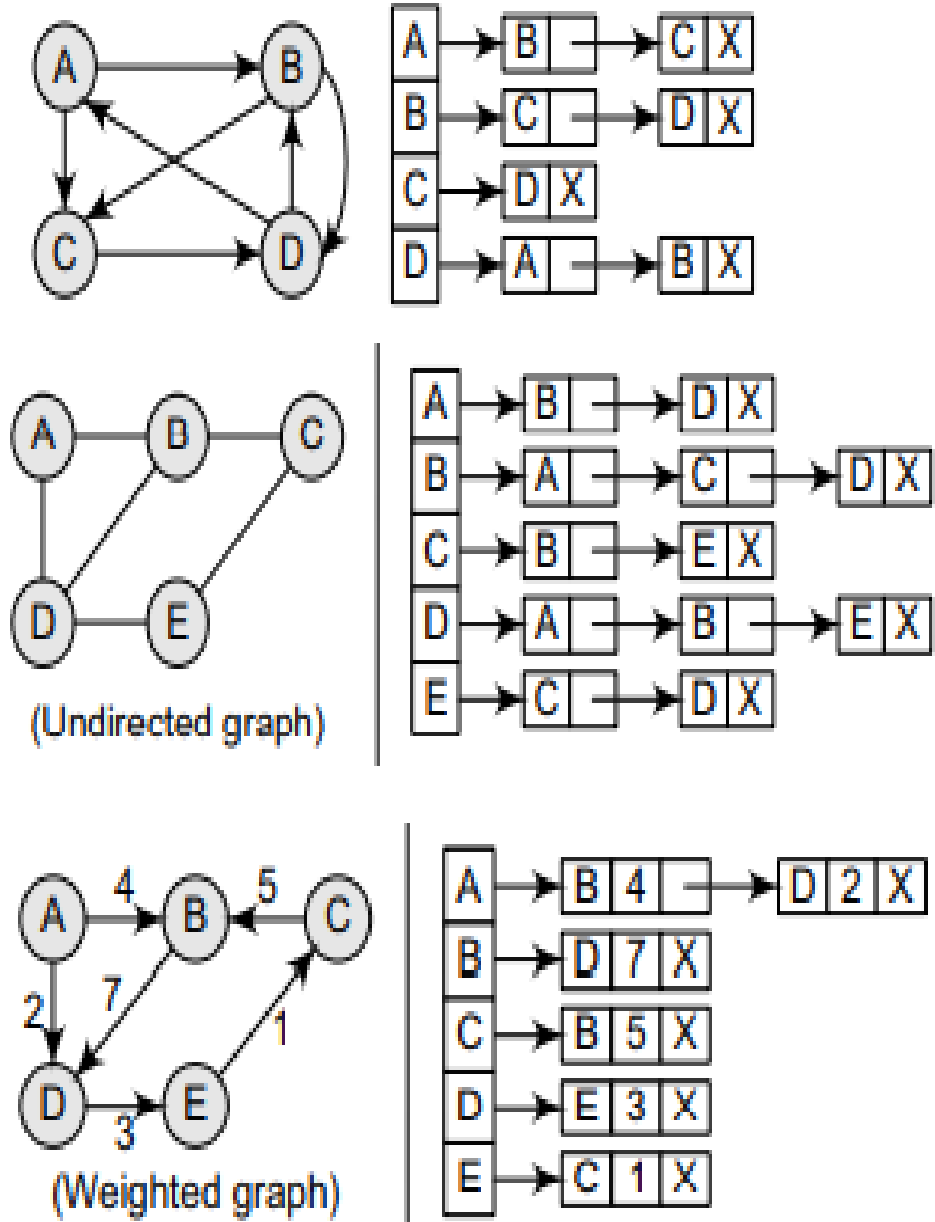
- If there is an edge (v_i, v_k) and (v_k, v_j) , then there is a path from v_i to v_j of length 2.
- In general, every entry in the i th row and j th column of A^n gives the number of paths of length n from node v_i to v_j .
- The path matrix is used to show whether there exists a simple path from node v_i to v_j or not.

Path matrix can be given as,

$$\begin{bmatrix} 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 \\ 1 & 1 & 0 & 0 \end{bmatrix} + \begin{bmatrix} 0 & 0 & 1 & 2 \\ 1 & 1 & 0 & 1 \\ 1 & 1 & 0 & 0 \\ 1 & 1 & 2 & 1 \end{bmatrix} + \begin{bmatrix} 2 & 2 & 0 & 1 \\ 1 & 2 & 2 & 1 \\ 0 & 1 & 2 & 1 \\ 1 & 2 & 2 & 3 \end{bmatrix} = \begin{bmatrix} 2 & 3 & 2 & 3 \\ 2 & 3 & 3 & 3 \\ 1 & 2 & 2 & 2 \\ 3 & 4 & 4 & 4 \end{bmatrix} = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \end{bmatrix}$$

Path matrix

Adjacency List Representation



- ▶ Every node is linked to its own list that contains the names of all other nodes that are adjacent to it.
- ▶ **Key advantages**
 - Adjacency list is preferred for **sparse graphs** (small-to-moderate number of edges); otherwise, adjacency matrix is a good choice.
 - Adding new nodes in **G** is easy and straightforward when **G** is represented using an adjacency list.
 - Adding new nodes in an adjacency matrix is a difficult task.
- ▶ For a directed graph, the sum of the lengths of all adjacency lists is equal to the number of edges.
- ▶ For an undirected graph, the sum of the lengths of all adjacency lists is equal to twice the number of edges.

```
createGraph(struct node *Adj[], int no_of_nodes)
{
    struct node *newnode;
    int i, j, n, val;
    for(i = 1; i <= no_of_nodes; i++){
        ptr = NULL;
        printf("\n Enter no of neighbours of %d: ", i);
        scanf("%d", &n);
        for(j = 1; j <= n; j++){
            printf("\n Enter neighbour %d of %d: ", j, i);
            scanf("%d", &val);
            newnode=(struct node*)malloc(sizeof(struct node));
            newnode -> data = val;
            newnode -> next = NULL;
            if (Adj[i] == NULL)
                Adj[i] = newnode;
                ptr=Adj[i];
            else
                ptr -> next = newnode;
                ptr = newnode
        }
    }
}
```

```
#define MAX 100
struct node{
    int data;
    struct node *next;
};
struct node *Adj[MAX];
```

```
displayGraph(struct node *Adj[],int no_of_nodes)
{
    struct node *ptr;
    int i;
    for(i = 0; i < no_of_nodes; i++){
        ptr = Adj[i];
        printf("\n Neighbours of node %d are:", i);
        while(ptr != NULL)
        {
            printf("\t%d", ptr -> val);
            ptr = ptr -> next;
        }
    }
}
```

- ▶ Graph traversal is a method of examining the nodes and edges of the graph.
- ▶ Traversal of graph $G=(V,E)$ and vertex v is finding all w in V , such that w connects v .
 - **Breadth First Traversal (BFT)** (level order traversal in trees)
 - **Depth First Traversal (DFT)**
- ▶ Breadth-first traversal algorithm uses a queue as an auxiliary data structure to store nodes for processing.
- ▶ Depth-first traversal algorithm uses a stack.

Breadth First Traversal (BFT) or Breadth First Search (BFS)

- ▶ BFS begins at the root node and explores all the neighboring nodes.
 - Then for each of the nearest nodes, the algorithm explores their unexplored neighbor nodes, and so on, until it finds the goal.

Algorithm

Step 1: Set **STATUS=1** (ready state)

for each node in **G**

Step 2: **Enqueue** the starting node **A** and set its **STATUS=2** (waiting state)

Step 3: Repeat **Steps 4** and **5** until **QUEUE** is empty

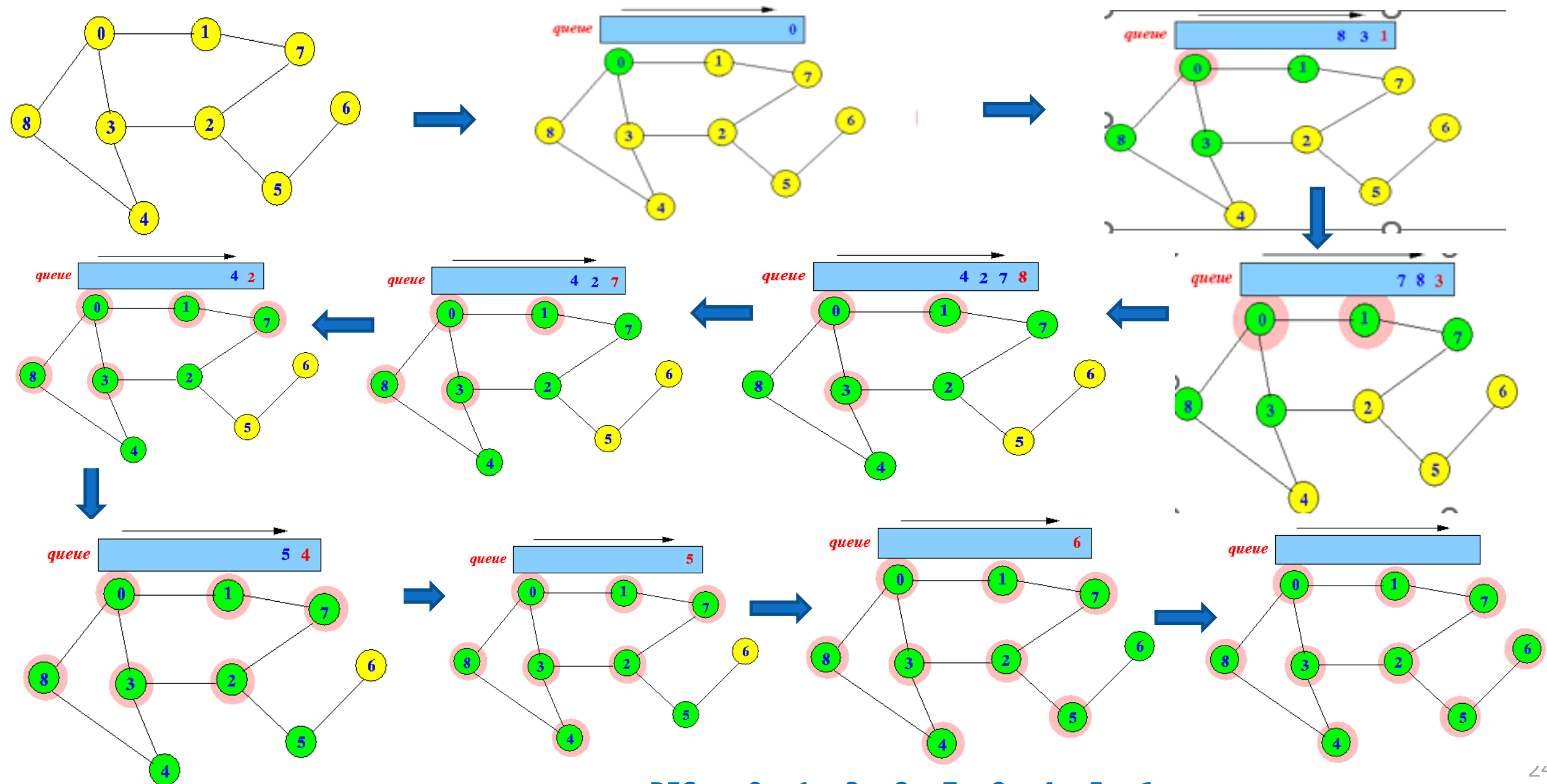
Step 4: **Dequeue** a node **N**. Process it and set its **STATUS=3** (processed state).

Step 5: **Enqueue** all the neighbors of **N** that are in the ready state (whose **STATUS=1**) and set their **STATUS=2** (nodes are **visited** but in **waiting state** for processing)

[End of Loop]

Step 6: EXIT

Breadth First Traversal (BFT)



Breadth First Traversal (BFT)

```
void breadth_first_traversal(int adj[][MAX],int visited[],int start)
{
    int queue[MAX],rear = -1,front =- 1, i;
    queue[++rear] = start;
    visited[start] = 1;
    while(rear != front)
    {
        start = queue[++front];
        printf("%d \t",start);
        for(i = 0; i < MAX; i++)
        {
            if(adj[start][i] == 1 && visited[i] == 0)
            {
                queue[++rear] = i;
                visited[i] = 1;
            }
        }
    }
}
```

```
#define MAX 10
int main()
{
    int visited[MAX] = {0};
    int adj[MAX][MAX], i, j;
    printf("Enter adjacency matrix:");
    for(i = 0; i < MAX; i++)
        for(j = 0; j < MAX; j++)
            scanf("%d", &adj[i][j]);
    breadth_first_traversal(adj,visited,0);
    return 0;
}
```

- The Time complexity of BFT is $O(|V| + |E|)$ when Adjacency List is used and $O(|V|^2)$ when Adjacency Matrix is used.

Depth First Traversal (DFT) or Depth First Search (DFS)

- ▶ DFS progresses by expanding the starting node of G
 - then going deeper and deeper until the goal node is found, or
 - until a node that has no children is encountered

Algorithm

Step 1: Set **STATUS=1** (ready state)

for each node in **G**

Step 2: **Push** the starting node **A** on the stack and set its **STATUS=2** (waiting state)

Step 3: Repeat Steps 4 and 5 until **STACK** is empty

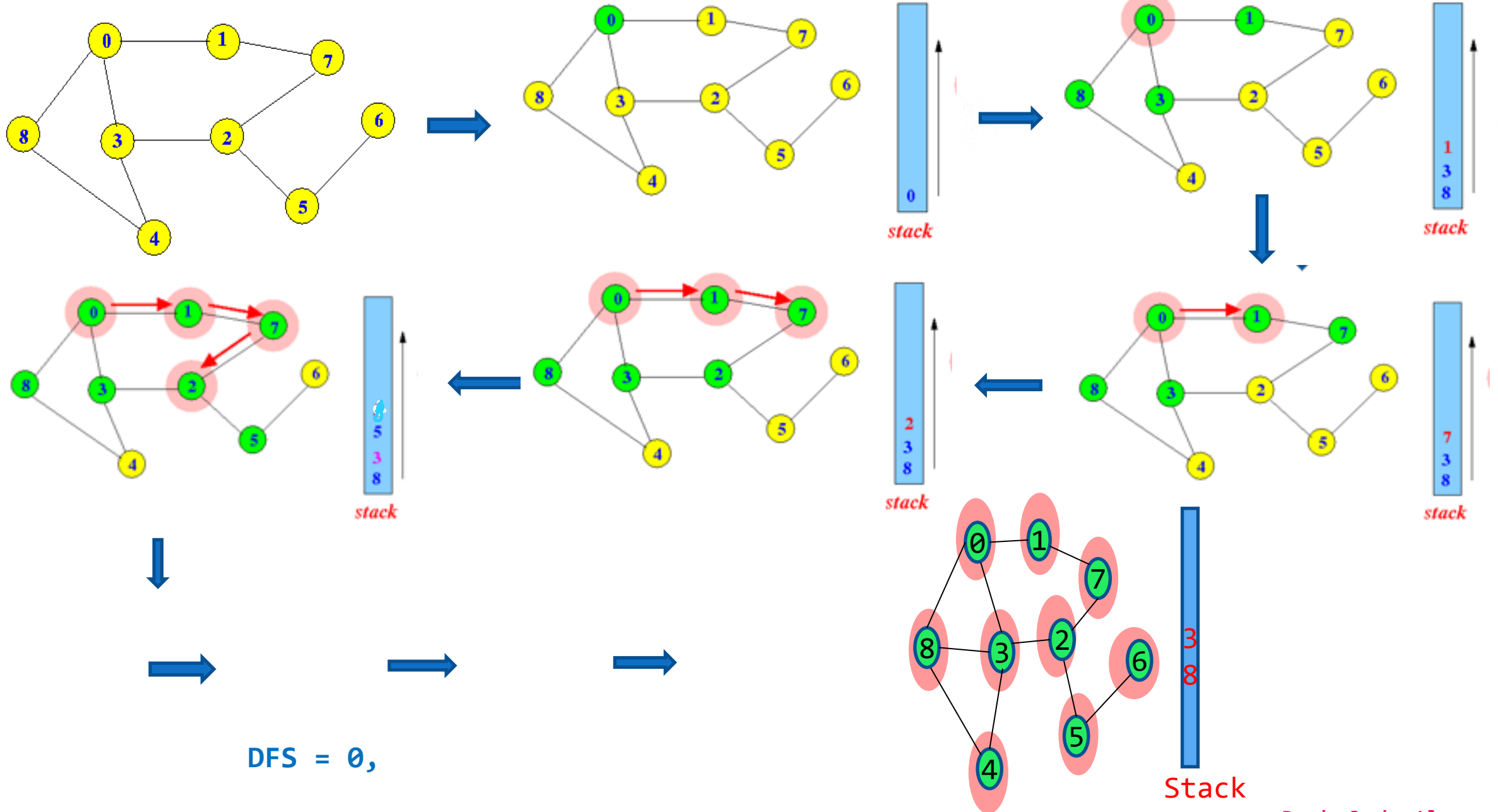
Step 4: **Pop** the **top** node **N**. Process it and set its **STATUS=3** (processed state)

Step 5: **Push** on the stack all the neighbors of **N** that are in the **ready state** (whose **STATUS=1**) and set their **STATUS=2** (waiting state)

[End of Loop]

Step 6: EXIT

Depth First Traversal (DFT)

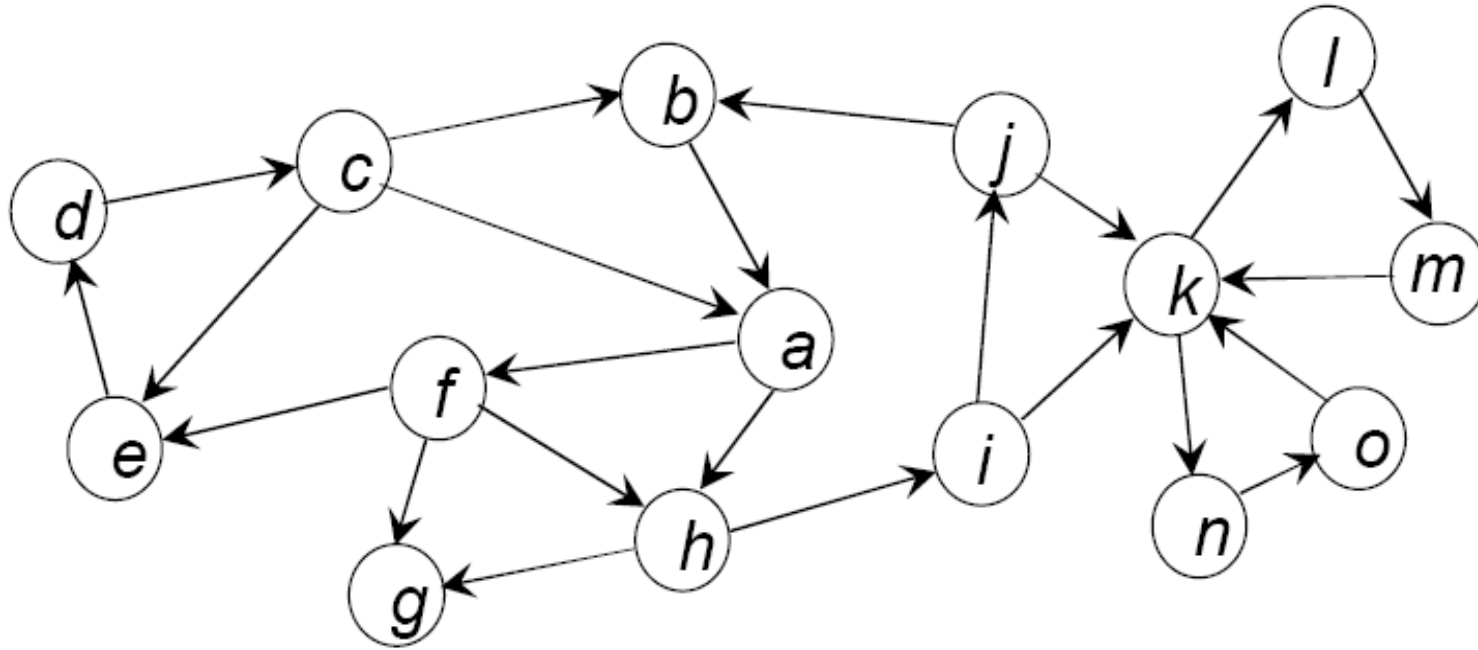


Depth First Traversal (DFT)

```
void depth_first_traversal(int adj[][MAX],int visited[],int start)
{
    int stack[MAX], top = -1, i;
    stack[++top] = start;
    visited[start] = 1;
    while(top != -1){
        start = stack[top--];
        printf("%c", start);
        for(i = 0; i < MAX; i++){
            if(adj[start][i] && visited[i] == 0)
            {
                stack[++top] = i;
                visited[i] = 1;
            }
        }
    }
}
```

```
#define MAX 5
int main()
{
    int adj[MAX][MAX];
    int visited[MAX] = {0}, i, j;
    printf("\nEnter adjacency matrix: ");
    for(i = 0; i < MAX; i++)
        for(j = 0; j < MAX; j++)
            scanf("%d", &adj[i][j]);
    printf("DFS Traversal: ");
    depth_first_traversal(adj,visited,0);
    printf("\n");
    return 0;
}
```

- The Time complexity of DFT is $O(|V| + |E|)$ when Adjacency List is used and $O(|V|^2)$ when Adjacency Matrix is used.



Find Breadth-First and Depth-First Traversal starting from node 'a'

Breadth-first traversal: a f h e g i d j k c l n b m o

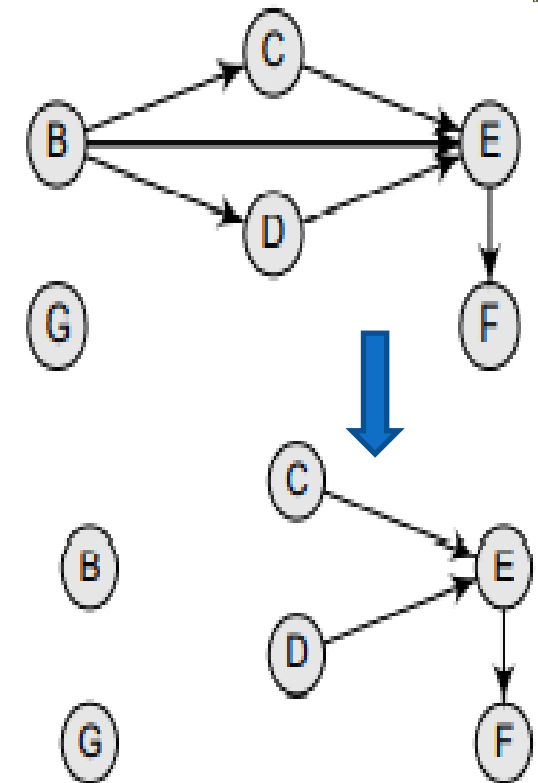
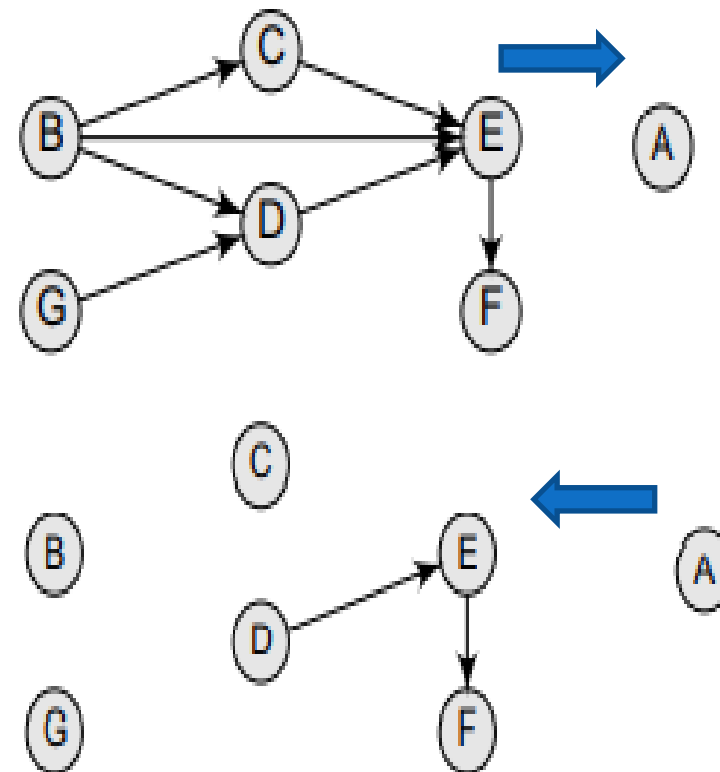
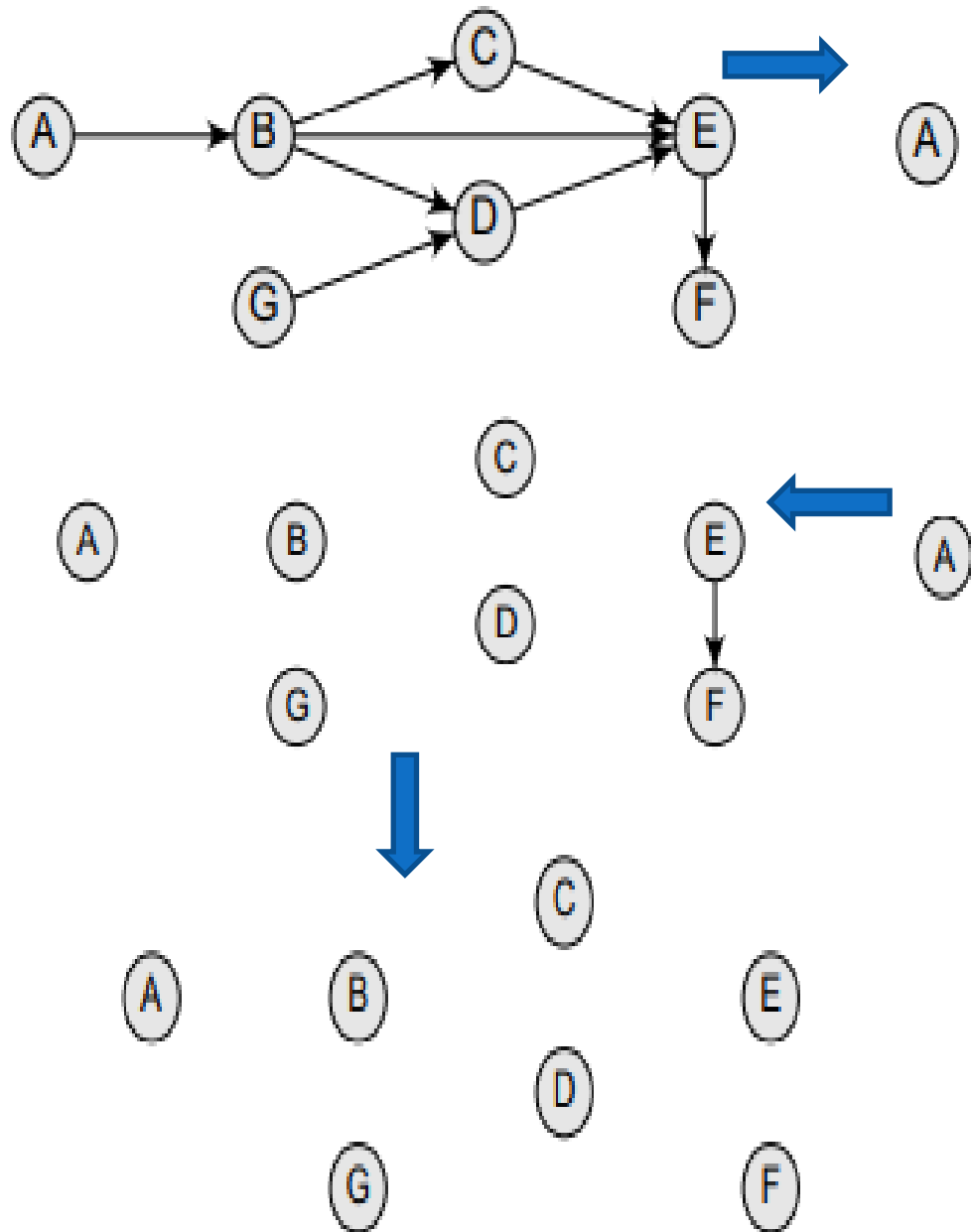
Depth-first traversal: a f e d c b g h i j k l m n o

- **Directed Acyclic Graph (DAG)** is a directed graph that does not contain the cycle
- A **topological sort** of a DAG G is an ordering of the vertices of G such that if G contains an edge (u, v) , then **u appears before v in the ordering.**
- Note that topological sort is possible only on DAGs
- **Every DAG has one or more number of topological sorts.**
- Topological sorting is widely used in scheduling applications, jobs, or tasks.

Steps in Topological sort Algorithm

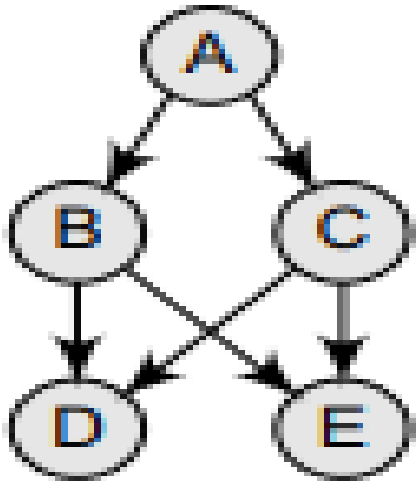
- Step-1: Selecting a node N with zero in-degree
- Step-2: Delete (Process or display) N from the graph along with its edges
- Step-3: Repeat steps 1 and 2

Topological sorting



Topological sort order: A, G, B, C, D, E, F

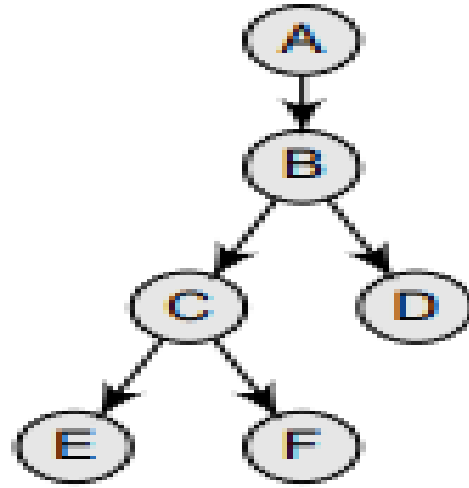
Topological sorting



Topological sort orders:

A, B, C, D, E

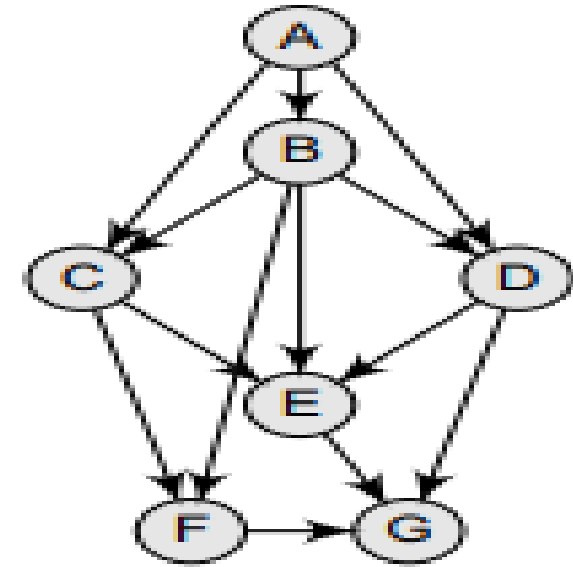
A, B, C, E, D



Topological sort orders:

A, B, D, C, E, F

A, B, D, C, F, E



Topological sort orders:

A, B, C, F, D, E, G

A, B, C, D, E, F, G

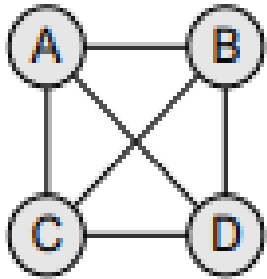
Shortest path algorithms

Different algorithms to calculate the shortest path between the vertices of a graph G.

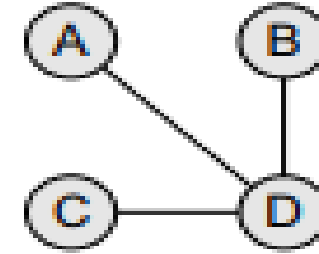
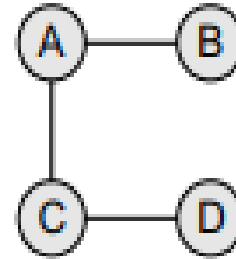
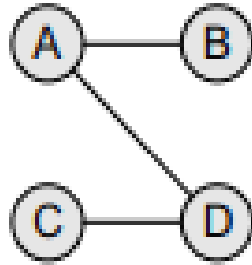
- ▶ Minimum spanning tree
- ▶ Dijkstra's algorithm
- ▶ Floyd-Warshall's algorithm

Minimum Spanning tree (MST)

- A spanning tree of a connected, undirected graph G is a sub-graph of G which is a tree that connects all the vertices together.
- A graph G can have many different spanning trees.



Graph G



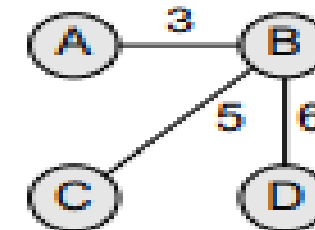
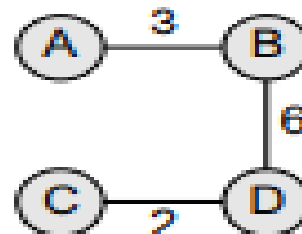
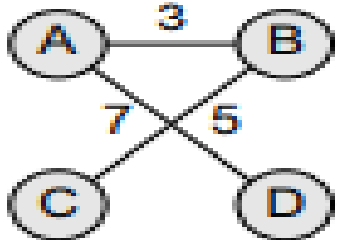
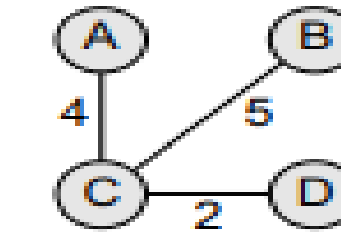
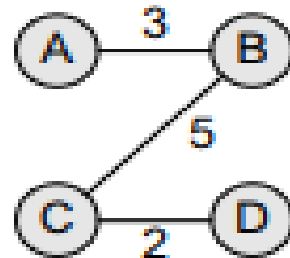
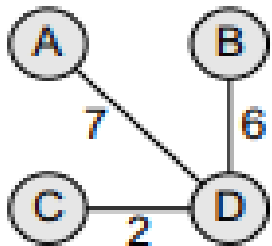
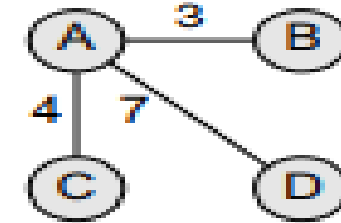
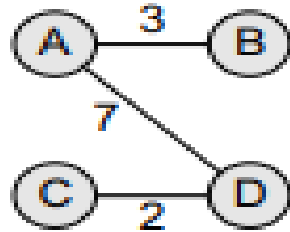
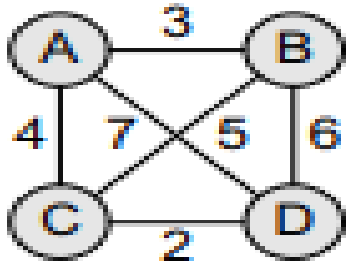
Spanning Trees of G

- Weight of a spanning tree is the sum of the weights of the edges in that spanning tree.
- A minimum spanning tree (MST or minimum weighted spanning tree) is defined as a spanning tree with weight less than or equal to the weight of every other spanning tree.

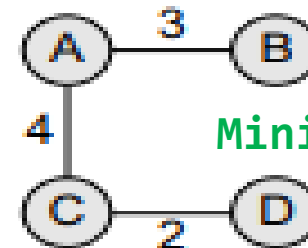
Let $G = (V, E)$ be an undirected connected graph.

MST of graph G is an acyclic subset $T \subseteq E$ that connects all of the vertices and whose total weight $W(T)$ is minimized, where $W(T) = \sum_{(u,v) \in T} w(u,v)$

Minimum Spanning tree (MST)



Note: No. of Spanning Trees for a given complete graph with n nodes = $n^{(n-2)}$



Minimum Spanning Tree (Weight = 9)

- **Tree vertices:** Vertices that are a part of the minimum spanning tree **T**.
- **Fringe vertices:** Vertices that are currently not a part of **T**, but are adjacent to some vertex of **T**.
- **Unseen vertices:** Vertices that are neither tree vertices nor fringe vertices fall under this category.

Algorithm

Step 1: Select a starting vertex.

Step 2: REPEAT Steps 3 and 4 until there are no fringe vertices.

Step 3: Select an edge **e** connecting the tree vertex and fringe vertex that has minimum weight.

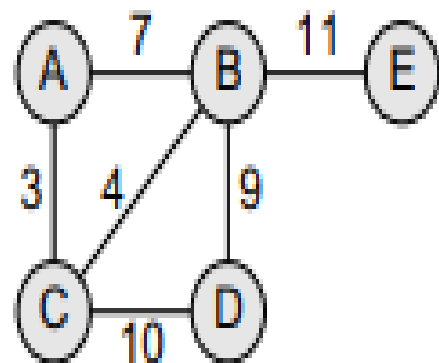
Step 4: Add the selected edge and the vertex to the minimum spanning tree **T**.

[END OF LOOP]

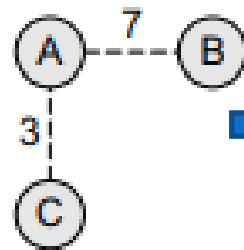
Step 5: EXIT

Example-1

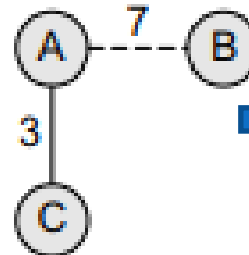
Prim's Algorithm



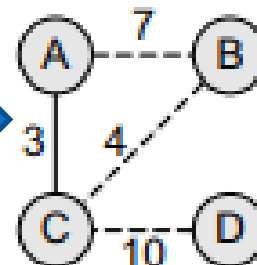
Step 1



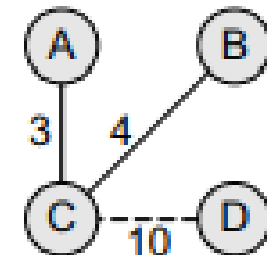
Step 2



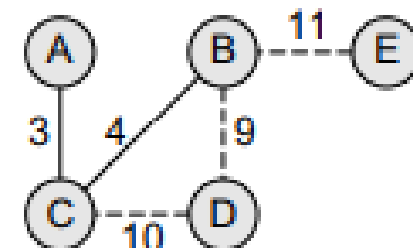
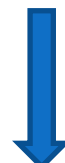
Step 3



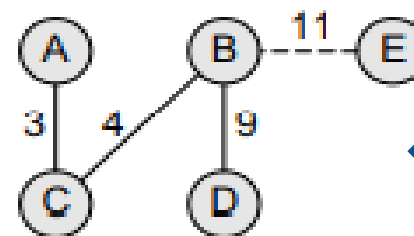
Step 4



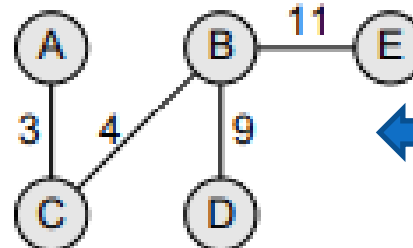
Step 5



Step 6

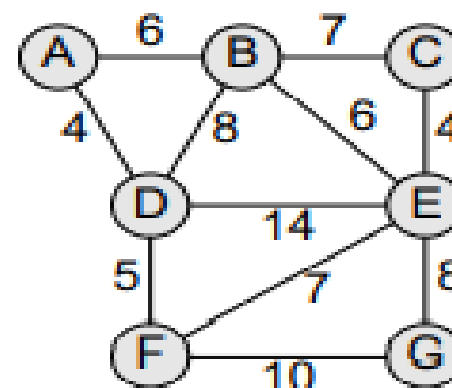


Step 7



Step 8

Minimum
Spanning Tree



Find Minimum Spanning Tree?

MST-PRIM(G, w, r) // using Min-Heap

```

1  for each  $u \in G.V$ 
2       $u.key = \infty$ 
3       $u.\pi = \text{NIL}$ 
4   $r.key = 0$ 
5   $Q = G.V$   $\longrightarrow$  Build heap:  $O(V \log V)$ 
6  while  $Q \neq \emptyset$   $\longrightarrow O(V)$ 
7       $u = \text{EXTRACT-MIN}(Q)$   $\longrightarrow O(\log V)$ 
8      for each  $v \in G.Adj[u]$   $\longrightarrow O(V)$ 
9          if  $v \in Q$  and  $w(u, v) < v.key$ 
10              $v.\pi = u$ 
11              $v.key = w(u, v)$   $\longrightarrow$  Decrease key:  $O(\log V)$ 
    
```

Total Time Complexity:

$$\begin{aligned}
 &= O(V) + O(V \log V) + O(V) * (O(\log V) + O(V) * O(\log V)) \\
 &= O(V) + O(V \log V) + O(V^2 \log V) \\
 &= O(E \log V)
 \end{aligned}$$

(Because, V^2 can be considered as no. of edges (E) (aggregate analysis))

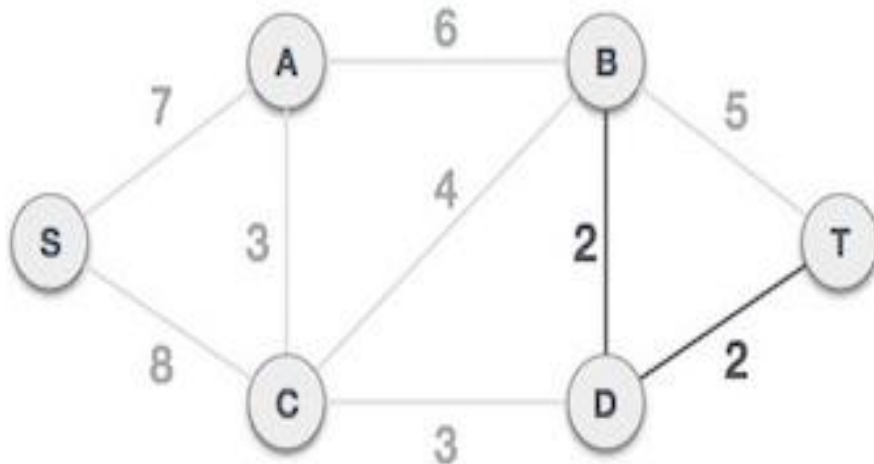
Kruskal's Algorithm

- ▶ Algorithm treats the graph as a forest and every node it has as an individual tree.
- ▶ A tree connects to another if and only if, it has the least cost among all available options and does not violate MST properties.

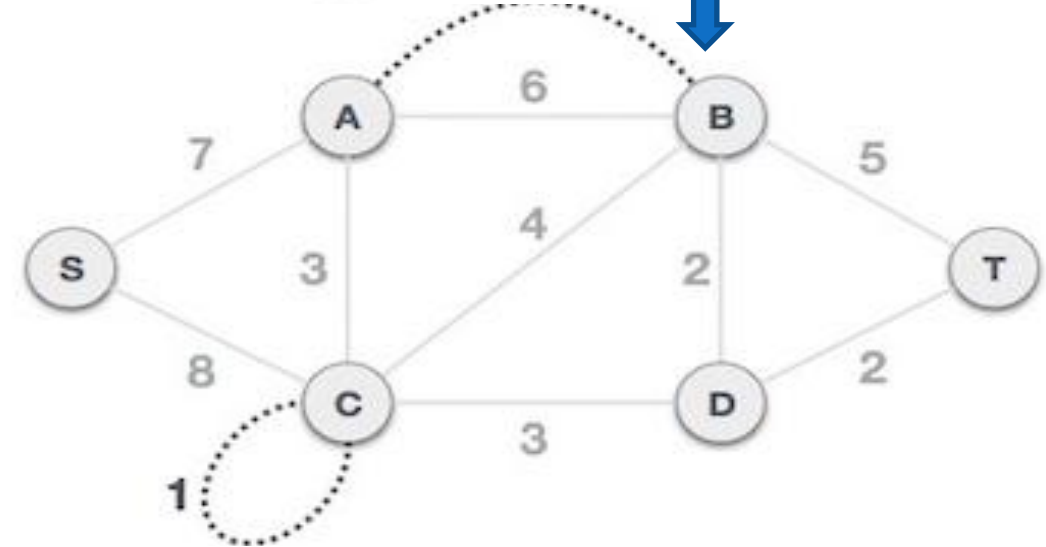
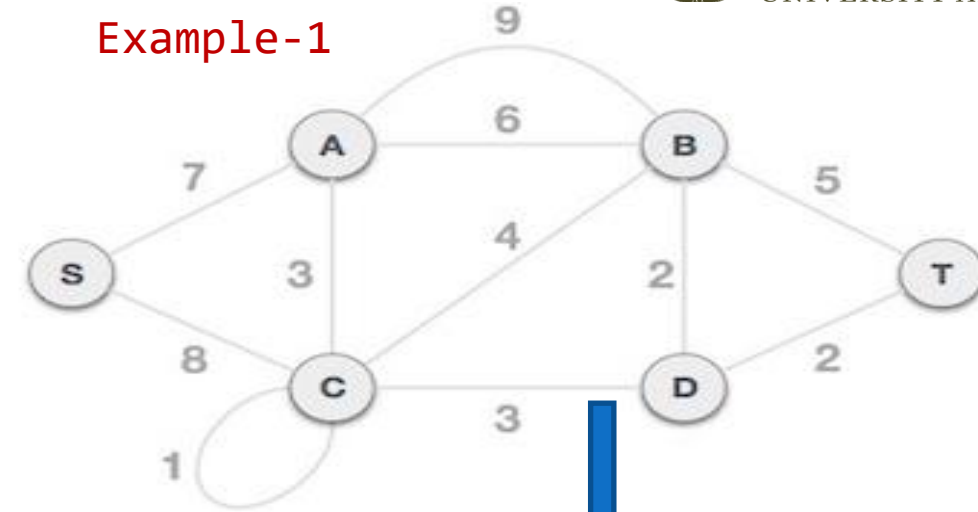
Step 1 - Remove all loops and Parallel Edges

Step 2 - Arrange all edges in their increasing order of weight, **REPEAT** Steps 3 until MST is formed.

Step 3 - Add the edge to MST which has the least weightage



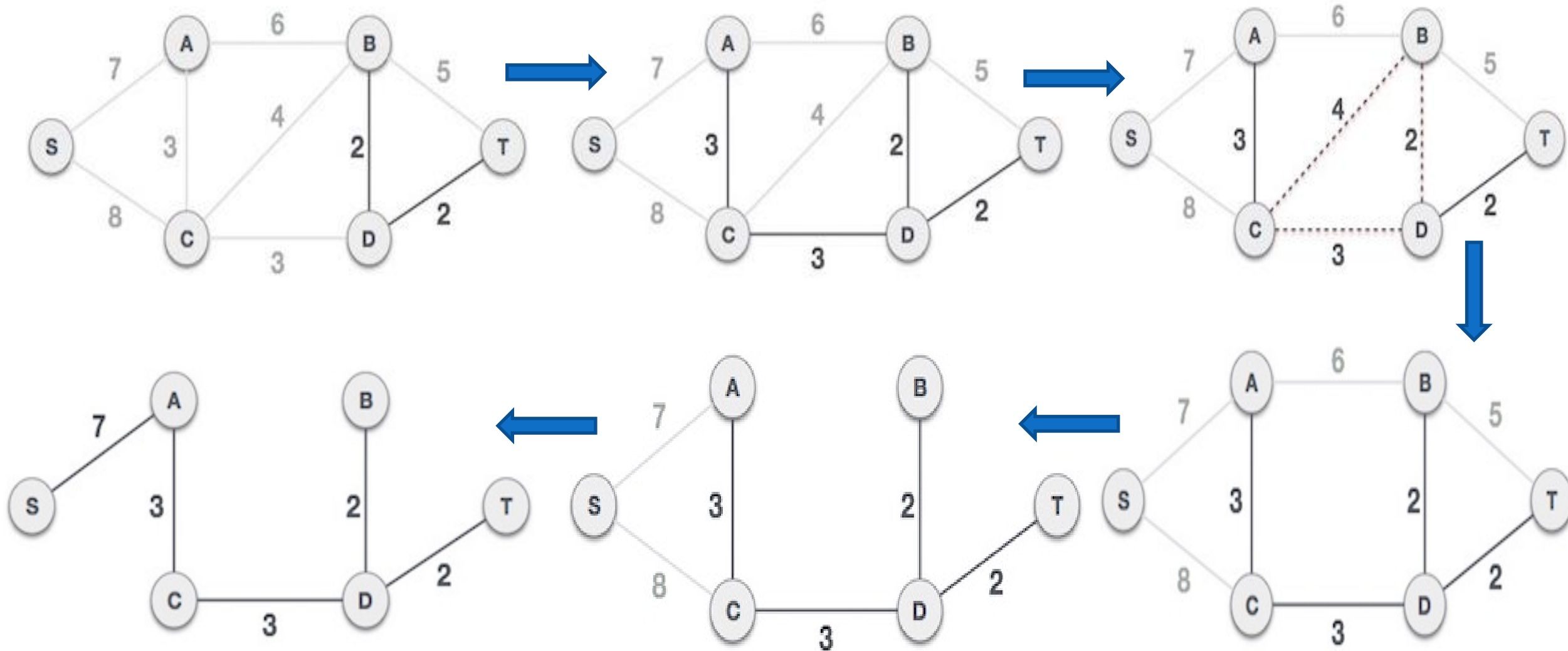
Example-1



B, D	D, T	A, C	C, D	C, B	B, T	A, B	S, A	S, C
2	2	3	3	4	5	6	7	8

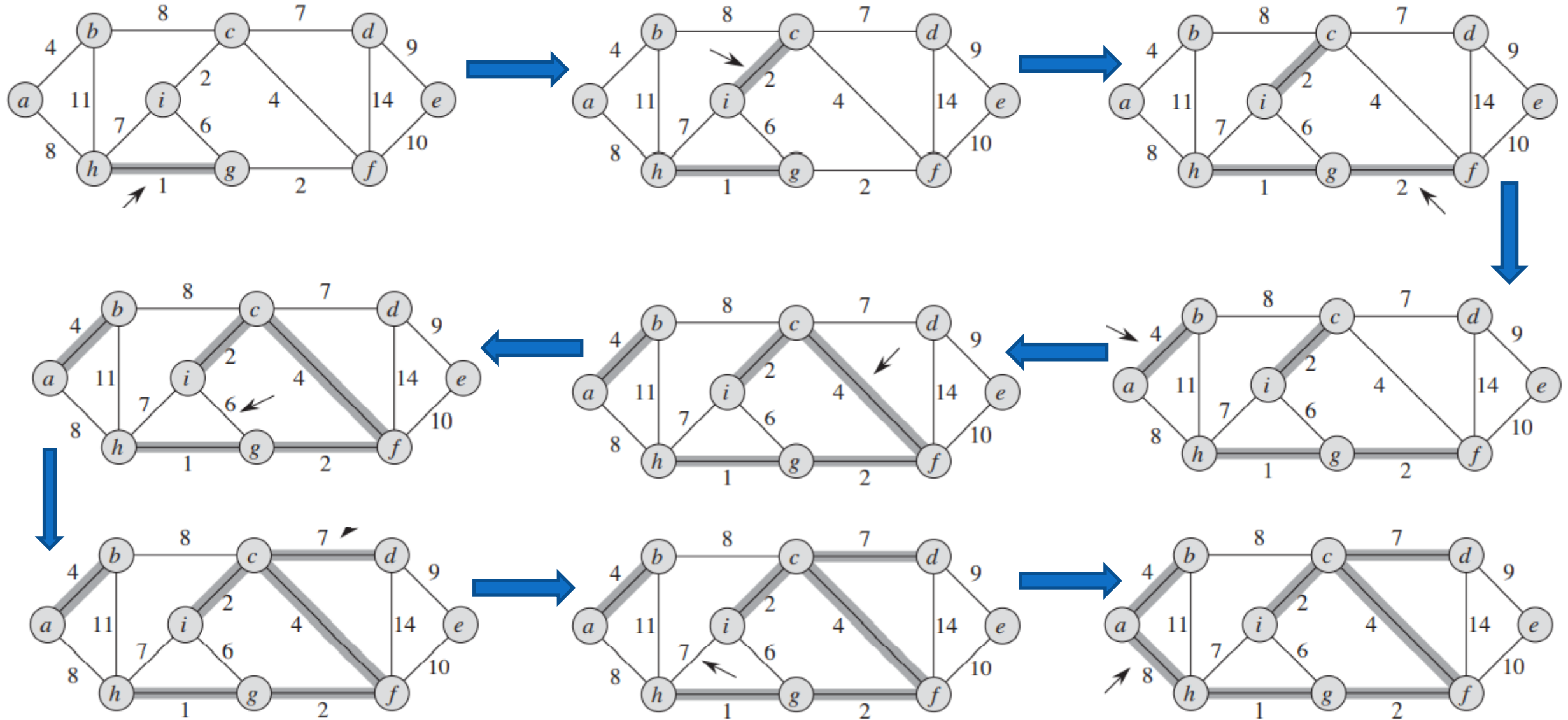
Kruskal's Algorithm

Step 3 - Add the edge which has the least weightage

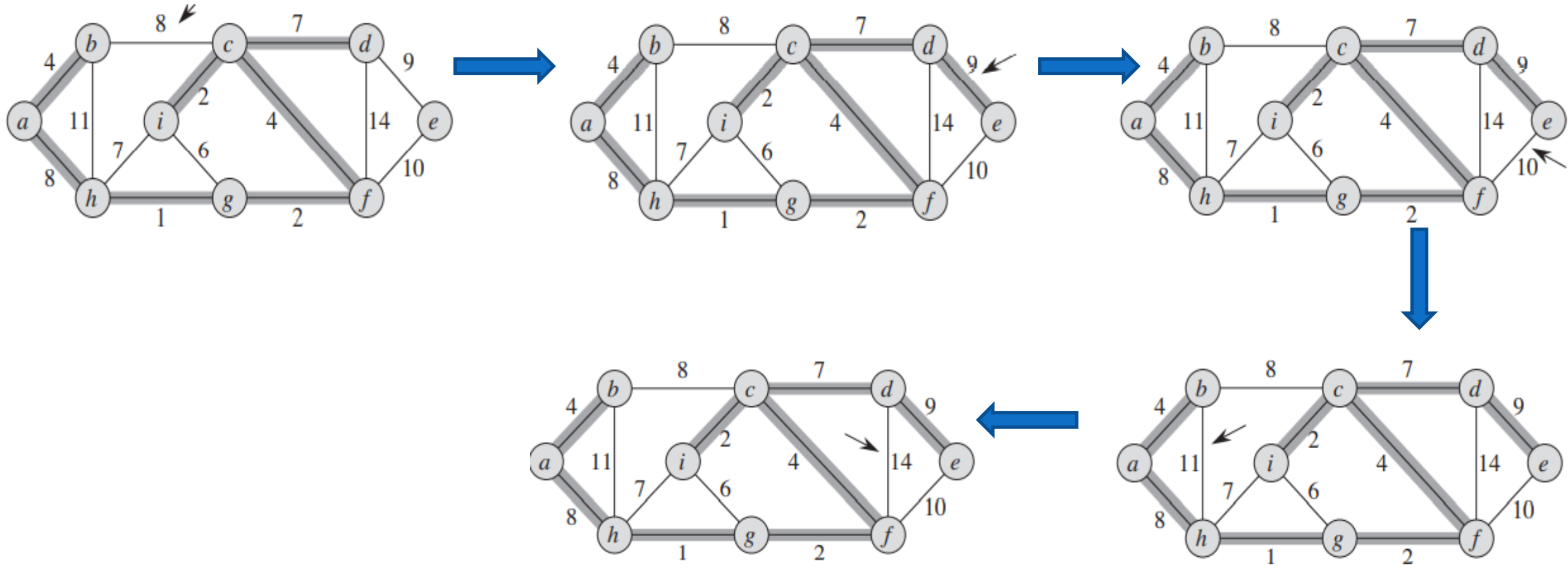


Kruskal's Algorithm

Example-2



Kruskal's Algorithm



MST-KRUSKAL(G, w)

```
1   $A = \emptyset$ 
2  for each vertex  $v \in G.V$   $\longrightarrow O(V)$ 
3      MAKE-SET( $v$ )
4  sort the edges of  $G.E$  into nondecreasing order by weight  $w \longrightarrow O(E \log E)$ 
5  for each edge  $(u, v) \in G.E$ , taken in nondecreasing order by weight
6      if FIND-SET( $u$ )  $\neq$  FIND-SET( $v$ )
7           $A = A \cup \{(u, v)\}$ 
8          UNION( $u, v$ )
9  return  $A$ 
```

$O(E)$

Total Time Complexity:

$$= O(V) + O(E \log E) + O(E)$$

$$= O(E \log E)$$

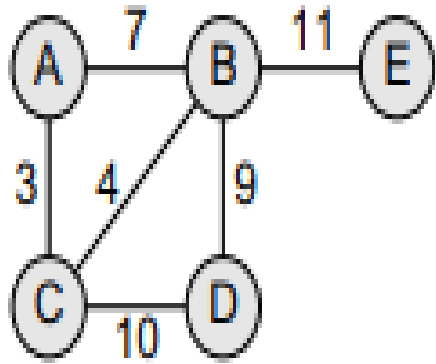
$$= O(E \log V^2) = O(E \log V)$$

(Because, V^2 can be considered as no. of edges (E) (aggregate analysis))

Space Complexity: $O(V)$ (To create V sets (MAKE-SET(v)))

Kruskal's Algorithm

Example-3



Shortest path algorithms

Different algorithms to calculate the shortest path between the vertices of a graph G .

- ▶ Minimum spanning tree
- ▶ Dijkstra's algorithm
- ▶ Floyd-Warshall's algorithm

Dijkstra's Algorithm (Single Source Shortest Path Algorithm)

Step-1: Maintain a list of unvisited vertices.

Step-2: Assign a vertex as “source” and also allocate a maximum possible cost (infinity) to every other vertex. (The cost of the source to itself will be zero)

Step-3: In every step, try to minimize the cost for each vertex.

Step-4: For every unvisited neighbor (V2, V3) of the current vertex (V1) calculate the new cost from V1.

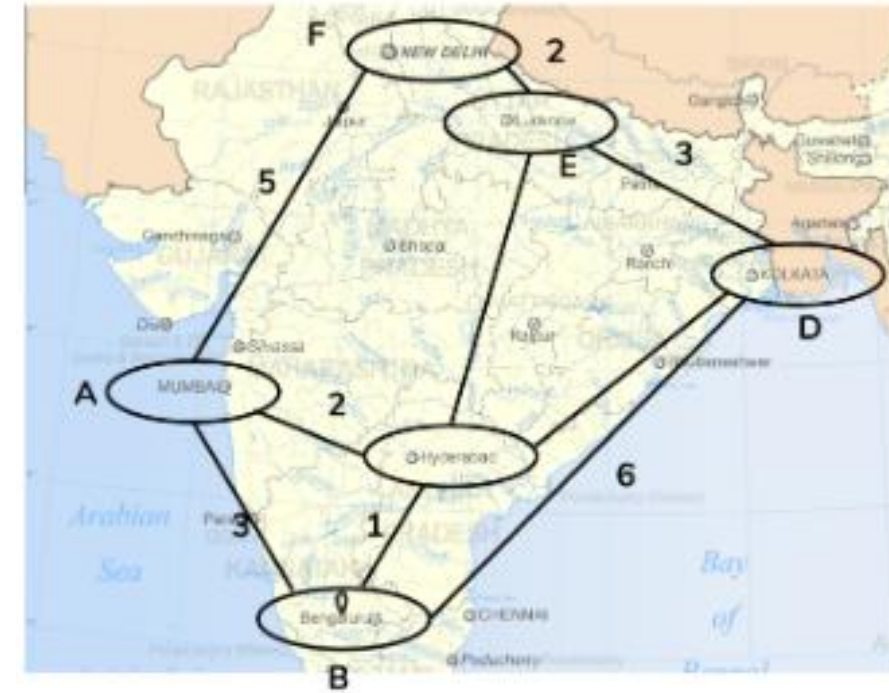
Step-5: The new cost of V2 is calculated as :

Minimum(existing cost of V2 , (sum of cost of V1 + the cost of edge from V1 to V2))

Step-6: Mark the current node V1 as visited and remove it from the unvisited list.

Step-7: Select next vertex with smallest cost from the unvisited list and repeat from step 4.

Step-8: The algorithm finally ends when there are no unvisited nodes left.

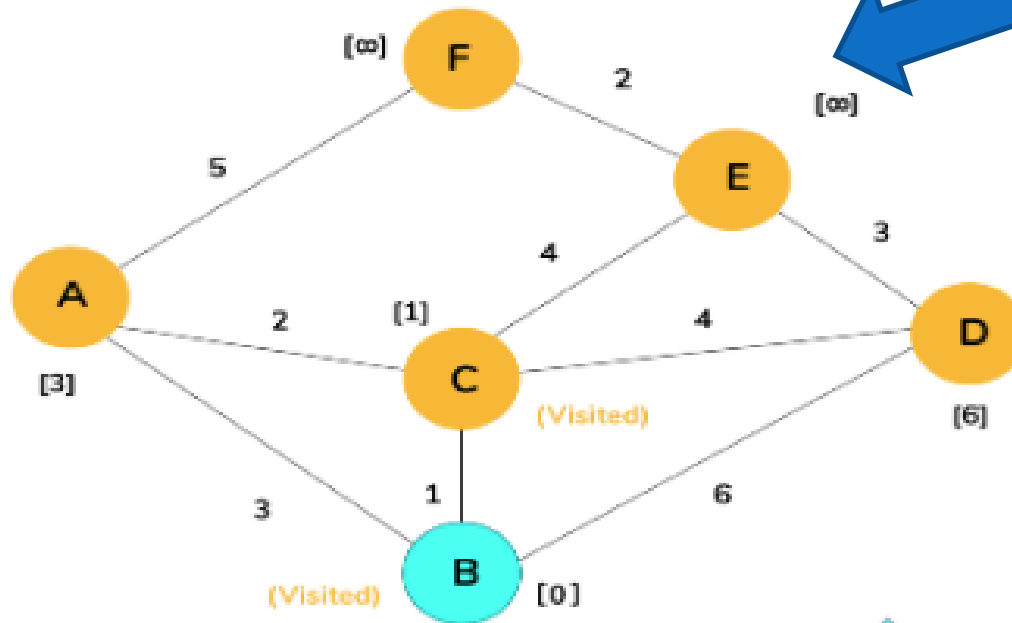
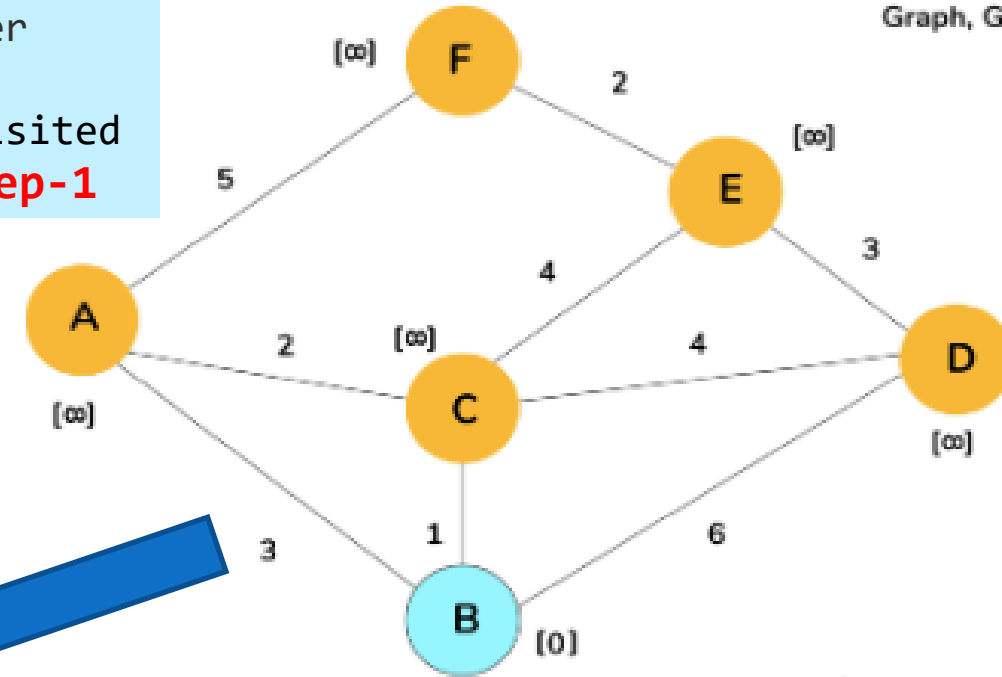
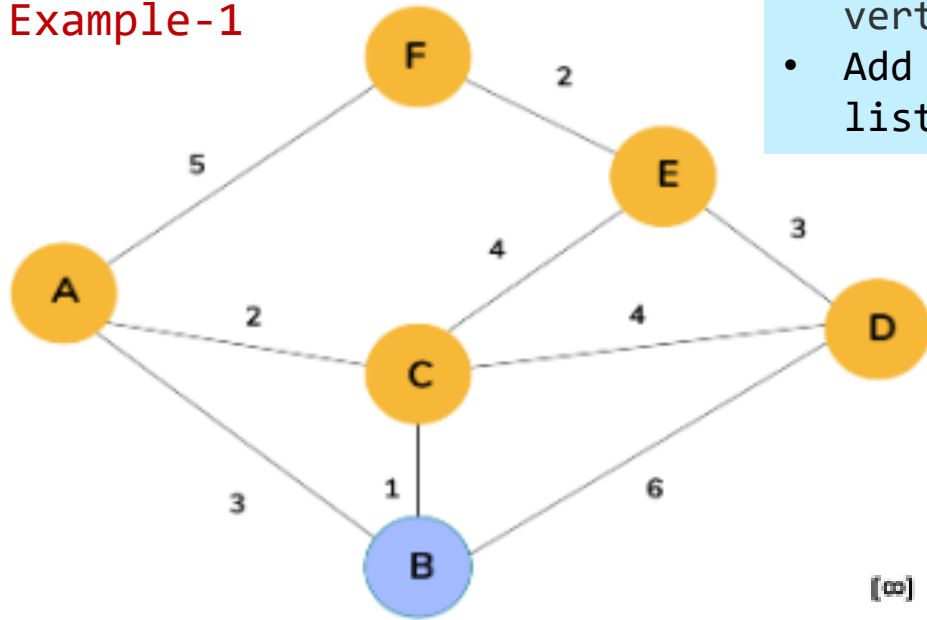


Dijkstra's Algorithm

Example-1

- Assign cost of 0 to source vertex and ∞ (Infinity) to all other vertices
- Add all the vertices to unvisited list

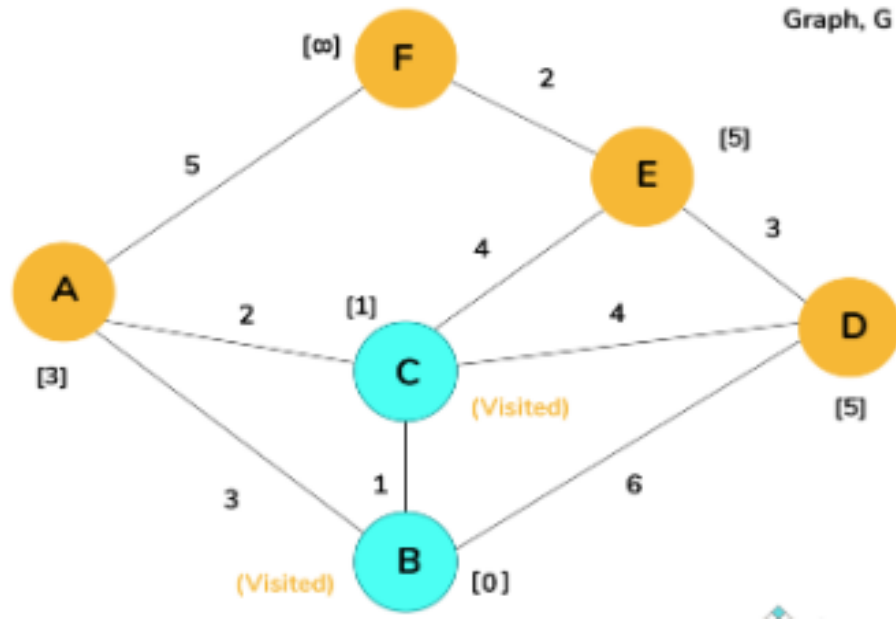
Step-1



- Step-2**
- For neighbor A:
cost = Minimum(∞ , $0+3$) = 3
 - For neighbor C:
cost = Minimum(∞ , $0+1$) = 1
 - For neighbor D:
cost = Minimum(∞ , $0+6$) = 6

Dijkstra's Algorithm

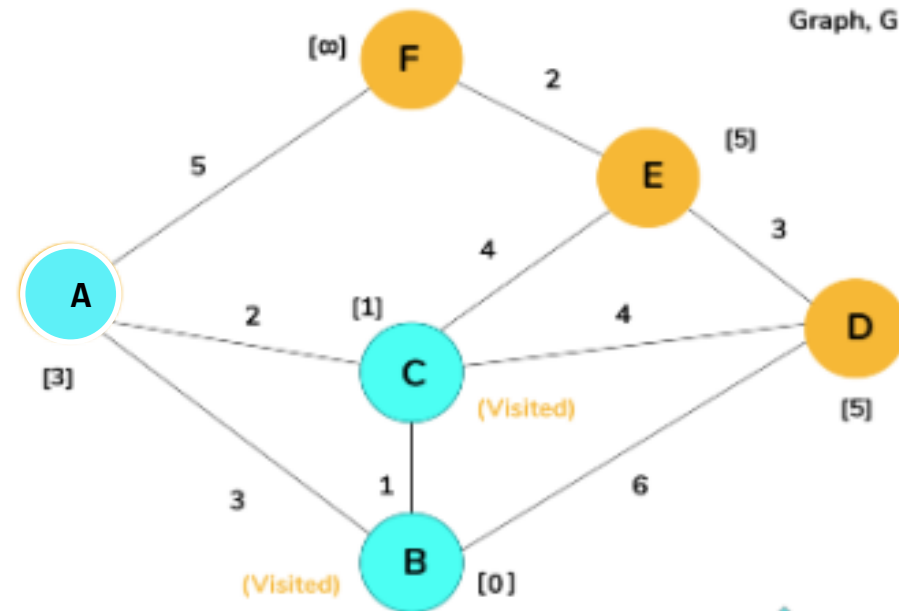
Step-3: Select next vertex with smallest cost from the unvisited list. (C)



- For neighbor A: cost = Minimum(3 , 1+2) = 3
- For neighbor D: cost = Minimum(6 , 1+4) = 5
- For neighbor E: cost = Minimum(∞ , 1+4) = 5

Step-4

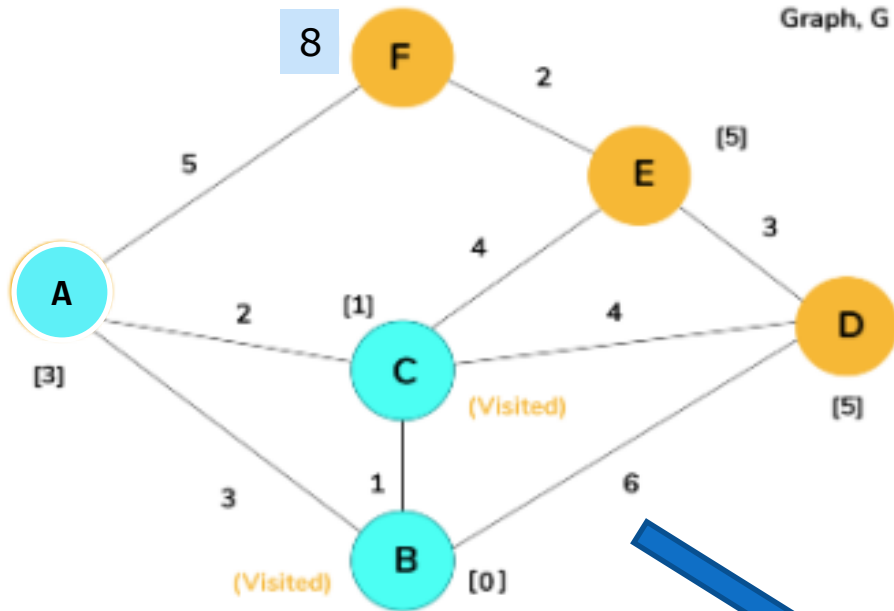
Step-5: Select next vertex with smallest cost from the unvisited list. (A)



- For neighbor F: cost = Minimum(∞ , 3+5) = 8

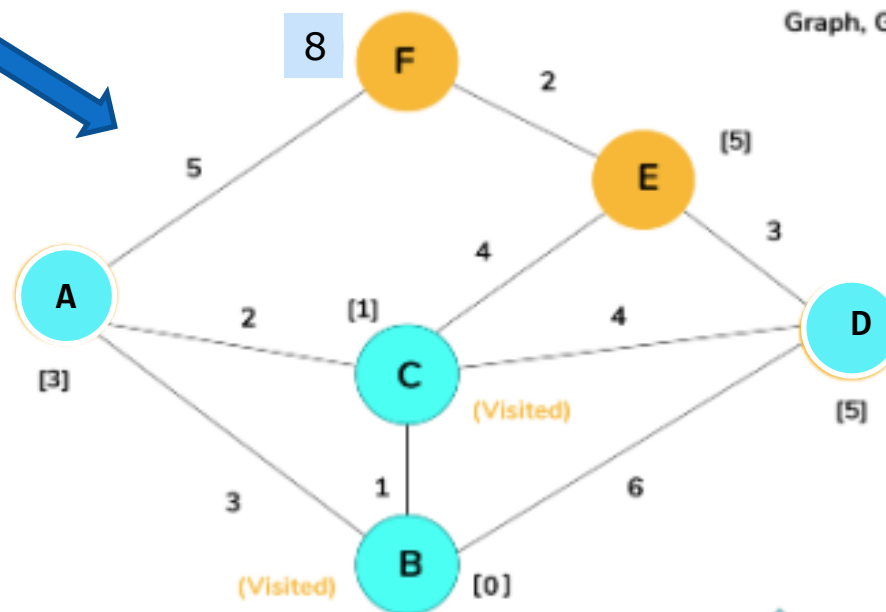
Step-6

Dijkstra's Algorithm



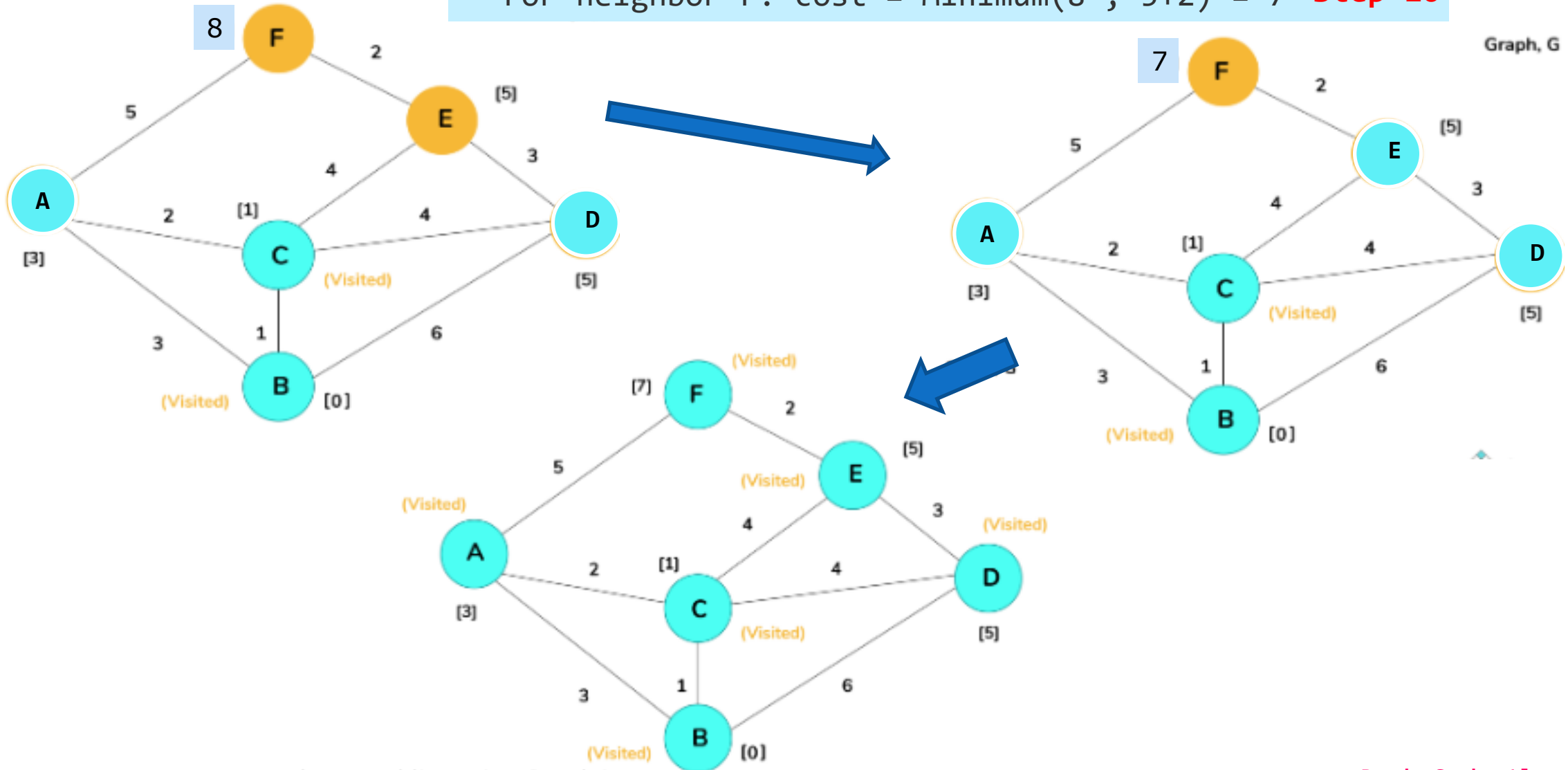
Step-7: Select next vertex with smallest cost from the unvisited list. **(D)**

• For neighbor E: cost = Minimum(5 , 3+5) = 8 **Step-8**



Step-9: Select next vertex with smallest cost from the unvisited list. **(E)**

• For neighbor F: cost = Minimum(8 , 5+2) = 7 **Step-10**



Dijkstra's Algorithm

MST-PRIM(G, w, r)

```

1  for each  $u \in G.V$ 
2       $u.key = \infty$ 
3       $u.\pi = \text{NIL}$ 
4   $r.key = 0$ 
5   $Q = G.V$ 
6  while  $Q \neq \emptyset$ 
7       $u = \text{EXTRACT-MIN}(Q)$ 
8      for each  $v \in G.Adj[u]$ 
9          if  $v \in Q$  and  $w(u, v) < v.key$ 
10              $v.\pi = u$ 
11              $v.key = w(u, v)$ 
  
```

INITIALIZE-SINGLE-SOURCE(G, s)

```

1  for each vertex  $v \in G.V$ 
2       $v.d = \infty$ 
3       $v.\pi = \text{NIL}$ 
4   $s.d = 0$ 
  
```

DIJKSTRA(G, w, s)

```

1  INITIALIZE-SINGLE-SOURCE( $G, s$ )
2   $S = \emptyset$ 
3   $Q = G.V$ 
4  while  $Q \neq \emptyset$ 
5       $u = \text{EXTRACT-MIN}(Q)$ 
6       $S = S \cup \{u\}$ 
7      for each vertex  $v \in G.Adj[u]$ 
8          RELAX( $u, v, w$ )
  
```

RELAX(u, v, w)

```

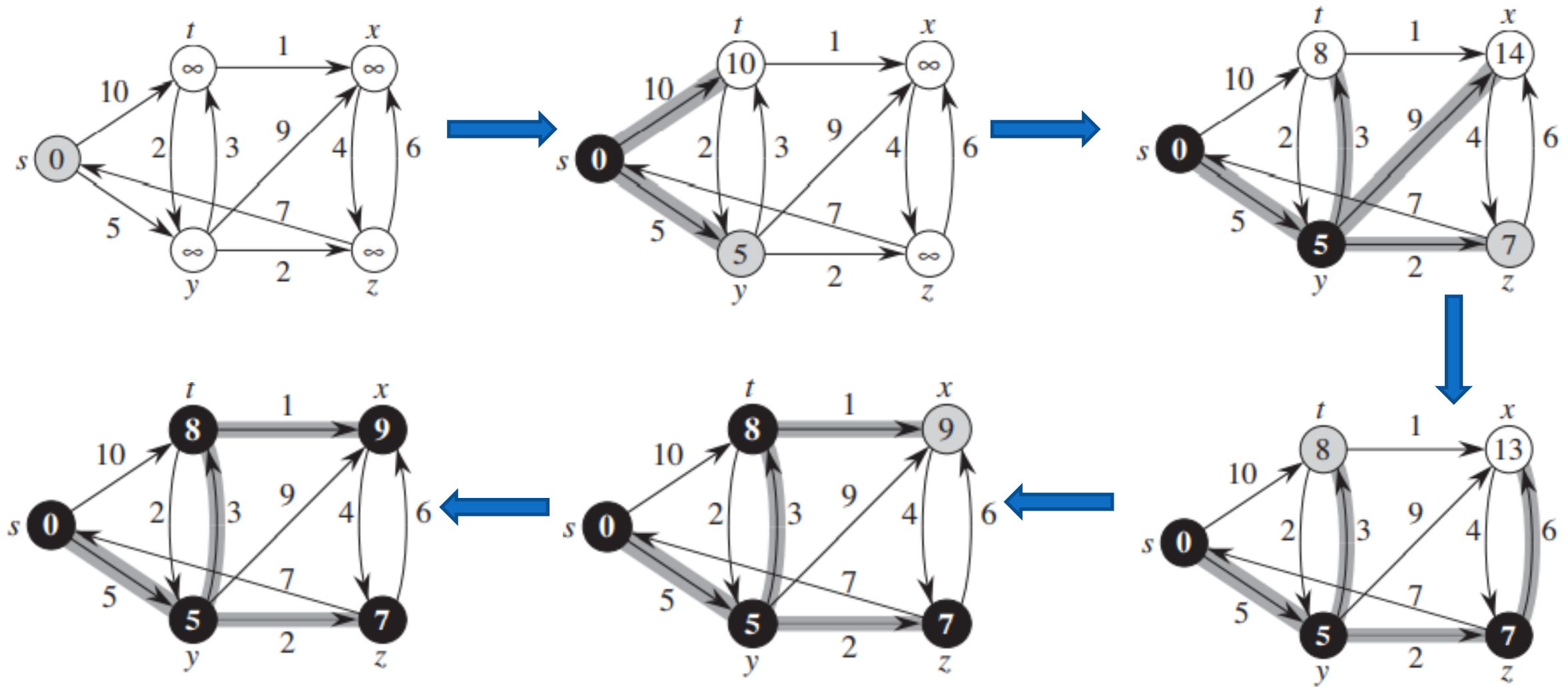
1  if  $v.d > u.d + w(u, v)$ 
2       $v.d = u.d + w(u, v)$ 
3       $v.\pi = u$ 
  
```

In the above algorithm
 S = the set which contains the
 vertices of shortest path,
 s = source vertex

Time complexity = $O(E \log V)$

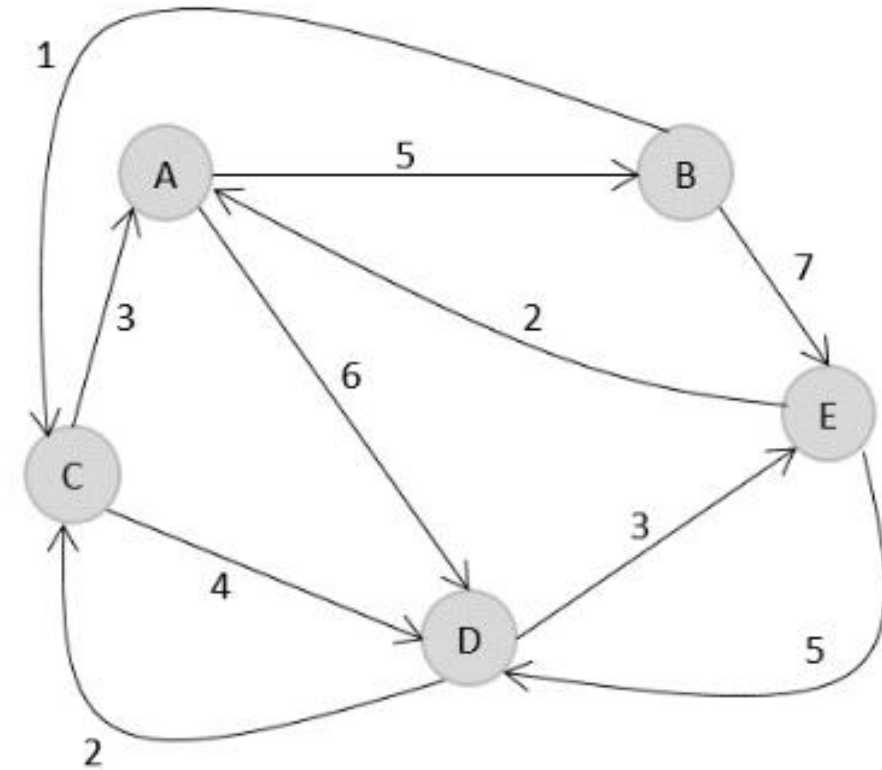
Dijkstra's Algorithm

Example-2



Example 3:

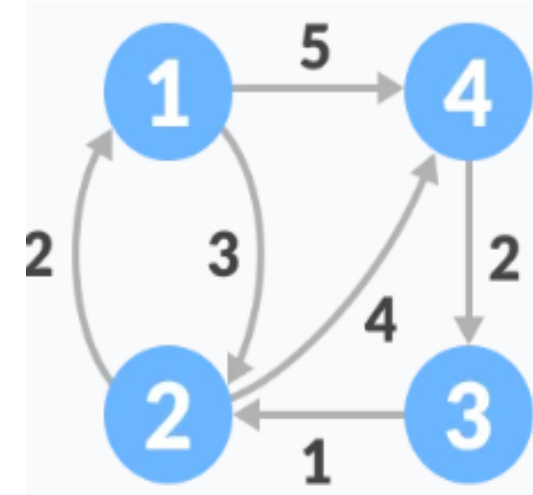
Find the single source shortest path for the given graph by using Dijkstra's algorithm, source vertex is A



Floyd-Warshall's algorithm (All pair shortest path Algorithm)

Step-1: Create a matrix A^0 of dimension $n \times n$ where n is the number of vertices.

Step-2: Each cell $A[i][j]$ is filled with the distance from the i^{th} vertex to the j^{th} vertex.
If there is no path from i^{th} vertex to j^{th} vertex, the cell is left as infinity.



Step-3: Now, create a matrix A^1 using matrix A^0 .
The elements in the first column and the first row are left as they are.
The remaining cells are filled in the following way.

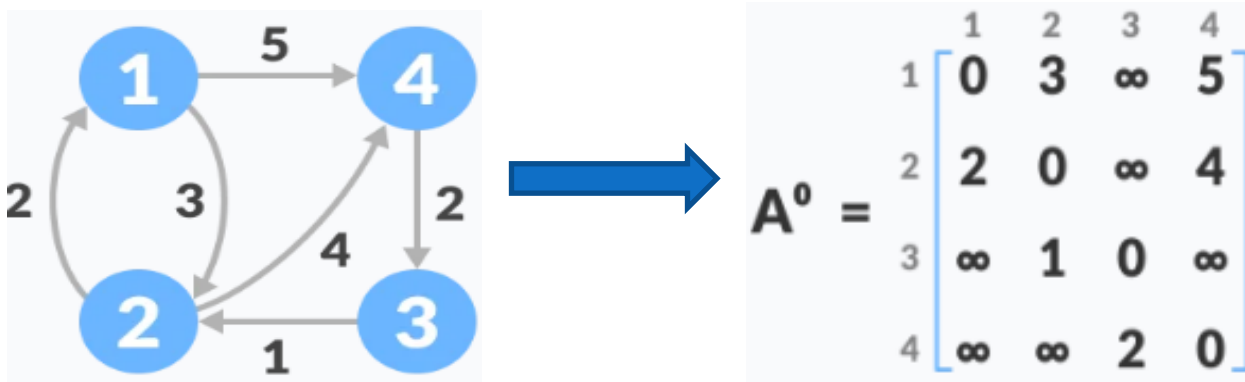
Let k be the intermediate vertex in the shortest path from source to destination.
(k starts from the first vertex).

$A[i][j]$ is filled with $(A[i][k] + A[k][j])$ if $(A[i][j] > A[i][k] + A[k][j])$

If the direct distance from the source to the destination is greater than the path through the vertex k , then the cell is filled with $A[i][k] + A[k][j]$.

Floyd-Warshall's algorithm

Step-1 and Step-2:



$$A^1 = \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{bmatrix} 0 & 3 & \infty & 5 \\ 2 & 0 & & \\ \infty & & 0 & \\ \infty & & & 0 \end{bmatrix} \end{matrix} \rightarrow \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{bmatrix} 0 & 3 & \infty & 5 \\ 2 & 0 & 9 & 4 \\ \infty & 1 & 0 & 8 \\ \infty & \infty & 2 & 0 \end{bmatrix} \end{matrix}$$

Step-3:

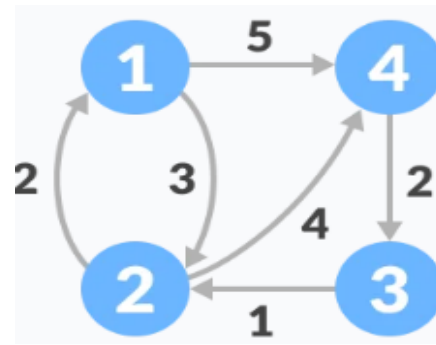
- Create a matrix A^1 using matrix A^0 . The elements in the first column and the first row are left as they are.

$$\begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{bmatrix} 0 & 3 & \infty & 5 \\ 2 & 0 & & \\ \infty & & 0 & \\ \infty & & & 0 \end{bmatrix} \end{matrix}$$

- Calculate the distance from the source vertex to destination vertex through the vertex k (in this step, k is vertex 1).

- For $A^1[2, 3]$, the direct distance from vertex 2 to 3 is ∞ (no edge) and the sum of the distance from vertex 2 to 3 through vertex 1 (from vertex 2 to 1 and from vertex 1 to 4, 4 to 3) is 9.
- Since $9 < \infty$, $A^0[2, 3]$ is filled with 9.

Floyd-Warshall's algorithm



Step-3 contd...

- A^2 is created using A^1 . The elements in the second column and the second row are left as they are.

$$A^2 = \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{bmatrix} 0 & 3 & & \\ 2 & 0 & 9 & 4 \\ & 1 & 0 & \\ & \infty & & 0 \end{bmatrix} \end{matrix}$$

- Calculate the distance from the source vertex to destination vertex through the vertex k (In this step, k is vertex 2).

$$A^2 = \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{bmatrix} 0 & 3 & & \\ 2 & 0 & 9 & 4 \\ & 1 & 0 & \\ & \infty & & 0 \end{bmatrix} \end{matrix} \rightarrow \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{bmatrix} 0 & 3 & 9 & 5 \\ 2 & 0 & 9 & 4 \\ 3 & 1 & 0 & 5 \\ \infty & \infty & 2 & 0 \end{bmatrix} \end{matrix}$$

Step-3 contd...

- A^3 is created using A^2

$$A^3 = \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{bmatrix} 0 & & \infty & \\ & 0 & 9 & \\ \infty & 1 & 0 & 8 \\ & & 2 & 0 \end{bmatrix} \end{matrix} \rightarrow \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{bmatrix} 0 & 3 & 9 & 5 \\ 2 & 0 & 9 & 4 \\ 3 & 1 & 0 & 5 \\ 5 & 3 & 2 & 0 \end{bmatrix} \end{matrix}$$

Step-3 contd...

- A^4 is created using A^3

$$A^4 = \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{bmatrix} 0 & & & 5 \\ & 0 & & 4 \\ & & 0 & 5 \\ 5 & 3 & 2 & 0 \end{bmatrix} \end{matrix} \rightarrow \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{bmatrix} 0 & 3 & 7 & 5 \\ 2 & 0 & 6 & 4 \\ 3 & 1 & 0 & 5 \\ 5 & 3 & 2 & 0 \end{bmatrix} \end{matrix}$$

A^4 gives the shortest path between each pair of vertices.

Floyd-Warshall's algorithm

Step 1: Repeat Step 2 for $I=0$ to $n-1$, where n is the no. of nodes in the graph G

Step 2: Repeat Step 3 for $J=0$ to $n-1$

Step 3: IF $G[I][J] = 0$, then $A[I][J] = \text{Infinity}$

ELSE

$A[I][J] = G[I][j]$

END OF LOOP]

[END OF LOOP]

Step 4: Repeat Step 5 for $K=0$ to $n-1$ [Calculate the shortest path matrix A]

Step 5: Repeat Step 6 for $I=0$ to $n-1$

Step 6: Repeat Step 7 for $J=0$ to $n-1$

Step 7: IF $A[I][J] > A[I][K] + A[K][J]$

$A[I][J] = A[I][K] + A[K][J]$

[END OF IF]

[END OF LOOP]

[END OF LOOP]

[END OF LOOP]

Step 8: EXIT

n = no of vertices, A = matrix of dimension $n*n$

for $k = 1$ to n

for $i = 1$ to n

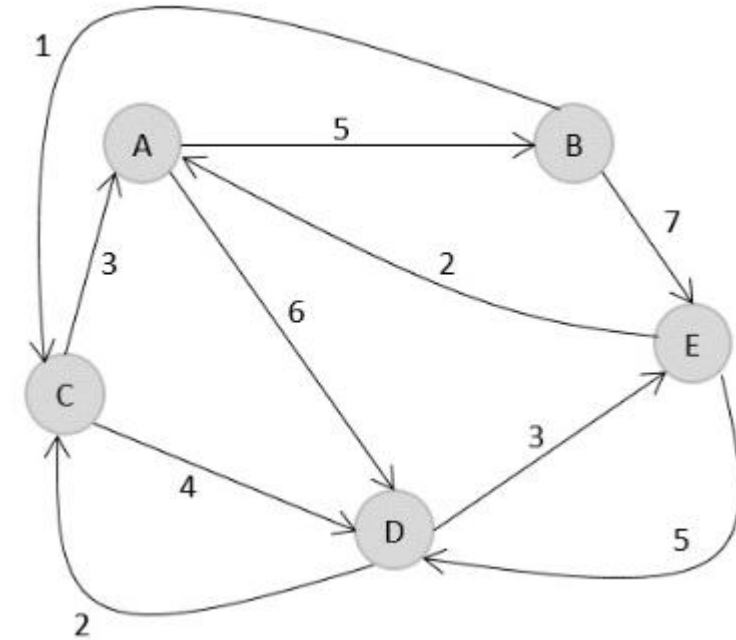
for $j = 1$ to n

$A^k[i, j] = \min(A^{k-1}[i, j], A^{k-1}[i, k] + A^{k-1}[k, j])$

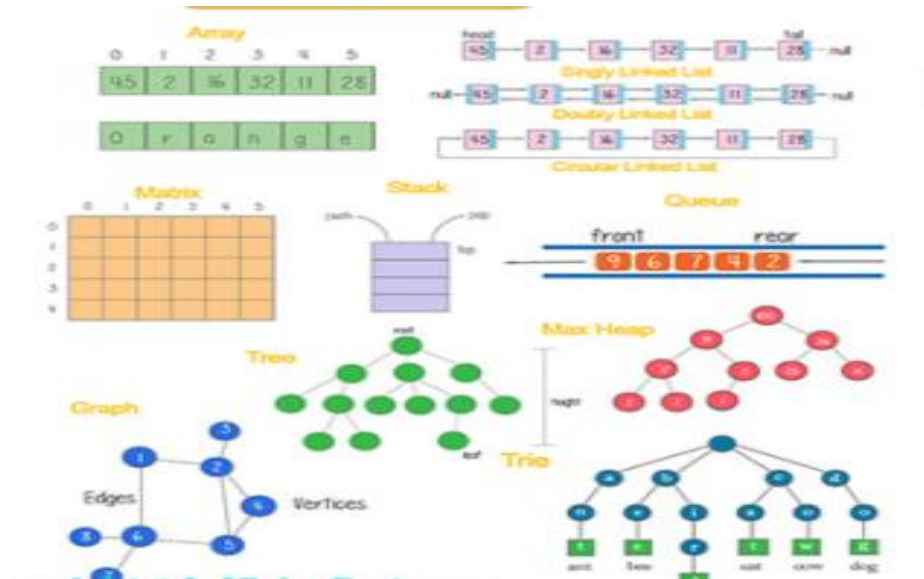
return A

Example 2:

Find the all-pairs shortest path by using Floyd-Warshall's algorithm



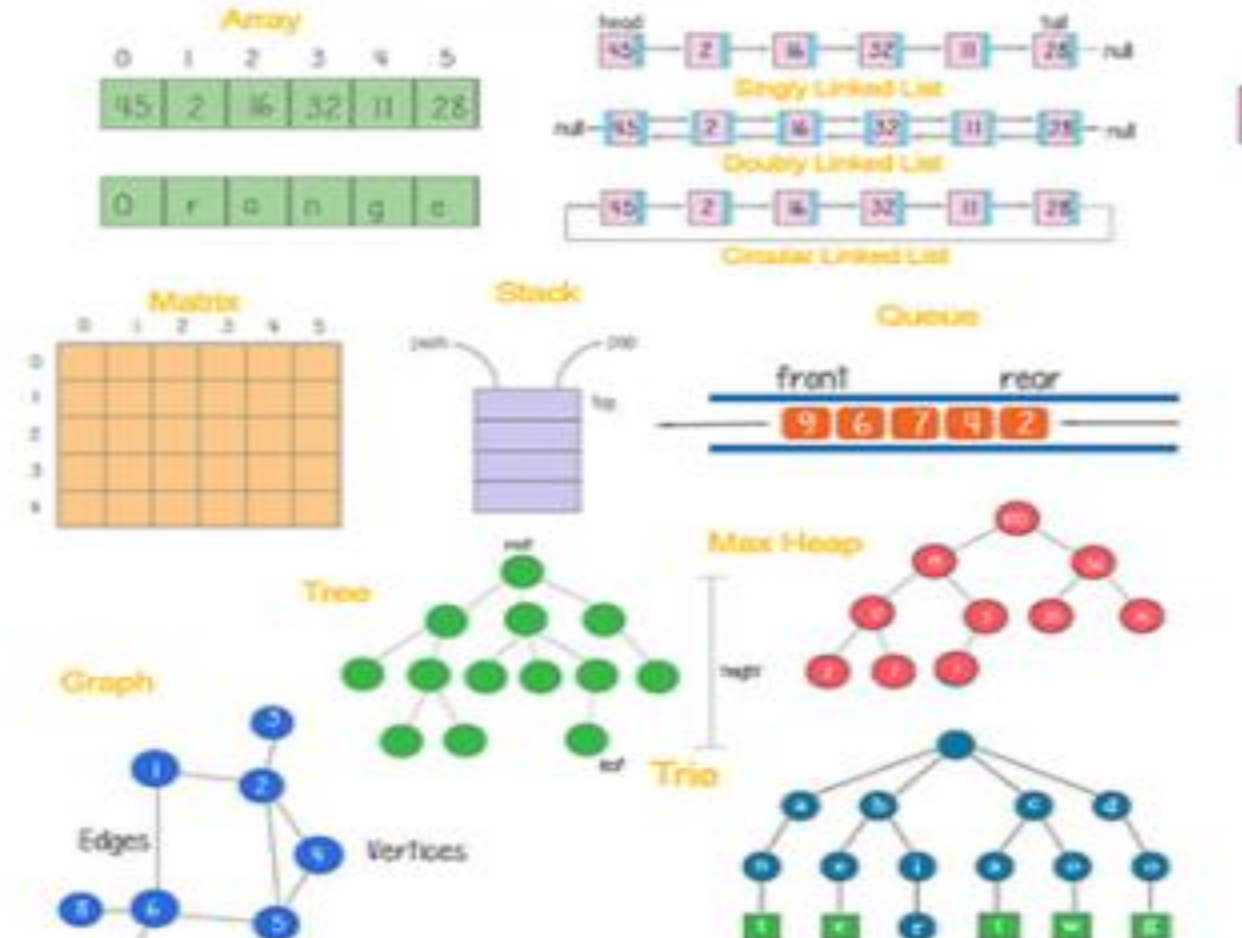
Data Structures



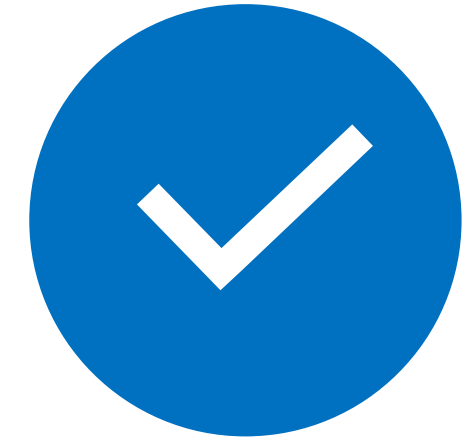
Mr. Pandu Sowkuntla
Asst. Professor,
Dept. of CSE, SRM University AP



Data structures



UNIT-V



SEARCHING AND SORTING

Aspects of Computer Programming (Data structures and Algorithms)

- Data organization also commonly called as data structures.
- Choosing the appropriate algorithm to manipulate the data to solve a problem efficiently.

Searching is used to find the location where an element is available.

- 1. Linear or sequential search
- 2. Binary search or half interval search

Sorting allows an efficient arrangement of elements within a given data structure.

- 1. Selection sort
- 2. Bubble sort
- 3. Insertion sort
- 4. Quick sort
- 5. Heap sort
- 6. Radix sort
- 7. Merge sort

Internal sorting techniques (data sorting process that takes place entirely within the main memory).

External sorting technique (some of the data (elements) to be sorted are kept on the secondary storage).

Searching techniques

Linear Search or Sequential Search

```
int main()
{
    int SIZE = 8, key;
    int a1[8]={8, 4, 5, 3, 2, 9, 4, 1};
    printf("Enter the key to search");
    scanf("%d", &key);
    if(linearSearch(a1, SIZE, key) == 1)
        printf("Search Found");
    else
        printf("Search Not Found");
}
```

Search key is compared with each element of the array linearly.

Linear search has complexity of $O(n)$.

```
int linearSearch(int arr[], int size, int key)
{
    int i;
    for(i = 0; i < size; i++)
        if(arr[i] == key)
            return 1;
    return 0;
}
```

Linear Search or Sequential Search

```
int main()
{
    int SIZE = 8;
    int a1[SIZE]={8, 4, 5, 3, 2, 9, 4, 1};
    linearSearch(a1, SIZE, 8);
    linearSearch(a1, SIZE, 4);
    linearSearch(a1, SIZE, 99);
}
```

```
int linearSearch(int a[], int size, int key)
{
    int flag=0;
    for (int i = 0; i < size; ++i)
    {
        if (a[i] == key)
        {
            flag=1;
            break;
        }
    }
    if(flag==1)
    {
        printf("Key found at %d", i);
        return;
    }
    else
        printf("Key not found");
}
```

Binary Search

- ▶ A **binary search** (or **half-interval search**) is **applicable only to a sorted array**.
- ▶ It compares the **search key** with the **middle element**.
- ▶ If there is a match,
it returns the element's index.
- ▶ If the search key is less than the middle element,
repeat searching on the **left half**;
otherwise, repeat searching on the **right half**.

Binary Search

Array

10	20	35	45	67	78	90	100
----	----	----	----	----	----	----	-----

Key

67

Start Index

End Index

mid

arr[mid]

0

1

2

3

4

5

6

7

0

7

$(0+7) / 2 = 3$

45

10

20

35

45

67

78

90

100

45 is less than 67. So skip first half array. Now start index will be mid + 1. i.e
 $3 + 1 = 4$

4

7

$(4+7) / 2 = 5$

78

67

78

90

100

78 is greater than 67. So skip second half array. Now end index will be mid - 1. i.e
 $5 - 1 = 4$

4

4

$(4+4) / 2 = 4$

67

67

Search Found

Binary Search

Array

10	20	35	45	67	78	90	100
----	----	----	----	----	----	----	-----

Key

50

Start Index

End Index

mid

arr[mid]

0

1

2

3

4

5

6

7

0

7

$(0+7) / 2 = 3$

45

10

20

35

45

67

78

90

100

45 is less than 50. So skip first half array. Now start index will be mid + 1. i.e
 $3 + 1 = 4$

4

7

$(4+7) / 2 = 5$

78

67

78

90

100

78 is greater than 50. So skip second half array. Now end index will be mid - 1. i.e
 $5 - 1 = 4$

4

4

$(4+4) / 2 = 4$

67

67

67 greater than 50. so end 4 - 1 which is 3

4

3

Invalid index range.

Search Not Found

```
int BinarySearch(int arr[],int start,int end,int key)
{
    int mid;
    while(start <= end)
    {
        mid = (start + end) / 2;
        if(arr[mid] == key)
            return 1;

        if(arr[mid] < key)
            start = mid + 1;
        else
            end = mid - 1;
    }
    return 0;
}
```

```
int main()
{
    int SIZE=8, key;
    int a[SIZE] = {10,23,45,70,90,100,111,123};
    printf("Enter the key to search");
    scanf("%d", &key)

    if(BinarySearch(a,0,SIZE - 1, key) == 1)
        printf("Search Found\n");
    else
        printf("Search Not Found\n");

}
```

Sorting techniques

Selection Sort

Method:

- ▶ First smallest element is selected from the unsorted array and placed at the first position.
- ▶ After that second smallest element is selected and placed in the second position.
- ▶ The process continues until the array is entirely sorted.

Unsorted array: {8, 4, 5, 3, 2, 9, 4, 1}

Assumed smallest element Actual smallest element

{ } {8, 4, 5, 3, 2, 9, 4, 1} => { } {1, 4, 5, 3, 2, 9, 4, 8}

Swap elements

{1} {4, 5, 3, 2, 9, 4, 8} => {1} {2, 5, 3, 4, 9, 4, 8}

{1, 2} {5, 3, 4, 9, 4, 8} => {1, 2} {3, 5, 4, 9, 4, 8}

{1, 2, 3} {5, 4, 9, 4, 8} => {1, 2, 3} {4, 5, 9, 4, 8}

{1, 2, 3, 4} {5, 9, 4, 8} => {1, 2, 3, 4} {4, 9, 5, 8}

{1, 2, 3, 4, 4} {9, 5, 8} => {1, 2, 3, 4, 4} {5, 9, 8}

{1, 2, 3, 4, 4, 5} {9, 8} => {1, 2, 3, 4, 4, 5} {8, 9}

{1, 2, 3, 4, 4, 5, 8, 9}

Selection Sort

```
int main()
{
    int SIZE=8;
    int a[SIZE] = {8,4,5,3,2,9,4,1},i;
    selectionSort(a,SIZE);
}
```

Time complexity = $O(n^2)$.

Note:

Stable sorting algorithms preserve the relative order of equal elements, while unstable sorting algorithms don't.

In other words, stable sorting maintains the position of two equals elements relative to one another.

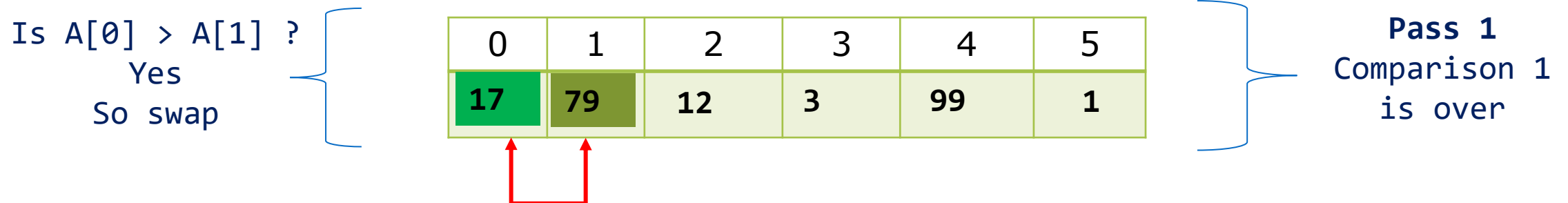
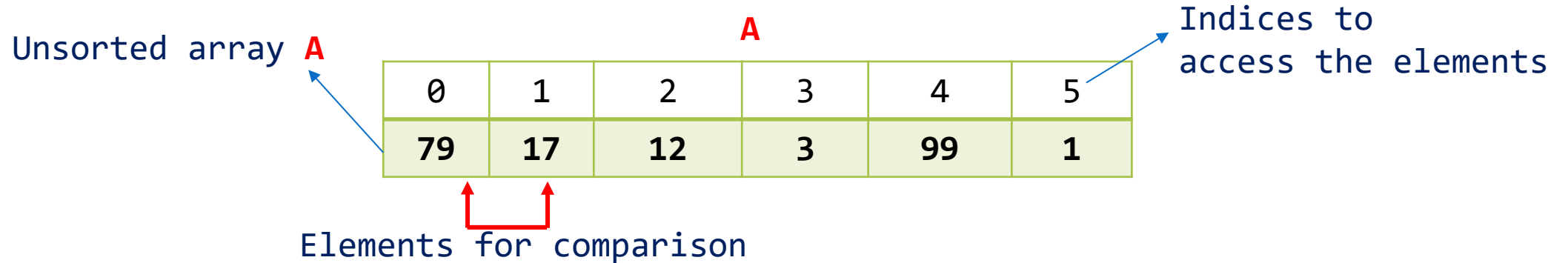
Selection sort is stable algorithm

```
void selectionSort(int a[], int size)
{
    int temp; // for swapping
    for (int i = 0; i < size - 1; ++i)
    {
        int minIndex = i; // assume first element is
                           // the smallest
        for (int j = i + 1; j < size; ++j)
        {
            if (a[j] < a[minIndex])
                minIndex = j;
        }
        if (minIndex != i)
        {
            temp = a[i];
            a[i] = a[minIndex];
            a[minIndex] = temp;
        }
    }
    printf("After selection sort\n");
    for(i = 0; i < size; i++)
        printf("%d ",a[i]);
}
```

Bubble Sort

Method:

- ▶ Pass through the list,
- ▶ Compare two adjacent elements
swap them if they are in the wrong order.
- ▶ Repeat the pass until no swaps are needed.



Bubble Sort

Is $A[1] > A[2]$?

Yes

So swap these two

0	1	2	3	4	5
17	12	79	3	99	1



Is $A[2] > A[3]$?

Yes

So swap these two

0	1	2	3	4	5
17	12	3	79	99	1



Is $A[3] > A[4]$?

No

Don't swap

0	1	2	3	4	5
17	12	3	79	99	1



Is $A[4] > A[5]$?

Yes

So swap these two

0	1	2	3	4	5
17	12	3	79	1	99



Pass 1
Comparison 2
is over

Pass 1
Comparison 3
is over

Pass 1
Comparison 4
is over

Pass 1
Comparison 5
is over

Bubble Sort

Is $A[0] > A[1]$?
Yes
So swap these two

Is $A[1] > A[2]$?
Yes
So swap these two

Is $A[2] > A[3]$?
No
Don't swap

Is $A[3] > A[4]$?
Yes
So swap these two

Is $A[4] > A[5]$?
No
Don't swap

0	1	2	3	4	5
12	17	3	79	1	99



0	1	2	3	4	5
12	3	17	79	1	99



0	1	2	3	4	5
12	3	17	79	1	99



0	1	2	3	4	5
12	3	17	1	79	99



0	1	2	3	4	5
12	3	17	1	79	99



Pass 2
Comparison 1
is over

Pass 2
Comparison 2
is over

Pass 2
Comparison 3
is over

Pass 2
Comparison 4
is over

Pass 2
Comparison 5
is over

Bubble Sort

Is $A[0] > A[1]$?
Yes
So swap these two

Is $A[1] > A[2]$?
No
Don't swap

Is $A[2] > A[3]$?
Yes
So swap these two

Is $A[3] > A[4]$?
No
Don't swap

Is $A[4] > A[5]$?
No
Don't swap

0	1	2	3	4	5
3	12	17	1	79	99



0	1	2	3	4	5
3	12	17	1	79	99



0	1	2	3	4	5
3	12	1	17	79	99



0	1	2	3	4	5
3	12	1	17	79	99



0	1	2	3	4	5
3	12	1	17	79	99



Pass 3
Comparison 1
is over

Pass 3
Comparison 2
is over

Pass 3
Comparison 3
is over

Pass 3
Comparison 4
is over

Pass 3
Comparison 5
is over

Bubble Sort

0	1	2	3	4	5
3	12	1	17	79	99



Pass 4
Comparison 1
is over

0	1	2	3	4	5
3	1	12	17	79	99



Pass 4
Comparison 2
is over

0	1	2	3	4	5
3	1	12	17	79	99



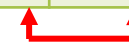
Pass 4
Comparison 3
is over

0	1	2	3	4	5
3	1	12	17	79	99



Pass 4
Comparison 4
is over

0	1	2	3	4	5
3	12	1	17	79	99



Pass 4
Comparison 5
is over

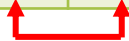
Bubble Sort

0	1	2	3	4	5
1	3	12	17	79	99



Pass 5
Comparison 1
is over

0	1	2	3	4	5
1	3	12	17	79	99



Pass 5
Comparison 2
is over

0	1	2	3	4	5
1	3	12	17	79	99



Pass 5
Comparison 3
is over

0	1	2	3	4	5
1	3	12	17	79	99



Pass 5
Comparison 4
is over

0	1	2	3	4	5
1	3	12	17	79	99



Pass 5
Comparison 5
is over

Bubble Sort

Method:

- Pass through the list,
- Compare two adjacent items
swap them if they are in the wrong order.
- Repeat the pass until no swaps are needed.

Unsorted array: {8,4,5,3,2,9,4,1}

PASS 1 ...

{8,4,5,3,2,9,4,1} => {4,8,5,3,2,9,4,1}
 {4,8,5,3,2,9,4,1} => {4,5,8,3,2,9,4,1}
 {4,5,8,3,2,9,4,1} => {4,5,3,8,2,9,4,1}
 {4,5,3,8,2,9,4,1} => {4,5,3,2,8,9,4,1}
 {4,5,3,2,8,9,4,1} => {4,5,3,2,8,4,9,1}
 {4,5,3,2,8,4,9,1} => {4,5,3,2,8,4,1,9}

PASS 2 ...

{4,5,3,2,8,4,1,9} => {4,3,5,2,8,4,1,9}
 {4,3,5,2,8,4,1,9} => {4,3,2,5,8,4,1,9}
 {4,3,2,5,8,4,1,9} => {4,3,2,5,4,8,1,9}
 {4,3,2,5,4,8,1,9} => {4,3,2,5,4,1,8,9}

PASS 3 ...

{4,3,2,5,4,1,8,9} => {3,4,2,5,4,1,8,9}
 {3,4,2,5,4,1,8,9} => {3,2,4,5,4,1,8,9}
 {3,2,4,5,4,1,8,9} => {3,2,4,4,5,1,8,9}
 {3,2,4,4,5,1,8,9} => {3,2,4,4,1,5,8,9}

PASS 4 ...

{3,2,4,4,1,5,8,9} => {2,3,4,4,1,5,8,9}
 {2,3,4,4,1,5,8,9} => {2,3,4,1,4,5,8,9}

PASS 5 ...

{2,3,4,1,4,5,8,9} => {2,3,1,4,4,5,8,9}

PASS 6 ...

{2,3,1,4,4,5,8,9} => {2,1,3,4,4,5,8,9}

PASS 7 ...

{2,1,3,4,4,5,8,9} => {1,2,3,4,4,5,8,9}

Sorted array: {1,2,3,4,4,5,8,9}

Bubble Sort

```
void bubbleSort(int arr[])
{
    int i,j;
    for{ j=1; j <= n-1; i++)// Pass number
    {
        for{ i=1; i <= n-1-j; i++)// Comparison
        {
            if(A[i-1] > A[i])
            {
                temp = A[i-1];
                A[i-1] = A[i];
                A[i] = temp;
            }
        }
    }
}
```

Number of passes in bubble sort with
“n” elements is (n-1)

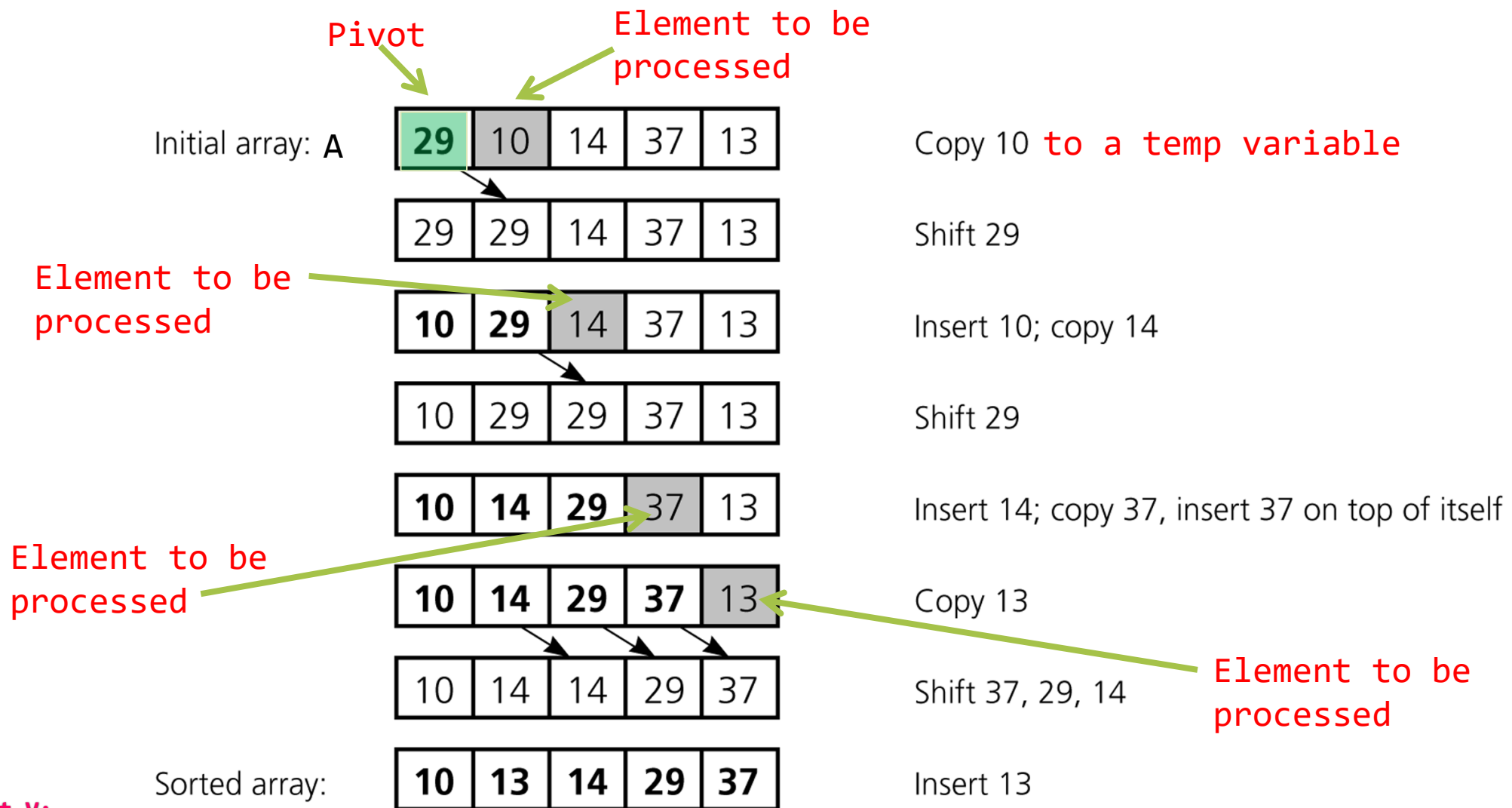
Time complexity = $O(n^2)$.

Bubble sort is stable algorithm

Insertion Sort

■ While some elements are unsorted ...

- Using linear search, find the location in the sorted portion where the 1st element of the unsorted portion should be inserted.
- Move all the elements after the insertion location up one position to make space for the new element.



```
void main()
{
    int n, arr[1000], i, d, temp;
    printf("Enter number of elements\n");
    scanf("%d", &n);
    printf("Enter %d integers\n", n);
    for (i = 0; i < n; i++)
        scanf("%d", &arr[i]);
    for (i = 1; i <= n - 1; i++) {
        d = i;
        while (d > 0 && arr[d-1] > arr[d]) {
            temp = arr[d];
            arr[d] = arr[d-1];
            arr[d-1] = temp;
            d--;
        }
    }
    printf("Sorted array in ascending order:\n");
    for (i = 0; i <= n - 1; i++) {
        printf("%d\n", arr[i]);
    }
}
```

Initial array:



Sorted array:



Time complexity is $O(n^2)$

Insertion sort is stable algorithm

Quick Sort

Quicksort is a **divide and conquer algorithm**.

It divides the large array into smaller sub-arrays.

And then quicksort recursively sort the sub-arrays.

Pivot

1. Picks an element called the **"pivot"**.

There are many ways to choose the **pivot element**.

i) The first element or second or middle or **end element** in the array

ii) We can also pick the element randomly.

Partition

2. Rearrange the array elements such that the all values lesser than the pivot should come before the pivot and all the values greater than the pivot should come after it.

At the end of the partition, the pivot element will be placed at its sorted position.

Recursive

3. Do the above process recursively to all the sub-arrays and sort the elements.

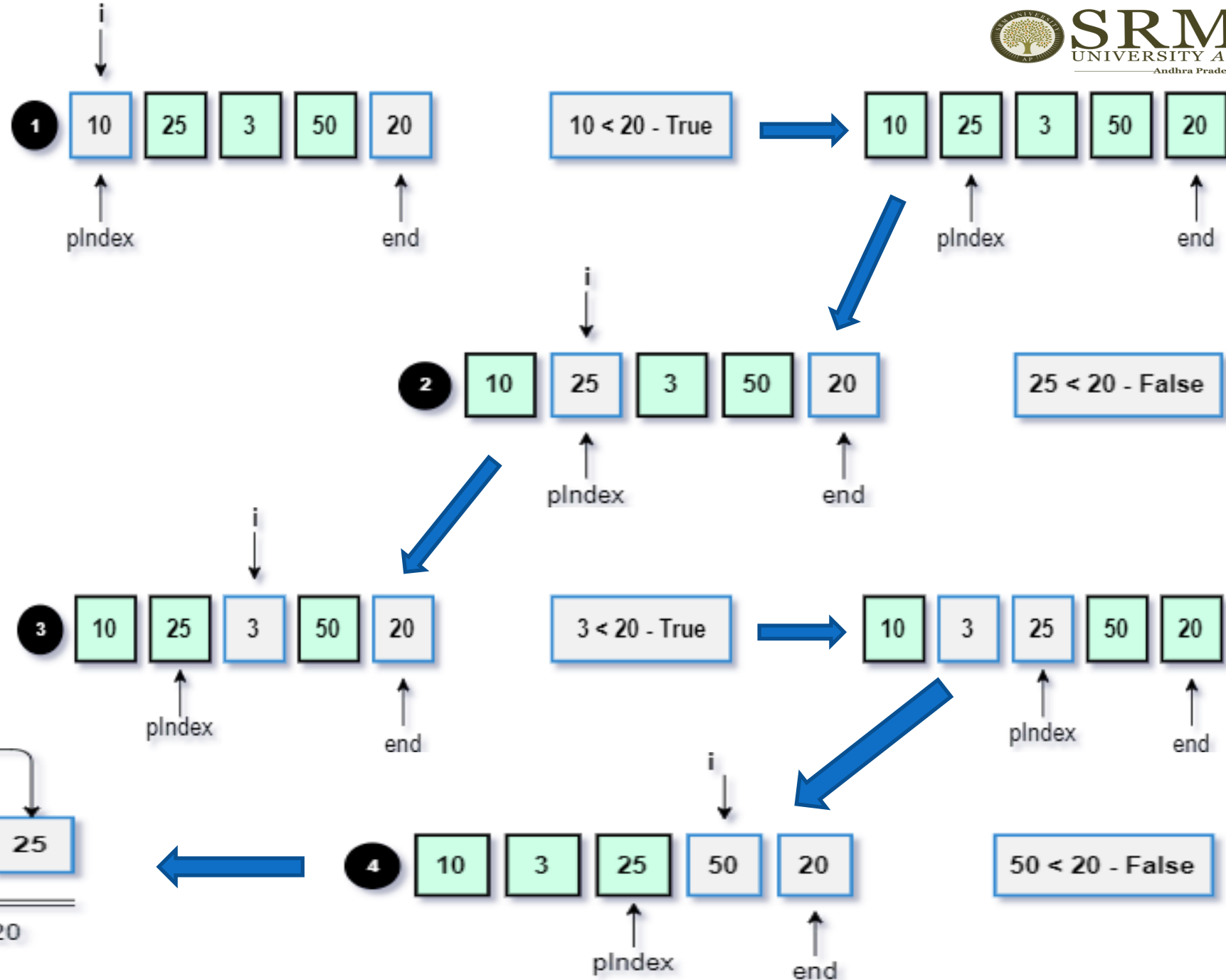
Base Case

If the array has zero or one element, there is no need to call the partition method.

Quick Sort

Example

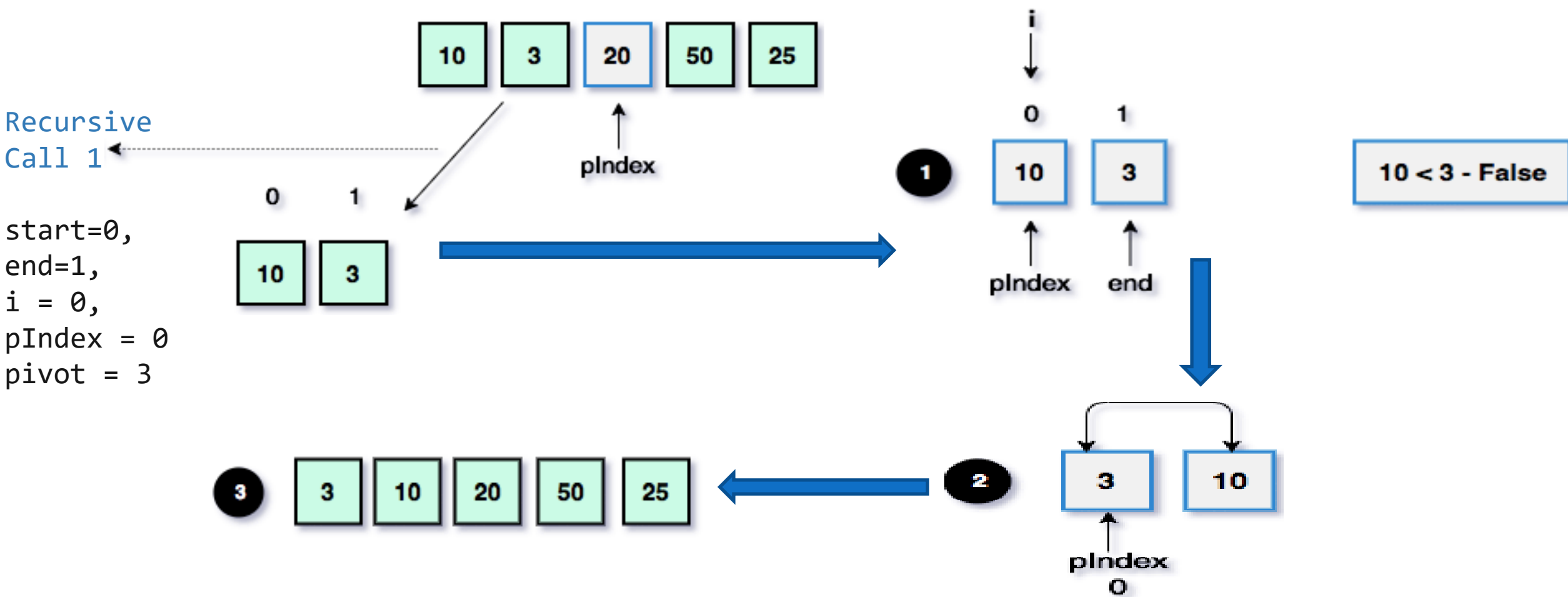
`arr[5] = {10, 25, 3, 50, 20}`
`start = 0, end = 4,`
`pindex = 0, pivot=arr[4]=20`



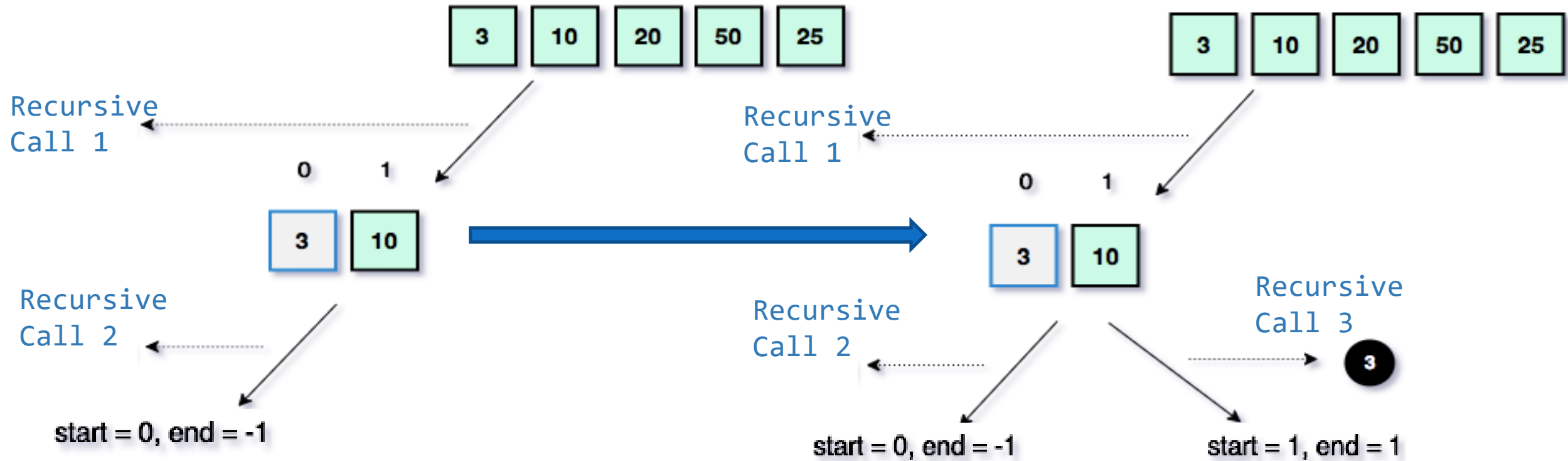
Quick Sort

Recursive calls:

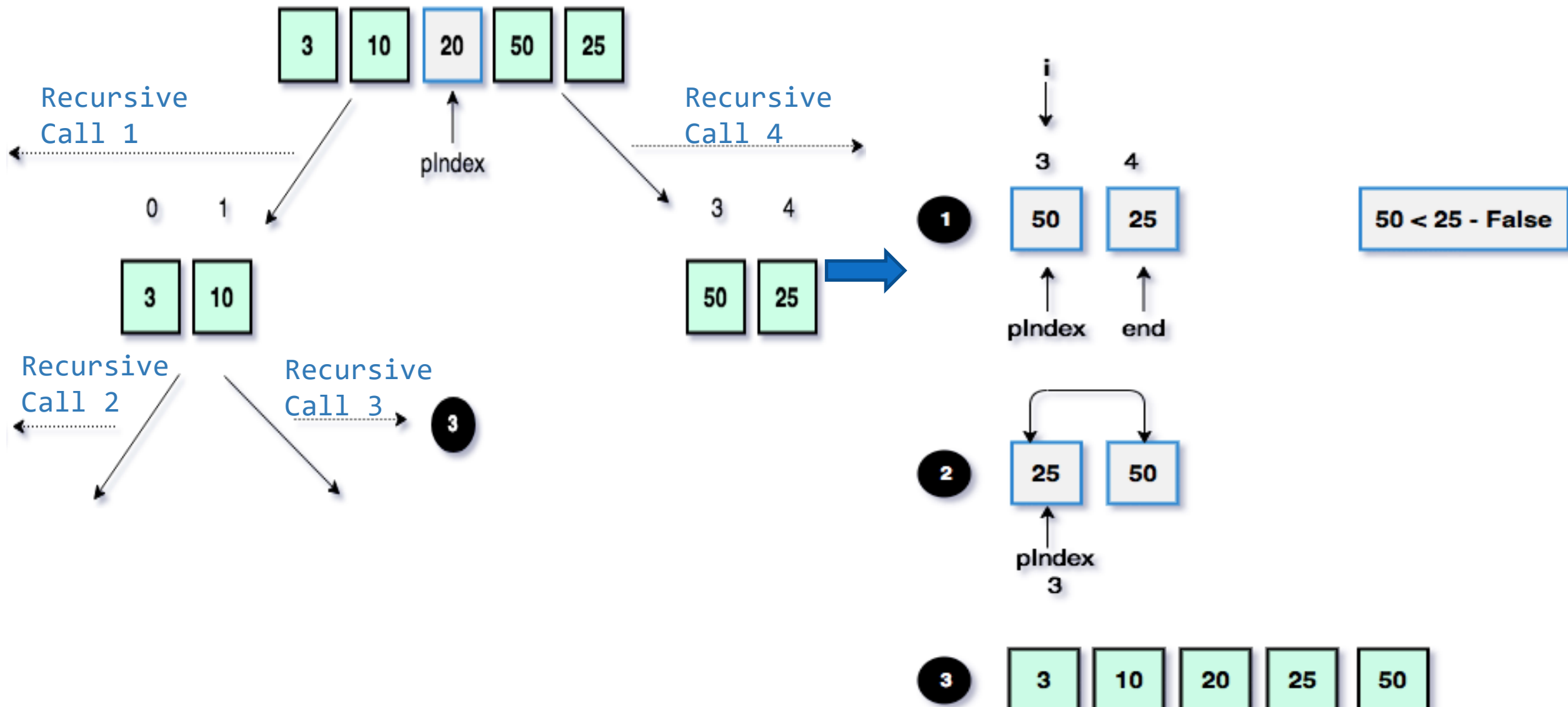
After splitting the array into 2 partitions, quicksort algorithm called recursively on each sub array



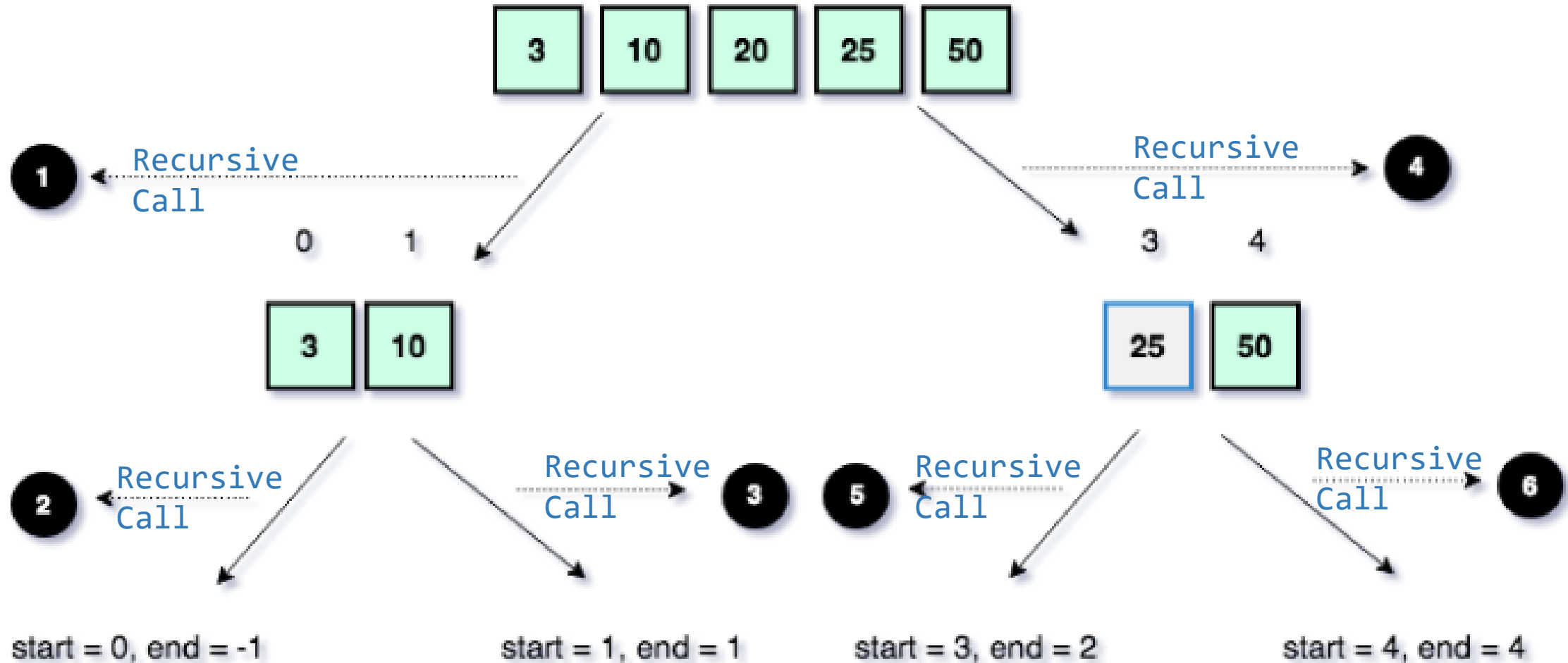
Quick Sort



Quick Sort



Quick Sort



Sorted array: {3, 10, 20, 25, 50}

Quick Sort

```
void quickSort(int arr[], int start, int end){
    if(start < end)
    {
        int pIndex = partition(arr, start, end);
        quickSort(arr, start, pIndex-1);
        quickSort(arr, pIndex+1, end);
    }
}

int partition(int arr[], int start, int end){
    int pIndex = start;
    int pivot = arr[end];
    int j;
    for(j = start; j < end; j++){
        if(arr[j] < pivot)
        {
            swap(arr[j], arr[pIndex]);
            pIndex++;
        }
    }
    swap(arr[end], arr[pIndex]);
    return pIndex;
}
```

```
int main(){
    int n,i ;
    printf("Enter Array Size\n");
    scanf("%d",&n);
    int arr[n];
    printf("Enter Array Elements\n");

    for(i=0;i<n;i++)
        scanf("%d",&arr[i]);

    quickSort(arr,0,n-1);
    printf("After the quickSort\n");

    for(i=0;i<n;i++)
        printf("%d ",arr[i]);

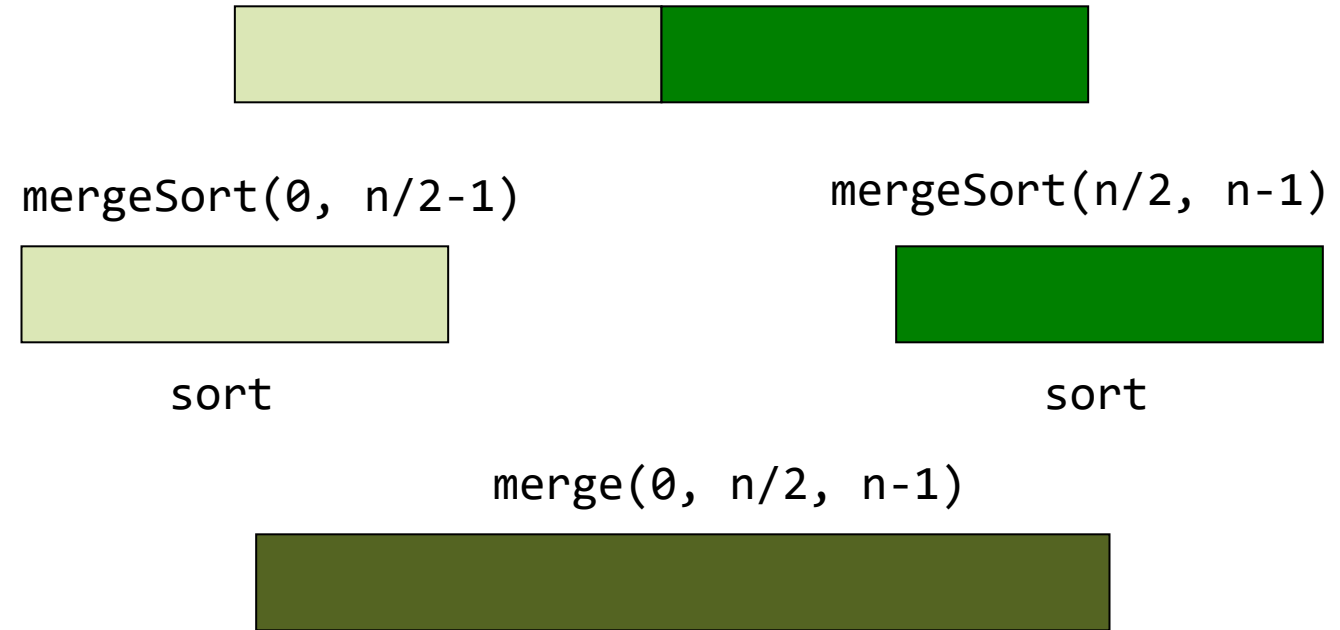
    printf("\n");
    return 0;
}
```

Merge Sort

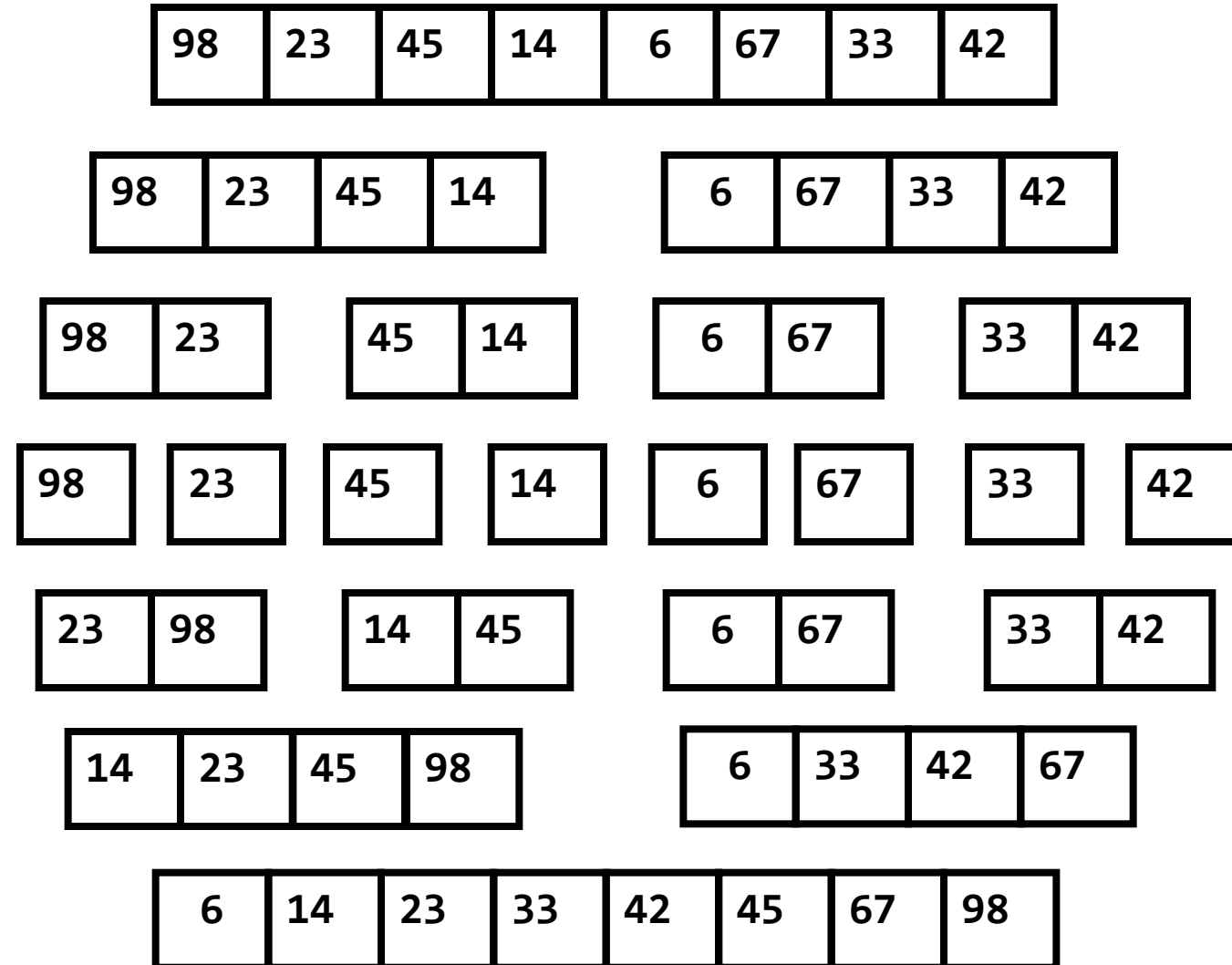
Method :

A "divide and conquer" algorithm

- Divides the array into two roughly equal parts
- Recursively divide each part in half, continuing until a part contains only one element
- Recursively sort the two halves, and merge the two parts into one sorted array.
- Continue to merge parts as the recursion unfolds

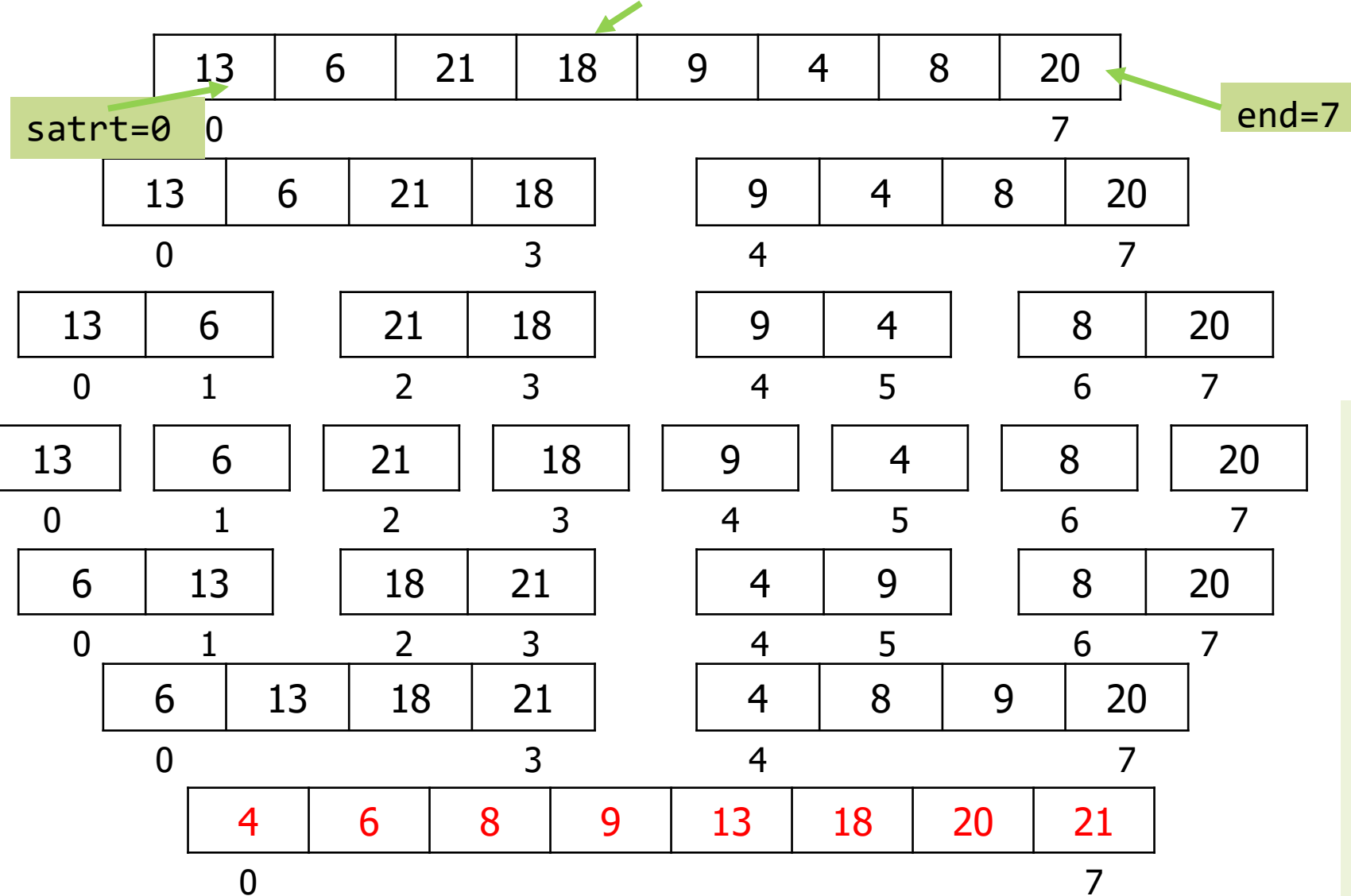


Merge Sort



Merge Sort

$$M = \text{floor}((L+H)/2)$$



Algorithm:

```
mergeSort(a[], start, end)
{
    if(start < end)
    {
        mid = floor((start+end)/2)
        mergeSort(a, start, mid)
        mergeSort(a, mid+1, end)
        merge(a, start, mid, mid+1, end)
    }
}
```

- *Merge operation:*
 - Given two sorted arrays, *merge* operation produces a sorted array with all the elements of the two arrays

A	6	13	18	21	B	4	8	9	20
C	4	6	8	9	13	18	20	21	

Running time of *merge*: $O(n)$, where n is the number of elements in the merged array.

Algorithm for merging two given sorted arrays:

```
Merge()
{
While( i < len1  && j < len2)
{
    if(a[i] < a[j])
    {
        mArr[k++] = a[i++];
    }
    else
    {
        mArr[k++] = a[j++];
    }
}
// Copy rest of the elements as it is...
While( i < len1)
    mArr[k++] = a[i++];
While( j < len2)
    mArr[k++] = a[j++];
}
```

Merge Sort

```
int main()
{
    int a[30],n,i;
    printf("Enter no of elements:");
    scanf("%d",&n);
    printf("Enter array elements:");

    for(i=0;i<n;i++)
        scanf("%d",&a[i]);

    mergesort(a,0,n-1);

    printf("\nSorted array is :");
    for(i=0;i<n;i++)
        printf("%d ",a[i]);

    return 0;
}

void mergesort(int a[],int i,int j)
{
    int mid;

    if(i<j)
    {
        mid=(i+j)/2;
        mergesort(a,i,mid); //left recursion
        mergesort(a,mid+1,j); //right recursion
        merge(a,i,mid,mid+1,j); //merging of two
    }
}
```

```
void merge(int a[],int i1,int j1,int i2,int j2)
{
    int temp[50]; //array used for merging
    int i,j,k;
    i=i1; //beginning of the first list
    j=i2; //beginning of the second list
    k=0;

    while(i<=j1 && j<=j2) //while elements in l
    {
        if(a[i]<a[j])
            temp[k++]=a[i++];
        else
            temp[k++]=a[j++];
    }

    while(i<=j1) //copy remaining elements of
        temp[k++]=a[i++];

    while(j<=j2) //copy remaining elements of
        temp[k++]=a[j++];

    //Transfer elements from temp[] back to a[]
    for(i=i1,j=0;i<=j2;i++,j++)
        a[i]=temp[j];
}
```

Merge Sort

Algorithm:

MergSort(A, L, H)

```
{
    if(L<H)
    {
        mid = floor((L+H)/2))
        MergSort(L, mid)      → T(n/2)
        MergSort(mid+1, H)   → T(n/2)
        Merge(L,H)           → T(n)
    }
}
```

- Divide the unsorted collection into two ($O(\log n)$)
-
- Until the sub-arrays only contain one element ($O(n)$)
- Then merge the sub-problem solutions together
- Total runtime $O(n \log n)$.

Note: An in-place algorithm is an algorithm that does not need an extra space and produces an output in the same memory that contains the data by transforming the input 'in-place'. However, a small constant extra space used for variables is allowed.

Merge Sort is not in-place algorithm, selection, bubble, insertion, quick, heap sort algorithms are in-place algorithms

In Radix sort, digit by digit sorting is performed starting from the least significant digit to the most significant digit.

Most significant digit ← 735 → Least significant digit

Radix sort works similar to the sorting of names, according to the alphabetical order.

0-9 (10) buckets required for sorting the numbers

A-Z (26) buckets required for sorting the numbers

Radix Sort

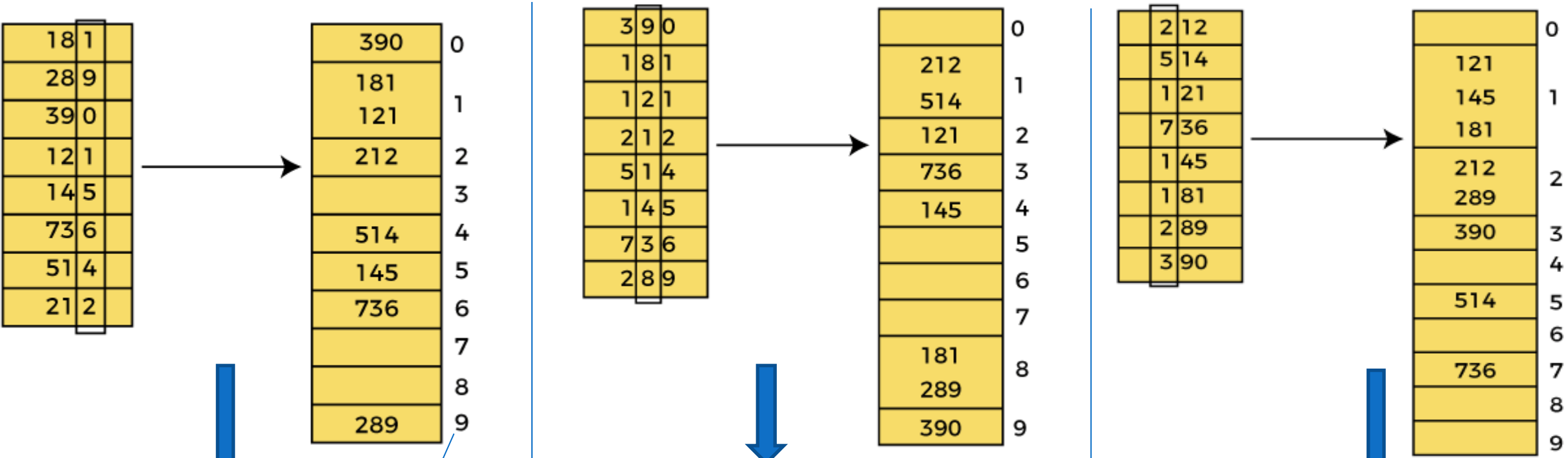
Unsorted array:

181	289	390	121	145	736	514	212
-----	-----	-----	-----	-----	-----	-----	-----

0-9 buckets required

Largest element: 736, three passes require

First pass: Second pass: Third pass:



Buckets

After the second pass:

212	514	121	736	145	181	289	390
-----	-----	-----	-----	-----	-----	-----	-----

After the first pass:

390	181	121	212	514	145	736	289
-----	-----	-----	-----	-----	-----	-----	-----

After the third pass:

121	145	181	212	289	390	514	736
-----	-----	-----	-----	-----	-----	-----	-----

Sorted array

Binary Heaps

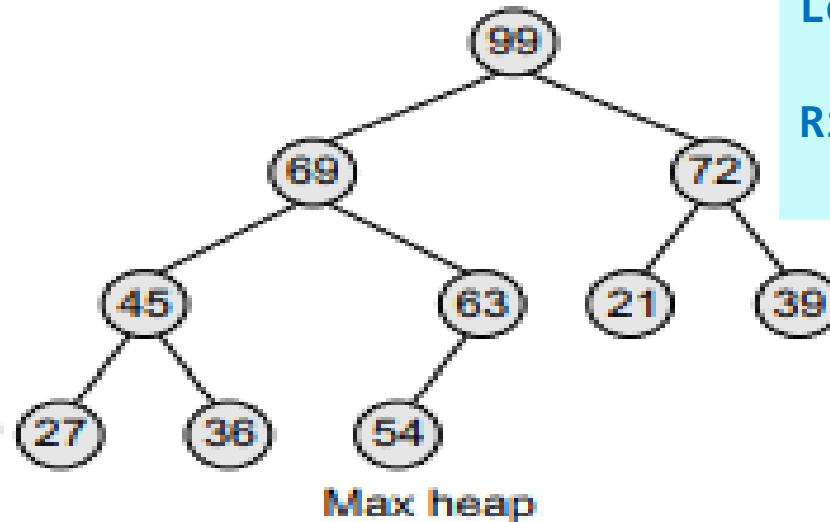
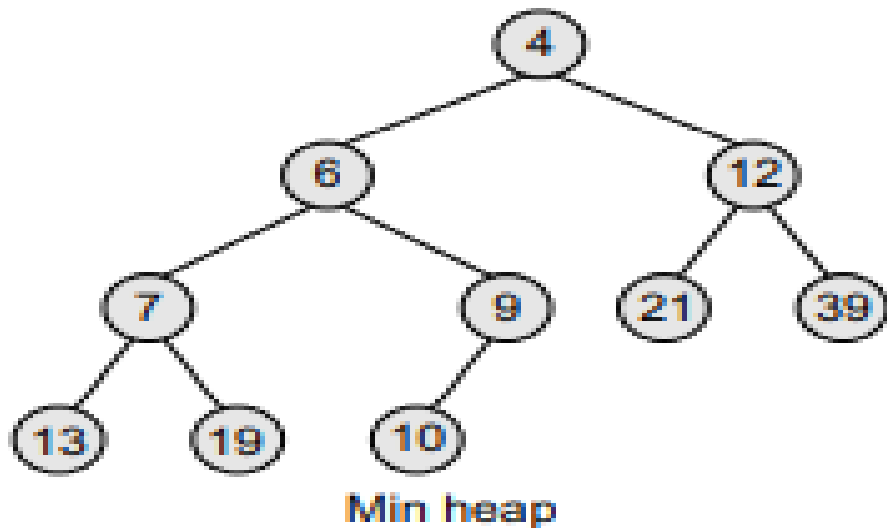
► A binary heap is a complete binary tree in which every node satisfies the heap property which states that:

- If B is a child of A, then $\text{key}(A) \geq \text{key}(B)$ (max-heap property)
- If B is a child of A, then $\text{key}(A) \leq \text{key}(B)$ (min-heap property)

Parent of $a[k] = a[(k-1)/2]$

Left child of $a[k] = a[2*k+1]$

Right child of $a[k] = a[2*k+2]$



Elements can be added **randomly** but only the element with the **highest value** is **removed** in case of max heap and **lowest value** in case of **min heap**.

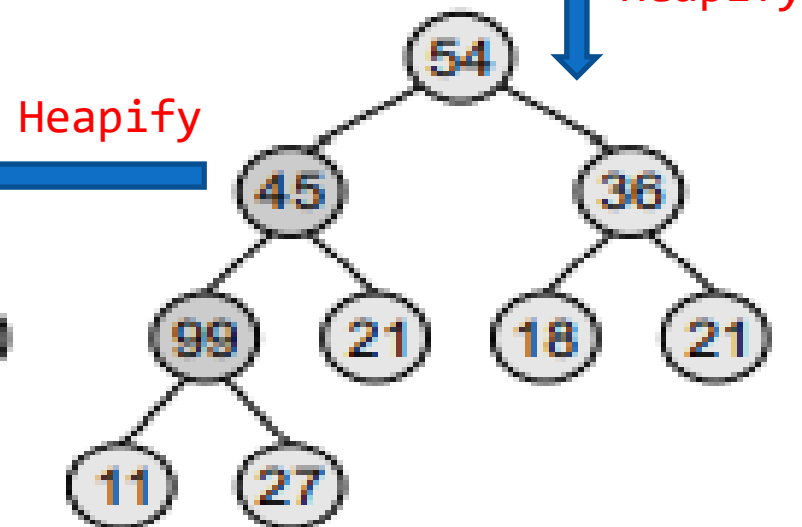
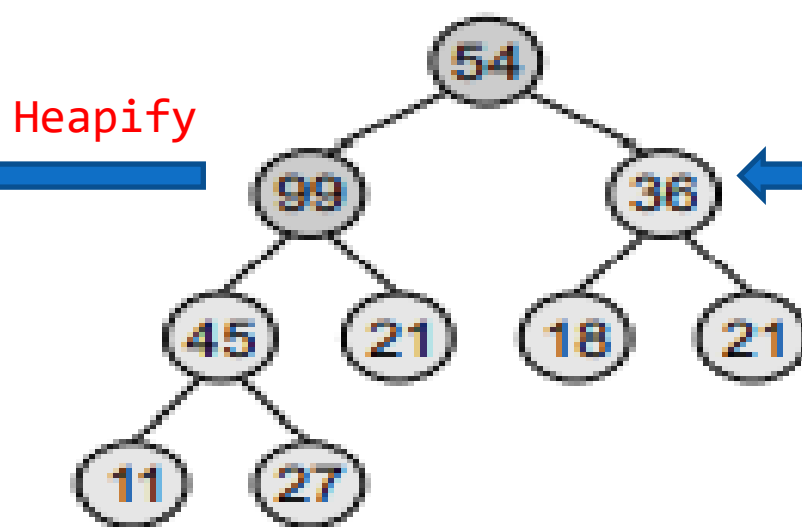
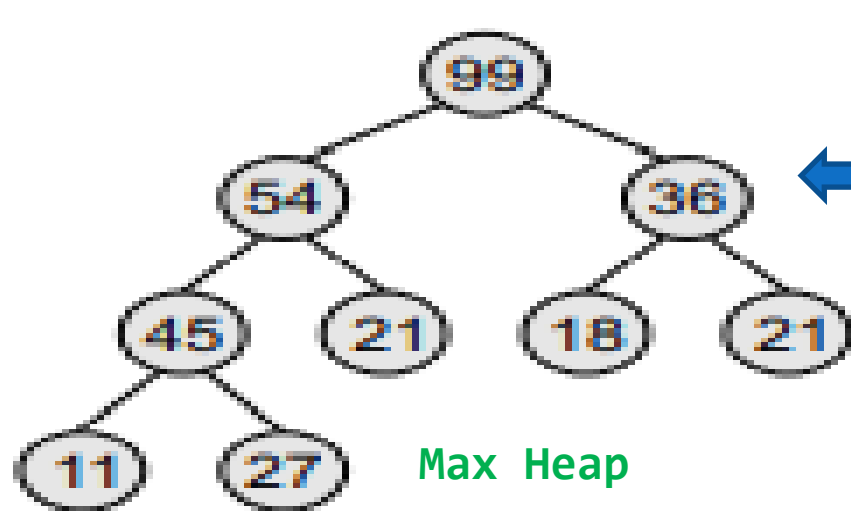
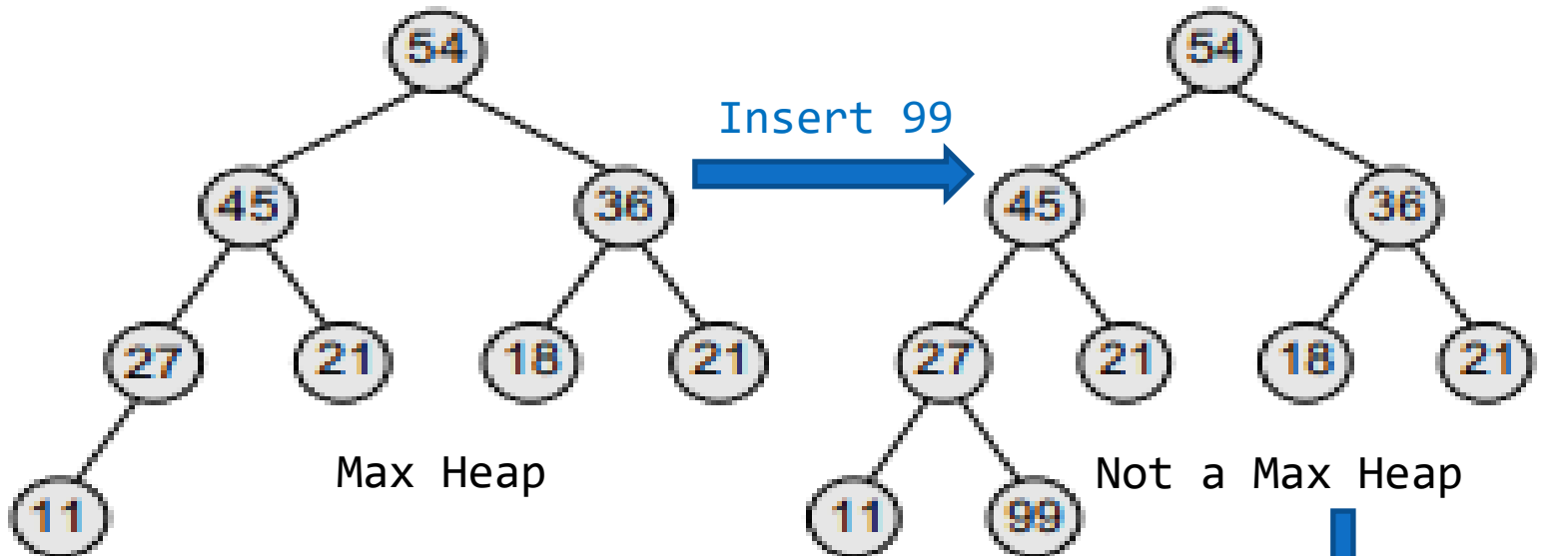
Binary Heaps

Inserting a new element in a Binary Heap (H)

Method:

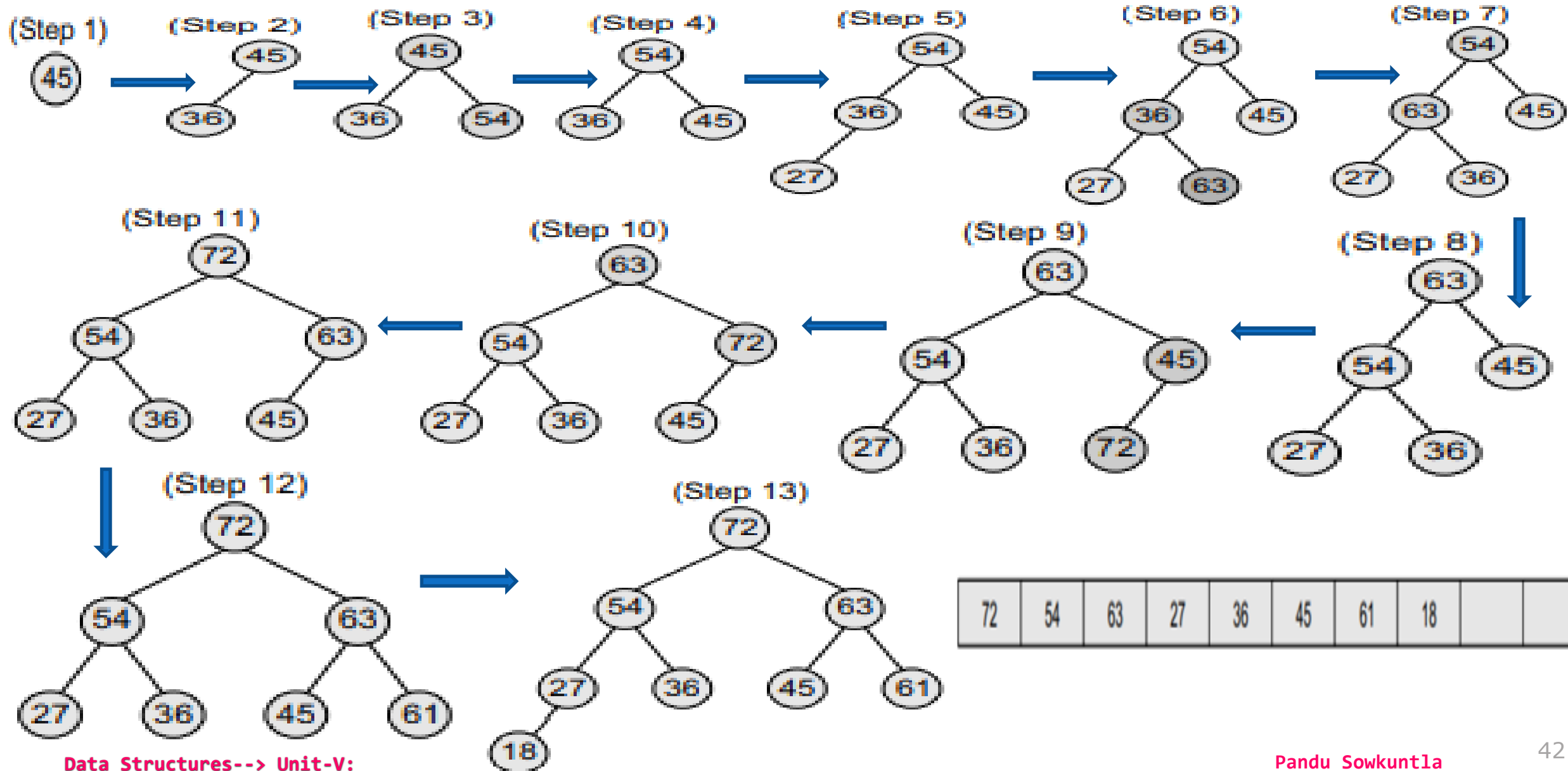
1. Add the new value at the bottom of H in such that H is still a complete binary tree but not necessarily a heap.

2. Let the new value rise to its appropriate place in H so that H now becomes a heap as well.



Binary Heaps

Build a max heap H from numbers: 45, 36, 54, 27, 63, 72, 61, 18.



Binary Heaps

Deleting an element from a Binary Heap (H)

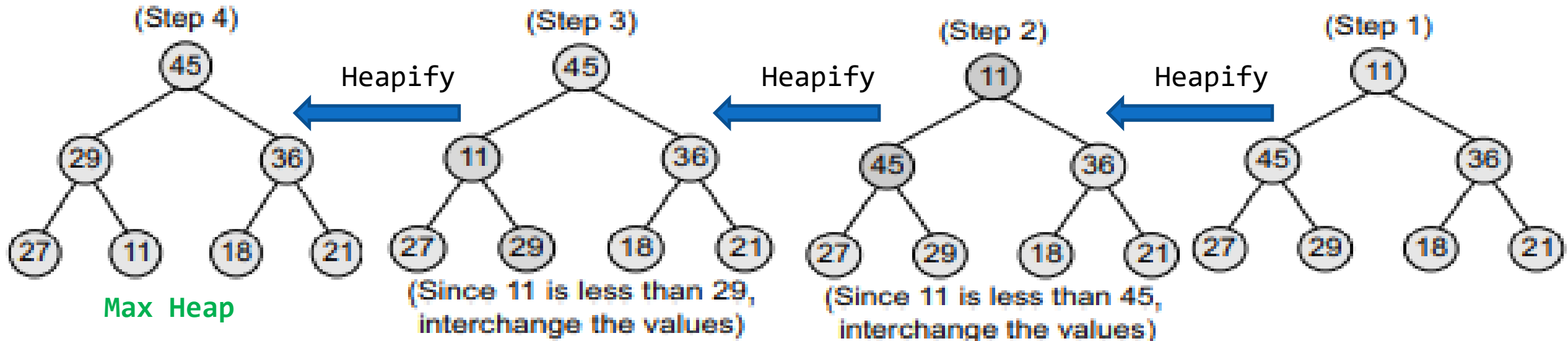
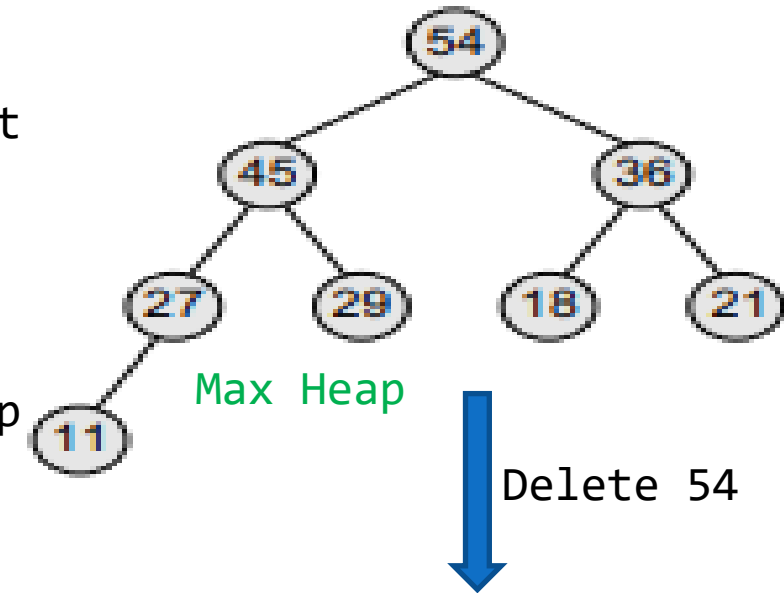
Method (Max Heap):

1. Replace the root node's value with the last node's value so that H is still a complete binary tree but not necessarily a heap.

1. Delete the root node.

2. Sink down the new root node's value so that H satisfies the heap property.

(interchange the root node's value with its child node's value).



Heap sort

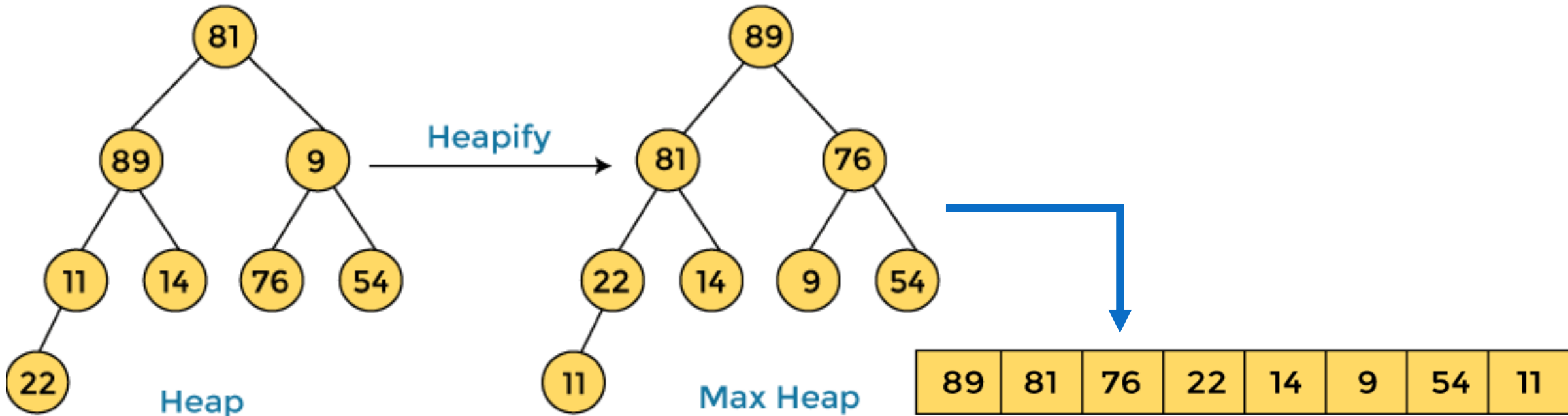
Two phases involved in the sorting of elements

1. Construct a heap by adjusting the array elements.
2. Once the heap is created repeatedly eliminate the root element of the heap by shifting it to the end of the array and then heapify the remaining elements.

Unsorted array:

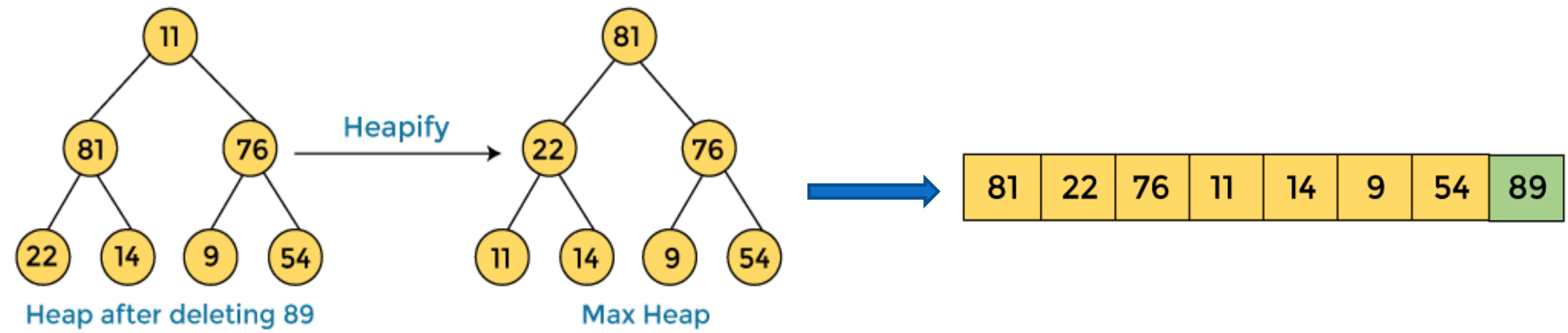
81	89	9	11	14	76	54	22
----	----	---	----	----	----	----	----

Construct a heap from the given array and convert it into max heap.

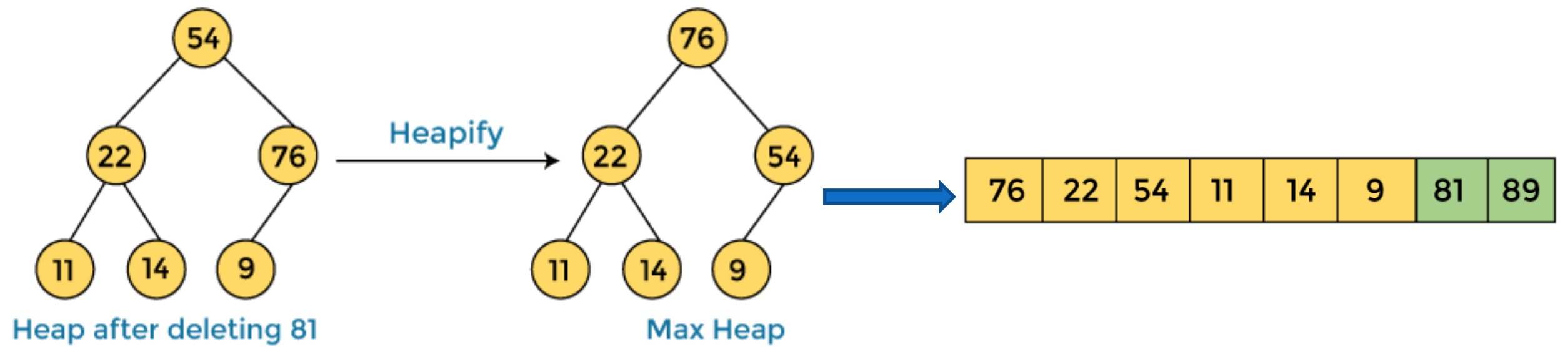


Heap sort

1. To delete the root element **89** from the max heap we have to swap it with the last node **11**.

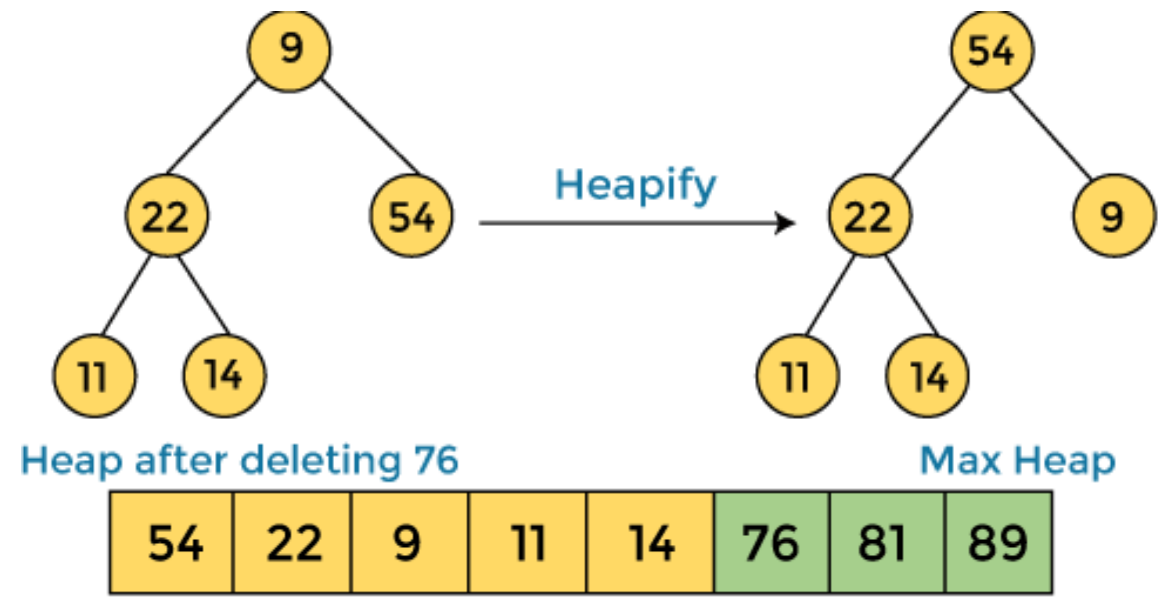


2. To delete **81** node, we have to swap it with the last node **54**.

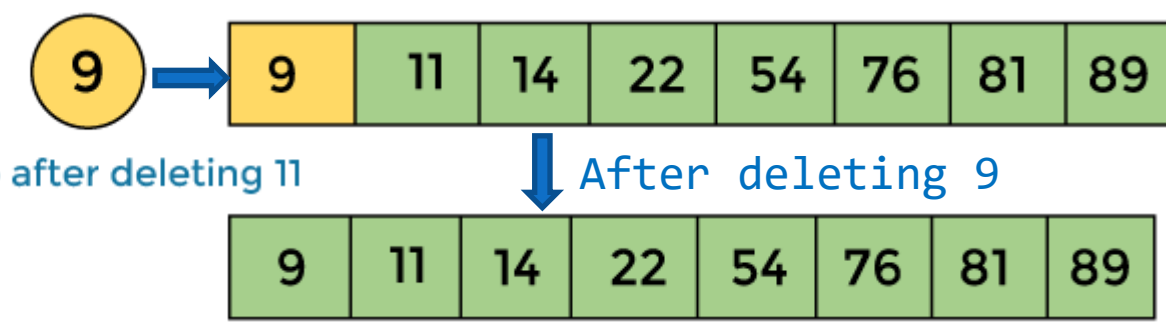
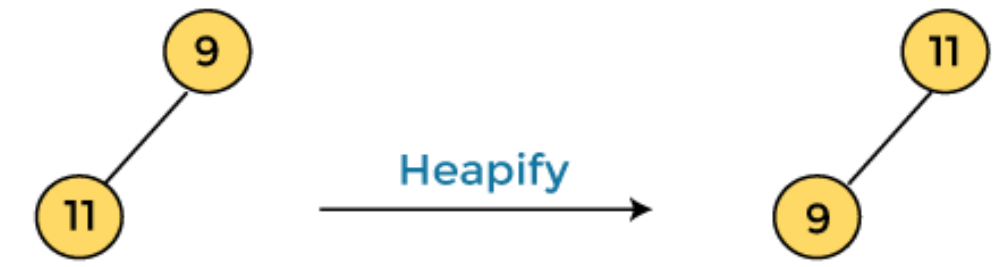
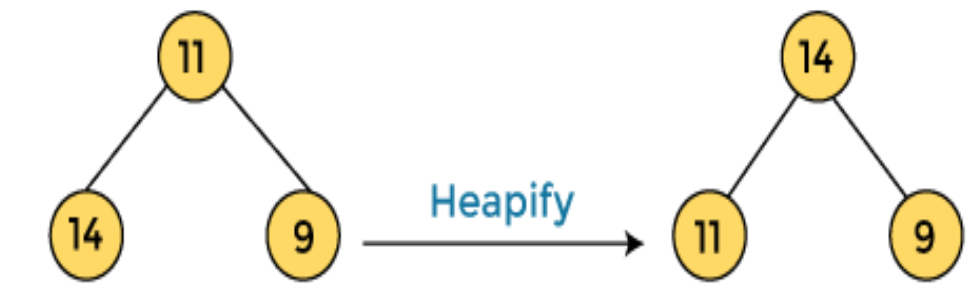
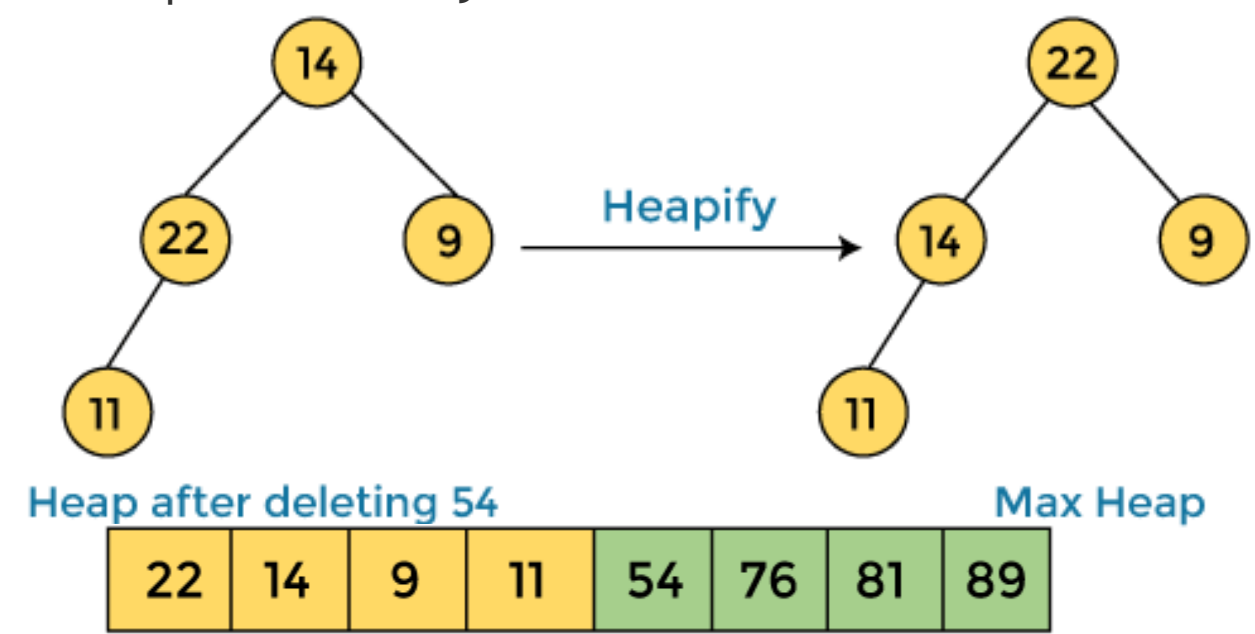


Heap sort

3. Swap 76 and 9, then delete 76



4. Swap 54 and 14, then delete 54



Time complexities of different algorithms

Algorithm	Runtime
Linear Search	$O(n)$
Binary Search	$O(\log n)$
Bubble Sort	$O(n^2)$
Insertion Sort	$O(n^2)$
Selection Sort	$O(n^2)$
Merge Sort	$O(n \log n)$
Quick Sort	$O(n^2)$
Heapsort	$O(n \log n)$
Radix sort	$O(n * k)$

Why hashing ?

Linear search has a running time proportional to $O(n)$

Binary search takes time proportional to $O(\log n)$

Is there a way to search an array in constant time $O(1)$, irrespective of its size?

Key	Array of Employees' Records
Key 0 → [0]	Employee record with Emp_ID 0
Key 1 → [1]	Employee record with Emp_ID 1
Key 2 → [2]	Employee record with Emp_ID 2
.....	
.....	
Key 98 → [98]	Employee record with Emp_ID 98
Key 99 → [99]	Employee record with Emp_ID 99

For single digit Emp_ID, we need an array of size 100, of which all 100 elements will be used.

Key	Array of Employees' Records
Key 00000 → [0]	Employee record with Emp_ID 00000
.....	
Key n → [n]	Employee record with Emp_ID n
.....	
Key 99998 → [99998]	Employee record with Emp_ID 99998
Key 99999 → [99999]	Employee record with Emp_ID 99999

For a five-digit Emp_ID, we need an array of size 100,000, of which only 100 elements will be used.

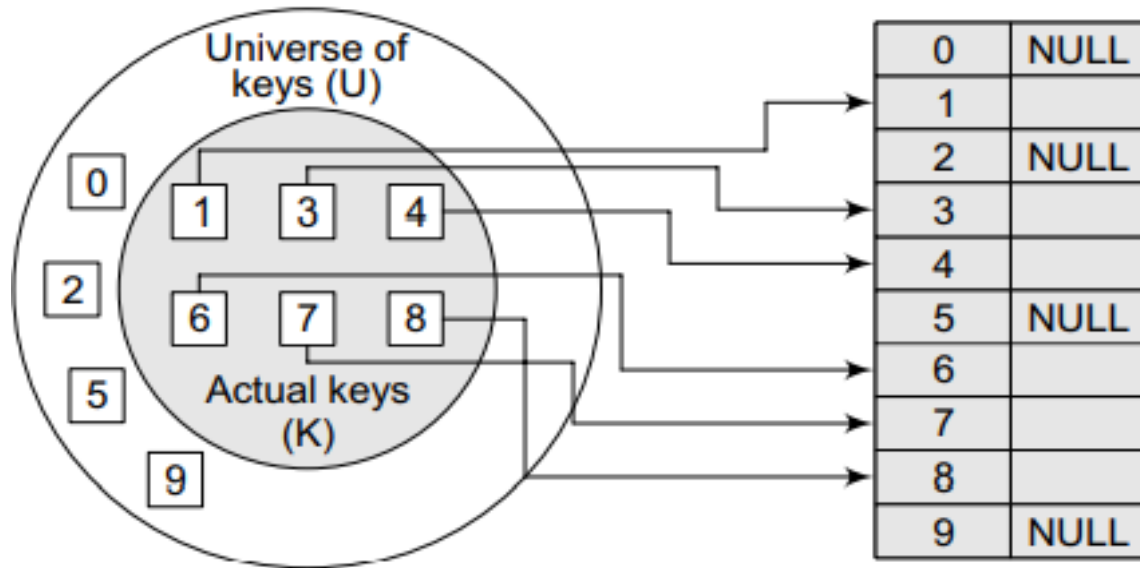
Whether we use a two-digit primary key (Emp_ID) or a five-digit key, there are just 100 employees in the company. Thus, we will be using only 100 locations in the array.

Solution:

- ▶ Elements are not stored according to the **value** of the **key**.
- ▶ We need a way to convert a **five-digit key** number to a **two-digit array index**.
- ▶ We need a **function** which will do the transformation.
- ▶ The term **hash table** is used for an array and the function that will carry out the transformation will be called a **hash function**.

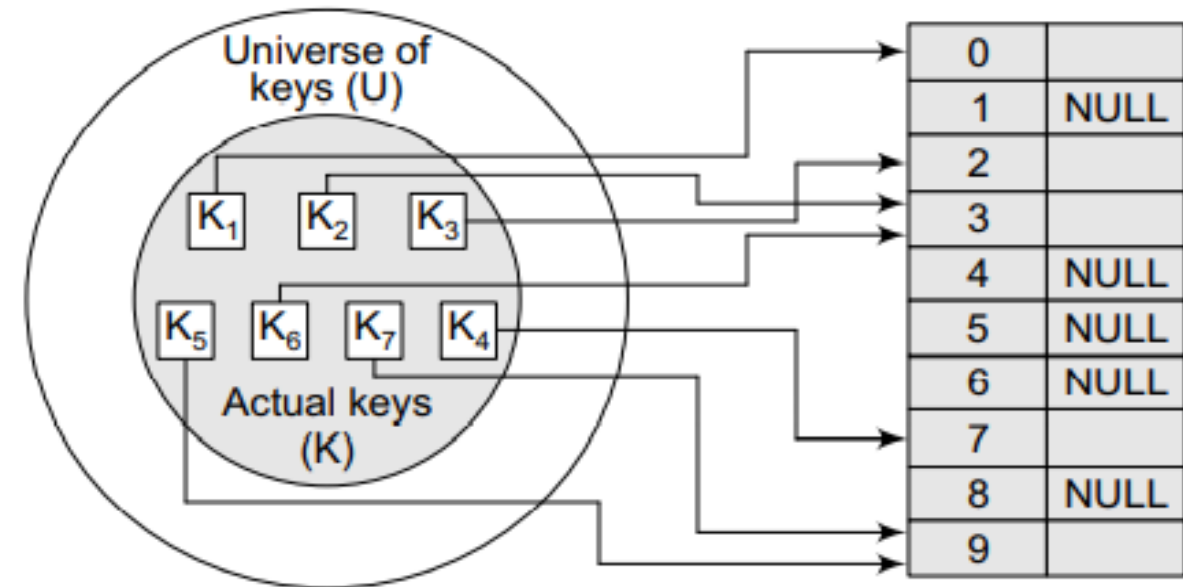
Hash table is a data structure in which keys are mapped to array positions by a hash function.

A **hash function** is a mathematical formula which, when applied to a key, produces an integer which can be used as an index for the key in the hash table.



Direct relationship between key and index in the array

- Universe of keys is small and when most of the keys are actually used from the whole set of keys.



Indirect Relationship between keys and hash table index

- In a hash table, an element with key k is stored at index $h(k)$ and not k .
- A hash function h is used to calculate the index at which the element with key k will be stored.
- The process of mapping the keys to appropriate locations (or indices) in a hash table is called hashing.

Different hashing functions

1. Division Method:

It is the most simple method of hashing an integer x .

This method divides x by M and then uses the remainder obtained.

The hash function can be given as $h(x) = x \bmod M$

Example: Hash values of keys 1234 and 5462.

$M = 97$,

$h(1234) = 1234 \% 97 = 70$

$h(5642) = 5642 \% 97 = 16$

Extra care should be taken to select a suitable value for M .

For example, suppose M is an even number then $h(x)$ is even if x is even and $h(x)$ is odd if x is odd

The division method will not spread the hashed values uniformly.

It is **best to choose M to be a prime number** because making M a prime number increases the likelihood that the keys are mapped with a uniformity in the output range of values.

Hashing functions

2. Multiplication Method:

Step 1: Choose a constant A such that $0 < A < 1$

Step 2: Multiply the key k by A.

Step 3: Extract the fractional part of kA.

Step 4: Multiply the result of Step 3 by the size of hash table (m).

The hash function can be given as:

$$h(k) = \text{floor}[m * (kA \bmod 1)]$$

Example:

Given a hash table of size 1000, map the key 12345 to an appropriate location in the hash table.

Solution:

Let $A = 0.618033$, $m = 1000$, and $k = 12345$

$$h(12345) = 1000 (12345 * 0.618033 \bmod 1)$$

$$h(12345) = 1000 (7629.617385 \bmod 1)$$

$$h(12345) = 1000 (0.617385)$$

$$h(12345) = 617.385$$

$$h(12345) = 617$$

3. Mid-Square Method:

Step 1: Square the value of the key. That is, find k^2 .

Step 2: Extract the middle r digits of the result obtained in Step 1

Example:

Calculate the hash value for keys 1234 and 5642 using the mid-square method. The hash table has 100 memory locations.

Solution:

Locations whose indices vary from 0 to 99.

Only two digits are needed to map the key to a location in the hash table, so $r = 2$.

When $k = 1234$, $k^2 = 152\textcolor{teal}{27}\textcolor{red}{56}$, $h(1234) = 27$

When $k = 5642$, $k^2 = 3183\textcolor{teal}{21}\textcolor{red}{64}$, $h(5642) = 21$

Note that the **3rd** and **4th** digits starting from the right are chosen

Collision Resolution Techniques

The two most popular methods of resolving collisions are:

1. Open addressing (or Closed hashing Technique)
2. Chaining (or Linked list-based Technique)

Open addressing:

- ▶ The hash table contains two types of values: *sentinel values* (e.g., -1 indicates no value in the location) and *data values*.
- ▶ When a key is mapped to a location, then the value it holds is checked.
- ▶ If it contains a sentinel value, then data can be stored.
- ▶ If the location already has some data, then other slots are examined in the forward direction to find a free slot.
- ▶ If even a single free location is not found, then it is **OVERFLOW** condition.
- ▶ The process of examining memory locations in the hash table is called **probing**

Open addressing can be implemented in the following ways:

- ▶ Linear Probing
- ▶ Quadratic Probing
- ▶ Double Hashing
- ▶ Rehashing

- ▶ If a value is already stored at a location generated by $h(k)$, then the following hash function is used to resolve the collision:

$$h(k, i) = [h'(k) + i] \bmod m$$

- ▶ Where m is the size of the hash $h'(k) = (k \bmod m)$ and i is the probe number that varies from 0 to $m - 1$
- ▶ Linear probing is known for its simplicity.
- ▶ When we have to store a value, we try the slots:

$[h'(k)] \bmod m$ (when $i = 0$), $[h'(k) + 1] \bmod m$ (when $i = 1$), $[h'(k) + 2] \bmod m$ (when $i = 2$),
 $[h'(k) + 3] \bmod m$, $[h'(k) + 4] \bmod m$, and so on, until a vacant location is found.

Example: Consider a hash table of size 10. Using linear probing, insert the keys 72, 27, 36, 24, 63, 81, 92, and 101 into the table.

Initially, the hash table (HT) can be given as:

0	1	2	3	4	5	6	7	8	9
-1	-1	-1	-1	-1	-1	-1	-1	-1	-1

Step-1: Key = 72, probe with $i=0$.

$$h(72, 0) = (72 \bmod 10 + 0) \bmod 10 = (2) \bmod 10 = 2$$

HT[2] is vacant, so 72 can be inserted

0	1	2	3	4	5	6	7	8	9
-1	-1	72	-1	-1	-1	-1	-1	-1	-1

Step-2: Key = 27

$$h(27, 0) = (27 \bmod 10 + 0) \bmod 10 = (7) \bmod 10 = 7$$

HT[7] is vacant, 27 can be inserted

0	1	2	3	4	5	6	7	8	9
-1	-1	72	-1	-1	-1	-1	27	-1	-1

Step 3: Key = 36

$$h(36, 0) = (36 \bmod 10 + 0) \bmod 10 = (6) \bmod 10 = 6$$

HT[6] is vacant, insert key 36 at this location

0	1	2	3	4	5	6	7	8	9
-1	-1	72	-1	-1	-1	36	27	-1	-1

Step 4: Key = 24

$$h(24, 0) = (24 \bmod 10 + 0) \bmod 10 = (4) \bmod 10 = 4$$

HT[4] is vacant, insert key 24

0	1	2	3	4	5	6	7	8	9
-1	-1	72	-1	24	-1	36	27	-1	-1

Step 5: Key = 63

$$h(63, 0) = (63 \bmod 10 + 0) \bmod 10 = (3) \bmod 10 = 3 \text{ HT}[3] \text{ is}$$

vacant, insert key 63.

0	1	2	3	4	5	6	7	8	9
-1	-1	72	63	24	-1	36	27	-1	-1

Step 6: Key = 81

$$h(81, 0) = (81 \bmod 10 + 0) \bmod 10 = (1) \bmod 10 = 1 \text{ HT}[1] \text{ is}$$

vacant, insert key 81

0	1	2	3	4	5	6	7	8	9
0	81	72	63	24	-1	36	27	-1	-1

Step 7: Key = 92

$$h(92, 0) = (92 \bmod 10 + 0) \bmod 10 = (2) \bmod 10 = 2$$

HT[2] is occupied, so we cannot store the key 92 in HT[2].

Therefore, try again for the next location. Thus, probe with $i = 1$.

Probing with $i = 1$, Key = 92

$$h(92, 1) = (92 \bmod 10 + 1) \bmod 10 = (2 + 1) \bmod 10 = 3$$

HT[3] is occupied, so we cannot store the key 92 in HT[3].

Therefore, try again for the next location.

Probing with $i = 2$, Key = 92

$$h(92, 2) = (92 \bmod 10 + 2) \bmod 10 = (2 + 2) \bmod 10 = 4$$

HT[4] is occupied, so we cannot store the key 92 in HT[4].

Therefore, try again for the next location.

Probing with $i = 3$, Key = 92

$$h(92, 3) = (92 \bmod 10 + 3) \bmod 10 = (2 + 3) \bmod 10 = 5$$

HT[5] is vacant, insert key 92

0	1	2	3	4	5	6	7	8	9
-1	81	72	63	24	92	36	27	-1	-1

Step 8: Key = 101

$$h(101, 0) = (101 \bmod 10 + 0) \bmod 10 = (1) \bmod 10 = 1$$

HT[1] is occupied, so we cannot store the key 101 in HT[1], try for the next location.

Probing with $i = 1$, Key = 101

$$h(101, 1) = (101 \bmod 10 + 1) \bmod 10 = (1 + 1) \bmod 10 = 2$$

HT[2] is also occupied, so we cannot store the key in this location.

The procedure will be repeated.....

Pros and Cons

- ▶ Linear probing finds an empty location by doing a **linear search** in the array beginning from position $h(k)$.
- ▶ In the worst case, the search operation may take $n-1$ comparisons, and the running time of the search algorithm may be $O(n)$.
- ▶ The performance of linear probing is sensitive to the distribution of input values.
- ▶ The drawback of this algorithm is that it results in **clustering (Primary clustering)**
- ▶ As the hash table fills, clusters of consecutive cells are formed, and the time required for a search increases with the size of the cluster
- ▶ Because of Primary clustering, there is a higher risk of more collisions where one collision has already taken place.

- ▶ If a value is already stored at a location generated by $h(k)$, then the following hash function is used to resolve the collision:

$$h(k, i) = [h'(k) + c_1i + c_2i^2] \bmod m$$

- ▶ Where m is the size of the hash $h'(k) = (k \bmod m)$ and i is the probe number that varies from 0 to $m - 1$, and c_1 and c_2 are constants such that $c_1, c_2 \neq 0$
- ▶ Quadratic probing does a quadratic search instead of doing a linear search
- ▶ It eliminates the primary clustering phenomenon of linear probing.

Quadratic Probing

Example: Consider a hash table of size 10. Using quadratic probing, insert the keys 72, 27, 36, 24, 63, 81, and 101 into the table. Take $c_1 = 1$ and $c_2 = 3$

Initially, the hash table (HT) can be given as:

$$h(k, i) = [h'(k) + c_1i + c_2i^2] \bmod m$$

0	1	2	3	4	5	6	7	8	9
-1	-1	-1	-1	-1	-1	-1	-1	-1	-1

Step 1: Key = 72

$$h(72, 0) = [72 \bmod 10 + 1 * 0 + 3 * 0] \bmod 10 = [72 \bmod 10] \bmod 10 = 2 \bmod 10 = 2$$

0	1	2	3	4	5	6	7	8	9
-1	-1	72	-1	-1	-1	-1	-1	-1	-1

Step 2: Key = 27

$$h(27, 0) = [27 \bmod 10 + 1 * 0 + 3 * 0] \bmod 10 = [27 \bmod 10] \bmod 10 = 7 \bmod 10 = 7$$

0	1	2	3	4	5	6	7	8	9
-1	-1	72	-1	-1	-1	-1	27	-1	-1

Step 3: Key = 36

$$h(36, 0) = [36 \bmod 10 + 1 * 0 + 3 * 0] \bmod 10 = [36 \bmod 10] \bmod 10 = 6 \bmod 10 = 6$$

0	1	2	3	4	5	6	7	8	9
-1	-1	72	-1	-1	-1	36	27	-1	-1

Step 4: Key = 24

$$h(24, 0) = [24 \bmod 10 + 1 * 0 + 3 * 0] \bmod 10 = [24 \bmod 10] \bmod 10 = 4 \bmod 10 = 4$$

0	1	2	3	4	5	6	7	8	9
-1	-1	72	-1	24	-1	36	27	-1	-1

Step 5: Key = 63

$$h(63, 0) = [63 \bmod 10 + 1 * 0 + 3 * 0] \bmod 10 = [63 \bmod 10] \bmod 10 = 3 \bmod 10 = 3$$

0	1	2	3	4	5	6	7	8	9
-1	-1	72	63	24	-1	36	27	-1	-1

Step 6: Key = 81

$$h(81, 0) = [81 \bmod 10 + 1 * 0 + 3 * 0] \bmod 10 = [81 \bmod 10] \bmod 10 = 1 \bmod 10 = 1$$

0	1	2	3	4	5	6	7	8	9
-1	81	72	63	24	-1	36	27	-1	-1

Step 7: Key = 101

$$h(101, 0) = [101 \bmod 10 + 1 * 0 + 3 * 0] \bmod 10 = [101 \bmod 10 + 0] \bmod 10 = 1 \bmod 10 = 1$$

Collision occurs,

therefore, probe with $i = 1$

$$\begin{aligned} h(101, 1) &= [101 \bmod 10 + 1 * 1 + 3 * 1] \bmod 10 = [101 \bmod 10 + 1 + 3] \bmod 10 \\ &= [101 \bmod 10 + 4] \bmod 10 = [1 + 4] \bmod 10 = 5 \bmod 10 = 5 \end{aligned}$$

0	1	2	3	4	5	6	7	8	9
-1	81	72	63	24	101	36	27	-1	-1

Step 8: Key = 92 ?

Pros and Cons

- ▶ Quadratic probing resolves the primary clustering problem that exists in the linear probing technique.
- ▶ Although quadratic probing is free from primary clustering, it is still liable to what is known as **secondary clustering**.
- ▶ If there is a collision between two keys, then the same probe sequence will be followed for both.
- ▶ With quadratic probing, the probability for multiple collisions increases as the table becomes full.

- ▶ In double hashing, we use two hash functions rather than a single function.

$$h(k, i) = [h_1(k) + i * h_2(k)] \text{ mod } m$$

- ▶ m is the size of the hash table, $h_1(k)$ and $h_2(k)$ are two hash functions where $h_1(k) = (k \text{ mod } m)$ and $h_2(k) = (k \text{ mod } m')$, here m' is some value less than m , usually $m' = m - 1$ or $m - 2$, and i is the probe number that varies from 0 to $m - 1$.
- ▶ If collision occurs, subsequent probes generate locations that are at an offset of $h_2(k) = (k \text{ mod } m')$ from the previous location.
- ▶ Double hashing is free from problems associated with **primary clustering** as well as **secondary clustering**.

Double Hashing

Example: Consider a hash table of size = 10. Using double hashing, insert the keys 72, 27, 36, 24, 63, 81, 92, and 101 into the table. Take $h_1 = (k \bmod 10)$ and $h_2 = (k \bmod 8)$.

Initially, the hash table (HT) can be given as:

$$h(k, i) = [h_1(k) + i * h_2(k)] \bmod m$$

0	1	2	3	4	5	6	7	8	9
-1	-1	-1	-1	-1	-1	-1	-1	-1	-1

Step 1: Key = 72

$$h(72, 0) = [72 \bmod 10 + (0 * (72 \bmod 8))] \bmod 10 = [2 + (0 * 0)] \bmod 10 = 2 \bmod 10 = 2$$

0	1	2	3	4	5	6	7	8	9
-1	-1	72	-1	-1	-1	-1	-1	-1	-1

Step 2: Key = 27

$$h(27, 0) = [27 \bmod 10 + (0 * (27 \bmod 8))] \bmod 10 = [7 + (0 * 3)] \bmod 10 = 7 \bmod 10 = 7$$

0	1	2	3	4	5	6	7	8	9
-1	-1	72	-1	-1	-1	-1	27	-1	-1

Step 3: Key = 36

$$h(36, 0) = [36 \bmod 10 + (0 * (36 \bmod 8))] \bmod 10 = [6 + (0 * 4)] \bmod 10 = 6 \bmod 10 = 6$$

0	1	2	3	4	5	6	7	8	9
-1	-1	72	-1	-1	-1	36	27	-1	-1

Double Hashing

Step 4: Key = 24

$$h(24, 0) = [24 \bmod 10 + (0 * (24 \bmod 8))] \bmod 10 = [4 + (0 * 0)] \bmod 10 = 4 \bmod 10 = 4$$

0	1	2	3	4	5	6	7	8	9
-1	-1	72	-1	24	-1	36	27	-1	-1

Step 5: Key = 63

$$h(63, 0) = [63 \bmod 10 + (0 * (63 \bmod 8))] \bmod 10 = [3 + (0 * 7)] \bmod 10 = 3 \bmod 10 = 3$$

0	1	2	3	4	5	6	7	8	9
-1	-1	72	63	24	-1	36	27	-1	-1

Step 6: Key = 81

$$h(81, 0) = [81 \bmod 10 + (0 * (81 \bmod 8))] \bmod 10 = [1 + (0 * 1)] \bmod 10 = 1 \bmod 10 = 1$$

0	1	2	3	4	5	6	7	8	9
-1	81	72	63	24	-1	36	27	-1	-1

Step 7: Key = 92

$$h(92, 0) = [92 \bmod 10 + (0 * (92 \bmod 8))] \bmod 10 = [2 + (0 * 4)] \bmod 10 = 2 \bmod 10 = 2$$

Collision occurs, try probing with $i = 1$: $h(92, 1) = [92 \bmod 10 + (1 * (92 \bmod 8))] \bmod 10$
 $= [2 + (1 * 4)] \bmod 10 = (2 + 4) \bmod 10 = 6 \bmod 10 = 6$

Again collision occurs, probe with $i=2$: $h(92, 2) = [92 \bmod 10 + (2 * (92 \bmod 8))] \bmod 10$
 $= [2 + (2 * 4)] \bmod 10 = [2 + 8] \bmod 10 = 10 \bmod 10 = 0$

0	1	2	3	4	5	6	7	8	9
92	81	72	63	24	-1	36	27	-1	-1

Step 8: Key = 101 ?

m value decides the performance of the Double hashing technique, m should be a prime number

Rehashing

- ▶ When the hash table becomes nearly full, the number of collisions increases.
- ▶ A better option is to create a new hash table with size double of the original hash table.
- ▶ All the entries in the original hash table will then have to be moved to the new hash table.
- ▶ Though rehashing seems to be a simple process, it is quite expensive and must therefore not be done frequently.
- ▶ Consider the following hash table which is almost full with the function: $h(x) = x \% 5$.

0	1	2	3	4
	26	31	43	17

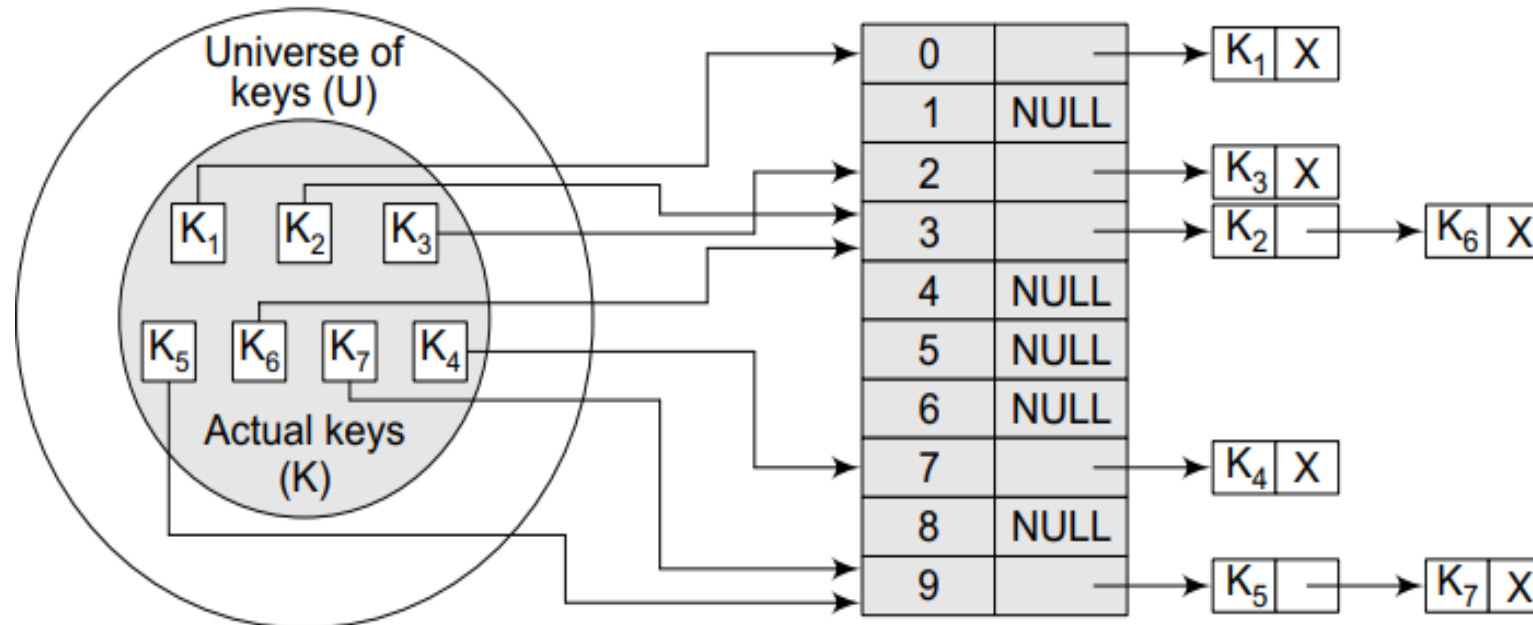
- ▶ New hash table is of 10 locations, double the size of the original table.

0	1	2	3	4	5	6	7	8	9

- ▶ Rehash the key values from the old hash table into the new one using hash function:
 $h(x) = x \% 10$.

0	1	2	3	4	5	6	7	8	9
	31		43			26	17		

- ▶ In chaining, each location in a hash table stores a pointer to a linked list that contains all the key values that were hashed to that location.
- ▶ For example, location *i* in the hash table points to the head of the linked list of all the key values that hashed to *i*



Collision resolution by chaining

Example: Insert the keys 7, 24, 18, 52, 36, 54, 11, and 23 in a chained hash table of 9 memory locations. Use $h(k) = k \bmod m$.

0	NULL
1	NULL
2	NULL
3	NULL
4	NULL
5	NULL
6	NULL
7	NULL
8	NULL

Step 1: Key = 7
 $h(k) = 7 \bmod 9 = 7$

0	NULL
1	NULL
2	NULL
3	NULL
4	NULL
5	NULL
6	NULL
7	→ [7 X]
8	NULL

Step 2: Key = 24
 $h(k) = 24 \bmod 9 = 6$

Step 3: Key = 18
 $h(k) = 18 \bmod 9 = 0$

Step 4: Key = 52
 $h(k) = 52 \bmod 9 = 7$

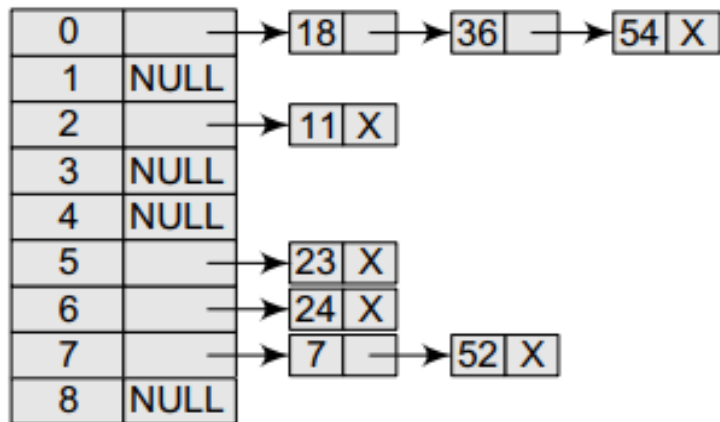
Collision does not occur, insert 52 at the end of the node with 7.

0	NULL
1	NULL
2	NULL
3	NULL
4	NULL
5	NULL
6	→ [24 X]
7	→ [7 X]
8	NULL

0	→ [18 X]
1	NULL
2	NULL
3	NULL
4	NULL
5	NULL
6	→ [24 X]
7	→ [7 X]
8	NULL

0	→ [18 X]
1	NULL
2	NULL
3	NULL
4	NULL
5	NULL
6	→ [24 X]
7	→ [7 X] → [52 X]
8	NULL

Collision resolution by chaining



```
MAX =101;
struct node
{
    int value;
    struct node *next;
};
```

```
initializeHT(node *HT[], int m)
{
    int i;
    for(i=0; i<MAX; i++)
        HT[i]=NULL;
}
```

```
node *insert_value( node *HT[MAX], int k)
{
    node *new_node;
    new_node = (node *)malloc(sizeof(node));
    h(k) = k%MAX;
    new_node→value = k;
    new_node→next = HT[h(k)];
    HT[h(k)] = new_node;
}
```

Pros and Cons

- ▶ Effective even when the number of key values to be stored is much higher than the number of locations in the hash table.
- ▶ Free from clustering problems and provides an efficient mechanism to handle collisions.
- ▶ Chained hash tables inherit the disadvantages of linked lists.

C1: Course Instructional Materials

Course Code	CSE 102
Course Title	Data Structures

The Course instructional material is available in Annexure-I

The shared Google folder **Annexure-I: Course Instructional Materials** link is given below.

https://drive.google.com/drive/folders/1HvQskzxweV_JW7lnjUvOaF0uhSRkb4dM?usp=drive_link